ویژه‌ی آمادگی کوییز

کتاب جامع درسنامه و تست برنامه‌نویسی وب

JavaScript، کار با DOM در صفحه‌ی وب، Node.js و TypeScript — از صفر تا تسلط، با درسنامه‌ی ساده، تست‌های چهارگزینه‌ای سطح‌بندی‌شده و پاسخنامه‌ی تشریحی

JavaScript · DOM · Node.js · npm · TypeScript

```javascript
console.log("5" + 1);       // ?
console.log("5" - 1);       // ?
console.log([] == false);   // ?
setTimeout(() => console.log("timeout"), 0);
Promise.resolve().then(() => console.log("promise"));
```

۳	۱۳	۲۷۳	۲۷۳
فصل	درسنامه	تست چهارگزینه‌ای	پاسخ تشریحی

مطابق سه مجموعه اسلاید درس: «JS in HTML»، «Introduction to JS» و «NodeJS & TS». پاسخ همه‌ی تست‌های کدنویسی با اجرای واقعی کد در Node.js، jsdom و کامپایلر TypeScript (tsc) بررسی شده است.

فهرست مطالب

راهنمای استفاده از کتاب و برنامه‌ی مطالعه ۳

فصل ۱ مقدمه‌ای بر JavaScript ۵

۱-۱ متغیرها: let، const و var ۶

فصل ۲ JavaScript در HTML (کار با DOM) ۱۷

۲-۱ مرور پیش‌نیازها: سه لایه‌ی وب، شیء، this و async ۱۸

۲-۲ زبان‌های نشانه‌گذاری و درخت DOM ۳۲

۲-۳ سلسله‌مراتب DOM در JavaScript و شیء document ۴۷

۲-۴ ساختن و افزودن عناصر جدید ۶۰

۲-۵ حذف و جایگزینی عناصر ۷۵

۲-۶ ویژگی‌ها (Attributes) و کلاس‌ها: setAttribute و classList ۸۸

فصل ۳ Node.js و TypeScript ۱۰۰

۳-۱ محدودیت‌های JavaScript و تولد Node.js ۱۰۱

۳-۲ ویژگی‌های Node.js ۱۱۳

۳-۳ مشکلات تایپ در JavaScript و معرفی TypeScript ۱۲۷

۳-۴ متغیرها و انواع داده در TypeScript ۱۳۷

۳-۵ کلاس‌ها در TypeScript ۱۴۹

۳-۶ Generics در TypeScript ۱۶۹

پاسخنامه‌ی تشریحی ۱۸۳

فصل ۱ پاسخ‌های مقدمه‌ای بر JavaScript ۱۸۳

فصل ۲ پاسخ‌های JavaScript در HTML (کار با DOM) ۱۸۶

فصل ۳ پاسخ‌های Node.js و TypeScript ۲۰۱

راهنمای استفاده از کتاب

این کتاب برای دانشجویی نوشته شده که **هیچ پیش‌زمینه‌ای** از JavaScript ندارد و می‌خواهد در چند روز برای کوییز درس برنامه‌نویسی وب (CE 40419) آماده شود. محدوده‌ی کتاب دقیقاً همان سه مجموعه اسلاید درس است: «Introduction to JS»، «JS in HTML» و «NodeJS & TS». همه‌ی مفاهیم اسلایدها پوشش داده شده‌اند؛ چه مفاهیمی که به‌صورت متن روی اسلایدند و چه کدها، نمودارها، میم‌ها و «Class Activity»هایی که فقط به‌صورت تصویر آمده‌اند. هر جا برای فهم واقعی یک اسلاید به پیش‌زمینه‌ی نزدیکی نیاز بوده (مثل TDZ، event loop یا bubbling رویدادها)، آن هم آموزش داده شده است.

این کتاب ۱۳ درسنامه و ۲۷۳ تست چهارگزینه‌ای با پاسخ تشریحی دارد.

ساختار کتاب

- کتاب سه فصل دارد: **فصل ۱** مقدمه‌ای بر JavaScript، **فصل ۲** JavaScript در HTML (کار با DOM) و **فصل ۳** Node.js و TypeScript.
- هر فصل به درس‌های کوچک تقسیم شده است. هر درس دو بخش دارد:
  1. **درسنامه:** توضیح ساده و قدم‌به‌قدم با مثال، تشبیه، نکته و دام تستی.
  2. **تست‌های تمرینی:** بلافاصله بعد از درسنامه، برای تثبیت همان درس.
- درس‌ها به ترتیب **یادگیری** چیده شده‌اند (نه لزوماً ترتیب اسلایدها). زیر عنوان هر درس نوشته شده که از کدام اسلایدها آمده است.
- آخر هر فصل یک **آزمون جامع فصل** با تست‌های ترکیبی و چالشی آمده است.
- **پاسخنامه‌ی تشریحی** همه‌ی تست‌ها در انتهای کتاب است؛ به ترتیب فصل و درس و با همان شماره‌ها. اول هر فصلِ پاسخنامه یک **کلید سریع** هم هست.
- شماره‌ی تست‌ها در کل کتاب پشت سر هم است. در فایل PDF روی شماره‌ی هر تست بزنید تا به پاسخش بروید و در پاسخنامه روی شماره بزنید تا به خود تست برگردید. فهرست مطالب هم قابل کلیک است.

سطح تست‌ها

برچسب	یعنی
آسان	سؤال مستقیم از خود مفهوم
متوسط	به‌کار بستن مفهوم در یک مثال
سخت	سؤال ترکیبی، دام‌دار یا پیش‌بینی خروجی کد

جعبه‌های درسنامه

> **نکته**
>
> نکته‌های مهمی که باید حفظ باشید.

> **دام تستی**
>
> اشتباه‌های رایجی که طراحان سؤال روی آن‌ها تست می‌سازند. قبل از کوییز همه‌ی این جعبه‌ها را دوباره بخوانید.

در درسنامه‌ها جعبه‌های «یک تشبیه ساده»، «مثال»، «بیشتر بدانیم» و «جمع‌بندی» هم می‌بینید.

خروجی کدها را چطور بخوانیم؟

در تست‌های «خروجی کد زیر چیست؟» هر خانه‌ی خاکستری **یک خط** از خروجی است و خانه‌ها از چپ به راست به همان ترتیبی که چاپ می‌شوند آمده‌اند. مثلاً گزینه‌ی `123` `33` `123` یعنی اول `33` چاپ می‌شود و در خط بعد `123` . خانه‌ی قرمز `Uncaught TypeError` یعنی برنامه در آن نقطه با این خطا متوقف می‌شود.

- خروجی‌ها مطابق `console.log` در **Node.js** نوشته شده‌اند: رشته‌ها در سطح اول بدون کوتیشن چاپ می‌شوند و رشته‌های داخل آرایه با کوتیشن تکی، مثل `['a', 'b']` .
- هر جا نتیجه در مرورگر و Node.js فرق دارد، در صورت سؤال گفته شده که منظور کدام محیط است.
- در فصل TypeScript فرض بر این است که تنظیم `strict` روشن است (پیش‌فرض پروژه‌هایی که با `tsc --init` ساخته می‌شوند).

> **درستی پاسخ‌ها**
>
> پاسخ همه‌ی تست‌هایی که به اجرای کد وابسته‌اند **با اجرای واقعی کد** بررسی شده است: کدهای JavaScript با Node.js، کدهای DOM با jsdom (شبیه‌ساز مرورگر) و کدهای TypeScript با کامپایلر `tsc` . یک اسکریپت خودکار تأیید کرده که در هر کدام از این تست‌ها فقط گزینه‌ی کلید با نتیجه‌ی واقعی یکی است.

روش پیشنهادی مطالعه‌ی هر درس

1. درسنامه را با دقت بخوانید و کدهای مثال را در ذهن (یا در Console مرورگر) اجرا کنید.
2. تست‌های همان درس را **بدون نگاه کردن به پاسخ** بزنید و کنار هر تست علامت بزنید: مطمئن، شک‌دار یا نمی‌دانم.
3. پاسخ تشریحی همه‌ی تست‌ها را بخوانید، حتی آن‌هایی که درست زده‌اید.
4. تست‌های غلط و شک‌دار را علامت بزنید تا روز آخر دوباره حل کنید.

برنامه‌ی مطالعه‌ی ۴ روزه

روز	چه بخوانیم	تمرکز ویژه
روز اول	فصل ۱، درس‌های ۱ تا ۱۷	انواع داده، Type coercion و `==` ، `let` / `const` / `var` ، scope، توابع
روز دوم	فصل ۱، درس‌های ۱۸ تا ۳۱ و آزمون جامع فصل ۱	hoisting و closure، TDZ، آرایه‌ها، `this` ، Promise، `async/await` و event loop
روز سوم	کل فصل ۲ و درس‌های ۱ تا ۷ فصل ۳	DOM، رویدادها، `preventDefault` ، `localStorage` ، npm و `package.json`
روز چهارم	درس‌های ۸ تا ۱۵ فصل ۳، آزمون‌های جامع فصل ۲ و ۳، مرور تست‌های غلط	TypeScript (generic، interface، کلاس، ماژول) و همه‌ی جعبه‌های «دام تستی»

> **نکته**
>
> در تست‌های خروجی، کد را خط به خط روی کاغذ دنبال کنید و مقدار متغیرها را بنویسید. بیشترِ اشتباه‌ها از عجله در همین مرحله است، نه از ندانستن مفهوم.

فصل اول

۱
مقدمه‌ای بر JavaScript

بر اساس اسلایدهای «3- Introduction to JS.pdf»

در این فصل از صفر شروع می‌کنیم: JavaScript چیست، از کجا آمده و چطور اجرا می‌شود. بعد سراغ داده‌ها، تبدیل نوع‌ها و دام‌های معروفش، متغیرها و scope، توابع و closure، hoisting، اشیاء و `this`، آرایه‌ها و متدهایشان، مدیریت خطا و JSON می‌رویم و فصل را با برنامه‌نویسی ناهمگام (callback، Promise، `async/await` و event loop) تمام می‌کنیم. بیشترِ تست‌های «خروجی کد» کوییز از همین فصل طرح می‌شوند.

۱ درس · ۲۳ تست · آزمون جامع پایان فصل

۱-۱ متغیرها: let، const و var

۱-۱ متغیرها: let، const و var

منبع: Introduction to JS، اسلایدهای ۲۳ و ۲۴

درسنامه

فرض کن در آشپزخانه چند ظرف داری و روی هر کدام یک برچسب چسبانده‌ای: «شکر»، «نمک»، «برنج». هر وقت شکر لازم داشته باشی، سراغ ظرفی می‌روی که برچسب «شکر» دارد. در برنامه‌نویسی، **متغیر (variable)** دقیقاً همین ظرف برچسب‌دار است: یک **نام** دارد (برچسب) و یک **مقدار** (چیزی که داخلش ریخته‌ایم).

اسلاید ۲۳ همین را این‌طور می‌گوید:

- متغیرها **ظرف‌هایی برای نگه‌داری داده** هستند (Variables are containers for storing data).
- با یکی از سه کلمه‌ی کلیدی `let`، `const` یا کلمه‌ی قدیمی‌تر `var` تعریف می‌شوند.
- `let` برای **مقدارهای قابل‌تغییر** است و `const` برای **مقدارهای ثابت** (که نمی‌شود دوباره به آن‌ها مقدار داد).
- `var` **قدیمی (outdated)** و **function-scoped** است و نباید در کد مدرن از آن استفاده کرد.
- متغیرها به ما اجازه می‌دهند داده‌ها را بارها **استفاده** و **دست‌کاری** کنیم.

تعریف، انتساب و مقداردهی اولیه

```JavaScript
let city;                // declaration: a box named "city" is created (value: undefined)
city = "Tehran";         // assignment: put a value into the box
let year = 2026;         // declaration + initialization in one line
console.log(city, year); // Tehran 2026
```

این سه اصطلاح را از هم جدا کن:

اصطلاح	یعنی چه؟	مثال
declaration (اعلان یا تعریف)	ساختن متغیر و انتخاب نام برای آن	`let city;`
assignment (انتساب)	ریختن یک مقدار در متغیر با عملگر `=`	`city = "Tehran";`
initialization (مقداردهی اولیه)	تعریف و اولین انتساب، با هم در یک خط	`let year = 2026;`

اگر متغیری را تعریف کنی ولی مقداری در آن نریزی، مقدارش `undefined` (یعنی «هنوز مقدار نگرفته») است:

```JavaScript
let score;
console.log(score); // undefined
score = 10;
console.log(score); // 10
```

`let` : متغیرِ قابل‌تغییر

با `let` متغیری می‌سازی که هر وقت خواستی می‌توانی مقدار تازه‌ای در آن بریزی. به این کار **reassignment** (انتساب دوباره) می‌گویند. این کد اسلاید ۲۴ است:

letconst.js

```
let age = 21;

age = 22;  // works

const pi = 3.14;

pi = 3.1415;  // error
```

خط `age = 22` بی‌دردسر اجرا می‌شود، ولی خط آخر در Node.js و Chrome این خطا را می‌دهد و برنامه همان‌جا متوقف می‌شود:

```
TypeError: Assignment to constant variable.
```

`const` : متغیرِ ثابت

`const` (مخفف constant یعنی «ثابت») متغیری می‌سازد که **بعد از مقداردهی، دیگر نمی‌شود به آن مقدار تازه داد.** هر تلاشی برای انتساب دوباره، چه با `=` و چه با `+=` یا `++`، خطای `TypeError` (خطای نوع) می‌دهد.

دو قانون مهم `const`:

1. **باید همان لحظه‌ی تعریف مقدار بگیرد.** چون بعداً نمی‌شود به آن مقدار داد، تعریفِ خالیِ آن بی‌معنی است:

JavaScript

```
const z; // SyntaxError: Missing initializer in const declaration
```

این خطا از نوع `SyntaxError` (خطای نحوی یا دستوری) است و **قبل از اجرای برنامه** پیدا می‌شود؛ یعنی وقتی در یک فایل باشد، **هیچ خطی از آن فایل اجرا نمی‌شود**، حتی خط‌های قبل از آن.

2. **فقط «اتصال» ثابت است، نه محتوای شیء.** اگر در یک `const` یک شیء (object) یا آرایه (array) بگذاری، نمی‌توانی متغیر را به شیء یا آرایه‌ی دیگری وصل کنی، ولی **می‌توانی محتوای همان شیء را تغییر دهی:**

JavaScript

```
const user = { name: "Ali" };
user.name = "Sara";   // OK: we change the content of the same object
const nums = [1, 2];
nums.push(3);         // OK: nums is now [1, 2, 3]
nums = [4, 5];        // TypeError: Assignment to constant variable.
```

یک تشبیه ساده

`const` مثل برچسبی است که با چسب قطره‌ای روی یک جعبه چسبانده شده: دیگر نمی‌توانی آن را بکَنی و روی جعبه‌ی دیگری بزنی (انتساب دوباره ممنوع است)، ولی اگر جعبه در داشته باشد (شیء یا آرایه)، هنوز می‌توانی وسایل داخلش را جابه‌جا کنی.

دموی کنسول اسلاید ۲۳

در اسلاید ۲۳ این دستورها یکی‌یکی در کنسول Firefox تایپ شده‌اند. علامت `»` یعنی چیزی که ما تایپ کردیم و `←` یعنی مقداری که کنسول به‌عنوان «نتیجه‌ی آن دستور» نشان می‌دهد:

```
» let a = 12
← undefined
» const b = 13
← undefined
» b = 12
Uncaught TypeError: invalid assignment to const 'b'
» a = "Hello!"
← "Hello!"
» console.log(a)
  Hello!
← undefined
```

بیا خط‌به‌خط بخوانیم:

- `let a = 12` و `const b = 13` : یک **تعریف** مقداری تولید نمی‌کند، پس کنسول `undefined` نشان می‌دهد. این یعنی «این دستور نتیجه‌ای نداشت»، **نه** این‌که مقدار `a` برابر `undefined` شده باشد. مقدار `a` همان `12` است.
- `b = 12` : انتساب دوباره به یک `const` ، پس `TypeError` . در Firefox (موتور SpiderMonkey) متن پیام `invalid assignment to const 'b'` است و در Chrome و Node.js (موتور V8) `Assignment to constant variable.` . **متن پیام به موتور بستگی دارد، ولی نوع خطا همیشه** `TypeError` **است.**
- `a = "Hello!"` : هر **انتساب** خودش یک عبارت (expression، یعنی تکه‌کدی که مقدار دارد) است و مقدارش همان چیزی است که منتسب شد؛ برای همین کنسول `"Hello!"` را نشان می‌دهد. این خط ضمناً نشان می‌دهد متغیر `let` که تا الان عدد بود، حالا رشته (string) شده است.
- `console.log(a)` : متن `Hello!` را **چاپ** می‌کند و چون خود `console.log` چیزی برنمی‌گرداند، بعدش `undefined` می‌آید.

> **یادداشت**
>
> کنسول مرورگر هر خط را جداگانه اجرا می‌کند؛ برای همین بعد از خطای خط سوم، دستورهای بعدی هم اجرا شدند. اما اگر همین پنج خط را در **یک فایل** بنویسی و با Node.js اجرا کنی، برنامه روی خط `b = 12` متوقف می‌شود و خط‌های بعدی اصلاً اجرا نمی‌شوند.

`var` : روش قدیمی (پیش‌نمایش)

قبل از ES6 (سال ۲۰۱۵) تنها راه تعریف متغیر `var` بود. اسلاید ۲۳ می‌گوید `var` قدیمی و **function-scoped** است؛ یعنی محدوده‌ی (scope) آن کل تابع است، نه بلوک `{ }` . فعلاً همین دو رفتار عجیبش را بدان:

JavaScript

```
var color = "red";
var color = "blue";  // var allows redeclaration: no error
console.log(color);  // blue

{
  var leaked = "I escape the block";
}
console.log(leaked); // I escape the block (var ignores { } blocks)
```

جزئیات کامل مشکلات `var` (و hoisting) را در درس‌های «Scope»، «مشکلات var» و «Hoisting و TDZ» می‌بینیم.

`let` و `const` هر دو block-scoped هستند (پیش‌نمایش)

اسلاید ۲۴ می‌گوید `let` و `const` هر دو **block-scoped** هستند؛ یعنی فقط داخل همان `{ ... }` (بلوکی) که در آن تعریف شده‌اند وجود دارند:

```JavaScript
{
  let inside = "hi";
  console.log(inside); // hi
}
console.log(inside);   // ReferenceError: inside is not defined
```

تعریف دوباره (redeclaration) در برابر انتساب دوباره (reassignment)

این دو را با هم قاطی نکن. **انتساب دوباره** یعنی ریختن مقدار تازه در متغیری که از قبل هست (`x = 2`). **تعریف دوباره** یعنی نوشتن دوباره‌ی `let x` یا `const x` برای نامی که در همان محدوده از قبل تعریف شده است.

	`let`	`const`	`var`
انتساب دوباره (`x = 2`)	مجاز	`TypeError`	مجاز
تعریف دوباره در همان محدوده	`SyntaxError`	`SyntaxError`	مجاز (بی‌صدا)
مقدار دادن هنگام تعریف	اختیاری	**اجباری**	اختیاری
محدوده (scope)	بلوک `{ }`	بلوک `{ }`	تابع

```JavaScript
let total = 5;
total = 10;     // reassignment: fine
let total = 20; // redeclaration: SyntaxError: Identifier 'total' has already been declared
```

قوانین نام‌گذاری متغیرها

- نام می‌تواند از **حروف**، **رقم‌ها**، `$` و `_` ساخته شود: `price`، `$price`، `_count`، `item2`.
- نام **نمی‌تواند با رقم شروع شود**: `2ndPlace` غلط است.
- فاصله و خط تیره مجاز نیست: `user name` و `first-name` غلط‌اند (خط تیره همان عملگر منها است!).
- **کلمه‌های رزروشده (reserved words)** مثل `class`، `const`، `function`، `return`، `if`، `new` و `typeof` را نمی‌توان نام متغیر گذاشت.
- JavaScript **به بزرگی و کوچکی حروف حساس است (case-sensitive)**: `userName` و `username` دو متغیر کاملاً جدا هستند.
- **قرارداد (convention)**: در JavaScript نام متغیرها را به سبک **camelCase** می‌نویسیم؛ کلمه‌ی اول با حرف کوچک و هر کلمه‌ی بعدی با حرف بزرگ شروع می‌شود: `firstName`، `totalPrice`، `isLoggedIn`. سبک `PascalCase` (مثل `TotalPrice`) معمولاً برای نام کلاس‌ها است و `snake_case` (مثل `total_price`) سبک رایج Python است. این‌ها خطا نمی‌دهند؛ فقط قرارداد نیستند.

Dynamic typing (نوع‌دهی پویا): پیش‌نمایش

در زبان‌هایی مثل Java یا C باید نوع متغیر را بنویسی (`int x = 5;`) و بعداً نمی‌توانی در آن رشته بریزی. در JavaScript متغیرها **نوع ثابت ندارند**؛ نوع به **مقدار** تعلق دارد، نه به متغیر. همان متغیر `let` می‌تواند اول عدد و بعد رشته باشد:

```JavaScript
let data = 42;
console.log(typeof data); // number
data = "forty-two";
console.log(typeof data); // string
```

(عملگر `typeof` نوع یک مقدار را به‌صورت یک رشته برمی‌گرداند؛ در درس بعد کامل با آن آشنا می‌شویم.)

بهترین روش (BEST PRACTICE)

بنر اسلاید ۲۴ می‌گوید: **به‌طور پیش‌فرض از `const` استفاده کن و فقط وقتی به انتساب دوباره نیاز داری، سراغ `let` برو.** از `var` هم در کد جدید استفاده نکن. این‌طوری اگر جایی به‌اشتباه به متغیری مقدار تازه بدهی، همان لحظه خطا می‌گیری و باگ پنهان نمی‌ماند.

> ⚠ **هشدار**
>
> هیچ‌وقت متغیر را بدون `let` / `const` به کار نبر. اگر بنویسی `total = 5` و `total` قبلاً تعریف نشده باشد، در حالت عادی (sloppy) بی‌صدا یک متغیر سراسری (global) ساخته می‌شود و در حالت `"use strict"` خطای `ReferenceError` می‌گیری. خط `a = "Hello!"` در اسلاید ۲۳ مشکلی ندارد، چون `a` قبلاً با `let` تعریف شده بود.

> 💡 **نکته**
>
> - `const` یعنی «انتساب دوباره ممنوع»، نه «محتوای تغییرناپذیر». `push` روی آرایه‌ی `const` و تغییر ویژگی شیء `const` کاملاً مجاز است.
> - هر چیزی که مقدار تازه‌ای در متغیر می‌ریزد انتساب دوباره است: `x = ...` ، `x += 1` ، `x++` . همه‌ی این‌ها روی `const` خطای `TypeError` می‌دهند.
> - متغیری که فقط تعریف شده و مقدار نگرفته `undefined` است. این برای `let` و `var` ممکن است، ولی `const` بدون مقدار اصلاً مجاز نیست.

⚠ دام تستی

- «تعریف `const` بدون مقدار» و «تعریف دوباره‌ی `let` / `const` در همان محدوده» از نوع `SyntaxError` هستند و چون قبل از اجرا کشف می‌شوند، **هیچ خطی از آن فایل اجرا نمی‌شود**، حتی `console.log` های قبل از آن. ولی «انتساب دوباره به `const` » از نوع `TypeError` است و در **زمان اجرا** رخ می‌دهد؛ پس خط‌های قبل از آن اجرا شده‌اند و خروجی‌شان چاپ شده است.
- `undefined` ی که کنسول بعد از `let a = 12` نشان می‌دهد نتیجه‌ی **دستور** است، نه مقدار `a` .
- متن پیام خطا در موتورهای مختلف فرق دارد (`invalid assignment to const 'b'` در Firefox و `Assignment to constant variable.` در Chrome و Node.js)؛ در تست‌ها به **نوع خطا** یعنی `TypeError` دقت کن.
- `let x` داخل یک بلوک جدید `{ }` تعریف دوباره حساب نمی‌شود؛ آن یک متغیر **جدید و جدا** در محدوده‌ی همان بلوک است.

✓ جمع‌بندی

- متغیر یعنی ظرف داده با یک نام. تعریف با `const` (پیش‌فرض)، `let` (وقتی مقدار عوض می‌شود) و `var` (قدیمی؛ استفاده نکن).
- `let` : انتساب دوباره مجاز. `const` : انتساب دوباره `TypeError` و مقداردهی هنگام تعریف اجباری (وگرنه `SyntaxError`).
- `let` و `const` block-scoped هستند و تعریف دوباره‌شان در همان محدوده `SyntaxError` است؛ `var` تابع‌محور است و تعریف دوباره را اجازه می‌دهد.
- نام‌ها: حرف، رقم، `$` و `_` ؛ بدون رقم در ابتدا؛ حساس به حروف بزرگ و کوچک؛ بدون کلمه‌های رزروشده؛ به سبک camelCase.

✓ تست‌های تمرینی

۲۳ تست · شماره‌ی ۱ تا ۲۳

۱ آسان

کدام کلمه‌ی کلیدی متغیری می‌سازد که **نمی‌توان** به آن مقدار تازه‌ای منتسب کرد؟

۱) `let`　۲) `var`　۳) `final`　۴) `const`

۲ آسان

طبق اسلاید ۲۳، کدام توصیف درباره‌ی `var` درست است؟

۱) قدیمی است، function-scoped است و بهتر است در کد مدرن از آن استفاده نشود.

۲) جدیدترین روش تعریف متغیر است که در ES6 به زبان اضافه شد.

۳) مثل `const` است و نمی‌توان به آن مقدار تازه داد.

۴) block-scoped است و فقط داخل `{ ... }` وجود دارد.

۳ آسان

خروجی کد زیر چیست؟

```JavaScript
let age = 21;
age = 22;
age = age + 1;
console.log(age);
```

۱) `23`
۲) `21`
۳) `22`
۴) `Uncaught TypeError`

۴ آسان

خروجی کد زیر چیست؟

```JavaScript
let score;
console.log(score);
score = 10;
console.log(score);
```

۱) `undefined` `10`
۲) `null` `10`
۳) `0` `10`
۴) `Uncaught ReferenceError`

۵ آسان

کد زیر (همان کد اسلاید ۲۴) در Node.js اجرا می‌شود. کدام خط خطا می‌دهد؟

```JavaScript
1 let age = 21;
2 age = 22;
3 const pi = 3.14;
4 pi = 3.1415;
5 console.log(age, pi);
```

۱) خط ۲
۲) خط ۴
۳) خط ۵
۴) هیچ خطایی رخ نمی‌دهد

۶ آسان

طبق «BEST PRACTICE» اسلاید ۲۴، کدام روش برای تعریف متغیرها توصیه می‌شود؟

۱) همیشه `let`، چون انعطاف بیشتری دارد.
۲) به‌طور پیش‌فرض `const` و فقط وقتی انتساب دوباره لازم است `let`.
۳) همیشه `var`، چون در همه‌ی مرورگرها کار می‌کند.
۴) `const` فقط برای ثابت‌های ریاضی مثل عدد پی، و `let` برای بقیه‌ی متغیرها.

۷ متوسط

اجرای کدام قطعه‌کد **خطا** می‌دهد؟

۱) `const user = { name: "Ali" }; user.name = "Sara";`

۲) `const nums = [1, 2]; nums.push(3);`

۳) `let count = 1; count = count + 1;`

۴) `const city = "Tehran"; city = "Shiraz";`

۸ متوسط

خروجی کد زیر چیست؟

```JavaScript
const nums = [1, 2, 3];
nums.push(4);
nums[0] = 10;
console.log(nums);
```

۱) `[1, 2, 3]`

۲) `[1, 2, 3, 4]`

۳) `Uncaught TypeError`

۴) `[10, 2, 3, 4]`

۹ سخت

خروجی کد زیر در Node.js چیست؟

```JavaScript
const settings = { theme: "dark" };
settings.theme = "light";
console.log(settings.theme);
settings = { theme: "blue" };
console.log(settings.theme);
```

۱) `light` `blue`

۲) `dark` `Uncaught TypeError`

۳) `light` `Uncaught TypeError`

۴) `Uncaught TypeError`

۱۰ سخت

کدام خط کد زیر خطا می‌دهد؟

```JavaScript
let a = 1;
const b;
b = 2;
console.log(a, b);
```

۱) خط ۲

۲) خط ۳

۳) خط ۴

۴) هیچ خطایی رخ نمی‌دهد

۱۱ متوسط

در کنسول مرورگر (مثل اسلاید ۲۳) دستور `let a = 12` را وارد می‌کنیم. کنسول بلافاصله بعد از آن، به‌عنوان نتیجه‌ی این دستور چه چیزی نشان می‌دهد؟

۱) `12`　۲) `undefined`　۳) `null`　۴) `"a"`

۱۲ متوسط

در کنسول مرورگر ابتدا `let a = 12` و سپس `a = "Hello!"` را وارد می‌کنیم. کنسول بعد از خط دوم چه مقداری را به‌عنوان نتیجه نشان می‌دهد؟

۱) `"Hello!"`　۲) `undefined`　۳) `12`　۴) `true`

۱۳ سخت

در اسلاید ۲۳، Firefox برای `b = 12` (که قبلاً با `const` تعریف شده بود) پیام `invalid assignment to const 'b'` را نشان داد. اگر همین کار در Node.js انجام شود، کدام گزینه درست است؟

۱) خطایی رخ نمی‌دهد و مقدار `b` برابر `12` می‌شود.

۲) خطا از نوع `SyntaxError` است، چون به `const` اصلاً نمی‌شود چیزی منتسب کرد.

۳) نوع خطا همان `TypeError` است، ولی متن پیام فرق می‌کند: `Assignment to constant variable.`

۴) خطا از نوع `ReferenceError` است، چون متغیر `b` پیدا نمی‌شود.

۱۴ متوسط

کدام گزینه درست است؟

۱) `const` اجازه نمی‌دهد محتوای آرایه‌ای که در آن ذخیره شده تغییر کند.

۲) `let` و `const` هر دو block-scoped هستند؛ یعنی فقط داخل همان `{ ... }` که در آن تعریف شده‌اند وجود دارند.

۳) متغیر `let` فقط یک بار مقدار می‌گیرد و بعد از آن قفل می‌شود.

۴) `var` جدیدتر از `let` است و block-scoped است.

۱۵ سخت

خروجی کد زیر در Node.js چیست؟

```javascript
{
  var a = 1;
  let b = 2;
}
console.log(a);
console.log(b);
```

۱) `1` `2`　۲) `Uncaught ReferenceError`

۳) `undefined` `2`　۴) `1` `Uncaught ReferenceError`

۱۶ سخت

خروجی کد زیر چیست؟

```JavaScript
let level = "outer";
{
  let level = "inner";
  console.log(level);
}
console.log(level);
```

۱) `inner` `inner`
۲) `inner` `outer`
۳) `outer` `outer`
۴) `Uncaught SyntaxError`

۱۷ سخت

کدام خط کد زیر خطا می‌دهد؟

```JavaScript
1 var x = 1;
2 var x = 2;
3 let y = 1;
4 y = 2;
5 let y = 3;
6 console.log(x, y);
```

۱) خط ۲
۲) خط ۴
۳) خط ۵
۴) هیچ خطایی رخ نمی‌دهد

۱۸ سخت

کد زیر به‌صورت یک فایل با Node.js اجرا می‌شود. خروجی کدام است؟

```JavaScript
console.log("start");
let total = 5;
total = 10;
let total = 20;
console.log(total);
```

۱) `start` `20`
۲) `start` `Uncaught SyntaxError`
۳) `start` `10`
۴) `Uncaught SyntaxError`

۱۹ متوسط

کدام تعریف متغیر **بدون خطا** اجرا می‌شود؟

۱) `let first-name = "Ali";`
۲) `let 2ndPlace = "Sara";`
۳) `let class = "A1";`
۴) `let _total$ = 0;`

۲۰ متوسط

خروجی کد زیر چیست؟

```javascript
let userName = "Ali";
let username = "Reza";
userName = "Sara";
console.log(username);
```

۱) `Ali`
۲) `Reza`
۳) `Sara`
۴) `Uncaught SyntaxError`

۲۱ آسان

طبق قرارداد رایج نام‌گذاری در JavaScript (camelCase)، کدام نام برای متغیری که «تعداد کل سفارش‌ها» را نگه می‌دارد مناسب‌تر است؟

۱) `total_orders`
۲) `TotalOrders`
۳) `totalOrders`
۴) `totalorders`

۲۲ متوسط

خروجی کد زیر چیست؟

```javascript
let data = 42;
console.log(typeof data);
data = "forty-two";
console.log(typeof data);
```

۱) `number` `string`
۲) `number` `number`
۳) `number` `Uncaught TypeError`
۴) `string` `string`

۲۳ متوسط

در جای خالی کدام کلمه قرار بگیرد تا کد زیر بدون خطا اجرا شود و `3` را چاپ کند؟

```javascript
___ count = 1;
count = count + 2;
console.log(count);
```

۱) `const`
۲) `int`
۳) `let`
۴) `final`

فصل دوم

۲

JavaScript در HTML (کار با DOM)

بر اساس اسلایدهای «4 - JS in HTML.pdf»

حالا JavaScript را به صفحه‌ی وب وصل می‌کنیم. می‌بینیم مرورگر چطور HTML را به درخت DOM تبدیل می‌کند، چطور عنصرها را پیدا، اضافه، حذف و ویرایش کنیم، به رویدادها (event) واکنش نشان دهیم، با فرم‌ها کار کنیم و از امکانات شیء `window` و حافظه‌های `localStorage` و `sessionStorage` استفاده کنیم.

۶ درس · ۱۲۳ تست · آزمون جامع پایان فصل

۲-۱ مرور پیش‌نیازها: سه لایه‌ی وب، شیء this و async
۲-۲ زبان‌های نشانه‌گذاری و درخت DOM
۲-۳ سلسله‌مراتب DOM در JavaScript و شیء document
۲-۴ ساختن و افزودن عناصر جدید
۲-۵ حذف و جایگزینی عناصر
۲-۶ ویژگی‌ها (Attributes) و کلاس‌ها: setAttribute و classList

۲-۱ مرور پیش‌نیازها: سه لایه‌ی وب، شیء، this و async

منبع: JS in HTML، اسلایدهای ۳ تا ۶

درسنامه

این درس، مرور چند پیش‌نیاز است که اسلایدهای ۳ تا ۶ پیش از شروع بحث DOM یادآوری می‌کنند. عنوان این جلسه «**Structure Meets Behaviour!**» است، یعنی «ساختار به رفتار می‌رسد»: قرار است JavaScript (لایه‌ی رفتار) را به HTML (لایه‌ی ساختار) وصل کنیم. طبق اسلاید ۲، این جلسه سه هدف دارد:

1. بفهمیم **DOM** چیست (از درس بعد کامل می‌بینیم).
2. راهی پیدا کنیم که صفحه‌هایمان **تعاملی (interactive)** شوند، یعنی به کارهای کاربر واکنش نشان دهند.
3. بفهمیم **برنامه‌نویسی ناهمگام (async)** در وب به چه کاری می‌آید.

۱) سه لایه‌ی یک وب‌سایت (اسلاید ۳)

یک تشبیه ساده

یک خانه را تصور کن. **اسکلت و دیوارها** مشخص می‌کنند خانه چند اتاق دارد و هر چیز کجاست. **رنگ و دکوراسیون** مشخص می‌کنند خانه چه شکلی دیده شود. **سیم‌کشی برق و کلیدها** مشخص می‌کنند وقتی کلیدی را می‌زنی چه اتفاقی بیفتد. یک صفحه‌ی وب هم دقیقاً همین سه لایه را دارد.

لایه	سؤالی که جواب می‌دهد (اسلاید ۳)	فناوری	مثال
Structure (ساختار)	What the **content** is: محتوا چیست؟	HTML	یک عنوان `<h1>` و یک دکمه‌ی `<button>`
Style (ظاهر)	How it **looks**: چه شکلی است؟	CSS	رنگ، اندازه‌ی فونت، فاصله‌ها و چیدمان
Behaviour (رفتار)	How it **reacts**: چه واکنشی نشان می‌دهد؟	JavaScript	با کلیک روی دکمه، متن عنوان عوض شود

اسلاید ۳ نام فناوری‌ها را ننوشته، ولی تناظر استاندارد (از جلسه‌های قبل) همین است: ساختار = HTML، ظاهر = CSS، رفتار = JavaScript. مثال کوچک زیر هر سه لایه را با هم دارد:

HTML

```
<!DOCTYPE html>
<html>
  <head>
    <!-- Style layer: CSS -->
    <style>
      h1 { color: navy; }
    </style>
  </head>
  <body>
    <!-- Structure layer: HTML -->
    <h1 id="title">Hello World</h1>
    <button id="btn">CLICK ME!</button>

    <!-- Behaviour layer: JavaScript -->
    <script>
      document.getElementById("btn").addEventListener("click", () => {
        document.getElementById("title").textContent = "Goodbye World";
      });
    </script>
  </body>
</html>
```

بدون JavaScript، صفحه فقط چیزی را **نمایش** می‌دهد؛ با JavaScript صفحه **واکنش** نشان می‌دهد (اینجا با کلیک، متن عنوان به `Goodbye World` تغییر می‌کند). البته لایه‌ی رفتار می‌تواند روی دو لایه‌ی دیگر هم اثر بگذارد؛ مثلاً در اسلاید ۱۹ با `title.style.color = 'red'` رنگ متن را از داخل JavaScript عوض می‌کنیم.

۲) اشیاء (Objects) — اسلاید ۴

شیء (object) مجموعه‌ای از داده‌ها و رفتارهای مرتبط است که کنار هم بسته‌بندی شده‌اند. شیء از **جفت‌های کلید–مقدار (key-value pairs)** ساخته می‌شود:

- اگر مقدارِ یک کلید **داده** باشد، به آن **property** (ویژگی) می‌گوییم.
- اگر مقدارِ یک کلید **تابع** باشد، به آن **method** (متد، یعنی تابعی که متعلق به شیء است) می‌گوییم.

JavaScript

```
const student = {
  name: "Ali",            // property (data)
  age: 20,                // property (data)
  greet() {               // method (function)
    return "Hi, I am " + this.name;
  },
};

console.log(student.name);    // "Ali"
console.log(student.greet()); // "Hi, I am Ali"
```

JSON (مخفف JavaScript Object Notation، یعنی «نشانه‌گذاری شیء JavaScript») یک قالب **متنی** رایج برای **ذخیره و به اشتراک گذاشتن** داده‌های شیء است؛ مثلاً برای فرستادن داده به سرور یا ذخیره در حافظه‌ی مرورگر. دو تابع اصلی آن:

```JavaScript
const text = JSON.stringify(student); // object -> JSON string
console.log(text);                    // {"name":"Ali","age":20}   (the method is gone!)

const copy = JSON.parse('{"name":"Sara","age":21}'); // JSON string -> object
console.log(copy.age + 1);            // 22   (age is a real number again)
```

در JSON کلیدها حتماً داخل `"` هستند و **تابع‌ها ذخیره نمی‌شوند**؛ JSON فقط داده نگه می‌دارد، نه رفتار.

«در JavaScript تقریباً همه‌چیز شیء است»

در اسلاید ۴ میم معروف دو فضانورد آمده: اولی به زمینی که رویش لوگوی JavaScript است نگاه می‌کند و می‌گوید «Wait it's all OBJECTS» (صبر کن، همه‌اش شیء است؟) و دومی پشت سرش می‌گوید «Always has been» (همیشه همین بوده). زیر آن نوشته: «از نظر فنی، در JavaScript **تقریباً** همه‌چیز شیء است». آرایه‌ها، تابع‌ها، تاریخ‌ها و (همان‌طور که در این فصل می‌بینیم) **همه‌ی عنصرهای صفحه** شیء هستند:

```JavaScript
console.log(typeof []);             // "object"
console.log(typeof function () {}); // "function"  (functions are special objects)
console.log(typeof null);           // "object"    (a famous historical bug!)
```

پس چرا «تقریباً»؟ چون مقدارهای **primitive** (مقدار اولیه: `string`، `number`، `boolean`، `null`، `undefined` و `bigint`، `symbol`) شیء **نیستند**. با این حال می‌توانی روی یک رشته property بخوانی یا متد صدا بزنی:

```JavaScript
console.log("abc".length);   // 3
console.log((5).toFixed(1)); // "5.0"
```

راز ماجرا **شیء پوششی (wrapper object)** است: وقتی روی یک primitive نقطه می‌گذاری، JavaScript برای یک لحظه یک شیء موقت (`String`، `Number` یا `Boolean`) دور آن می‌سازد، property یا متد را از آن می‌خواند و بلافاصله آن شیء موقت را دور می‌اندازد. به همین دلیل نمی‌توانی روی یک primitive چیزی ذخیره کنی:

```JavaScript
const s = "abc";
s.extra = 42;          // goes to a temporary wrapper (strict mode: TypeError)
console.log(s.extra);  // undefined
console.log(typeof s); // "string"  -> still a primitive
```

ارتباط با این فصل: مرورگر هر تگ HTML را به یک **شیء** تبدیل می‌کند که property دارد (مثل `id` و `textContent`) و method دارد (مثل `addEventListener`). پس هرچه درباره‌ی اشیاء بلدی، اینجا به کارت می‌آید.

۳) کلمه‌ی کلیدی `this` (اسلاید ۵)

قوانین اسلاید ۵ درباره‌ی `this`:

1. `this` به شیئی اشاره می‌کند که تابع را **صدا زده** است.
2. در متدِ یک شیء، `this` خودِ همان شیء است.
3. در کلاس‌ها، `this` همان **نمونه‌ی (instance)** ساخته‌شده با `new` است.

4. در arrow function، `this` از **scope (محدوده‌ی) بیرونی** به ارث می‌رسد؛ arrow function `this` مخصوص به خودش را ندارد.
5. مقدار `this` به **نحوه‌ی صدا زدن** تابع بستگی دارد، نه به جایی که تابع تعریف شده است.

نحوه‌ی فراخوانی	مقدار `this`
`obj.method()`	`obj` (شیءِ قبل از نقطه)
`new Person()` و متدهای کلاس	نمونه‌ی ساخته‌شده
arrow function	همان `this` بیرونِ arrow function
فراخوانی ساده‌ی `f()`	حالت عادی (sloppy): شیء سراسری (`window` در مرورگر، `globalThis` در Node.js)؛ حالت strict و بدنه‌ی کلاس‌ها: `undefined`
listener با `function` معمولی	همان عنصری که listener روی آن ثبت شده است

JavaScript

```
const obj = {
  name: "obj",
  regular() {
    return this.name;
  },
};
const other = { name: "other", regular: obj.regular }; // the same function!
const f = obj.regular;                                 // detached from obj

console.log(obj.regular());   // "obj"
console.log(other.regular()); // "other"  -> depends on the caller
console.log(f());             // Node.js: undefined | browser: "" (see the trap box)
```

arrow functionی که مستقیماً به‌عنوان property در یک object literal نوشته شود، `this` را از **بیرونِ** آن شیء می‌گیرد؛ ولی arrow functionی که **داخل یک متد** نوشته شود، `this` همان متد را به ارث می‌برد که معمولاً همان چیزی است که می‌خواهیم:

JavaScript

```
const counter = {
  count: 10,
  regular() { return this.count; },
  arrow: () => this.count,   // `this` comes from OUTSIDE the object literal
};
console.log(counter.regular()); // 10
console.log(counter.arrow());   // undefined

class Timer {
  constructor(label) {
    this.label = label;            // `this` = the new instance
  }
  report() {
    const show = () => this.label; // arrow: inherits report's `this`
    return show();
  }
}
console.log(new Timer("T1").report()); // "T1"
```

`this` در صفحه‌ی وب: داخل event listener

در درس رویدادها می‌بینیم که با `addEventListener` به مرورگر می‌گوییم «هر وقت روی این عنصر کلیک شد، این تابع را اجرا کن». اینجا هم قانون «نحوه‌ی فراخوانی» برقرار است: مرورگرِ listener معمولی (`function`) را طوری صدا می‌زند که `this` همان **عنصری** باشد که listener روی آن ثبت شده (یعنی `event.currentTarget`). ولی arrow function خودش `this` را ندارد و `this` بیرونی را برمی‌دارد؛ در سطح بالای یک اسکریپت مرورگر، آن `this` همان `window` است.

HTML

```
<button id="a">First</button>
<button id="b">Second</button>
<script>
  document.getElementById("a").addEventListener("click", function (event) {
    console.log(this.id, this === event.currentTarget); // a true
  });
  document.getElementById("b").addEventListener("click", (event) => {
    console.log(this === window, event.currentTarget.id); // true b
  });
</script>
```

نکته

اگر داخل listener به خودِ عنصر نیاز داری، مطمئن‌ترین راه `event.currentTarget` است؛ هم در `function` معمولی جواب می‌دهد و هم در arrow function.

دام تستی: `this.name` در مرورگر و Node.js

در کد بالا `f()` (متدی که از شیء جدا شده) در **Node.js** مقدار `undefined` می‌دهد، ولی در **مرورگر** رشته‌ی خالی `""`! چون در فراخوانی ساده (حالت sloppy)، `this` همان `window` است و `window` از قبل یک property به نام `name` دارد که مقدار پیش‌فرضش `""` است. پس در تست‌ها همیشه دقت کن کد در کدام محیط اجرا می‌شود.

> ⚠ **دام تستی**
>
> arrow functionی که مستقیماً به‌عنوان property در object literal نوشته شود، `this` را **شیء** نمی‌گیرد. برای متدهایی که به `this` نیاز دارند از نحو `method() { }` یا `function` استفاده کن.

۴) یادآوری JavaScript ناهمگام (اسلاید ۶)

چرا async در وب این‌قدر مهم است؟

JavaScript در مرورگر روی یک **thread اصلی** (رشته‌ی اجرای اصلی) اجرا می‌شود و همین thread کارهای دیگری مثل رسم صفحه و پاسخ به کلیک و تایپ کاربر را هم انجام می‌دهد. اگر یک کد همگام (synchronous) چند ثانیه منتظر بماند، در تمام این مدت صفحه **یخ می‌زند**: دکمه‌ها کار نمی‌کنند و حتی اسکرول هم گیر می‌کند. در وب سه نوع کارِ «منتظرماندنی» زیاد داریم:

- **زمان‌سنج‌ها (timers)**: مثل `setTimeout` و `setInterval` (در درس شیء `window` می‌بینیم).
- **رویدادهای کاربر**: معلوم نیست کاربر کی کلیک می‌کند؛ پس تابعی می‌دهیم که «هر وقت کلیک شد» اجرا شود.
- **درخواست‌های شبکه**: گرفتن داده از سرور ممکن است از چند صدم ثانیه تا چند ثانیه طول بکشد.

راه‌حل این است که کار زمان‌بر را شروع کنیم، تابعی برای «بعداً» ثبت کنیم و فوراً به کار خودمان ادامه دهیم تا صفحه پاسخ‌گو بماند. این همان هدف سوم جلسه است: «Understand async programming usages in web».

سه سبک: callback، Promise و async/await

۱. Callback (تابع برگشتی): تابعی را به تابع دیگر می‌دهیم تا **بعداً** اجرا شود. خطاها را باید **دستی** بررسی کنیم؛ معمولاً اولین پارامتر callback خطاست (`err`):

JavaScript

```
function loadData(ok, callback) {
  setTimeout(() => {
    if (ok) callback(null, "data");
    else callback(new Error("failed"));
  }, 10);
}

loadData(false, (err, data) => {
  if (err) {                                  // manual error handling
    console.log("callback error:", err.message);
    return;
  }
  console.log("callback data:", data);
});
// callback error: failed
```

وقتی چند کار ناهمگام باید پشت سر هم انجام شوند، callbackها تو در تو می‌شوند و کد به شکل یک هرمِ کج درمی‌آید که خواندن و مدیریت خطایش سخت است. به این وضعیت **callback hell** («جهنم callbackها») می‌گویند:

```JavaScript
// Illustration only: getUser, getOrders and getDetails are imaginary functions.
getUser(1, (err, user) => {
  getOrders(user, (err, orders) => {
    getDetails(orders[0], (err, details) => {
      console.log(details); // deeper and deeper...
    });
  });
});
```

۲. Promise (وعده): شیئی که نماینده‌ی یک **مقدار آینده** است و یکی از سه **حالت (state)** را دارد: `pending` (در انتظار)، `fulfilled` (انجام‌شده) یا `rejected` (ردشده). نتیجه را با `.then()` و خطا را با `.catch()` می‌گیریم؛ زنجیره‌ی `.then()` ها کد را از حالت تو در تو بیرون می‌آورد:

```JavaScript
function fetchUser(ok) {
  return new Promise((resolve, reject) => {
    setTimeout(() => (ok ? resolve("user loaded") : reject(new Error("network down"))), 30);
  });
}

fetchUser(true)
  .then((value) => console.log("then:", value))    // then: user loaded
  .catch((err) => console.log("catch:", err.message));
```

۳. async/await: بر پایه‌ی همان Promiseهاست، ولی نحو (syntax) تمیزتری دارد و کد را **شبیه کد همگام** نشان می‌دهد. `await` فقط اجرای **همان تابع async** را تا آماده شدن نتیجه متوقف می‌کند، نه کل برنامه را. خطاها با `try...catch` گرفته می‌شوند:

```JavaScript
async function main() {
  try {
    console.log(await fetchUser(true));  // user loaded
    console.log(await fetchUser(false)); // rejects -> jumps to catch
    console.log("after");                // never runs
  } catch (err) {
    console.log("caught:", err.message); // caught: network down
  }
}
main();
```

سبک	ایده	مدیریت خطا (اسلاید ۶)	ویژگی
Callback	تابعی را برای اجرا در آینده می‌دهیم	دستی (مثلاً بررسی `err`)	تو در تو شدن: callback hell
Promise	نماینده‌ی یک مقدار آینده با چند حالت	`.catch()`	زنجیره با `.then()` / `.catch()`
async/await	نحو ساده‌تر روی Promise	`try...catch`	ظاهر همگام و خوانایی بیشتر

ترتیب اجرا (یادآوری فصل اول)

ابتدا همه‌ی کد همگام اجرا می‌شود، بعد **microtask**ها (کارهای کوچکِ فوری، مثلِ callback `.then` و ادامه‌ی تابع بعد از `await`) و بعد **macrotask**ها (مثلِ callback `setTimeout`، حتی با تأخیر `0`). جزئیات event loop در فصل اول آمده است:

```JavaScript
async function load() {
  console.log("load start");
  const data = await Promise.resolve("data ready");
  console.log(data);
}
console.log("script start");
setTimeout(() => console.log("timeout"), 0);
load();
console.log("script end");
// script start -> load start -> script end -> data ready -> timeout
```

راستی، listenerهای رویداد هم نوعی callback هستند: در `btn.addEventListener("click", handler)` تابع `handler` «بعداً»، یعنی هر وقت کاربر کلیک کرد، اجرا می‌شود.

نکته

بدنه‌ی تابع async تا رسیدن به اولین `await` **همگام** اجرا می‌شود؛ برای همین در مثال بالا `load start` قبل از `script end` چاپ شد.

دام تستی

- `setTimeout(fn, 0)` یعنی «حداقل بعد از ۰ میلی‌ثانیه، **هر وقت نوبتش شد**»، نه «همین الان». callbackهای Promise (microtask) همیشه زودتر از آن اجرا می‌شوند.
- `JSON.stringify` متدها را حذف می‌کند، و `typeof null` برابر `"object"` است، با اینکه `null` شیء نیست.

جمع‌بندی

- ساختار = HTML (محتوا چیست)، ظاهر = CSS (چه شکلی است)، رفتار = JavaScript (چه واکنشی نشان می‌دهد).
- شیء = جفت‌های کلید–مقدار؛ داده = property و تابع = method؛ JSON قالب متنی برای ذخیره و تبادل داده است.
- primitiveها شیء نیستند، ولی با wrapper object موقت property و متد دارند (`"abc".length`).
- `this` به نحوه‌ی فراخوانی بستگی دارد؛ arrow function، `this` بیرونی را به ارث می‌برد؛ در listener معمولی، `this` همان عنصر است.
- callback (خطای دستی)، async/await، Promise (`.catch()`)، (`try...catch`)؛ ترتیب: همگام، microtask، macrotask.

تست‌های تمرینی

۱۷ تست · شماره‌ی ۲۴ تا ۴۰

۲۴ آسان

در مدل سه‌لایه‌ی یک وب‌سایت (اسلاید ۳)، لایه‌ای که به سؤال «صفحه چه واکنشی نشان می‌دهد؟» (How it reacts) پاسخ می‌دهد کدام است و معمولاً با چه فناوری‌ای ساخته می‌شود؟

۱) Structure، با HTML
۲) Style، با CSS
۳) Behaviour، با JavaScript
۴) Behaviour، با CSS

۲۵ آسان

کدام عبارت درباره‌ی سه لایه‌ی وب نادرست است؟

۱) HTML مشخص می‌کند محتوای صفحه چیست؛ مثلاً یک عنوان و یک دکمه.

۲) CSS مشخص می‌کند صفحه چه شکلی است؛ مثلاً رنگ، اندازه و چیدمان.

۳) JavaScript مشخص می‌کند صفحه به کارهای کاربر چه واکنشی نشان دهد.

۴) تغییر رنگ متن فقط با CSS ممکن است و JavaScript نمی‌تواند ظاهر عنصرها را تغییر دهد.

۲۶ آسان

خروجی کد زیر در Node.js کدام است؟

```JavaScript
const student = {
  name: "Ali",
  age: 20,
  greet() {
    return "Hi, I am " + this.name;
  },
};
console.log(typeof student.name, typeof student.greet);
console.log(student.greet());
```

۱) `string function` `Hi, I am Ali`

۲) `string object` `Hi, I am Ali`

۳) `property method` `Hi, I am Ali`

۴) `string function` `Hi, I am undefined`

۲۷ متوسط

خروجی کد زیر کدام است؟

```JavaScript
const student = {
  name: "Ali",
  age: 20,
  greet() {
    return "Hi";
  },
};
const text = JSON.stringify(student);
console.log(text);
console.log(typeof text, JSON.parse(text).age + 1);
```

۱) `{"name":"Ali","age":20,"greet":"Hi"}` `string 21`

۲) `{"name":"Ali","age":20}` `string 21`

۳) `{name:"Ali",age:20}` `string 21`

۴) `{"name":"Ali","age":20}` `string 201`

۲۸ متوسط

با توجه به جمله‌ی «در JavaScript تقریباً همه‌چیز شیء است» (اسلاید ۴)، کدام گزینه درست است؟

۱) `typeof "abc"` برابر `"object"` است.

۲) `typeof null` برابر `"null"` است.

۳) `typeof []` برابر `"object"` است.

۴) `typeof function () {}` برابر `"object"` است.

۲۹ سخت

خروجی کد زیر در Node.js (حالت عادی، نه strict) کدام است؟

```JavaScript
const s = "abc";
console.log(s.length, s.toUpperCase());
s.extra = 42;
console.log(s.extra, typeof s);
```

۱) `3 ABC` `undefined string`

۲) `3 ABC` `42 object`

۳) `3 ABC` `42 string`

۴) `Uncaught TypeError`

۳۰ متوسط

خروجی کد زیر در Node.js کدام است؟

```JavaScript
const obj = {
  name: "obj",
  regular() {
    return this.name;
  },
};
const other = { name: "other", regular: obj.regular };
const f = obj.regular;
console.log(obj.regular());
console.log(other.regular());
console.log(f());
```

۱) `obj` `obj` `obj`

۲) `obj` `other` `undefined`

۳) `obj` `obj` `undefined`

۴) `obj` `other` `obj`

۳۱ سخت

این کد در یک تگ `<script>` معمولی **در مرورگر** اجرا می‌شود (نه در Node.js). خروجی کدام است؟

```javascript
const user = {
  name: "Ali",
  getName() {
    return this.name;
  },
};
const f = user.getName;
console.log(user.getName());
console.log(JSON.stringify(f()));
```

۱) `Ali` `"Ali"`
۲) `Ali` `undefined`
۳) `Ali` `""`
۴) `Ali` `Uncaught TypeError`

۳۲ متوسط

خروجی کد زیر در Node.js کدام است؟

```javascript
const counter = {
  count: 10,
  regular() {
    return this.count;
  },
  arrow: () => this.count,
};
console.log(counter.regular(), counter.arrow());
```

۱) `10 10`
۲) `undefined 10`
۳) `Uncaught TypeError`
۴) `10 undefined`

۳۳ سخت

خروجی کد زیر کدام است؟

```JavaScript
class Timer {
  constructor(label) {
    this.label = label;
  }
  report() {
    const show = () => this.label;
    return show();
  }
}
const t = new Timer("T1");
console.log(t.report());
const r = t.report;
console.log(r());
```

۱) `T1` `Uncaught TypeError`

۲) `T1` `T1`

۳) `T1` `undefined`

۴) `undefined` `Uncaught TypeError`

۳۴ متوسط

در صفحه‌ی زیر، کاربر ابتدا روی دکمه‌ی First و سپس روی دکمه‌ی Second کلیک می‌کند. خروجی Console کدام است؟

```HTML
<button id="a">First</button>
<button id="b">Second</button>
```

```JavaScript
document.getElementById("a").addEventListener("click", function () {
  console.log(this.textContent);
});
document.getElementById("b").addEventListener("click", () => {
  console.log(this === window);
});
```

۱) `First` `false`

۲) `First` `true`

۳) `undefined` `true`

۴) `First` `Second`

۳۵ آسان

طبق اسلاید ۶، روش مدیریت خطا در هر یک از سه سبک برنامه‌نویسی ناهمگام کدام است؟

۱) Promise — `try...catch` :callback؛ async/await — دستی `.catch()`

۲) async/await — `try...catch` :Promise؛ callback: دستی `.catch()`

۳) هر سه سبک فقط با `try...catch` مدیریت می‌شوند.

۴) callback: دستی (مثلاً بررسی پارامتر `err`) — async/await — Promise: `.catch()` `try...catch`

۳۶ سخت

خروجی کد زیر کدام است؟

```JavaScript
async function load() {
  console.log("load start");
  const data = await Promise.resolve("data ready");
  console.log(data);
}
console.log("script start");
setTimeout(() => console.log("timeout"), 0);
load();
console.log("script end");
```

۱) `script start` `load start` `data ready` `script end` `timeout`

۲) `script start` `script end` `load start` `data ready` `timeout`

۳) `script start` `load start` `script end` `data ready` `timeout`

۴) `script start` `load start` `script end` `timeout` `data ready`

۳۷ متوسط

خروجی کد زیر کدام است؟

```JavaScript
function fetchUser(ok) {
  return new Promise((resolve, reject) => {
    setTimeout(() => (ok ? resolve("user loaded") : reject(new Error("network down"))), 10);
  });
}
async function main() {
  try {
    console.log(await fetchUser(true));
    console.log(await fetchUser(false));
    console.log("after");
  } catch (err) {
    console.log("caught:", err.message);
  } finally {
    console.log("done");
  }
}
main();
```

۱) `user loaded` `caught: network down` `done`

۲) `user loaded` `caught: network down` `after` `done`

۳) `user loaded` `Uncaught Error`

۴) `caught: network down` `done`

۳۸ آسان

چرا در صفحه‌ی وب کارهای زمان‌بر مثل درخواست شبکه را به‌صورت ناهمگام (async) انجام می‌دهیم؟

۱) چون JavaScript در مرورگر هم‌زمان روی چند thread اجرا می‌شود و اجرای همگام در آن ممکن نیست.

۲) چون اگر کد همگام منتظر بماند، صفحه تا پایان آن کار «یخ می‌زند» و به کلیک و تایپ کاربر پاسخ نمی‌دهد.

۳) چون کد ناهمگام همیشه سریع‌تر از کد همگام اجرا می‌شود.

۴) چون مرورگر اجرای توابع همگام را بعد از بارگذاری کامل صفحه ممنوع می‌کند.

۳۹ آسان

کد زیر نمونه‌ای از کدام مشکل است و راه‌حل مدرن آن چیست؟

```JavaScript
getUser(1, (err, user) => {
  getOrders(user, (err, orders) => {
    getDetails(orders[0], (err, details) => {
      console.log(details);
    });
  });
});
```

۱) خطای TDZ؛ باید تعریف تابع‌ها را بالاتر برد.

۲) سایه‌اندازی متغیرها (shadowing)؛ باید نام پارامترهای `err` را عوض کرد.

۳) callback hell؛ بازنویسی با زنجیره‌ی Promise (`.then()`) یا `async/await`.

۴) نشت حافظه در DOM؛ باید listenerها را حذف کرد.

۴۰ متوسط

خروجی کد زیر کدام است؟

```JavaScript
Promise.resolve(5)
  .then((n) => n * 2)
  .then((n) => {
    if (n > 5) throw new Error("too big");
    return n;
  })
  .then((n) => console.log("value:", n))
  .catch((err) => console.log("error:", err.message));
console.log("sync");
```

۱) `sync` `value: 10`

۲) `value: 10` `sync`

۳) `sync` `error: too big`

۴) `error: too big` `sync`

۲-۲ زبان‌های نشانه‌گذاری و درخت DOM

منبع: JS in HTML، اسلایدهای ۷ تا ۱۱

درسنامه

تا اینجا HTML را فقط به چشم «متنی پر از تگ» دیده‌ایم. اما مرورگر وقتی صفحه را باز می‌کند، این متن را به یک **درخت از اشیاء** تبدیل می‌کند که JavaScript می‌تواند آن را بخواند و تغییر دهد. این درخت همان **DOM** است و پایه‌ی کل این فصل.

زبان نشانه‌گذاری (Markup language) چیست؟ (اسلاید ۷)

زبان نشانه‌گذاری (markup language) زبانی است که با **تگ‌ها (tags)** یا نشانه‌ها، **ساختار** یک سند را توصیف می‌کند: کدام بخش عنوان است، کدام بخش پاراگراف است و چه چیزی داخل چه چیزی قرار دارد. اسلاید ۷ سه نمونه نام می‌برد: XML، HTML و Markdown.

HTML

```
<!-- HTML: the structure of a web page -->
<h1>Shop</h1>
<p>Welcome to <b>our</b> shop!</p>
```

XML

```
<!-- XML: the structure of any data, with our own tag names -->
<student>
  <name>Ali</name>
  <age>20</age>
</student>
```

Markdown

```
# Shop
Welcome to **our** shop!
```

(Markdown به‌جای تگ‌های `<...>` از نشانه‌هایی مثل `#` برای عنوان و `**` برای متن پررنگ استفاده می‌کند، ولی کارش همان است: توصیف ساختار.) زبان نشانه‌گذاری، زبان برنامه‌نویسی **نیست**؛ شرط، حلقه و محاسبه ندارد و فقط می‌گوید هر بخش «چیست». منطق و رفتار کار JavaScript است.

سند، یک درخت سلسله‌مراتبی است

چون تگ‌ها **تو در تو (nested)** نوشته می‌شوند، سند به شکل یک **درخت سلسله‌مراتبی (hierarchical tree)** درمی‌آید: هر تگی که داخل تگ دیگری باشد **فرزند (child)** آن است و تگ بیرونی **والد (parent)** آن. به گفته‌ی اسلاید ۷، رابطه‌ی والد و فرزند نشان می‌دهد محتوا چطور تو در تو شده است، و همین مدل درختی **پایه‌ی DOM** است.

> **یک تشبیه ساده**
>
> شجره‌نامه‌ی یک خانواده یا ساختار پوشه‌های کامپیوترت را در نظر بگیر: داخل پوشه‌ی `Documents` پوشه‌ی `University` است و داخل آن چند فایل. هر پوشه (به‌جز پوشه‌ی ریشه) دقیقاً یک والد دارد و ممکن است چند فرزند داشته باشد. درخت یک سند HTML هم دقیقاً همین‌طور است.

نمودار اسلاید ۷ یک درخت عمومی است (A به B و C و D فلش دارد، B به E و F، و E و F هر کدام یک فرزند دارند):

```
        A
      /  |  \
     B   C   D
    / \
   E   F
   |   |
   G   H
```

واژه	معنی	در نمودار بالا
ریشه (root)	تنها گره‌ای که والد ندارد	`A`
والد (parent)	گره‌ی بلافاصله بالای یک گره	والدِ `E` گره‌ی `B` است
فرزند (child)	گره‌ی بلافاصله پایین یک گره	فرزندان `A` : `B` ، `C` و `D`
هم‌نیا (sibling)	گره‌هایی که والد مشترک دارند	`B` ، `C` و `D` ؛ همچنین `E` و `F`
جد (ancestor)	والد، والدِ والد و ... تا ریشه	اجداد `G` : `E` ، `B` و `A`
نواده (descendant)	فرزند، فرزندِ فرزند و ...	نوادگان `B` : `E` ، `F` ، `G` و `H`
برگ (leaf)	گره‌ای که هیچ فرزندی ندارد	`C` ، `D` ، `G` و `H`

از نشانه‌گذاری تا DOM (اسلاید ۸)

مرورگر متن HTML را **parse** می‌کند (یعنی می‌خواند و تجزیه می‌کند) و از آن یک **درخت از گره‌ها (nodes)** می‌سازد. قوانین اسلاید ۸:

- هر **تگ** به یک **گره‌ی عنصر (element node)** در درخت تبدیل می‌شود.
- **متنِ داخل تگ‌ها** به **گره‌ی متنی (text node)** تبدیل می‌شود.
- **attributeها** (ویژگی‌های داخل تگ، مثل `href` و `id`) به **propertyهای گره** تبدیل می‌شوند؛ یعنی به خودِ عنصر چسبیده‌اند و **فرزند آن نیستند**.
- همین درخت DOM است که JavaScript می‌تواند به آن دسترسی پیدا کند و تغییرش دهد.

برای HTML زیر:

```HTML
<html>
  <head>
    <title>My title</title>
  </head>
  <body>
    <a href="#">My link</a>
    <h1>My header</h1>
  </body>
</html>
```

نمودار W3Schools در اسلاید ۸ این درخت را نشان می‌دهد (اینجا به شکل متنی بازکشیده شده است):

```
Document
`-- Root element: <html>
    |-- Element: <head>
    |   `-- Element: <title>
    |       `-- Text: "My title"
    `-- Element: <body>
        |-- Element: <a>   ---- Attribute: "href"
        |   `-- Text: "My link"
        `-- Element: <h1>
            `-- Text: "My header"
```

دقت کن که در نمودار، جعبه‌ی `Attribute: "href"` **کنار** `<a>` و با یک خط افقی به آن وصل شده، نه زیر آن؛ چون attribute فرزند نیست، ویژگیِ خودِ عنصر است. جمله‌ی کنار نمودار هم همین را می‌گوید: «ببین درخت DOM عنصرها را چطور مرتب می‌کند».

DOM چیست؟ (اسلاید ۹)

DOM مخفف **Document Object Model** (مدل شیءِ سند) است. هر کلمه‌اش معنایی دارد:

- **Document**: سند، یعنی همان صفحه‌ی وب.
- **Object**: هر بخش صفحه (هر عنصر، متن و ...) به یک **شیء** JavaScript تبدیل می‌شود که property و method دارد.
- **Model**: این اشیاء با هم یک **مدل** (نمایش ساختاریافته) از صفحه می‌سازند که شکلش درخت است.

طبق اسلاید ۹، DOM:

- یک **نمایش درختی (tree-based representation)** از صفحه‌ی وب است؛
- **ساختار HTML را به JavaScript وصل می‌کند**؛
- به برنامه‌ها اجازه می‌دهد عنصرها را **بخوانند، تغییر دهند، اضافه یا حذف کنند**؛
- مثل یک **پل** بین محتوا و تعامل‌پذیری (interactivity) عمل می‌کند.

نمودار Milne Publishing در اسلاید ۹ همان ایده را با رنگ‌بندی نشان می‌دهد. این بار `<h1>` قبل از `<a>` آمده و ریشه با حروف کوچک `document` نوشته شده، درست مثل نام شیء در JavaScript:

```
document
`-- Root element: <html>
    |-- Element: <head>
    |   `-- Element: <title>
    |       `-- Text: "My title"
    `-- Element: <body>
        |-- Element: <h1>
        |   `-- Text: "A heading"
        `-- Element: <a>   ---- Attribute: href
            `-- Text: "Link text"
```

> **DOM همان فایل HTML نیست**
>
> فایل `.html` فقط **دستور ساخت** است. مرورگر از روی آن، درخت DOM را **در حافظه** می‌سازد. وقتی JavaScript عنصری را تغییر می‌دهد، فقط DOM (همان درختِ داخل حافظه) عوض می‌شود و فایل روی دیسک یا سرور دست نمی‌خورد؛ با refresh صفحه همه‌چیز از روی فایل اصلی دوباره ساخته می‌شود. در DevTools، زبانه‌ی Elements (یا Inspector) همین DOM زنده را نشان می‌دهد.

فعالیت کلاسی اسلاید ۱۰: درخت DOM این HTML را رسم کن

کد اسلاید ۱۰ (فایل `sample.html`) دقیقاً این است:

```HTML
<html>
    <head>
        <title>sample page here :)</sample>
        <link rel="stylesheet" href="./style.css">
    </head>
    <body>
        <h1>SAMPLE PAGE!</h1>
        <div class="sampleclass">
            <p>sample paragraph in sample class in sample page!</p>
            <h3 id="2223">wait wha-?</h3>
        </div>
    </body>
</html>
```

پاسخ مورد انتظار (با فرض اینکه خط ۳ با `</title>` بسته شده باشد؛ گره‌های متنیِ فقط‌فاصله را نشان نداده‌ایم و attributeها داخل `{ }` کنار عنصرشان آمده‌اند، چون فرزند نیستند):

```
#document
`-- html
    |-- head
    |   |-- title
    |   |   `-- #text "sample page here :)"
    |   `-- link   {rel="stylesheet", href="./style.css"}
    `-- body
        |-- h1
        |   `-- #text "SAMPLE PAGE!"
        `-- div   {class="sampleclass"}
            |-- p
            |   `-- #text "sample paragraph in sample class in sample page!"
            `-- h3   {id="2223"}
                `-- #text "wait wha-?"
```

چند نکته درباره‌ی این درخت:

- `head` و `body` هم‌نیا هستند و هر دو فرزند `html` اند؛ `p` و `h3` هم‌نیا و فرزند `div` اند؛ `div` والد `p` و جدِ متن `"wait wha-?"` است.
- `link` فرزندی ندارد (تگ خالی است) و `rel` و `href` ویژگی‌های آن هستند.
- در درخت واقعی، بین تگ‌ها گره‌های متنی‌ای هم هست که فقط شامل «خط جدید و فاصله» (whitespace) است. مثلاً `head` در واقع ۵ گره‌ی فرزند دارد: فاصله، `title`، فاصله، `link`، فاصله.

> ⚠ **دامِ تایپیِ اسلاید: `</sample>` به‌جای `</title>`**
>
> روی اسلاید، `<title>` با `</sample>` بسته شده که تگی بی‌معنی است. تگ `<title>` در HTML حالت خاصی دارد: parser همه‌ی محتوای آن را تا رسیدن به یک `</title>` واقعی **متن خام** حساب می‌کند و تگ‌های داخلش را تشخیص نمی‌دهد. چون در این فایل هیچ `</title>` ی وجود ندارد، **کل باقی فایل** (از `</sample>` تا `</html>`) به متن عنوان تبدیل می‌شود و `<body>` **خالی** می‌ماند: صفحه سفید دیده می‌شود و یک عنوان خیلی طولانی در زبانه‌ی مرورگر می‌نشیند.
>
> نکته‌ی مهم‌تر: **parser هیچ‌وقت به‌خاطر HTML نادرست خطا (exception) نمی‌دهد**؛ همیشه سعی می‌کند سند را «تعمیر» کند، حتی اگر نتیجه آن چیزی نباشد که انتظار داشتی.

دو نکته‌ی دیگر درباره‌ی این فایل: اول اینکه `<!DOCTYPE html>` ندارد، پس مرورگر صفحه را در «حالت سازگاری» (quirks mode) نشان می‌دهد؛ همیشه سند را با `<!DOCTYPE html>` شروع کن. دوم اینکه id عنصر `h3` با رقم شروع شده (`2223`)؛ در درس «پیدا کردن عناصر» می‌بینیم این موضوع برای `querySelector` دردسر می‌سازد.

ساختار DOM (اسلاید ۱۱)

خلاصه‌ی اسلاید ۱۱:

- **ریشه‌ی درخت (root node)، `document` است.**
- **عنصرها (elements)**: همان تگ‌ها، مثل `<div>` و `<p>`.
- **attributeها**: propertyهای عنصر، مثل `id` و `class`.
- **متن**: گره‌های متنی (text nodes).
- همه در یک درخت سلسله‌مراتبی به هم وصل‌اند.

دقت کن: `document` **گره‌ی ریشه** است، ولی **عنصر ریشه (root element)** تگ `<html>` است که فرزند `document` است و در JavaScript با `document.documentElement` به آن می‌رسیم.

سلسله‌مراتب کلاس‌ها (نمودار StackOverflow در اسلاید ۱۱)

هر گره در DOM نمونه‌ای از یک **کلاس** است و این کلاس‌ها از هم ارث می‌برند. نمودار اسلاید ۱۱ (از چپ به راست کشیده شده) را اینجا از بالا به پایین می‌بینی؛ `EventTarget` در نمودار اسلاید نیست ولی بالای همه قرار دارد:

```
EventTarget                 (not on the slide: the top of every chain)
`-- Node
    |-- Document
    |   `-- HTMLDocument
    |-- CharacterData
    |   |-- Text
    |   `-- Comment
    |-- Element
    |   `-- HTMLElement
    |       |-- HTMLHeadElement
    |       |-- HTMLBodyElement
    |       |-- HTMLTitleElement
    |       |-- HTMLParagraphElement
    |       |-- HTMLInputElement
    |       |-- HTMLTableElement
    |       `-- ... etc.  (many more element types)
    `-- Attr
```

رابطه‌ی ارث‌بری یعنی رابطه‌ی «**هست یک**»: یک `<p>` یک `HTMLParagraphElement` است، پس یک `HTMLElement`، یک `Element`، یک `Node` (و یک `EventTarget`) هم هست و همه‌ی propertyها و متدهای آن‌ها را دارد: از `Node` چیزهایی مثل `parentNode` و `childNodes`، از `Element` چیزهایی مثل `id` و `children`، از `HTMLElement` چیزهایی مثل `style`، و از `EventTarget` متد `addEventListener`. به همین دلیل روی هر گره‌ای می‌توان listener گذاشت.

HTML

```
<p id="p">Hi<!-- note --></p>
<script>
  const p = document.getElementById("p");
  console.log(p instanceof HTMLParagraphElement, p instanceof Element, p instanceof Node); // true
true true
  console.log(p.firstChild instanceof Text, p.firstChild instanceof CharacterData);       // true
true
  console.log(p.firstChild instanceof Element);                                           // false
  console.log(p.lastChild instanceof Comment, document instanceof HTMLDocument);          // true
true
</script>
```

- `Text` و `Comment` (گره‌ی توضیح `<!-- ... -->`) هر دو زیرکلاس `CharacterData` هستند، **نه** `Element`.
- `Attr` (کلاس attributeها) مستقیماً از `Node` ارث می‌برد، ولی در `childNodes` هیچ عنصری ظاهر نمی‌شود.
- `document` نمونه‌ای از `HTMLDocument` است که از `Document` و آن هم از `Node` ارث می‌برد.

پیمایش درخت با JavaScript (پیش‌زمینه‌ی لازم)

هر گره دو property شناسایی دارد: `nodeType` (یک عدد) و `nodeName` (یک رشته):

گره	`nodeType`	`nodeName`
عنصر (مثلاً `<p>`)	`1`	نام تگ با حروف بزرگ، مثلاً `"P"`
attribute	`2`	نام attribute، مثلاً `"href"`
متن	`3`	`"#text"`
comment	`8`	`"#comment"`
`document`	`9`	`"#document"`

برای حرکت در درخت این propertyها را داریم:

property	چه برمی‌گرداند	گره‌های متنی و comment را هم حساب می‌کند؟
`parentNode`	گره‌ی والد	—
`childNodes`	**همه‌ی** گره‌های فرزند	بله
`children`	فقط **عنصرهای** فرزند	خیر
`firstChild` / `lastChild`	اولین / آخرین گره‌ی فرزند	بله
`firstElementChild` / `lastElementChild`	اولین / آخرین عنصر فرزند	خیر
`previousSibling` / `nextSibling`	گره‌ی هم‌نیای بعدی / قبلی	بله
`previousElementSibling` / `nextElementSibling`	عنصر هم‌نیای بعدی / قبلی	خیر

مهم‌ترین دام اینجاست: **خط جدید و فاصله‌هایی که برای مرتب نوشتن HTML می‌گذاریم، گره‌ی متنی می‌سازند.**

HTML

```
<ul id="list">
  <li>A</li>
  <li>B</li>
</ul>
<script>
  const ul = document.getElementById("list");
  console.log(ul.childNodes.length, ul.children.length);              // 5 2
  console.log(ul.firstChild.nodeName, ul.firstElementChild.nodeName); // #text LI
  console.log(ul.parentNode.nodeName);                                // BODY
  console.log(ul.children[1].previousElementSibling.textContent);     // A
</script>
```

چرا ۵؟ چون فرزندان `ul` این‌ها هستند: متنِ فاصله، `li`، متنِ فاصله، `li`، متنِ فاصله. اگر همین لیست را در یک خط و بدون فاصله بنویسیم (`<ul><li>A</li><li>B</li></ul>`)، `childNodes.length` هم `2` می‌شود.

> **نکته**
>
> اگر فقط با تگ‌ها کار داری، از نسخه‌های «Element» استفاده کن: `children`، `firstElementChild`، `nextElementSibling`. نسخه‌های بدون Element (`childNodes`، `firstChild`، `nextSibling`) گره‌های متنی، حتی فاصله‌ها، و commentها را هم می‌شمارند.

⚠ دام تستی

- attributeها فرزند نیستند: برای `<a href="#">Link</a>` مقدار `a.childNodes.length` برابر `1` است (فقط گره‌ی متنی)، نه `2`.
- گره‌ی ریشه `document` است ولی عنصر ریشه `<html>` (`document.documentElement`)؛ خودش `document` عنصر نیست (`nodeType` آن `9` است، نه `1`).
- `nodeName` عنصرها با حروف بزرگ است (`"LI"`)، و `nodeName` گره‌ی متنی `"#text"` است، نه `"text"`.
- `Text` زیرکلاس `CharacterData` است، نه `Element`؛ پس `instanceof Element` برای گره‌ی متنی `false` است.

جمع‌بندی

- زبان نشانه‌گذاری (HTML، XML، Markdown) ساختار را با تگ یا نشانه توصیف می‌کند؛ سند یک درخت است.
- تگ ← گره‌ی عنصر، متن ← گره‌ی متنی، attribute ← property عنصر (فرزند نیست).
- DOM = Document Object Model: نمایش درختیِ صفحه در حافظه، پل بین HTML و JavaScript.
- ریشه `document` است؛ کلاس‌ها: Document/CharacterData/Element/Attr ← Node و ← Element ← HTMLParagraphElement ← HTMLElement و ...
- `children` فقط عنصرها، `childNodes` همه‌ی گره‌ها (از جمله فاصله‌ها).

تست‌های تمرینی

۲۱ تست · شماره‌ی ۴۱ تا ۶۱

۴۱ آسان

کدام گزینه یک زبان نشانه‌گذاری (markup language) نیست؟

۱) HTML　۲) XML　۳) Markdown　۴) JavaScript

۴۲ آسان

DOM مخفف چیست؟

۱) Data Object Model　۲) Document Object Model
۳) Document Oriented Markup　۴) Dynamic Object Method

۴۳ آسان

در درخت زیر (نمودار اسلاید ۷)، کدام گره‌ها برگ (leaf) هستند؟



۱) فقط `G` و `H`　۲) `E`، `F`، `G` و `H`　۳) `C`، `D`، `G` و `H`　۴) `A` و `B`

۴۴ متوسط

در همان درخت اسلاید ۷، کدام عبارت نادرست است؟

```
        A
      /  |  \
     B   C   D
    / \
   E   F
   |   |
   G   H
```

۱) `B`، `C` و `D` هم‌نیا (sibling) هستند.

۲) `A` جدِ (ancestor) `H` است.

۳) `E` فرزند `B` و والد `G` است.

۴) `F` و `G` هم‌نیا هستند.

۴۵ آسان

طبق اسلاید ۸، وقتی مرورگر HTML را به DOM تبدیل می‌کند، attributeهای یک تگ (مثل `href` در `<a>`) به چه چیزی تبدیل می‌شوند؟

۱) به یک گره‌ی متنی (text node) که فرزند همان عنصر است

۲) به propertyهای همان گره‌ی عنصر

۳) به یک گره‌ی عنصر جداگانه که هم‌نیای همان عنصر است

۴) دور ریخته می‌شوند و در DOM باقی نمی‌مانند

۴۶ متوسط

خروجی کد زیر در مرورگر کدام است؟

HTML

```html
<a id="home" href="/">Home page</a>
```

JavaScript

```javascript
const a = document.getElementById("home");
console.log(a.childNodes.length, a.firstChild.nodeName, a.attributes.length);
```

۱) `1 #text 2`

۲) `2 #text 2`

۳) `3 A 2`

۴) `1 HREF 1`

۴۷ متوسط

خروجی کد زیر کدام است؟

HTML

```html
<p id="p">Hello <b>big</b> world</p>
```

JavaScript

```javascript
const p = document.getElementById("p");
console.log(p.childNodes.length, p.children.length);
```

۱) `1 1`

۲) `2 1`

۳) `3 3`

۴) `3 1`

۴۸ متوسط

خروجی کد زیر کدام است؟

```HTML
<ul id="list">
  <li>A</li>
  <li>B</li>
</ul>
```

```JavaScript
const ul = document.getElementById("list");
console.log(ul.childNodes.length, ul.children.length);
```

۱) `2 2`
۲) `5 2`
۳) `3 2`
۴) `5 5`

۴۹ سخت

خروجی کد زیر کدام است؟

```HTML
<ul id="list">
  <li>A</li>
  <li>B</li>
</ul>
```

```JavaScript
const ul = document.getElementById("list");
console.log(ul.firstChild.nodeName, ul.firstElementChild.nodeName,
ul.lastElementChild.textContent);
```

۱) `LI LI B`
۲) `#text LI A`
۳) `#text LI B`
۴) `LI #text B`

۵۰ متوسط

خروجی کد زیر کدام است؟

```HTML
<p id="p">Hi</p>
```

```JavaScript
const p = document.getElementById("p");
console.log(document.nodeType, document.body.nodeType, p.firstChild.nodeType);
```

۱) `9 1 3`
۲) `1 1 3`
۳) `0 1 2`
۴) `9 1 1`

۵۱ متوسط

خروجی کد زیر کدام است؟

```HTML
<p id="p">Hi</p>
```

```JavaScript
const p = document.getElementById("p");
console.log(document.nodeName, document.documentElement.nodeName, p.firstChild.nodeName);
```

۱) `document html text`
۲) `#document BODY #text`
۳) `DOCUMENT HTML TEXT`
۴) `#document HTML #text`

۵۲ آسان

طبق اسلاید ۱۱، ریشه‌ی درخت DOM (root node) کدام است؟

۱) `window`
۲) عنصر `<html>`
۳) `document`
۴) عنصر `<body>`

۵۳ سخت

خروجی کد زیر کدام است؟

```HTML
<!DOCTYPE html>
<html>
<head><title>Root</title></head>
<body><p>Hi</p></body>
</html>
```

```JavaScript
const root = document.documentElement;
console.log(root.tagName, root.parentNode === document, document.childNodes.length);
```

۱) `HTML true 1`
۲) `HTML true 2`
۳) `HTML false 2`
۴) `BODY true 2`

۵۴ سخت

HTML فعالیت کلاسی اسلاید ۱۰ را با یک اصلاح (بستن `<title>` با `</title>`) در مرورگر باز می‌کنیم و کد زیر در انتهای `<body>` اجرا می‌شود. خروجی کدام است؟

HTML

```
<html>
    <head>
        <title>sample page here :)</title>
        <link rel="stylesheet" href="./style.css">
    </head>
    <body>
        <h1>SAMPLE PAGE!</h1>
        <div class="sampleclass">
            <p>sample paragraph in sample class in sample page!</p>
            <h3 id="2223">wait wha-?</h3>
        </div>
    </body>
</html>
```

JavaScript

```
const div = document.querySelector(".sampleclass");
const h3 = document.getElementById("2223");
console.log(div.children.length, div.parentNode.tagName);
console.log(h3.previousElementSibling.tagName, document.head.children.length);
```

۱) `2 BODY` `P 2`

۲) `2 HTML` `P 2`

۳) `3 BODY` `H1 2`

۴) `2 BODY` `P 1`

۵۵ سخت

HTML اسلاید ۱۰ را **دقیقاً همان‌طور که روی اسلاید است** (با `</sample>` به‌جای `</title>`) در مرورگر باز می‌کنیم:

```HTML
<html>
    <head>
        <title>sample page here :)</sample>
        <link rel="stylesheet" href="./style.css">
    </head>
    <body>
        <h1>SAMPLE PAGE!</h1>
        <div class="sampleclass">
            <p>sample paragraph in sample class in sample page!</p>
            <h3 id="2223">wait wha-?</h3>
        </div>
    </body>
</html>
```

سپس در Console این دستورها را اجرا می‌کنیم. خروجی کدام است؟

```JavaScript
console.log(document.body.children.length);
console.log(document.getElementById("2223") === null);
console.log(document.title.includes("SAMPLE PAGE!"));
```

۱) `2` `false` `false`

۲) `0` `true` `true`

۳) `Uncaught SyntaxError`

۴) `2` `false` `true`

۵۶ متوسط

طبق نمودار سلسله‌مراتب کلاس‌ها در اسلاید ۱۱، کدام رابطه‌ی ارث‌بری نادرست است؟

۱) `HTMLParagraphElement` از `HTMLElement`، آن از `Element` و آن از `Node` ارث می‌برد.

۲) `Text` و `Comment` هر دو زیرکلاس `CharacterData` هستند.

۳) `Text` زیرکلاس `Element` است، چون متن هم داخل تگ‌ها قرار دارد.

۴) `Attr` (کلاس attributeها) مستقیماً از `Node` ارث می‌برد.

۵۷ سخت

خروجی کد زیر کدام است؟

HTML

```
<p id="p">Hi</p>
```

JavaScript

```
const p = document.getElementById("p");
console.log(
  p instanceof HTMLElement,
  p.firstChild instanceof Element,
  p.firstChild instanceof CharacterData,
  document instanceof Element
);
```

۱) `true true true false`
۲) `true false false true`
۳) `false false true false`
۴) `true false true false`

۵۸ متوسط

کدام عبارت درباره‌ی DOM درست است؟

۱) DOM نمایش درختیِ صفحه در حافظه‌ی مرورگر است که JavaScript از طریق آن می‌تواند عنصرها را بخواند، تغییر دهد، اضافه یا حذف کند.

۲) DOM یک زبان برنامه‌نویسی جدا از JavaScript است که برای ساخت صفحه به کار می‌رود.

۳) DOM همان فایل `.html` روی دیسک است و تغییر DOM با JavaScript، فایل را هم تغییر می‌دهد.

۴) DOM فقط برای خواندن صفحه است و نمی‌توان با آن عنصر جدیدی ساخت یا حذف کرد.

۵۹ آسان

در نمودار W3Schools (اسلاید ۸)، والدِ گره‌ی متنی `"My title"` کدام است؟

۱) عنصر `<title>`
۲) عنصر `<head>`
۳) عنصر `<html>`
۴) `Document`

۶۰ سخت

خروجی کد زیر کدام است؟

HTML

```
<div id="box"><!-- note --><span>A</span>text</div>
```

JavaScript

```
const box = document.getElementById("box");
console.log(box.childNodes.length, box.children.length, box.firstChild.nodeType);
```

۱) `2 1 8`
۲) `3 1 3`
۳) `3 1 8`
۴) `1 1 1`

۶۱ متوسط

خروجی کد زیر کدام است؟

HTML

```
<h1 id="head">A heading</h1><a id="link" href="#">Link text</a>
```

JavaScript

```
const h1 = document.getElementById("head");
const a = document.getElementById("link");
console.log(a.parentNode.tagName, a.previousElementSibling.id,
h1.nextElementSibling.getAttribute("href"));
```

۱) `BODY head #`

۲) `H1 head #`

۳) `BODY link #`

۴) `BODY head null`

۲-۳ سلسله‌مراتب DOM در JavaScript و شیء document

منبع: JS in HTML، اسلایدهای ۱۲ تا ۱۵

درسنامه

در درس قبل دیدیم مرورگر از HTML یک درخت می‌سازد. حالا ببینیم این درخت در JavaScript از کجا شروع می‌شود و چطور به آن دست پیدا می‌کنیم.

یک تشبیه ساده

مرورگر را یک **قاب عکس** فرض کن (`window`) که داخلش یک **نقاشی** قرار دارد (`document`)؛ هر جزء نقاشی (درخت، خانه، آسمان) یک **عنصر** است. قاب چیزهای دیگری هم دارد که به خودِ نقاشی ربطی ندارند: مثلاً برچسب آدرسِ پشت قاب (`location`) یا فهرست نقاشی‌هایی که قبلاً در این قاب بوده‌اند (`history`).

سلسله‌مراتب DOM در JavaScript (اسلاید ۱۲)

سطح	توضیح اسلاید ۱۲	در کد
`window`	بالاترین شیء مرورگر؛ همان **global context** (بافت سراسری) که در فصل اول دیدیم	`window`
`document`	صفحه‌ی وبِ داخل پنجره	`window.document` یا فقط `document`
عنصرها	تگ‌ها (`<div>` ، `<p>` ، `<h1>`) به‌صورت شیء در درخت DOM	`document.getElementById("title")`
attributeها و متن	propertyهای گره‌های عنصر	`h1.id` ، `h1.className` ، `h1.textContent`
رویدادها (events)	متدها و listenerهایی که JavaScript را به کارهای کاربر وصل می‌کنند	`btn.addEventListener("click", ...)`

`window` شیء سراسری (global object) مرورگر است؛ پس هر چیزی را که روی `window` باشد می‌توان بدون نوشتن `window.` هم استفاده کرد: `document` همان `window.document` است و `alert` همان `window.alert` . (جزئیات `window` را در درس «شیء window» می‌بینیم.)

HTML

```
<h1 id="title" class="big">Hello</h1>
<script>
  console.log(window.document === document);                    // true
  const h1 = document.querySelector("h1");
  console.log(h1.id, h1.className, h1.textContent, h1.tagName); // title big Hello H1
</script>
```

نمودار اسلاید ۱۲

نمودار اسلاید ۱۲ (از منبع CIS) را اینجا به شکل متنی می‌بینی. روی اسلاید، فرزندان `document` در دو ستون دو طرف یک خط عمودی کشیده شده‌اند (anchors و applets، links و embeds، plugins و frames، scripts و images، stylesheets و forms) و `forms` خودش به `elements` وصل است:

```
window
|-- document
|   |-- anchors
|   |-- links
|   |-- applets
|   |-- plugins
|   |-- embeds
|   |-- scripts
|   |-- frames
|   |-- stylesheets
|   |-- images
|   `-- forms
|       `-- elements
|-- event
|-- frames
|-- location
|-- history
|-- navigator
|   |-- plugins
|   `-- mimeTypes
`-- screen
```

فرزند `window` در نمودار	چیست؟
`document`	خودِ صفحه، یعنی درخت DOM
`event`	رویدادی که همین حالا در حال پردازش است (قدیمی؛ امروز از پارامتر `event` در listener استفاده می‌کنیم)
`frames`	قاب‌های داخلی صفحه (`<iframe>` ها)
`location`	آدرس (URL) صفحه‌ی فعلی
`history`	تاریخچه‌ی صفحه‌های بازشده در این زبانه (برای back و forward)
`navigator`	اطلاعات خودِ مرورگر؛ در نمودار دو فرزند دارد: `plugins` (افزونه‌ها) و `mimeTypes` (نوع فایل‌هایی که مرورگر می‌شناسد)
`screen`	اطلاعات صفحه‌نمایش (مانیتور) کاربر

فرزندان `document` در نمودار، **مجموعه‌هایی از عنصرهای خاص صفحه** هستند: `images` همه‌ی تصاویر، `links` لینک‌ها، `forms` فرم‌ها (و `elements` هر فرم، کادرهای ورودیِ آن فرم)، `scripts` تگ‌های `<script>`، `embeds` و `plugins` محتوای جاسازی‌شده، `anchors` لنگرها، و `stylesheets` شیوه‌نامه‌ها (فایل‌های CSS).

> **بیشتر بدانیم: این نمودار قدیمی است**
>
> این نمودار مربوط به «مدل کلاسیک» مرورگرهاست و پیام اصلی‌اش این است: `window` در بالاترین سطح است، `document` یکی از فرزندان آن است، و `window` فرزندان دیگری هم دارد. جزئیات نام‌ها را حفظ نکن: appletها سال‌هاست از مرورگرها حذف شده‌اند، نام property واقعی در مرورگرهای امروزی `document.styleSheets` (با S بزرگ) است، `frames` در عمل property خودِ `window` است و `window.event` هم منسوخ (deprecated) شده است.

شیء `document` (اسلاید ۱۳)

طبق اسلاید ۱۳، شیء `document` :

- نماینده‌ی **کل صفحه‌ی وب** در مرورگر است؛
- **نقطه‌ی ورود (entry point)** به درخت DOM است، یعنی تقریباً هر کاری با صفحه با `document.` شروع می‌شود؛
- متدهایی برای **پیدا کردن عنصرها** دارد، مثل `getElementById` و `querySelector` (درس بعد)؛
- به JavaScript اجازه می‌دهد گره‌ها را **بسازد، حذف کند یا به‌روزرسانی کند**.

چند property پرکاربرد آن:

property	چه چیزی را برمی‌گرداند
`document.documentElement`	عنصر `<html>` (عنصر ریشه)
`document.head`	عنصر `<head>`
`document.body`	عنصر `<body>`
`document.title`	متن عنوان صفحه (همان متنِ زبانه‌ی مرورگر)؛ قابل خواندن و تغییر

مثال اسلاید ۱۳ (فایل `doc.html`):

doc.html

```
<!DOCTYPE html>
<html>
  <body>
    <h1 id="title">Hello World</h1>
    <button id="btn">Change Text</button>

    <script>
      // Access an element
      const heading = document.getElementById("title");

      // Add event to button
      document.getElementById("btn").addEventListener("click", () => {
        heading.textContent = "Text Changed via document!";
      });
    </script>
  </body>
</html>
```

خط به خط:

1. `document.getElementById("title")` شیء عنصر `<h1>` را پیدا می‌کند و در `heading` نگه می‌دارد.
2. `document.getElementById("btn")` دکمه را پیدا می‌کند و `addEventListener("click", ...)` تابعی (اینجا یک arrow function) ثبت می‌کند که با هر کلیک اجرا شود.
3. با کلیک، `heading.textContent = "..."` متن `<h1>` را عوض می‌کند و صفحه فوراً به‌روز می‌شود.
4. کلیک دوم همان متن را دوباره می‌نویسد، پس تغییر دیگری دیده نمی‌شود.

`<script>` در **انتهای** `<body>` و بعد از عنصرهایی که لازم دارد آمده است؛ برای همین `getElementById` آن‌ها را پیدا می‌کند (پایین‌تر علتش را می‌بینیم). در ضمن تابعِ داخل listener متغیر `heading` را از بیرون به خاطر سپرده است (همان closure فصل

اول)، پس عنصر فقط یک بار جست‌وجو می‌شود.

پیش‌بینی خروجی Console (اسلاید ۱۴ و ۱۵)

اسلاید ۱۴ کد زیر را اجرا می‌کند؛ کدی که JavaScript را داخل HTML جاسازی کرده است، و می‌پرسد: «می‌توانی حدس بزنی چه چیزی در Console چاپ می‌شود؟»

doc.html

```
<!DOCTYPE html>
<html>
  <body>
    <h1 id="title">Hello World</h1>

    <script>
      const heading = document.getElementById("title");
      console.log(heading)
      </script>
  </body>
</html>
```

پاسخ اسلاید ۱۵: **خودِ عنصر** چاپ می‌شود، که منطقی است چون خودِ عنصر را به `console.log` داده‌ایم. در Firefox، زبانه‌ی Console سطری مثل `▶ <h1 id="title">` نشان می‌دهد و سمت راستش لینک `doc.html:8:15` آمده؛ یعنی «فایل `doc.html`، خط ۸، ستون ۱۵»، همان جایی که `console.log` نوشته شده است. با زدن مثلث `▶` فرزندان عنصر باز می‌شوند و با بردن ماوس روی آن، عنصر در صفحه هایلایت می‌شود.

- چیزی که چاپ شده **شیء گره** است، نه رشته‌ی `"Hello World"`. برای متن باید `heading.textContent` و برای نام تگ `heading.tagName` را چاپ کنیم.
- `null` هم چاپ نمی‌شود، چون اسکریپت بعد از `<h1>` آمده و عنصر از قبل در درخت وجود دارد.
- نبودن `;` بعد از `console.log(heading)` مشکلی ایجاد نمی‌کند؛ JavaScript خودش سمی‌کالن را درج می‌کند (automatic semicolon insertion). فاصله‌ی اضافه‌ی قبل از `</script>` هم فقط ظاهری است.

اسلاید ۱۵ با این سؤال تمام می‌شود: «`document` چه کارهای جالب دیگری می‌تواند بکند؟!»؛ جواب، باقی همین فصل است: پیدا کردن عنصرها، تغییر متن و استایل، رویدادها و ساختن و حذف عنصرها.

چرا جای تگ `<script>` مهم است؟ (پیش‌زمینه‌ی لازم)

مرورگر HTML را **از بالا به پایین** می‌خواند و درخت را کم‌کم می‌سازد. هر جا به یک `<script>` معمولی برسد، ساختن درخت را متوقف می‌کند، اسکریپت را اجرا می‌کند و بعد ادامه می‌دهد. پس اسکریپت فقط عنصرهایی را می‌بیند که **بالای آن** هستند:

HTML

```
<!DOCTYPE html>
<html>
<head>
  <script>
    const heading = document.getElementById("title");
    console.log(heading);            // null  (the <h1> is not parsed yet)
    heading.textContent = "Changed"; // TypeError: Cannot set properties of null
  </script>
</head>
<body>
  <h1 id="title">Hello World</h1>
</body>
</html>
```

`getElementById` وقتی عنصری پیدا نکند `null` برمی‌گرداند، و هر تلاشی برای خواندن یا نوشتن property روی `null` خطای `TypeError` می‌دهد. سه راه‌حل:

راه‌حل	توضیح
`<script>` در انتهای `<body>`	روشی که اسلایدها استفاده می‌کنند؛ وقتی اسکریپت اجرا می‌شود همه‌ی عنصرهای بالایی ساخته شده‌اند
رویداد `DOMContentLoaded`	`document.addEventListener("DOMContentLoaded", () => { ... })`؛ کد وقتی اجرا می‌شود که کل HTML خوانده شده و درخت DOM آماده است
ویژگی `defer`	`<script src="app.js" defer></script>`؛ فایل خارجی هم‌زمان دانلود می‌شود ولی **بعد از** پایان parse اجرا می‌شود. `defer` فقط روی اسکریپت‌های **خارجی (با** `src` **)** اثر دارد

رویداد `load` روی `window` از این هم دیرتر است: منتظر می‌ماند تا **همه‌ی منابع** (تصاویر، فایل‌های CSS و ...) هم بارگیری شوند. ترتیب واقعی در این مثال:

HTML

```
<!DOCTYPE html>
<html>
<head>
  <script>
    console.log("head", document.getElementById("title"));
    document.addEventListener("DOMContentLoaded", () => {
      console.log("DOMContentLoaded", document.getElementById("title").textContent);
    });
    window.addEventListener("load", () => console.log("load"));
  </script>
</head>
<body>
  <h1 id="title">Hello World</h1>
  <script>
    console.log("end of body", document.getElementById("title").tagName);
  </script>
</body>
</html>
```

```
head null
end of body H1
DOMContentLoaded Hello World
load
```

نکته

`console.log(element)` در مرورگر خودِ گره را (به‌صورت قابل باز شدن) نشان می‌دهد. اگر متن را می‌خواهی `element.textContent` و اگر نوع عنصر را می‌خواهی `element.tagName` را چاپ کن. تبدیل عنصر به رشته (`String(element)`) فقط چیزی مثل `"[object HTMLHeadingElement]"` می‌دهد.

دام تستی

- اسکریپتِ داخل `<head>` (بدون `defer` یا `DOMContentLoaded`) عنصرهای `<body>` را نمی‌بیند: `getElementById` مقدار `null` می‌دهد و استفاده از آن `TypeError` می‌دهد.
- گذاشتن `defer` روی یک اسکریپت **درون‌خطی** (بدون `src`) هیچ اثری ندارد.
- رابطه برعکس نیست: `document` یکی از propertyهای `window` است، نه اینکه `window` داخل `document` باشد.
- اگر عنصری را **بیرون** از `DOMContentLoaded` (زودتر از موعد) جست‌وجو کنی و فقط listener را داخل آن بگذاری، باز هم `null` در دست مانده است.

جمع‌بندی

- `window` (بالاترین شیء و global context) ← `document` (صفحه) ← عنصرها (اشیاء درخت)؛ attributeها و متن، propertyهای عنصرها هستند و رویدادها JavaScript را به کاربر وصل می‌کنند.
- `document` نقطه‌ی ورود به DOM است: پیدا کردن، ساختن، حذف و به‌روزرسانی گره‌ها.
- `console.log(heading)` خودِ عنصر را چاپ می‌کند، نه متنش را.
- اسکریپت فقط عنصرهای بالای خودش را می‌بیند؛ راه‌حل‌ها: انتهای `<body>` ، `DOMContentLoaded` یا `defer` (فقط برای فایل خارجی). ترتیب: `DOMContentLoaded` قبل از `load` .

تست‌های تمرینی

۱۸ تست · شماره‌ی ۶۲ تا ۷۹

۶۲ آسان

طبق اسلاید ۱۲، بالاترین شیء (top-level object) مرورگر که همان global context است کدام است؟

۱) `document` ۲) `window` ۳) عنصر `<html>` ۴) عنصر `<body>`

۶۳ آسان

طبق اسلاید ۱۳، شیء `document` نماینده‌ی چیست؟

۱) پنجره‌ی مرورگر، همراه با تاریخچه و آدرس صفحه

۲) فقط محتوای تگ `<body>`

۳) کل صفحه‌ی وبِ بارگذاری‌شده در پنجره؛ نقطه‌ی ورود به درخت DOM

۴) فایل HTML روی دیسک، که با تغییر DOM به‌روز می‌شود

۶۴ متوسط

در نمودار اسلاید ۱۲، کدام مورد فرزند مستقیم `window` نیست؟

۱) `location` ۲) `history` ۳) `navigator` ۴) `forms`

۶۵ متوسط

در نمودار اسلاید ۱۲، `mimeTypes` زیرمجموعه‌ی کدام شیء است؟

۱) `navigator` ۲) `document` ۳) `screen` ۴) `location`

۶۶ آسان

خروجی کد زیر در مرورگر کدام است؟

HTML

```html
<h1>Hi</h1>
```

JavaScript

```javascript
console.log(
  window.document === document,
  document.documentElement.tagName,
  document.body.tagName,
  document.head.tagName
);
```

۱) `false HTML BODY HEAD` ۲) `true BODY BODY HEAD`

۳) `true HTML BODY HEAD` ۴) `true DOCUMENT BODY HEAD`

۶۷ آسان

صفحه‌ی زیر همان مثال اسلاید ۱۳ است. کاربر **دو بار** روی دکمه کلیک می‌کند و سپس در Console دستور `console.log(heading.textContent)` را اجرا می‌کنیم. خروجی کدام است؟

HTML

```
<h1 id="title">Hello World</h1>
<button id="btn">Change Text</button>
```

JavaScript

```
// Access an element
const heading = document.getElementById("title");

// Add event to button
document.getElementById("btn").addEventListener("click", () => {
  heading.textContent = "Text Changed via document!";
});
```

۱) `Hello World`

۲) `Text Changed via document!`

۳) `Text Changed via document!Text Changed via document!`

۴) `Uncaught TypeError`

۶۸ متوسط

این کد اسلاید ۱۴ است. با باز کردن صفحه در Firefox، چه چیزی در Console نمایش داده می‌شود؟

HTML

```
<!DOCTYPE html>
<html>
  <body>
    <h1 id="title">Hello World</h1>

    <script>
      const heading = document.getElementById("title");
      console.log(heading)
      </script>
  </body>
</html>
```

۱) خودِ عنصر `<h1 id="title">`، به‌صورت یک گره‌ی قابل باز شدن

۲) رشته‌ی `"Hello World"`

۳) `null`، چون `heading` هنوز ساخته نشده است

۴) یک خطای نحوی (SyntaxError)، چون بعد از `console.log(heading)` سمی‌کالن نیامده است

۶۹ سخت

خروجی Console برای صفحه‌ی زیر کدام است؟

HTML

```
<!DOCTYPE html>
<html>
<head>
  <script>
    const heading = document.getElementById("title");
    console.log(heading);
    heading.textContent = "Changed";
    console.log("done");
  </script>
</head>
<body>
  <h1 id="title">Hello World</h1>
</body>
</html>
```

۱) `[object HTMLHeadingElement]` `done`

۲) `undefined` `done`

۳) `null` `Uncaught TypeError`

۴) `Uncaught ReferenceError`

۷۰ سخت

ترتیب خروجی‌های Console برای صفحه‌ی زیر کدام است؟

```HTML
<!DOCTYPE html>
<html>
<head>
  <script>
    console.log("head", document.getElementById("title"));
    document.addEventListener("DOMContentLoaded", () => {
      console.log("DOMContentLoaded", document.getElementById("title").textContent);
    });
    window.addEventListener("load", () => console.log("load"));
  </script>
</head>
<body>
  <h1 id="title">Hello World</h1>
  <script>
    console.log("end of body", document.getElementById("title").tagName);
  </script>
</body>
</html>
```

۱) `head null` `DOMContentLoaded Hello World` `end of body H1` `load`

۲) `head null` `end of body H1` `DOMContentLoaded Hello World` `load`

۳) `head H1` `end of body H1` `DOMContentLoaded Hello World` `load`

۴) `head null` `end of body H1` `load` `DOMContentLoaded Hello World`

۷۱ متوسط

یک اسکریپت درون‌خطی (بدون `src`) داخل `<head>` قرار دارد و با `getElementById` دنبال عنصری در `<body>` می‌گردد، ولی `null` می‌گیرد. کدام تغییر به‌تنهایی مشکل را حل می‌کند؟

۱) اضافه کردن ویژگی `defer` به همان تگ `<script>` درون‌خطی

۲) تبدیل `const` به `var` در تعریف متغیر

۳) قرار دادن کد داخل `document.addEventListener("DOMContentLoaded", () => { ... })`

۴) نوشتن `getElementById("#title")` به‌جای `getElementById("title")`

۷۲ متوسط

خروجی کد زیر کدام است؟

HTML

```
<h1 id="title">Hello World</h1>
```

JavaScript

```
const heading = document.getElementById("title");
console.log(String(heading), typeof heading, heading.textContent);
```

۱) `Hello World string Hello World`

۲) `<h1 id="title">Hello World</h1> string Hello World`

۳) `[object HTMLHeadingElement] object Hello World`

۴) `[object HTMLHeadingElement] element Hello World`

۷۳ آسان

خروجی کد زیر کدام است؟

HTML

```
<h1 id="title" class="big">Hello</h1>
```

JavaScript

```
const h = document.getElementById("title");
console.log(h.id, h.className, h.textContent, h.tagName);
```

۱) `#title .big Hello h1`

۲) `title undefined Hello H1`

۳) `title big <h1 id="title" class="big">Hello</h1> H1`

۴) `title big Hello H1`

۷۴ متوسط

خروجی کد زیر کدام است؟

```HTML
<!DOCTYPE html><html><head><title>Shop</title></head><body><h1>Home</h1></body></html>
```

```JavaScript
console.log(document.title);
document.title = "Cart (2)";
console.log(document.title, document.querySelector("title").textContent);
```

۱) `Shop` `Cart (2) Cart (2)`

۲) `Home` `Cart (2) Shop`

۳) `Shop` `Cart (2) Shop`

۴) `undefined` `Cart (2) Cart (2)`

۷۵ آسان

کدام عبارت درباره‌ی شیء `document` نادرست است؟

۱) نقطه‌ی ورود (entry point) برای دسترسی به درخت DOM است.

۲) متدهایی مثل `getElementById` و `querySelector` برای پیدا کردن عنصرها دارد.

۳) با آن می‌توان گره‌ها را ساخت، حذف یا به‌روزرسانی کرد.

۴) بالاترین شیء مرورگر است و `window` یکی از propertyهای آن است.

۷۶ سخت

خروجی Console برای صفحه‌ی زیر کدام است؟

```HTML
<h1 id="a">A</h1>
<script>
  console.log(document.getElementById("a") !== null, document.getElementById("b") !== null);
</script>
<h2 id="b">B</h2>
```

```JavaScript
console.log(document.getElementById("b").textContent);
```

۱) `true false` `B`

۲) `true true` `B`

۳) `false false` `B`

۴) `true false` `Uncaught TypeError`

۷۷ متوسط

کدام عبارت درباره‌ی رویدادهای `DOMContentLoaded` و `load` درست است؟

۱) `load` زودتر از `DOMContentLoaded` رخ می‌دهد، چون فقط منتظر HTML است.

۲) `DOMContentLoaded` وقتی رخ می‌دهد که HTML کامل parse شده و درخت DOM آماده است؛ `load` علاوه بر آن منتظر بارگیری منابعی مثل تصاویر و stylesheetها هم می‌ماند.

۳) هر دو دقیقاً هم‌زمان رخ می‌دهند و فقط نامشان فرق دارد.

۴) `DOMContentLoaded` قبل از اجرای هر اسکریپتی در صفحه رخ می‌دهد.

۷۸ سخت

کاربر بعد از بارگذاری صفحه‌ی زیر روی دکمه‌ی Change Text کلیک می‌کند. خروجی Console کدام است؟

HTML

```
<!DOCTYPE html>
<html>
<head>
  <script>
    const heading = document.getElementById("title");
    document.addEventListener("DOMContentLoaded", () => {
      document.getElementById("btn").addEventListener("click", () => {
        heading.textContent = "Changed";
        console.log("changed");
      });
    });
  </script>
</head>
<body>
  <h1 id="title">Hello World</h1>
  <button id="btn">Change Text</button>
</body>
</html>
```

۱) `changed`

۲) (بدون خروجی)

۳) `null` `changed`

۴) `Uncaught TypeError`

۷۹ متوسط

کدام تگ، یک فایل JavaScript **خارجی** را در `<head>` طوری بارگذاری می‌کند که اجرای آن تا پایان parse شدن کل HTML عقب بیفتد؟

۱) `<script defer>console.log("hi")</script>`

۲) `<script src="app.js"></script>`

۳) `<script src="app.js" delay></script>`

۴) `<script src="app.js" defer></script>`

۲-۴ ساختن و افزودن عناصر جدید

منبع: JS in HTML، اسلایدهای ۲۳ تا ۲۶

درسنامه

تا این‌جا یاد گرفتیم عنصرهایی را که **از قبل** در صفحه هستند پیدا کنیم و متن یا ظاهرشان را عوض کنیم. اما در خیلی از صفحه‌ها باید چیزی را **از صفر** بسازیم: یک آیتم تازه در لیست کارها، یک پیام خطا زیر فرم، یک کارت محصول جدید و... . در این درس می‌بینیم JavaScript چطور یک عنصر (element) تازه می‌سازد و آن را در درخت DOM (همان مدل درختیِ صفحه که مرورگر از روی HTML می‌سازد) جا می‌دهد.

> **یک تشبیه ساده**
>
> ساختن عنصر جدید مثل ساختن یک قطعه‌ی لگو در دستت است. قطعه وجود دارد و رنگ و شکل دارد، ولی تا آن را به ساختمان لگو **نچسبانی** جزو ساختمان نیست و کسی آن را در ساختمان نمی‌بیند. در DOM هم `createElement` قطعه را می‌سازد و متدهایی مثل `appendChild` آن را به درخت صفحه می‌چسبانند.

چهار قدمِ ساختن یک عنصر

اسلاید ۲۳ کل کار را این‌طور خلاصه می‌کند:

1. **ساختن:** `document.createElement("tag")`
2. **گذاشتن محتوا:** `.innerText` یا `.innerHTML` (یا `.textContent`)
3. **تنظیم attributeها** (ویژگی‌های داخل تگ، مثل `id` و `class`): `.setAttribute("name", "value")`
4. **افزودن به صفحه:** `.appendChild()` یا `.append()`؛ و برای گذاشتن قبل از یک عنصر دیگر: `.insertBefore()`

و در آخر این کارها را با رویدادها (event) ترکیب می‌کنیم تا صفحه با کلیک و تایپ کاربر تغییر کند.

قدم ۱: ساختن با `createElement`

```JavaScript
const p = document.createElement("p");   // a brand-new <p> element
console.log(p.tagName);       // "P"
console.log(p.isConnected);   // false -> it is not in the page yet
console.log(p.parentNode);    // null  -> it has no parent yet
```

- ورودی فقط **نام تگ** است، بدون `<` و `>`. `createElement("div")` درست است ولی `createElement("<div>")` خطای `InvalidCharacterError` می‌دهد.
- خروجی یک شیء عنصر واقعی است (مثلاً `HTMLParagraphElement`) که همه‌ی propertyها و متدهای یک عنصر معمولی را دارد.
- این عنصر فعلاً فقط در **حافظه** است: پدر ندارد، `isConnected` (آیا به سند وصل است؟) برابر `false` است و روی صفحه دیده نمی‌شود.

قدم ۲: گذاشتن محتوا

```JavaScript
const div = document.createElement("div");
div.innerText = "Hello DOM!";          // plain text (this is what slide 24 uses)
div.textContent = "Hello DOM!";        // plain text as well
div.innerHTML = "<b>Hello</b> DOM!";   // parsed as HTML: a real <b> element is created
```

همان‌طور که در درس مربوط به `textContent` و `innerHTML` دیدیم، `innerHTML` رشته را **به‌عنوان کد HTML تفسیر می‌کند** و تگ‌ها را به عنصر تبدیل می‌کند، ولی `textContent` و `innerText` متن را **همان‌طور که هست** می‌گذارند (اگر در متن `<b>` باشد، خودِ این علامت‌ها روی صفحه دیده می‌شوند). برای متنی که کاربر وارد کرده همیشه `textContent` (یا `innerText`) را انتخاب کن تا کسی نتواند HTML یا اسکریپت مخرب در صفحه‌ات تزریق کند.

قدم ۳: تنظیم attributeها

```JavaScript
div.setAttribute("class", "box");   // <div class="box">
div.setAttribute("id", "msg");      // <div class="box" id="msg">
```

برای کلاس سه راه رایج داریم (جزئیاتشان در درس «ویژگی‌ها و کلاس‌ها» می‌آید):

روش	کار
`div.setAttribute("class", "box")`	کل attribute کلاس را برابر `"box"` می‌گذارد
`div.className = "box"`	همان کار، با property رشته‌ای `className`
`div.classList.add("box")`	کلاس `box` را **اضافه** می‌کند و کلاس‌های قبلی را نگه می‌دارد

property مربوط به کلاس `className` نام دارد، نه `class`، چون `class` در JavaScript یک کلمه‌ی رزروشده است.

قدم ۴: افزودن به درخت

`appendChild` : **یک گره، به‌عنوان آخرین فرزند.** `parent.appendChild(node)` گره (node) را **آخرین فرزند** `parent` می‌کند و **همان گره را برمی‌گرداند**:

```JavaScript
const list = document.querySelector("ul");
const li = document.createElement("li");
li.textContent = "Milk";
const result = list.appendChild(li);
console.log(result === li);   // true
```

`appendChild` فقط **یک** آرگومان می‌گیرد و آن هم باید یک **گره** باشد. اگر رشته بدهی (`list.appendChild("Milk")`) خطای `TypeError` می‌گیری.

`append` : **چند گره و رشته با هم.**

```JavaScript
const box = document.querySelector("#box");
const b = document.createElement("b");
b.textContent = "World";
box.append("Hello ", b, "!");   // strings become text nodes
console.log(box.innerHTML);     // Hello <b>World</b>!
```

`append` جدیدتر و راحت‌تر است: هر تعداد آرگومان می‌گیرد، هر رشته را به یک **گره‌ی متنی** (text node؛ گره‌ای که فقط متن است و تگ ندارد) جداگانه تبدیل می‌کند و **چیزی برنمی‌گرداند** (`undefined`).

	`appendChild(node)`	`append(...items)`
تعداد آرگومان	دقیقاً یکی	هر تعداد
رشته قبول می‌کند؟	خیر (`TypeError`)	بله؛ هر رشته یک text node می‌شود
مقدار برگشتی	همان گره‌ی اضافه‌شده	`undefined`
محل افزودن	آخرین فرزند	آخرِ فرزندان، به همان ترتیب آرگومان‌ها

`document.body` میان‌بُری برای عنصر `<body>` است؛ پس `document.body.appendChild(div)` یعنی «div را آخرین فرزند body کن».

`insertBefore` : افزودن قبل از یک فرزند مشخص.

```JavaScript
parent.insertBefore(newNode, referenceNode);
```

- روی **پدر** صدا زده می‌شود، نه روی گره‌ی مرجع.
- ترتیب آرگومان‌ها را این‌طور به خاطر بسپار: «**این** (گره‌ی جدید) را بگذار **قبلِ آن** (گره‌ی مرجع)».
- گره‌ی مرجع باید **فرزند مستقیم** همان پدر باشد؛ وگرنه خطای `NotFoundError` رخ می‌دهد.
- اگر مرجع `null` باشد، `insertBefore` درست مثل `appendChild` گره را به **انتها** اضافه می‌کند.
- مقدار برگشتی همان گره‌ی درج‌شده است.

```HTML
<ul id="list"><li>A</li><li id="c">C</li></ul>
```

```JavaScript
const list = document.getElementById("list");
const b = document.createElement("li");
b.textContent = "B";
list.insertBefore(b, document.getElementById("c"));   // A, B, C

const first = document.createElement("li");
first.textContent = "Start";
list.insertBefore(first, list.firstChild);            // Start, A, B, C
```

متدهای مدرن: `prepend` ، `before` و `after`

متد	گره‌ی جدید کجا می‌رود؟
`parent.prepend(x)`	**اولین** فرزند `parent`
`parent.append(x)`	**آخرین** فرزند `parent`
`el.before(x)`	هم‌سطحِ (sibling) `el` ، درست **قبل** از آن
`el.after(x)`	هم‌سطحِ `el` ، درست **بعد** از آن

این چهار متد مثل `append` چند گره یا رشته می‌گیرند و `undefined` برمی‌گردانند. توجه کن که `prepend` و `append` روی **پدر** صدا زده می‌شوند ولی `before` و `after` روی **خودِ عنصر مرجع**.

گره‌ی متنی با `createTextNode`

JavaScript

```
const text = document.createTextNode("<b>Hi</b>");
document.body.appendChild(text);   // shows the characters <b>Hi</b>, no bold element
```

`document.createTextNode` یک گره‌ی متنی می‌سازد. چون یک **گره** است، `appendChild` آن را قبول می‌کند. محتوایش هیچ‌وقت HTML تفسیر نمی‌شود؛ این همان کاری است که `append("…")` پشت صحنه با رشته‌ها می‌کند.

جابه‌جا کردن، نه کپی کردن!

هر گره در هر لحظه فقط **یک جا** در درخت می‌تواند باشد. اگر گره‌ای را که الان در صفحه است دوباره append یا insert کنی، از جای قبلی **برداشته می‌شود** و به جای جدید **منتقل** می‌شود؛ کپی ساخته نمی‌شود.

HTML

```
<ul id="todo"><li id="task">Read slides</li></ul>
<ul id="done"></ul>
```

JavaScript

```
const task = document.getElementById("task");
document.getElementById("done").appendChild(task);   // moved, not copied
console.log(document.getElementById("todo").children.length); // 0
console.log(document.getElementById("done").children.length); // 1
```

برای کپی واقعی از `cloneNode` استفاده کن:

- `el.cloneNode(true)` : کپی **عمیق** (deep)؛ خود عنصر به‌همراه همه‌ی فرزندان و متن داخلش.
- `el.cloneNode()` یا `el.cloneNode(false)` : کپی **سطحی** (shallow)؛ فقط خود تگ و attributeهایش، بدون فرزندان و حتی بدون متن.

کپی، attributeها و از جمله `id` را هم کپی می‌کند؛ پس اگر عنصر اصلی `id` داشت، کپی را عوض کن تا دو عنصر با یک `id` نداشته باشی. listenerهایی که با `addEventListener` به عنصر اصلی وصل شده‌اند کپی **نمی‌شوند**.

مثال اسلایدهای ۲۴ تا ۲۶: New Div!

doc.html

```
<!DOCTYPE html>
<html>
<body>
    <h1 id="title">Hello World</h1>
    <button>CLICK ME!</button>
    <script>
        const btn = document.querySelector('button');
        btn.addEventListener('click', () => {
            const div = document.createElement("div");
            div.innerText = "Hello DOM!";
            div.setAttribute("class", "box");
            document.body.appendChild(div);
        });
    </script>
</body>
</html>
```

خط به خط:

- **خط ۷:** اولین `<button>` صفحه را پیدا می‌کنیم.
- **خط ۸:** به رویداد `click` آن یک listener (تابع شنونده) وصل می‌کنیم.
- **خط ۹:** در **هر** کلیک یک `<div>` **تازه** ساخته می‌شود (هنوز روی صفحه نیست).
- **خط ۱۰:** متن داخلش `Hello DOM!` می‌شود.
- **خط ۱۱:** کلاس `box` را به آن می‌دهیم؛ حالا `<div class="box">Hello DOM!</div>` داریم.
- **خط ۱۲:** آن را آخرین فرزند `<body>` می‌کنیم و از همین لحظه روی صفحه دیده می‌شود.

نتیجه (اسلایدهای ۲۵ و ۲۶): قبل از کلیک فقط عنوان «Hello World» و دکمه‌ی «CLICK ME!» دیده می‌شود. بعد از کلیک، درست زیر دکمه یک خط تازه با متن «Hello DOM!» ظاهر می‌شود. چون `<div>` عنصری بلوکی (block؛ عنصری که از خط جدید شروع می‌شود و کل عرض را می‌گیرد) است، در خط جدید می‌آید.

دو نکته‌ی ظریف این مثال:

- div در واقع **بعد از تگ** `<script>` به body اضافه می‌شود (آخرین فرزند body)، ولی چون `<script>` چیزی روی صفحه نشان نمی‌دهد، div درست زیر دکمه دیده می‌شود.
- چون `createElement` **داخل** listener است، هر کلیک یک div **جدید** می‌سازد: بعد از ۳ کلیک، ۳ خط «Hello DOM!» داریم. برای `.box` در این مثال CSSی تعریف نشده، پس کلاس فعلاً اثر ظاهری ندارد و فقط نشان می‌دهد attribute چطور تنظیم می‌شود.

ترکیب با رویدادها: یک لیست کار ساده

HTML

```
<input id="task" placeholder="New task">
<button id="add">Add</button>
<ul id="list"></ul>
```

JavaScript

```
const input = document.getElementById("task");
const list = document.getElementById("list");

document.getElementById("add").addEventListener("click", () => {
  if (input.value.trim() === "") return;   // ignore empty tasks
  const li = document.createElement("li");
  li.textContent = input.value;            // safe: plain text
  list.appendChild(li);
  input.value = "";                        // clear the box for the next task
});
```

ساختن لیست از روی یک آرایه

JavaScript

```
const fruits = ["Apple", "Banana", "Cherry"];
const ul = document.createElement("ul");

fruits.forEach((name) => {
  const li = document.createElement("li");
  li.textContent = name;
  ul.appendChild(li);
});

document.body.appendChild(ul);
console.log(ul.children.length);   // 3
```

راه کوتاه‌تر: با `map` آرایه‌ای از عنصرها بساز و همه را با spread (`...`؛ پخش کردن عضوهای آرایه به‌صورت آرگومان‌های جدا) یک‌جا به `append` بده:

JavaScript

```
const items = fruits.map((name) => {
  const li = document.createElement("li");
  li.textContent = name;
  return li;
});
ul.append(...items);   // append accepts many nodes at once
```

راه سوم ساختن یک رشته‌ی HTML و دادن آن به `innerHTML` است: `ul.innerHTML = fruits.map((f) => "<li>" + f + "</li>").join("")`. کوتاه است، ولی اگر داده از کاربر بیاید خطرناک است و هر بار کل محتوای قبلی را دور می‌ریزد و از نو می‌سازد.

> **نکته**
>
> - `createElement` فقط **می‌سازد**؛ تا `append`، `appendChild`، `insertBefore`، `prepend` یا `before` یا `after` صدا نزنی، چیزی روی صفحه نمی‌آید.
> - `appendChild`: دقیقاً یک **گره**، و خروجی‌اش همان گره است. `append`: چند گره یا رشته، و خروجی‌اش `undefined` است.
> - `parent.insertBefore(جدید, مرجع)` روی **پدر** صدا زده می‌شود؛ مرجعِ `null` یعنی «به انتها اضافه کن».
> - برای «اولین فرزند کردن» دو راه داری: `parent.prepend(x)` یا `parent.insertBefore(x, parent.firstChild)`.

⚠ دام تستی

- اگر `createElement` را **بیرون** از listener بنویسی و داخل listener فقط `appendChild` کنی، با هر کلیک **همان یک عنصر** جابه‌جا می‌شود و تعداد عنصرها زیاد نمی‌شود!
- `list.appendChild("text")` خطای `TypeError` می‌دهد، ولی `list.append("text")` درست کار می‌کند.
- `const x = list.append(li); x.textContent = "…";` خطا می‌دهد، چون `x` برابر `undefined` است. با `appendChild` همین کد درست است.
- `createElement("<p>")` غلط است (`InvalidCharacterError`)؛ فقط نام تگ را بده: `createElement("p")`.
- اگر خروجی `map` را **بدون** `join("")` به `innerHTML` بدهی، آرایه با **کاما** به رشته تبدیل می‌شود و کاماها هم روی صفحه می‌آیند.
- append کردنِ گره‌ای که در صفحه هست آن را **منتقل** می‌کند، نه کپی؛ برای کپی `cloneNode(true)` لازم است.

✓ جمع‌بندی

ساختن: `createElement` (یا `createTextNode` برای متن خالی) ← محتوا: `textContent` / `innerText` / `innerHTML` ← attribute: `setAttribute` یا `className` / `classList` ← افزودن: `appendChild` / `append` (آخر)، `prepend` (اول)، `before` / `insertBefore` / `after` (وسط). هر گره فقط یک جا در درخت دارد و افزودن دوباره یعنی جابه‌جایی.

✓ تست‌های تمرینی

۲۵ تست · شماره‌ی ۸۰ تا ۱۰۴

۸۰ آسان

بلافاصله بعد از اجرای `const li = document.createElement("li");` کدام گزینه درست است؟

۱) یک `<li>` به انتهای `<body>` اضافه می‌شود و روی صفحه دیده می‌شود.

۲) اولین `<li>` موجود در صفحه پیدا و در `li` ذخیره می‌شود.

۳) یک عنصر `<li>` تازه در حافظه ساخته می‌شود که تا به درخت DOM اضافه نشود روی صفحه نمی‌آید.

۴) یک `<li>` خالی به‌عنوان آخرین فرزندِ اولین `<ul>` صفحه قرار می‌گیرد.

۸۱ آسان

خروجی کد زیر در مرورگر کدام است؟

HTML

```
<p>Old paragraph</p>
```

JavaScript

```
const p = document.createElement("p");
p.textContent = "New paragraph";
console.log(p.isConnected, document.querySelectorAll("p").length);
```

۱) `true 2`　　۲) `false 1`　　۳) `false 2`　　۴) `true 1`

۸۲ متوسط

کدام دستور هنگام اجرا در مرورگر **خطا** می‌دهد؟

۱) `document.createElement("section")`

۲) `document.createElement("<div>")`

۳) `document.body.append("Hi", "!")`

۴) `document.body.insertBefore(document.createElement("p"), null)`

۸۳ آسان

می‌خواهیم عنصر `div` که با `createElement` ساخته‌ایم به‌عنوان آخرین فرزند `<body>` روی صفحه بیاید. در جای خالی `document.body._____(div);` کدام گزینه قرار می‌گیرد؟

۱) `createElement`　۲) `querySelector`　۳) `setAttribute`　۴) `appendChild`

۸۴ آسان

کدام جمله درباره‌ی `append` و `appendChild` **درست** است؟

۱) `append` می‌تواند چند گره و رشته را یک‌جا اضافه کند، ولی `appendChild` فقط یک گره می‌گیرد.

۲) `appendChild` رشته‌ها را خودکار به گره‌ی متنی تبدیل می‌کند.

۳) `append` گره‌ی اضافه‌شده را برمی‌گرداند، پس می‌توان پشت آن `.textContent` نوشت.

۴) هر دو متد گره را به‌عنوان اولین فرزند پدر اضافه می‌کنند.

۸۵ متوسط

خروجی کد زیر کدام است؟

HTML

```html
<ul id="list"></ul>
```

JavaScript

```javascript
const list = document.querySelector("#list");
list.append(document.createElement("li"), "note", "!");
console.log(list.children.length, list.childNodes.length);
```

۱) `1 3`　۲) `3 3`　۳) `1 1`　۴) `1 2`

۸۶ سخت

خروجی کد زیر کدام است؟

HTML

```
<ul></ul>
```

JavaScript

```
const ul = document.querySelector("ul");
const first = ul.appendChild(document.createElement("li"));
first.textContent = "One";
console.log(ul.textContent);
const second = ul.append(document.createElement("li"));
second.textContent = "Two";
console.log(ul.textContent);
```

۱) `One` `OneTwo`

۲) `One` `Uncaught TypeError`

۳) `Uncaught TypeError`

۴) `One` `One`

۸۷ متوسط

اجرای کدام خط از کد زیر باعث خطا می‌شود؟

HTML

```
<div id="box"></div>
```

JavaScript

```
1  const box = document.getElementById("box");
2  const p = document.createElement("p");
3  box.append("Title: ", p);
4  console.log("A");
5  box.insertBefore(document.createTextNode("!"), null);
6  console.log("B");
7  box.appendChild("Done");
8  console.log("C");
```

۱) خط ۳

۲) خط ۵

۳) خط ۷

۴) هیچ خطایی رخ نمی‌دهد

۸۸ آسان

کدام دستور یک **گره‌ی متنی** (text node) می‌سازد که بعداً بتوان آن را با `appendChild` به صفحه اضافه کرد؟

۱) `document.createElement("text")`

۲) `document.createText("Hello")`

۳) `document.textNode("Hello")`

۴) `document.createTextNode("Hello")`

۸۹ متوسط

خروجی کد زیر کدام است؟

```HTML
<div id="box"></div>
```

```JavaScript
const box = document.getElementById("box");
box.appendChild(document.createTextNode("<b>Hi</b>"));
console.log(box.children.length, box.textContent);
```

۱) `1 Hi`
۲) `1 <b>Hi</b>`
۳) `0 Hi`
۴) `0 <b>Hi</b>`

۹۰ آسان

`ul` یک لیست است و `oldLi` یکی از فرزندان مستقیم آن. کدام دستور، `newLi` را درست **قبل از** `oldLi` قرار می‌دهد؟

۱) `ul.insertBefore(oldLi, newLi)`
۲) `ul.insertBefore(newLi, oldLi)`
۳) `oldLi.insertBefore(newLi)`
۴) `newLi.insertBefore(ul, oldLi)`

۹۱ متوسط

خروجی کد زیر کدام است؟

```HTML
<ul id="list"><li>A</li><li>C</li></ul>
```

```JavaScript
const list = document.getElementById("list");
const b = document.createElement("li");
b.textContent = "B";
list.insertBefore(b, list.children[1]);
console.log(list.textContent);
```

۱) `BAC`
۲) `ACB`
۳) `AC`
۴) `ABC`

۹۲ متوسط

می‌خواهیم `newItem` **اولین** فرزندِ `list` شود. در جای خالی `list.insertBefore(newItem, ______);` کدام گزینه قرار می‌گیرد؟

۱) `null`
۲) `list.firstChild`
۳) `list.lastChild`
۴) `list`

۹۳ سخت

خروجی کد زیر کدام است؟

HTML

```
<div id="box"><p>P1</p></div>
<span id="s">S</span>
```

JavaScript

```
const box = document.getElementById("box");
const s = document.getElementById("s");
box.insertBefore(document.createElement("em"), null);
console.log(box.children.length);
box.insertBefore(document.createElement("b"), s);
console.log(box.children.length);
```

۱) `2` `Uncaught NotFoundError`

۲) `2` `3`

۳) `Uncaught TypeError`

۴) `1` `Uncaught NotFoundError`

۹۴ متوسط

خروجی کد زیر کدام است؟

HTML

```
<ul id="list"><li id="b">B</li></ul>
```

JavaScript

```
const list = document.getElementById("list");
const b = document.getElementById("b");
const make = (t) => {
  const li = document.createElement("li");
  li.textContent = t;
  return li;
};
b.before(make("A"));
b.after(make("C"));
list.prepend(make("0"));
console.log(list.textContent);
```

۱) `A0BC`

۲) `ABC0`

۳) `0ABC`

۴) `0BAC`

۹۵ متوسط

خروجی کد زیر کدام است؟

```html
<ul id="todo"><li id="task">Read slides</li></ul>
<ul id="done"></ul>
```

```javascript
const task = document.getElementById("task");
document.getElementById("done").appendChild(task);
console.log(
  document.getElementById("todo").children.length,
  document.getElementById("done").children.length
);
```

۱) `0 1`
۲) `1 1`
۳) `1 0`
۴) `0 0`

۹۶ متوسط

کدام جمله درباره‌ی افزودن عناصر **نادرست** است؟

۱) اگر عنصری را که الان در صفحه است دوباره با `appendChild` به جای دیگری اضافه کنیم، یک کپی از آن ساخته می‌شود و نسخه‌ی قبلی سر جایش می‌ماند.

۲) `insertBefore(node, null)` گره را به انتهای فرزندان پدر اضافه می‌کند.

۳) `appendChild` همان گره‌ای را که اضافه کرده برمی‌گرداند.

۴) `append` رشته‌ها را به گره‌ی متنی تبدیل می‌کند و اضافه می‌کند.

۹۷ سخت

خروجی کد زیر کدام است؟

```html
<ul id="a"><li id="x">X</li></ul>
<ul id="b"></ul>
```

```javascript
const x = document.getElementById("x");
const deep = x.cloneNode(true);
const shallow = x.cloneNode();
document.getElementById("b").append(deep, shallow);
console.log(
  document.getElementById("a").children.length,
  document.getElementById("b").children.length
);
console.log(deep.textContent, shallow.textContent === "");
```

۱) `0 2` `X true`
۲) `1 2` `X false`
۳) `0 2` `X false`
۴) `1 2` `X true`

۹۸ سخت

خروجی کد زیر کدام است؟ (یادآوری `children` یک HTMLCollection **زنده** است؛ یعنی با هر تغییر در DOM به‌روز می‌شود.)

HTML

```
<ul id="list"><li>A</li><li>B</li><li>C</li></ul>
```

JavaScript

```
const list = document.getElementById("list");
const items = list.children;
const z = document.createElement("li");
z.textContent = "Z";
list.insertBefore(z, items[1]);
list.insertBefore(items[0], null);
console.log([...list.children].map((li) => li.textContent).join(""));
```

۱) `AZBCA`　۲) `AZBC`　۳) `ZBCA`　۴) `ZABC`

۹۹ آسان

در مثال «New Div!» اسلاید ۲۴، کاربر **یک بار** روی دکمه‌ی «CLICK ME!» کلیک می‌کند. طبق تصویر اسلاید ۲۶، صفحه چه شکلی می‌شود؟

۱) عنوان «Hello World» با «Hello DOM!» جایگزین می‌شود.

۲) «Hello DOM!» بالای عنوان «Hello World» ظاهر می‌شود.

۳) عنوان و دکمه سر جایشان می‌مانند و زیر دکمه یک خط تازه با متن «Hello DOM!» ظاهر می‌شود.

۴) هیچ تغییری دیده نمی‌شود، چون برای کلاس `box` هیچ CSSی تعریف نشده است.

۱۰۰ متوسط

صفحه‌ی زیر همان کد اسلاید ۲۴ است. کاربر **سه بار** روی دکمه کلیک می‌کند. خروجی کدام است؟

HTML

```
<h1 id="title">Hello World</h1>
<button>CLICK ME!</button>
```

JavaScript

```
const btn = document.querySelector('button');
btn.addEventListener('click', () => {
    const div = document.createElement("div");
    div.innerText = "Hello DOM!";
    div.setAttribute("class", "box");
    document.body.appendChild(div);
});
```

بعد از سه کلیک، این دستور در کنسول اجرا می‌شود:

```
console.log(document.querySelectorAll(".box").length, document.body.lastElementChild.tagName);
```

۱) `1 DIV`　۲) `3 DIV`　۳) `3 SCRIPT`　۴) `0 SCRIPT`

۱۰۱ سخت

کاربر **سه بار** روی دکمه کلیک می‌کند. خروجی کد زیر کدام است؟

HTML

```
<button>Add</button>
```

JavaScript

```
const btn = document.querySelector("button");
const div = document.createElement("div");
div.textContent = "Hello DOM!";
btn.addEventListener("click", () => {
  document.body.appendChild(div);
  console.log(document.querySelectorAll("div").length);
});
```

۱) `1` `2` `3`

۲) `0` `1` `2`

۳) `1` `1` `1`

۴) `1` `Uncaught TypeError`

۱۰۲ سخت

کاربر این کارها را به ترتیب انجام می‌دهد: در کادر متن `Read` را تایپ می‌کند و Add را می‌زند؛ دوباره Add را می‌زند (کادر خالی است)؛ `Write` را تایپ می‌کند و Add را می‌زند. خروجی کدام است؟

HTML

```
<input id="task">
<button id="add">Add</button>
<ul id="list"></ul>
```

JavaScript

```
const input = document.getElementById("task");
const list = document.getElementById("list");
document.getElementById("add").addEventListener("click", () => {
  if (input.value.trim() === "") return;
  const li = document.createElement("li");
  li.textContent = input.value;
  list.prepend(li);
  input.value = "";
});
```

در پایان این دستور اجرا می‌شود: `console.log(list.children.length, list.firstElementChild.textContent);`

۱) `3 Write`

۲) `3 Read`

۳) `2 Read`

۴) `2 Write`

۱۰۳ متوسط

خروجی کد زیر کدام است؟

HTML

```
<ul id="menu"></ul>
```

JavaScript

```
const data = ["HTML", "CSS", "JS"];
const ul = document.getElementById("menu");
data.forEach((item, i) => {
  const li = document.createElement("li");
  li.textContent = i + 1 + ". " + item;
  ul.appendChild(li);
});
console.log(ul.children.length, ul.lastElementChild.textContent);
```

۱) `3 21. JS`
۲) `3 3. JS`
۳) `1 3. JS`
۴) `3 2. JS`

۱۰۴ سخت

خروجی کد زیر کدام است؟

HTML

```
<ul></ul>
```

JavaScript

```
const fruits = ["Apple", "Banana"];
const ul = document.querySelector("ul");
ul.innerHTML = fruits.map((f) => "<li>" + f + "</li>");
console.log(ul.children.length, ul.textContent);
```

۱) `2 Apple,Banana`
۲) `2 AppleBanana`
۳) `1 Apple,Banana`
۴) `0 <li>Apple</li>,<li>Banana</li>`

۲-۵ حذف و جایگزینی عناصر

منبع: JS in HTML، اسلایدهای ۲۷ تا ۳۰

درسنامه

در درس قبل عنصر ساختیم و به صفحه اضافه کردیم. حالا نوبت برعکسش است: **برداشتن** عنصر از صفحه یا **عوض کردن** آن با عنصری دیگر. بستن یک اعلان (notification)، حذف یک کار از لیست کارها (todo list) و پاک کردن پیام خطا بعد از اصلاح فرم همه از همین جنس‌اند.

یک تشبیه ساده

درخت DOM را مثل یک قفسه‌ی کتاب در نظر بگیر. وقتی کتابی را از قفسه برمی‌داری، کتاب **نابود نمی‌شود**؛ فقط دیگر در قفسه نیست و اگر هنوز در دستت باشد (یعنی در یک متغیر نگهش داشته باشی) می‌توانی دوباره سر جایش یا جای دیگری بگذاری. حذف عنصر از DOM هم دقیقاً همین است.

سه ابزار اصلی (اسلاید ۲۷)

اسلاید ۲۷ می‌گوید:

- با `.remove()` عنصر را **مستقیم** حذف کن؛
- یا `.removeChild()` را روی **عنصر پدر** صدا بزن؛
- `replaceChild()` یک عنصر را با عنصر دیگری **عوض** می‌کند؛
- **همیشه اول عنصر را پیدا کن** (query کن)، بعد حذفش کن؛
- برای رابط‌های پویا مثل لیست کارها و اعلان‌ها کاربرد دارد؛
- با رویدادها ترکیبش کن تا کاربر با کلیک چیزی را حذف کند.

`remove()` : عنصر خودش را حذف می‌کند

```JavaScript
const note = document.getElementById("note");
note.remove();   // the element removes itself, together with all its children
```

- به پدر احتیاجی ندارد و آرگومان هم نمی‌گیرد.
- عنصر را **همراه با همه‌ی فرزندانش** (کل زیردرخت) از صفحه برمی‌دارد.
- مقدار برگشتی‌اش `undefined` است.
- اگر عنصر از قبل در صفحه نباشد (مثلاً قبلاً حذف شده یا هیچ‌وقت اضافه نشده)، **هیچ اتفاقی نمی‌افتد** و خطایی هم رخ نمی‌دهد.

`removeChild()` : پدر فرزندش را بیرون می‌کند

```JavaScript
const list = document.getElementById("list");
const first = list.firstElementChild;
const removed = list.removeChild(first);
console.log(removed === first);    // true  -> it returns the removed node
console.log(removed.isConnected);  // false -> no longer in the page
```

- روی **پدر** صدا زده می‌شود و فرزند را به‌عنوان آرگومان می‌گیرد.
- **گره‌ی حذف‌شده را برمی‌گرداند.**
- آرگومان باید **فرزند مستقیم** همان پدر باشد؛ وگرنه خطای `NotFoundError` رخ می‌دهد (مثلاً اگر قبلاً حذف شده باشد یا نوه باشد نه فرزند).
- روش قدیمی‌تر است؛ الگوی معروف `el.parentNode.removeChild(el)` همان کار `el.remove()` را می‌کند.

`replaceChild()` : عوض کردن یک فرزند با گره‌ای دیگر

```JavaScript
parent.replaceChild(newChild, oldChild);
```

- روی **پدر** صدا زده می‌شود.
- ترتیب آرگومان‌ها مثل `insertBefore` است: **اول جدید، بعد قدیمی**.
- **گره‌ی قدیمی** (همان که بیرون رفت) را برمی‌گرداند.
- اگر `oldChild` فرزند این پدر نباشد، خطای `NotFoundError` می‌گیری.
- اگر `newChild` از قبل جای دیگری در صفحه باشد، به جای جدید **منتقل** می‌شود (کپی نمی‌شود).

```HTML
<ul id="list"><li>A</li><li id="b">B</li><li>C</li></ul>
```

```JavaScript
const list = document.getElementById("list");
const fresh = document.createElement("li");
fresh.textContent = "X";
const old = list.replaceChild(fresh, document.getElementById("b"));
console.log(list.textContent);  // "AXC"
console.log(old.textContent);   // "B"  -> the replaced node is returned
```

نسخه‌ی مدرن‌تر `oldEl.replaceWith(newEl)` است که روی **خودِ عنصر قدیمی** صدا زده می‌شود و `undefined` برمی‌گرداند؛ یعنی `old.replaceWith(fresh)` همان `old.parentNode.replaceChild(fresh, old)` است.

متد	روی چه چیزی صدا زده می‌شود؟	مقدار برگشتی	اگر گره فرزندِ مورد انتظار نباشد
`el.remove()`	خود عنصر	`undefined`	خطا نمی‌دهد؛ کاری نمی‌کند
`parent.removeChild(child)`	پدر	گره‌ی حذف‌شده	`NotFoundError`
`parent.replaceChild(newC, oldC)`	پدر	گره‌ی قدیمی	`NotFoundError`
`oldEl.replaceWith(newEl)`	عنصر قدیمی	`undefined`	—

اول پیدا کن، بعد حذف کن!

`getElementById` و `querySelector` اگر چیزی پیدا نکنند `null` برمی‌گردانند. صدا زدن متد روی `null` خطاست:

```JavaScript
document.getElementById("nope").remove();
// Uncaught TypeError: Cannot read properties of null (reading 'remove')
```

پس قبل از حذف مطمئن شو عنصر پیدا شده:

```JavaScript
const el = document.getElementById("nope");
if (el) el.remove();   // classic guard
el?.remove();          // optional chaining: does nothing when el is null
```

`?.` (optional chaining؛ «اگر سمت چپ `null` یا `undefined` بود، ادامه نده») در این حالت بدون خطا `undefined` برمی‌گرداند.

مثال اسلایدهای ۲۸ تا ۳۰: Title Goes Away!

doc.html

```html
<!DOCTYPE html>
<html>
<body>
    <h1 id="title">Hello World</h1>
    <button>CLICK ME!</button>
    <script>
        const btn = document.querySelector('button');
        btn.addEventListener('click', () => {
            const element = document.getElementById("title");
            element.remove();
        });
    </script>
</body>
</html>
```

- **خط ۷:** دکمه را پیدا می‌کنیم.
- **خط ۸:** listener کلیک را وصل می‌کنیم.
- **خط ۹:** در **هر** کلیک، عنوانی با `id="title"` را جست‌وجو می‌کنیم (همان «اول پیدا کن»).
- **خط ۱۰:** عنوان را حذف می‌کنیم.

قبل از کلیک (اسلاید ۲۹): عنوان بزرگ «Hello World» و زیرش دکمه‌ی «CLICK ME!». **بعد از کلیک (اسلاید ۳۰):** عنوان ناپدید شده و فقط دکمه مانده که حالا بالای صفحه قرار گرفته (چون جای خالیِ عنوان هم از بین رفته است).

کلیک دوم چه می‌شود؟ این‌بار `getElementById("title")` چیزی پیدا نمی‌کند و `null` برمی‌گرداند؛ پس `element.remove()` در کنسول خطای `TypeError: Cannot read properties of null (reading 'remove')` می‌دهد. ظاهر صفحه تغییری نمی‌کند و دکمه هم سر جایش است؛ خطا فقط همان اجرای listener را متوقف می‌کند و با هر کلیک بعدی دوباره تکرار می‌شود. راه‌های رفع:

- `element?.remove();` یا `if (element) element.remove();`

- غیرفعال کردن دکمه بعد از کلیک اول: `btn.disabled = true;`
- اجرای listener فقط یک بار: `btn.addEventListener('click', handler, { once: true });` (گزینه‌ی `once` باعث می‌شود listener بعد از اولین اجرا خودبه‌خود حذف شود.)

حذف‌شده یعنی نابودشده؟ نه!

```JavaScript
const msg = document.getElementById("msg");
msg.remove();                  // gone from the page...
console.log(msg.textContent);  // ...but still alive in the variable
document.body.append(msg);     // put it back (an "undo" feature)
```

بعد از `remove()` متغیر `msg` **نمی‌شود**؛ همان شیء با همه‌ی متن، attributeها و فرزندانش در حافظه است و فقط به صفحه وصل نیست (`isConnected` برابر `false`). هر وقت هیچ متغیری دیگر به آن اشاره نکند، garbage collector (بخش خودکار پاک‌سازی حافظه‌ی JavaScript) آن را پاک می‌کند.

خالی کردن یک عنصر (حذف همه‌ی فرزندان)

```JavaScript
list.innerHTML = "";           // 1) simplest
list.textContent = "";         // 2) also removes every child
list.replaceChildren();        // 3) modern: no arguments = no children
while (list.firstChild) {      // 4) classic loop
  list.removeChild(list.firstChild);
}
```

در هر چهار روش خودِ `list` سر جایش می‌ماند و فقط **محتوایش** خالی می‌شود؛ اما `list.remove()` خودِ لیست را از صفحه برمی‌دارد. در روش ۴ دقت کن که `firstChild` گره‌های متنیِ فاصله و خط جدید بین تگ‌ها را هم شامل می‌شود؛ حلقه آن‌قدر ادامه می‌دهد تا هیچ فرزندی (از هر نوع) نماند.

دام حلقه روی مجموعه‌ی زنده

`getElementsByClassName` یک HTMLCollection **زنده** (live؛ با هر تغییر DOM خودش به‌روز می‌شود) برمی‌گرداند. حذف در یک حلقه‌ی معمولی روی آن **نصف عنصرها را جا می‌اندازد**:

```JavaScript
const items = document.getElementsByClassName("item"); // live collection of 4 items
for (let i = 0; i < items.length; i++) {
  items[i].remove();   // BUG: after each removal the collection shrinks and shifts
}
// only items 1 and 3 are removed; 2 and 4 stay
```

بعد از حذف `items[0]` ، عنصر دوم به خانه‌ی ۰ می‌لغزد ولی `i` به ۱ رفته است؛ پس از رویش می‌پرد. راه درست:

```JavaScript
document.querySelectorAll(".item").forEach((el) => el.remove()); // static NodeList
// or
while (items.length > 0) items[0].remove();
```

کاربردها: لیست کارها و اعلان‌ها (ترکیب با رویداد)

دکمه‌ی حذف برای هر کار با event delegation (یک listener روی پدر برای همه‌ی فرزندان، که در درس رویدادها دیدیم):

HTML

```
<ul id="todos">
  <li>Read slides <button class="del">x</button></li>
  <li>Do homework <button class="del">x</button></li>
</ul>
```

JavaScript

```
document.getElementById("todos").addEventListener("click", (e) => {
  if (e.target.classList.contains("del")) {
    e.target.parentElement.remove();   // remove the <li> that holds this button
  }
});
```

`e.target` خودِ دکمه است؛ اگر `e.target.remove()` بنویسیم فقط دکمه حذف می‌شود و متن کار می‌ماند. برای همین سراغ پدرش (`parentElement`) یا نزدیک‌ترین `<li>` با `e.target.closest("li")` می‌رویم.

اعلانی که خودش بعد از ۳ ثانیه بسته می‌شود:

JavaScript

```
function showToast(text) {
  const toast = document.createElement("div");
  toast.className = "toast";
  toast.textContent = text;
  document.body.append(toast);
  setTimeout(() => toast.remove(), 3000);   // auto-dismiss after 3 seconds
}
showToast("Saved!");
```

پنهان کردن با حذف کردن فرق دارد: `el.style.display = "none"` یا `el.hidden = true` فقط عنصر را **نامرئی** می‌کنند؛ عنصر هنوز در DOM است و `querySelector` پیدایش می‌کند. `remove()` واقعاً آن را از درخت بیرون می‌برد.

> **نکته**
>
> - `el.remove()` ساده‌ترین راه است: نه پدر می‌خواهد نه آرگومان، و روی عنصرِ از قبل حذف‌شده هم خطا نمی‌دهد.
> - `removeChild` و `replaceChild` روی **پدر** صدا زده می‌شوند و هر دو گره‌ای را که بیرون رفته **برمی‌گردانند**.
> - `replaceChild(جدید, قدیمی)` ؛ ترتیبش مثل `insertBefore(جدید, مرجع)` است: «اول جدید».
> - گره‌ی حذف‌شده در متغیرت زنده می‌ماند و می‌شود دوباره اضافه‌اش کرد.

⚠ دام تستی

- در مثال «Title Goes Away!»، کلیک دوم خطای `TypeError` (خواندن `remove` از `null`) می‌دهد، چون عنوان دیگر وجود ندارد.
- `removeChild` روی گره‌ای که فرزند مستقیم آن پدر نیست (یا قبلاً حذف شده) خطای `NotFoundError` می‌دهد، ولی `remove()` در همین وضعیت بی‌صدا هیچ کاری نمی‌کند.
- جابه‌جا نوشتن آرگومان‌ها در `replaceChild(old, fresh)` معمولاً به `NotFoundError` می‌رسد، چون `fresh` فرزند آن پدر نیست.
- حذف در حلقه‌ی `for` روی HTMLCollection زنده فقط نصف عنصرها را حذف می‌کند.
- حذف پدر یعنی حذف کل زیردرختش؛ فرزندان جداگانه در صفحه نمی‌مانند.

✓ جمع‌بندی

حذف: `el.remove()` (ساده) یا `parent.removeChild(child)` (برمی‌گرداند، سخت‌گیر). جایگزینی: `parent.replaceChild(newC, oldC)` یا `oldC.replaceWith(newC)` . خالی کردن: `innerHTML = ""` ، `textContent = ""` ، `replaceChildren()` یا حلقه‌ی `while (firstChild)` . قبل از حذف، عنصر را پیدا کن و `null` بودنش را بررسی کن.

✓ تست‌های تمرینی

۲۱ تست · شماره‌ی ۱۰۵ تا ۱۲۵

۱۰۵ آسان

کدام متد عنصر را **بدون نیاز به دسترسی به پدرش** و بدون هیچ آرگومانی از صفحه حذف می‌کند؟

۱) `el.removeChild()`
۲) `el.remove()`
۳) `el.replaceChild()`
۴) `delete el`

۱۰۶ آسان

خروجی کد زیر کدام است؟

HTML

```
<ul id="list"><li id="a">A</li><li>B</li></ul>
```

JavaScript

```
const list = document.getElementById("list");
const a = document.getElementById("a");
const removed = list.removeChild(a);
console.log(removed === a, list.children.length, removed.textContent);
```

۱) `true 1 A`
۲) `Uncaught TypeError`
۳) `true 1`
۴) `false 1 AB`

۱۰۷ آسان

در جای خالی `parent.replaceChild(_____)` برای این‌که `oldChild` با `newChild` عوض شود، کدام گزینه قرار می‌گیرد؟

۱) `oldChild, newChild`
۲) `parent, newChild`
۳) `newChild, oldChild`
۴) `newChild`

۱۰۸ متوسط

خروجی کد زیر کدام است؟

```HTML
<ul id="list"><li id="a">A</li></ul>
<p id="note">N</p>
```

```JavaScript
const list = document.getElementById("list");
const note = document.getElementById("note");
list.removeChild(document.getElementById("a"));
console.log(list.children.length);
list.removeChild(note);
console.log("done");
```

۱) `0` `done`
۲) `Uncaught NotFoundError`
۳) `0` `Uncaught TypeError`
۴) `0` `Uncaught NotFoundError`

۱۰۹ متوسط

خروجی کد زیر کدام است؟

```HTML
<ul id="list"><li>A</li><li id="b">B</li><li>C</li></ul>
```

```JavaScript
const list = document.getElementById("list");
const old = document.getElementById("b");
const fresh = document.createElement("li");
fresh.textContent = "X";
const r = list.replaceChild(fresh, old);
console.log(list.textContent, r.textContent);
```

۱) `AXC X`
۲) `AXC B`
۳) `ABC X`
۴) `AXBC B`

۱۱۰ سخت

خروجی کد زیر کدام است؟

```HTML
<ul id="list"><li>A</li><li>B</li></ul>
```

```JavaScript
const list = document.getElementById("list");
const old = list.querySelector("li");
const fresh = document.createElement("li");
fresh.textContent = "NEW";
list.replaceChild(old, fresh);
console.log(list.textContent);
```

۱) `NEWB`
۲) `AB`
۳) `Uncaught NotFoundError`
۴) `Uncaught TypeError`

۱۱۱ سخت

خروجی کد زیر کدام است؟

```HTML
<ul id="list"><li id="a">A</li><li id="b">B</li><li id="c">C</li></ul>
```

```JavaScript
const list = document.getElementById("list");
const a = document.getElementById("a");
const c = document.getElementById("c");
list.replaceChild(a, c);
console.log(list.textContent, list.children.length);
```

۱) `BA 2`
۲) `ABA 3`
۳) `ABC 3`
۴) `AB 2`

۱۱۲ آسان

دستور `old.replaceWith(fresh)` معادل کدام دستور است؟

۱) `old.parentNode.replaceChild(old, fresh)`
۲) `fresh.parentNode.replaceChild(old, fresh)`
۳) `old.removeChild(fresh)`
۴) `old.parentNode.replaceChild(fresh, old)`

۱۱۳ آسان

در مثال «Title Goes Away!» (اسلاید ۲۸)، طبق تصویر اسلاید ۳۰ بعد از **اولین** کلیک روی دکمه چه چیزی روی صفحه دیده می‌شود؟

۱) عنوان «Hello World» و دکمه، بدون تغییر

۲) فقط دکمه‌ی «CLICK ME!»

۳) فقط عنوان «Hello World»

۴) یک صفحه‌ی کاملاً خالی

۱۱۴ متوسط

صفحه‌ی زیر همان کد اسلاید ۲۸ است. کاربر **دو بار** روی دکمه کلیک می‌کند. خروجی کنسول کدام است؟

HTML

```html
<h1 id="title">Hello World</h1>
<button>CLICK ME!</button>
```

JavaScript

```javascript
const btn = document.querySelector('button');
btn.addEventListener('click', () => {
    const element = document.getElementById("title");
    element.remove();
});
```

بعد از دو کلیک این دستور اجرا می‌شود:

```
console.log(document.querySelectorAll("h1").length, document.querySelectorAll("button").length);
```

۱) `0 1`

۲) `0 0` `Uncaught TypeError`

۳) `Uncaught TypeError` `0 1`

۴) `0 0`

۱۱۵ متوسط

در مثال اسلاید ۲۸، برای این‌که کلیک‌های دوم به بعد **هیچ خطایی** ندهند، خط `element.remove();` را با کدام گزینه عوض کنیم؟

۱) `element?.remove();`

۲) `element.remove(true);`

۳) `document.removeChild(element);`

۴) `element.innerHTML = "";`

۱۱۶ سخت

اجرای کدام خط از کد زیر باعث خطا می‌شود؟

HTML

```
<ul id="list"><li>1</li><li>2</li></ul>
```

JavaScript

```
1  const list = document.querySelector("#list");
2  const first = list.firstElementChild;
3  first.remove();
4  console.log("A");
5  first.remove();
6  console.log("B");
7  list.removeChild(first);
8  console.log("C");
```

۱) خط ۳
۲) خط ۵
۳) خط ۷
۴) هیچ خطایی رخ نمی‌دهد

۱۱۷ متوسط

خروجی کد زیر کدام است؟

HTML

```
<div id="box"><p id="msg">Saved!</p></div>
```

JavaScript

```
const box = document.getElementById("box");
const msg = document.getElementById("msg");
msg.remove();
console.log(box.children.length, msg.textContent);
box.append(msg);
console.log(box.children.length, document.getElementById("msg") === msg);
```

۱) `0` `Uncaught TypeError`
۲) `1 Saved!` `1 true`
۳) `0 Saved!` `1 false`
۴) `0 Saved!` `1 true`

۱۱۸ سخت

خروجی کد زیر کدام است؟

```HTML
<div id="card"><h2>Title</h2><p>Text</p></div>
```

```JavaScript
const card = document.getElementById("card");
const title = card.querySelector("h2");
card.remove();
console.log(document.querySelectorAll("h2, p").length, title.isConnected,
card.children.length);
```

۱) `2 true 0`
۲) `0 false 2`
۳) `0 false 0`
۴) `0 true 2`

۱۱۹ متوسط

کدام جمله **نادرست** است؟

۱) `el.remove()` به دسترسی به پدر نیاز ندارد.
۲) `parent.removeChild(child)` گره‌ی حذف‌شده را برمی‌گرداند.
۳) `parent.replaceChild(newC, oldC)` گره‌ی قدیمی را برمی‌گرداند.
۴) بعد از اجرای `el.remove()`، متغیر `el` خودبه‌خود برابر `null` می‌شود.

۱۲۰ آسان

بعد از اجرای کدام دستور، `document.querySelector("#ad")` دیگر عنصر `#ad` را **پیدا نمی‌کند**؟ (`ad` همان عنصر `#ad` است.)

۱) `ad.remove()`
۲) `ad.style.display = "none"`
۳) `ad.hidden = true`
۴) `ad.textContent = ""`

۱۲۱ آسان

هدف این است که همه‌ی فرزندان لیست `list` حذف شوند ولی خودِ `<ul>` در صفحه بماند. کدام گزینه این کار را **انجام نمی‌دهد**؟

۱) `list.innerHTML = ""`
۲) `list.replaceChildren()`
۳) `while (list.firstChild) list.removeChild(list.firstChild);`
۴) `list.remove()`

۱۲۲ سخت

خروجی کد زیر کدام است؟ (به فاصله‌ها و خطوط جدیدِ داخل HTML دقت کن.)

HTML

```
<ul id="list">
  <li>A</li>
  <li>B</li>
</ul>
```

JavaScript

```
const list = document.getElementById("list");
let count = 0;
while (list.firstChild) {
  list.removeChild(list.firstChild);
  count++;
}
console.log(count, list.childNodes.length);
```

۱) `2 0`
۲) `5 0`
۳) `2 3`
۴) `3 0`

۱۲۳ سخت

خروجی کد زیر کدام است؟

HTML

```
<ul>
<li class="item">1</li><li class="item">2</li><li class="item">3</li><li class="item">4</li>
</ul>
```

JavaScript

```
const items = document.getElementsByClassName("item");
for (let i = 0; i < items.length; i++) {
  items[i].remove();
}
console.log(items.length, document.querySelector("ul").textContent.trim());
```

۱) `2 24`
۲) `0`
۳) `2 13`
۴) `3 234`

۱۲۴ متوسط

کاربر روی دکمه‌ی x **دومین** آیتم لیست کلیک می‌کند. خروجی کدام است؟

HTML

```
<ul id="todos">
  <li>Read <button class="del">x</button></li>
  <li>Code <button class="del">x</button></li>
  <li>Sleep <button class="del">x</button></li>
</ul>
```

JavaScript

```
const todos = document.getElementById("todos");
todos.addEventListener("click", (e) => {
  if (e.target.classList.contains("del")) {
    e.target.parentElement.remove();
  }
});
```

بعد از کلیک این دستور اجرا می‌شود: `console.log(todos.children.length, todos.querySelectorAll(".del").length);`

۱) `3 2`
۲) `2 3`
۳) `2 2`
۴) `0 0`

۱۲۵ متوسط

خروجی کد زیر کدام است؟

JavaScript

```
const note = document.createElement("div");
note.className = "toast";
note.textContent = "Saved!";
document.body.append(note);
setTimeout(() => note.remove(), 0);
console.log(document.querySelectorAll(".toast").length);
setTimeout(() => console.log(document.querySelectorAll(".toast").length), 10);
```

۱) `0` `0`
۲) `1` `1`
۳) `0` `1`
۴) `1` `0`

۲-۶ ویژگی‌ها (Attributes) و کلاس‌ها: setAttribute و classList

منبع: JS in HTML، اسلایدهای ۳۱ تا ۳۴

درسنامه

عنوان اسلاید ۳۱ این است: «Everything …Modifying!»؛ یعنی با JavaScript تقریباً **همه‌چیزِ** یک عنصر را می‌شود عوض کرد. در این درس سراغ attributeها (ویژگی‌ها؛ همان جفت‌های `name="value"` که داخل تگ HTML می‌نویسیم) و مهم‌ترینشان، یعنی `class` ، می‌رویم:

HTML

```
<img src="cat.png" alt="A cat" width="300">
<a href="/about" target="_blank">About</a>
<input type="text" value="Ali" disabled>
```

یک تشبیه ساده

هر عنصر را مثل یک جعبه با چند **برچسب** در نظر بگیر: «src: cat.png»، «width: 300». برچسب `setAttribute` می‌چسباند (یا برچسب قبلی را عوض می‌کند)، `getAttribute` برچسب را می‌خواند و `removeAttribute` آن را می‌کَنَد. برچسب `class` هم یک برچسب ویژه است که فهرستی از کلمه‌ها را نگه می‌دارد و ابزار مخصوص خودش، `classList` ، را دارد.

چهار متد کار با attributeها

JavaScript

```
const link = document.querySelector("a");
link.setAttribute("href", "https://sharif.edu");  // add or change
console.log(link.getAttribute("href"));           // "https://sharif.edu"
link.removeAttribute("target");                   // delete it
console.log(link.hasAttribute("target"));         // false
console.log(link.getAttribute("title"));          // null -> no such attribute
```

متد	کار	خروجی
`setAttribute(name, value)`	اگر attribute نباشد **اضافه** می‌کند و اگر باشد مقدارش را **عوض** می‌کند	`undefined`
`getAttribute(name)`	مقدار را **می‌خواند**	همیشه **رشته**، یا `null` اگر وجود نداشته باشد
`removeAttribute(name)`	attribute را **حذف** می‌کند؛ اگر نباشد خطایی نمی‌دهد	`undefined`
`hasAttribute(name)`	**وجود** attribute را بررسی می‌کند	`true` یا `false`

مقداری که به `setAttribute` می‌دهی همیشه **به رشته تبدیل** می‌شود: `setAttribute("title", 42)` مقدار `"42"` را ذخیره می‌کند و `setAttribute("title", null)` مقدار رشته‌ی `"null"` را؛ یعنی `null` دادن، attribute را **حذف نمی‌کند**.

attributeهای بولی (Boolean attributes)

attributeهایی مثل `disabled` ، `checked` ، `hidden` ، `required` و `readonly` فقط با **حضورشان** کار می‌کنند و مقدارشان مهم نیست. پس:

```JavaScript
btn.setAttribute("disabled", "false");  // STILL disabled! the attribute exists
console.log(btn.disabled);              // true
btn.removeAttribute("disabled");        // now it is enabled
btn.disabled = false;                   // same thing, using the property
```

attribute یا property؟

مرورگر برای بیشترِ attributeهای استاندارد یک **property** (خاصیت) هم‌نام روی شیء عنصر می‌سازد: `a.href` ، `img.src` ، `el.id` ، `input.value` و... . اما این دو همیشه یکی نیستند:

موضوع	`getAttribute(...)`	property
نوع مقدار	همیشه رشته (یا `null`)	نوع واقعی: `img.width` عدد `300` ، `btn.disabled` بولی
آدرس‌ها (`src` ، `href`)	همان متنی که نوشته شده: `"dambooldor2.png"`	آدرس کامل و مطلق: `"file:///…/dom/dambooldor2.png"`
`value` در `<input>`	مقدار **اولیه** (نوشته‌شده در HTML)	مقدار **فعلی** (چیزی که کاربر تایپ کرده)
نام	`class` ، `for`	`className` ، `htmlFor`

```HTML
<input id="name" value="guest">
```

```JavaScript
const input = document.getElementById("name");
// ...the user types "Sara" into the box...
console.log(input.value);                  // "Sara"  (current value)
console.log(input.getAttribute("value"));  // "guest" (initial value from HTML)
```

attributeهای ساختگی (غیراستاندارد) مثل `<div foo="1">` property نمی‌سازند (`div.foo` برابر `undefined` است)؛ برای آن‌ها یا `getAttribute` به کار ببر یا از `data-*` استفاده کن.

attributeهای `data-*` و `dataset`

برای چسباندن داده‌ی دلخواه به یک عنصر، استاندارد این است که نام attribute با `data-` شروع شود. JavaScript همه را در شیء `dataset` می‌بیند و نامِ خط‌تیره‌دار را به camelCase (چسباندن کلمه‌ها و بزرگ کردن حرف اول هر کلمه‌ی بعدی) تبدیل می‌کند:

```HTML
<li id="item" data-product-id="42" data-price="9.5">Pen</li>
```

```JavaScript
const item = document.getElementById("item");
console.log(item.dataset.productId);   // "42"  (a string!)
console.log(item.dataset.price);       // "9.5"
item.dataset.inStock = "yes";          // adds data-in-stock="yes" to the tag
```

`classList` : ابزار مخصوص کلاس‌ها

```JavaScript
const box = document.querySelector(".card");   // <div class="card">
box.classList.add("active");              // class="card active"
box.classList.add("big", "shadow");       // several at once: "card active big shadow"
box.classList.remove("shadow");           // "card active big" (no error if missing)
box.classList.toggle("dark");             // added -> returns true
box.classList.toggle("dark");             // removed -> returns false
box.classList.contains("active");         // true
box.classList.replace("big", "small");    // "card active small"
console.log(box.classList.length);        // 3
```

- `add` کلاس تکراری اضافه نمی‌کند و می‌تواند چند کلاس را یک‌جا بگیرد.
- `remove` هم چندتایی کار می‌کند و اگر کلاس وجود نداشته باشد خطا نمی‌دهد.
- `toggle(name)` اگر کلاس نباشد اضافه می‌کند و اگر باشد حذف؛ **خروجی‌اش وضعیت جدید است**: `true` یعنی «الان هست» و `false` یعنی «الان نیست».
- `toggle(name, force)` : با `force` برابر `true` فقط اضافه می‌کند و با `false` فقط حذف. مثلاً `el.classList.toggle("hidden", !isVisible)` کلاس را دقیقاً مطابق یک شرط تنظیم می‌کند.
- `contains(name)` فقط بررسی می‌کند و چیزی را تغییر نمی‌دهد.
- نام کلاس را **بدون نقطه** بنویس: `add("active")` . نقطه مال انتخابگرهای CSS است؛ `add(".active")` کلاسی به نام عجیبِ `.active` می‌سازد!
- هر آرگومان فقط **یک** کلاس است: `add("btn primary")` خطای `InvalidCharacterError` می‌دهد؛ درستش `add("btn", "primary")` است.

`className` ، `classList` و `setAttribute("class", …)`

	نوع	نوشتن با آن
`el.className`	یک رشته مثل `"card active"`	**همه‌ی** کلاس‌ها را جایگزین می‌کند
`el.setAttribute("class", …)`	—	**همه‌ی** کلاس‌ها را جایگزین می‌کند
`el.classList`	شیئی فهرست‌مانند (DOMTokenList) با متدهای بالا	فقط همان کلاس را اضافه/حذف می‌کند

```JavaScript
// <p class="note big">
p.setAttribute("class", "red");   // class="red"  (note and big are gone!)
p.classList.add("bold");          // class="red bold"
p.className += " small";          // class="red bold small" (do not forget the space)
```

استایل پویا و تغییر وضعیت با کلاس‌ها

آخرین نکته‌ی اسلاید ۳۱ می‌گوید این ابزارها برای **استایل پویا** و **تغییر وضعیت** (state) به کار می‌روند. به‌جای این‌که در JavaScript ده‌ها استایل درون‌خطی (inline style؛ مثل `el.style.color = "red"`) بنویسی، ظاهر را در CSS به شکل کلاس تعریف کن و در JavaScript فقط کلاس را اضافه یا حذف کن:

CSS

```
.dark   { background: #222; color: #eee; }
.hidden { display: none; }
.error  { border: 2px solid red; }
```

JavaScript

```
themeBtn.addEventListener("click", () => {
  const isDark = document.body.classList.toggle("dark");  // state: dark on or off
  themeBtn.textContent = isDark ? "Light mode" : "Dark mode";
});
```

این کار سه لایه‌ی صفحه (ساختار با HTML، ظاهر با CSS، رفتار با JavaScript؛ یادآوری اسلاید ۳) را از هم جدا نگه می‌دارد، برگرداندنش با یک `remove` یا `toggle` ساده است و یک کلاس را می‌شود روی عنصرهای زیادی به کار برد. کلاس‌ها **وضعیت** را هم نشان می‌دهند: تب `active`، منوی `open`، آیتم `selected`، فیلد `error`؛ و `contains` می‌گوید الان در کدام وضعیت هستیم.

مثال اسلایدهای ۳۲ تا ۳۴: Change Image

doc.html

```
1  <!DOCTYPE html>
2  <html>
3  <body>
4      <img src="dambooldor.png" id="dambooldor" width="300" height="300">
5      <button>clickme</button>
6      <script>
7          const btn = document.querySelector('button');
8          btn.addEventListener('click', () => {
9              const img = document.getElementById("dambooldor");
10             img.setAttribute("src", "dambooldor2.png");
11         });
12     </script>
13 </body>
14 </html>
```

- **خط ۴:** تصویری ۳۰۰ در ۳۰۰ با `src="dambooldor.png"` و `id="dambooldor"` (شوخیِ اسلاید با نام «دامبلدور»).
- **خطوط ۷ و ۸:** دکمه را پیدا می‌کنیم و listener کلیک را وصل می‌کنیم.
- **خط ۹:** تصویر را با `id` پیدا می‌کنیم.
- **خط ۱۰:** مقدار attribute `src` را عوض می‌کنیم؛ مرورگر بلافاصله تصویر جدید را بار می‌کند و نشان می‌دهد.

قبل از کلیک (اسلاید ۳۳): تصویر اول (دامبلدور با رنگ‌های قرمز) و دکمه‌ی «clickme» درست کنار آن، چون `<img>` و `<button>` هر دو عنصرهای inline (عنصرهایی که در یک خط کنار هم می‌نشینند) هستند. **بعد از کلیک (اسلاید ۳۴):** تصویر دیگری (`dambooldor2.png`، با رنگ‌های آبی‌سبز) به‌جایش آمده؛ اندازه همچنان ۳۰۰ در ۳۰۰ است، چون به attributeهای `width` و `height` دست نزدیم، و دکمه هم سر جایش است.

چند نکته درباره‌ی همین مثال:

- `img.src = "dambooldor2.png";` (کار با property) همین اثر را دارد.
- بعد از کلیک، `img.getAttribute("src")` برابر `"dambooldor2.png"` است ولی `img.src` آدرس کامل را می‌دهد: `file:///home/mani/Projects/University/web-ta/sampleprojects/dom/dambooldor2.png`.
- کلیک دوم همان مقدار را دوباره می‌نویسد؛ نه تغییری دیده می‌شود و نه خطایی رخ می‌دهد.

برای اینکه هر کلیک تصویر را **بین دو حالت** عوض کند، باید با `getAttribute` (نه `img.src`) مقایسه کنیم:

```JavaScript
btn.addEventListener("click", () => {
  const img = document.getElementById("dambooldor");
  const next = img.getAttribute("src") === "dambooldor.png" ? "dambooldor2.png" : "dambooldor.png";
  img.setAttribute("src", next);
});
```

نکته

- `setAttribute` هم اضافه می‌کند هم تغییر می‌دهد؛ `getAttribute` همیشه رشته یا `null` برمی‌گرداند؛ `removeAttribute` روی attribute ناموجود خطا نمی‌دهد.
- برای کلاس‌ها `classList` را ترجیح بده: `add`، `remove` و `toggle` فقط همان کلاس را دست می‌زنند و بقیه را نگه می‌دارند.
- `toggle` وضعیت جدید را برمی‌گرداند؛ پس `const isDark = body.classList.toggle("dark")` هم وضعیت را عوض می‌کند و هم می‌گوید الان در چه وضعیتی هستیم.
- ظاهر را در CSS تعریف کن و در JavaScript فقط کلاس را عوض کن.

دام تستی

- `getAttribute` برای attribute ناموجود `null` برمی‌گرداند، نه `undefined` و نه رشته‌ی خالی.
- `getAttribute("width")` رشته‌ی `"300"` است ولی `img.width` عدد `300`؛ مقدار `dataset` هم همیشه رشته است (`"42" + 1` می‌شود `"421"`).
- `img.src` و `a.href` آدرس **کامل** را برمی‌گردانند؛ مقایسه‌ی `img.src === "a.png"` تقریباً هیچ‌وقت درست نیست.
- `setAttribute("disabled", "false")` دکمه را غیرفعال **می‌کند**؛ برای فعال کردن `removeAttribute("disabled")` لازم است.
- `setAttribute("class", …)` و `className = …` همه‌ی کلاس‌های قبلی را پاک می‌کنند.
- `classList.add(".active")` کلاسی با نقطه می‌سازد و `classList.add("a b")` خطای `InvalidCharacterError` می‌دهد.
- بعد از تایپ کاربر، `input.value` و `input.getAttribute("value")` با هم فرق دارند.

جمع‌بندی

attributeها: `setAttribute` (افزودن/تغییر)، `getAttribute` (خواندن؛ رشته یا `null`)، `removeAttribute` (حذف)، `hasAttribute` (وجود). property همان مقدار را با نوع واقعی و وضعیت فعلی نشان می‌دهد (`img.src` مطلق، `input.value` فعلی). کلاس‌ها: `classList.add/remove/toggle/contains/replace` ؛ `className` و `setAttribute("class")` همه را جایگزین می‌کنند. استایل پویا = تعریف کلاس در CSS + عوض کردن کلاس در JavaScript.

تست‌های تمرینی

۲۱ تست · شماره‌ی ۱۲۶ تا ۱۴۶

۱۲۶ آسان

خروجی کد زیر کدام است؟

```HTML
<img id="pic" src="cat.png">
```

```JavaScript
const pic = document.getElementById("pic");
console.log(pic.getAttribute("alt"));
```

۱) `undefined`
۲) `null`
۳) `false`
۴) `Uncaught TypeError`

۱۲۷ آسان

کدام دستور attribute `title` را به‌طور **کامل** از عنصر `el` حذف می‌کند؟

۱) `el.setAttribute("title", null)`
۲) `el.setAttribute("title", "")`
۳) `el.removeAttribute("title")`
۴) `el.classList.remove("title")`

۱۲۸ آسان

خروجی کد زیر کدام است؟

```HTML
<a id="link" href="/home">Home</a>
```

```JavaScript
const link = document.getElementById("link");
link.setAttribute("href", "/about");
link.setAttribute("target", "_blank");
console.log(link.getAttribute("href"), link.getAttribute("target"), link.attributes.length);
```

۱) `/home _blank 3`
۲) `/about _blank 2`
۳) `/about null 3`
۴) `/about _blank 3`

۱۲۹ متوسط

خروجی کد زیر کدام است؟

```HTML
<div id="box">Box</div>
```

```JavaScript
const box = document.getElementById("box");
box.removeAttribute("title");
console.log(box.hasAttribute("title"), box.hasAttribute("id"));
```

۱) `Uncaught NotFoundError`
۲) `false true`
۳) `true true`
۴) `undefined true`

۱۳۰ متوسط

خروجی کد زیر کدام است؟

```HTML
<img src="a.png" width="300">
```

```JavaScript
const img = document.querySelector("img");
console.log(typeof img.getAttribute("width"), typeof img.width);
```

۱) `string number`
۲) `number number`
۳) `string string`
۴) `number string`

۱۳۱ سخت

خروجی کد زیر کدام است؟

```HTML
<button id="send">Send</button>
```

```JavaScript
const send = document.getElementById("send");
send.setAttribute("disabled", "false");
console.log(send.disabled);
send.removeAttribute("disabled");
console.log(send.disabled);
```

۱) `false` `false`
۲) `false` `true`
۳) `true` `false`
۴) `true` `true`

۱۳۲ آسان

در مثال «Change Image» (اسلاید ۳۲) بعد از کلیک روی دکمه‌ی «clickme»، طبق اسلاید ۳۴ چه تغییری روی صفحه دیده می‌شود؟

۱) تصویر و دکمه هر دو حذف می‌شوند و تصویر جدیدی جای آن‌ها می‌آید.

۲) فقط تصویر عوض می‌شود؛ اندازه‌ی ۳۰۰ در ۳۰۰ و دکمه بدون تغییر می‌مانند.

۳) تصویر عوض می‌شود ولی اندازه‌اش به اندازه‌ی واقعی فایل جدید برمی‌گردد.

۴) تصویر جدید زیر تصویر قبلی اضافه می‌شود.

۱۳۳ متوسط

صفحه‌ی زیر همان کد اسلاید ۳۲ است. کاربر روی دکمه کلیک می‌کند. خروجی کدام است؟

HTML

```html
<img src="dambooldor.png" id="dambooldor" width="300" height="300">
<button>clickme</button>
```

JavaScript

```javascript
const btn = document.querySelector('button');
btn.addEventListener('click', () => {
    const img = document.getElementById("dambooldor");
    img.setAttribute("src", "dambooldor2.png");
});
```

بعد از کلیک این دستور اجرا می‌شود: `console.log(pic.getAttribute("src"), pic.getAttribute("width"));` که در آن `pic` همان تصویر `#dambooldor` است.

۱) `dambooldor2.png 300`

۲) `dambooldor.png 300`

۳) `dambooldor2.png null`

۴) `dambooldor.png dambooldor2.png`

۱۳۴ سخت

صفحه از آدرس `https://quiz.example/gallery/index.html` باز شده است. خروجی کد زیر کدام است؟

HTML

```html
<img src="day.png">
```

JavaScript

```javascript
const img = document.querySelector("img");
img.setAttribute("src", "cat.png");
console.log(img.getAttribute("src"));
console.log(img.src);
```

۱) `cat.png` `cat.png`

۲) `cat.png` `https://quiz.example/gallery/cat.png`

۳) `https://quiz.example/gallery/cat.png` `cat.png`

۴) `https://quiz.example/cat.png` `cat.png`

۱۳۵ متوسط

`<input>` زیر را در نظر بگیر. کاربر محتوای کادر را پاک می‌کند و `Sara` را تایپ می‌کند. خروجی کدام است؟

```HTML
<input id="name" value="guest">
```

```JavaScript
const input = document.getElementById("name");
input.addEventListener("input", () => {
  console.log(input.value, input.getAttribute("value"));
});
```

۱) `Sara Sara`
۲) `guest guest`
۳) `guest Sara`
۴) `Sara guest`

۱۳۶ سخت

صفحه از آدرس `https://quiz.example/` باز شده و کاربر **سه بار** روی دکمه کلیک می‌کند. خروجی کدام است؟

```HTML
<img id="pic" src="day.png">
<button id="switch">Switch</button>
```

```JavaScript
const pic = document.getElementById("pic");
document.getElementById("switch").addEventListener("click", () => {
  if (pic.src === "day.png") {
    pic.src = "night.png";
  } else {
    pic.src = "day.png";
  }
  console.log(pic.getAttribute("src"));
});
```

۱) `night.png` `day.png` `night.png`
۲) `night.png` `night.png` `night.png`
۳) `day.png` `night.png` `day.png`
۴) `day.png` `day.png` `day.png`

۱۳۷ سخت

خروجی کد زیر کدام است؟

```html
<li id="item" data-product-id="42" data-price="9.5">Pen</li>
```

```javascript
const item = document.getElementById("item");
console.log(item.dataset.productId + 1);
item.dataset.inStock = "yes";
console.log(item.getAttribute("data-in-stock"));
```

۱) `43` `yes`

۲) `NaN` `null`

۳) `421` `null`

۴) `421` `yes`

۱۳۸ آسان

خروجی کد زیر کدام است؟

```html
<div id="box" class="card"></div>
```

```javascript
const box = document.getElementById("box");
box.classList.add("active", "big");
box.classList.add("active");
console.log(box.className, box.classList.length);
```

۱) `card active big 3`

۲) `card active big active 4`

۳) `active big 2`

۴) `active big active 3`

۱۳۹ متوسط

خروجی کد زیر کدام است؟

```html
<p class="a b c">Text</p>
```

```javascript
const el = document.querySelector("p");
el.classList.remove("ghost");
el.classList.remove("a", "c");
console.log(el.className, el.classList.contains("b"));
```

۱) `Uncaught NotFoundError`

۲) `a b c true`

۳) `b false`

۴) `b true`

۱۴۰ متوسط

خروجی کد زیر کدام است؟

```javascript
const body = document.body;
console.log(body.classList.toggle("dark"));
console.log(body.classList.toggle("dark"));
console.log(body.classList.toggle("dark", true));
console.log(body.className);
```

۱) `undefined` `undefined` `undefined` `dark`

۲) `true` `false` `true` `dark`

۳) `false` `true` `false` `dark`

۴) `true` `true` `true` `dark`

۱۴۱ متوسط

خروجی کد زیر کدام است؟

```html
<p id="p" class="note big">Hi</p>
```

```javascript
const p = document.getElementById("p");
p.setAttribute("class", "red");
p.classList.add("bold");
p.className += " small";
console.log(p.className);
```

۱) `note big red bold small`

۲) `red small`

۳) `red bold small`

۴) `bold small`

۱۴۲ سخت

خروجی کد زیر کدام است؟

```html
<div id="tab">Home</div>
```

```javascript
const tab = document.querySelector("#tab");
tab.classList.add(".active");
console.log(tab.classList.contains("active"), tab.className);
```

۱) `false .active`

۲) `true active`

۳) `true .active`

۴) `Uncaught SyntaxError`

۱۴۳ سخت

خروجی کد زیر کدام است؟

```HTML
<button>Save</button>
```

```JavaScript
const btn = document.querySelector("button");
btn.classList.add("btn primary");
console.log(btn.className);
```

۱) `btn primary`
۲) `Uncaught InvalidCharacterError`
۳) `btn`
۴) `btn,primary`

۱۴۴ متوسط

می‌خواهیم هر کلیک روی دکمه، حالت تاریک صفحه را **روشن یا خاموش** کند (اگر کلاس `dark` هست برداشته شود و اگر نیست اضافه شود). در جای خالی `document.body.classList.______("dark");` کدام گزینه قرار می‌گیرد؟

۱) `add`
۲) `contains`
۳) `toggle`
۴) `replace`

۱۴۵ سخت

کدام جمله **نادرست** است؟

۱) `el.classList.contains("x")` اگر کلاس `x` وجود نداشته باشد، آن را اضافه می‌کند.
۲) `el.className = "x"` همه‌ی کلاس‌های قبلی را با `x` جایگزین می‌کند.
۳) `el.classList.add("x")` کلاس‌های قبلی را نگه می‌دارد.
۴) `el.classList.toggle("x")` اگر `x` وجود داشته باشد آن را حذف می‌کند و `false` برمی‌گرداند.

۱۴۶ آسان

طبق اسلاید ۳۱ کلاس‌ها برای «استایل پویا و تغییر وضعیت» به کار می‌روند. کدام گزینه بهترین دلیل برای اضافه و حذف کردن **کلاس‌های CSS** به‌جای نوشتن چندین `el.style.…` در JavaScript است؟

۱) تغییر `style` از طریق JavaScript در مرورگرهای جدید کار نمی‌کند.
۲) `classList` تنها راه تغییر رنگ یک عنصر است.
۳) با کلاس‌ها، صفحه بدون CSS هم ظاهر درستی خواهد داشت.
۴) ظاهر در CSS تعریف می‌شود و JavaScript فقط وضعیت را عوض می‌کند؛ برگرداندن آن ساده است و یک کلاس را می‌شود روی عنصرهای زیادی به کار برد.

فصل سوم

۳

Node.js و TypeScript

بر اساس اسلایدهای «5- NodeJS & TS.pdf»

در این فصل از مرورگر بیرون می‌آییم: Node.js چرا و چطور به وجود آمد، npm و `package.json` و اسکریپت‌ها چه هستند، bundler به چه دردی می‌خورد و پکیج‌های معروف کدام‌اند. نیمه‌ی دوم فصل TypeScript است: نوع‌ها، interface، توابع، کلاس‌ها، generic، ماژول‌ها و بهترین شیوه‌ها.

۶ درس · ۱۲۷ تست · آزمون جامع پایان فصل

۳-۱ محدودیت‌های JavaScript و تولد Node.js
۳-۲ ویژگی‌های Node.js
۳-۳ مشکلات تایپ در JavaScript و معرفی TypeScript
۳-۴ متغیرها و انواع داده در TypeScript
۳-۵ کلاس‌ها در TypeScript
۳-۶ Generics در TypeScript

۳-۱ محدودیت‌های JavaScript و تولد Node.js

منبع: NodeJS & TS، اسلایدهای ۲ تا ۵

درسنامه

تا اینجای کتاب، JavaScript را فقط **داخل مرورگر** دیدیم: دکمه‌ها را کلیک‌پذیر کردیم، فرم‌ها را بررسی کردیم و DOM را تغییر دادیم. عنوان اسلاید اول این جلسه «**Beyond JavaScript**» است، یعنی «فراتر از JavaScript». در این درس می‌بینیم JavaScript اولش چه ضعف‌هایی داشت، چرا برنامه‌نویس‌ها خواستند آن را از مرورگر بیرون بیاورند و **Node.js** چطور این کار را ممکن کرد.

در این جلسه چه یاد می‌گیریم؟ (اسلاید ۲)

اسلاید ۲ سه هدف را روی سه کارت نشان می‌دهد:

کارت	متن اسلاید	یعنی…	کجای این فصل؟
a	Make console applications with JS	ساختن **برنامه‌های کنسولی** (console application: برنامه‌ای که در ترمینال اجرا می‌شود و صفحه‌ی گرافیکی ندارد) با JavaScript	Node.js و npm
b	Learn how JS runs on servers	یاد گرفتن اینکه JavaScript چطور روی **سرور** اجرا می‌شود	Node.js
c	Solve the old JS typing issues	حل مشکلات قدیمی **نوع‌دهی** (typing) در JavaScript	TypeScript (نیمه‌ی دوم فصل)

چند مفهوم پایه قبل از شروع

زبان، موتور و محیط اجرا

این سه کلمه را زیاد می‌شنوی و نباید قاطی‌شان کنی:

- **زبان (language)**: خودِ JavaScript؛ یعنی قواعد نوشتن کد (`let`، تابع، حلقه و…).
- **موتور (engine)**: برنامه‌ای که کد JavaScript را می‌خواند، به زبان ماشین تبدیل و اجرا می‌کند. معروف‌ترینش **V8** است که Google برای مرورگر Chrome ساخت.
- **محیط اجرا (runtime)**: موتور به‌اضافه‌ی ابزارهای آماده‌ای که آن محیط در اختیار کد می‌گذارد. مرورگر به کد ما `document` و `window` و `alert` می‌دهد؛ Node.js به جای آن‌ها ابزارهایی مثل `fs` (کار با فایل)، `http` (ساختن سرور) و `process` (اطلاعات برنامه‌ی در حال اجرا) می‌دهد.

> **یک تشبیه ساده**
>
> زبان مثل **دستور پخت** است، موتور مثل **آشپزی** است که دستور را می‌خواند و انجام می‌دهد، و محیط اجرا مثل **کل آشپزخانه** است: آشپز به‌اضافه‌ی وسایلش. آشپزی به اسم V8 هم در آشپزخانه‌ی Chrome کار می‌کند و هم در آشپزخانه‌ی Node.js؛ آشپز یکی است ولی وسایل دو آشپزخانه فرق دارد. برای همین در Node.js خبری از `document` نیست، ولی می‌توانی فایل بخوانی.

	مرورگر (مثلاً Chrome)	Node.js
زبان	JavaScript	JavaScript
موتور	V8	V8
ابزارهای مخصوص	`window`، `document`، `alert`، `localStorage`	`fs`، `http`، `process`، `require`
کاربرد اصلی	رابط کاربری صفحه‌ی وب	سرور، برنامه‌ی کنسولی، ابزارهای برنامه‌نویسی

client و server، frontend و backend

وقتی آدرس یک سایت را در مرورگر می‌زنی، مرورگرت یک **client** (کلاینت: برنامه‌ای که درخواست می‌فرستد) است و کامپیوتری که جواب می‌دهد یک **server** (سرور: برنامه یا کامپیوتری که به درخواست‌ها پاسخ می‌دهد). بخشی از برنامه که کاربر می‌بیند و در مرورگر اجرا می‌شود **frontend** (سمت کاربر) است و بخش پشت صحنه که روی سرور اجرا می‌شود (ذخیره در پایگاه داده، ورود کاربر، پرداخت و...) **backend** (سمت سرور).

I/O یعنی چه؟

I/O (Input/Output، ورودی/خروجی) هر کاری است که برنامه با «بیرون از خودش» انجام می‌دهد: خواندن و نوشتن فایل، فرستادن درخواست در شبکه، پرسیدن از پایگاه داده. این کارها در مقایسه با محاسبه‌ی خودِ CPU **خیلی کُند** هستند؛ پس یک سرور بیشتر وقتش را **منتظر I/O** است، نه مشغول حساب کردن. این نکته کلید فهمیدن Node.js است.

محدودیت‌های JavaScript (اسلاید ۳)

JavaScript در سال ۱۹۹۵ برای کارهای ساده‌ی صفحه‌ی وب ساخته شد. اسلاید ۳ پنج محدودیت آن را می‌شمارد:

۱. فقط برای کارهای ساده‌ی مرورگر طراحی شده بود. هدفش چیزهایی مثل بررسی یک فرم یا واکنش به کلیک بود، نه **برنامه‌های کامل و بزرگ** (full-scale applications) یا **منطق سمت سرور** (backend logic).

۲. اجرای تک‌نخی (single-threaded) باعث مسدود شدن (blocking) می‌شد. **thread** (نخ اجرا) را مثل یک کارگر در نظر بگیر. JavaScript فقط **یک** کارگر دارد که در هر لحظه یک کار انجام می‌دهد. اگر این کارگر درگیر یک محاسبه‌ی سنگین یا یک عملیات I/O طولانی شود، بقیه‌ی کارها **منتظر می‌مانند**؛ به این وضعیت **blocking** (مسدود شدن) می‌گوییم. مثال زیر را ببین:

```JavaScript
setTimeout(() => console.log("timer"), 0);

const start = Date.now();
while (Date.now() - start < 200) {} // busy wait: blocks the only thread for 200 ms
console.log("loop done");
// Output:
// loop done
// timer
```

با اینکه زمان تایمر `0` است، `timer` **بعد از** `loop done` چاپ می‌شود؛ چون تنها thread برنامه ۲۰۰ میلی‌ثانیه درگیر حلقه بود و نوبت به callback تایمر نمی‌رسید. (جزئیات صف‌ها و event loop را در فصل اول خواندیم.)

۳. نوع‌دهی پویا (dynamic typing) خطاهای غیرقابل‌پیش‌بینی می‌ساخت. در JavaScript نوع متغیر هنگام اجرا مشخص می‌شود و می‌تواند عوض شود. در پروژه‌های بزرگ این یعنی باگ‌هایی که فقط **هنگام اجرا** (runtime) پیدا می‌شوند و نگه‌داری کد را سخت می‌کنند:

```JavaScript
function total(price, count) {
  return price + count;
}
console.log(total(5, 2));   // 7
console.log(total("5", 2)); // "52" (a hidden bug: string concatenation!)
```

جواب این مشکل **TypeScript** است که در نیمه‌ی دوم همین فصل می‌خوانیم.

۴. نبودِ سیستم ماژول (module system). **ماژول** (module) یعنی یک فایل کد با مرز مشخص که فقط چیزهایی را که خودش بخواهد به بقیه نشان می‌دهد. JavaScript اولیه چنین چیزی نداشت: همه‌ی فایل‌های `<script>` یک **محدوده‌ی سراسری** (global scope) مشترک داشتند و اسم‌ها با هم تداخل می‌کردند:

```HTML
<script src="cart.js"></script>  <!-- var count = 5; -->
<script src="likes.js"></script> <!-- var count = 0; overwrites the cart count! -->
```

پس سازمان‌دهی کد و استفاده‌ی دوباره از آن سخت بود. بعدها Node.js سیستم ماژول CommonJS (`require`) را آورد و خود زبان هم ES Modules (`import` و `export`) را به دست آورد.

۵. ابزارهای محدود و مقیاس‌پذیری ضعیف. ابزار درست‌وحسابی برای مدیریت کتابخانه‌ها، ساختن (build) و تست پروژه نبود و کد با بزرگ شدن پروژه به‌سختی قابل نگه‌داری بود؛ یعنی **scalability** (مقیاس‌پذیری: توانایی رشد کردن بدون به هم ریختن) ضعیفی داشت.

محدودیت (اسلاید ۳)	جوابش در همین فصل
فقط مرورگر، نه برنامه‌ی کامل یا backend	Node.js (اجرای JS روی سرور و در ترمینال)
تک‌نخی و blocking در I/O	مدل event-driven و non-blocking در Node.js
dynamic typing	TypeScript
نبودِ module system	ماژول‌های CommonJS و ES Modules، پکیج‌های npm
ابزار محدود و مقیاس‌پذیری ضعیف	npm، bundlerها، TypeScript

جرقه‌ی اولیه (Inception، اسلاید ۴)

چه شد که کسی به فکر بردن JavaScript به بیرون از مرورگر افتاد؟ اسلاید ۴ پنج دلیل می‌آورد:

۱. برنامه‌نویس‌ها می‌خواستند مهارت JavaScript خود را **بیرون از مرورگر** هم به کار ببرند، مخصوصاً برای منطق **backend**؛ به جای اینکه برای سرور یک زبان دیگر (مثل PHP یا Java) هم یاد بگیرند.

۲. برنامه‌های وب **پویاتر** شدند و تبادل داده بین client و server باید **سریع‌تر و یکپارچه‌تر** (unified) انجام می‌شد؛ اگر هر دو طرف یک زبان و یک قالب داده داشته باشند، این کار خیلی ساده‌تر است.

۳. ظهور **AJAX** و **JSON** نشان داد JavaScript می‌تواند کارهای **ناهمگام** (asynchronous: کاری که شروع می‌شود و نتیجه‌اش بعداً می‌رسد، بدون اینکه برنامه منتظر بماند) را به‌خوبی مدیریت کند.

- (Asynchronous JavaScript And XML) **AJAX**: روشی که صفحه‌ی وب **بدون بارگذاری دوباره** و در پس‌زمینه از سرور داده بگیرد؛ مثلاً وقتی در نقشه‌ی آنلاین جابه‌جا می‌شوی و تکه‌های جدید نقشه بدون refresh ظاهر می‌شوند.

- (JavaScript Object Notation) **JSON**: یک قالب متنی برای جابه‌جایی داده که دقیقاً شبیه شیءهای JavaScript است:

```JSON
{ "name": "Ali", "age": 21, "courses": ["Web", "AI"] }
```

۴. **بهبود موتورها** (engine improvements)، مخصوصاً **V8** ساخت Google، JavaScript را آن‌قدر سریع کرد که برای بار کاری سرور هم کافی بود. V8 کد JavaScript را مستقیم به **کد ماشین** تبدیل می‌کند.

۵. همه‌ی این‌ها به ایده‌ی اجرای JavaScript **بیرون از مرورگر** رسید؛ یعنی شروع حرکت به سمت **server-side JavaScript** (JavaScript سمت سرور).

> **تاریخچه‌ی کوتاه**
>
> این تاریخ‌ها در اسلایدها نیست و فقط برای تصویر ذهنی بهتر آمده است: ۱۹۹۵ تولد JavaScript در شرکت Netscape؛ ۲۰۰۵ رواج نام AJAX؛ ۲۰۰۸ عرضه‌ی Chrome و موتور V8؛ ۲۰۰۹ معرفی Node.js توسط Ryan Dahl؛ ۲۰۱۰ عرضه‌ی npm.

سرانجام Node.js آمد (اسلاید ۵)

Node.js یک **runtime** (محیط اجرا) برای JavaScript است که روی موتور V8 ساخته شده و اجازه می‌دهد کد JavaScript را **بیرون از مرورگر** (روی سرور یا در ترمینال کامپیوتر خودت) اجرا کنی. دقت کن: Node.js **زبان جدید نیست** (زبانش همان JavaScript است)، **فریم‌ورک** (framework) هم نیست و **مرورگر** هم نیست.

اسلاید ۵ پنج ویژگی آن را می‌گوید:

۱. **Powered by Google's fast V8 engine:** همان موتور سریع Chrome.

۲. **Event-driven, non-blocking I/O model:** مهم‌ترین ایده‌ی Node.js.

- **non-blocking I/O** (ورودی/خروجی غیرمسدودکننده): Node.js کار I/O (مثلاً خواندن فایل) را **شروع می‌کند و منتظر نمی‌ماند**؛ می‌رود سراغ ادامه‌ی کد. وقتی نتیجه آماده شد، خبرش می‌کنند.
- **event-driven** (رویدادمحور): کد ما به **رویدادها** (event) واکنش نشان می‌دهد: «درخواستی رسید»، «خواندن فایل تمام شد»، «زمان تایمر رسید». ما از قبل یک تابع **callback** (تابعی که بعداً صدا زده می‌شود) ثبت می‌کنیم و Node.js هر وقت آن رویداد رخ داد، آن را اجرا می‌کند.

> **گارسون رستوران**
>
> یک رستوران را با **یک گارسون** تصور کن (همان thread واحد).
>
> - **گارسون blocking**: سفارش میز ۱ را می‌گیرد، به آشپزخانه می‌دهد و **دم در آشپزخانه می‌ایستد** تا غذا آماده شود؛ در این مدت میزهای دیگر معطل‌اند.
> - **گارسون non-blocking (مثل Node.js)**: سفارش میز ۱ را به آشپزخانه می‌دهد و **فوراً** سراغ میز ۲ و ۳ می‌رود. هر وقت آشپزخانه **زنگ زد** (رویداد)، غذای آماده را می‌برد (اجرای callback).
>
> با همان یک گارسون، رستوران دوم به مشتری‌های خیلی بیشتری می‌رسد. آشپزخانه اینجا همان سیستم‌عامل و بخش‌های داخلی Node.js است که کار I/O را در پس‌زمینه انجام می‌دهند.

این تفاوت را در کد ببین. `fs.readFile` غیرمسدودکننده است و `fs.readFileSync` مسدودکننده:

non-blocking.js

```
const fs = require("fs");

console.log("start");
fs.readFile(__filename, "utf8", () => {
  console.log("file read"); // runs later, when the file is ready
});
console.log("end");
// Output: start, end, file read
```

blocking.js

```
const fs = require("fs");

console.log("start");
const text = fs.readFileSync(__filename, "utf8"); // waits here until the file is read
console.log("file read");
console.log("end");
// Output: start, file read, end
```

(`__filename` در Node.js مسیر همین فایلِ در حال اجراست؛ یعنی برنامه دارد خودش را می‌خواند.)

۳. **Same language for frontend and backend:** یک زبان برای هر دو سمت. یک برنامه‌نویس می‌تواند هم frontend و هم backend بنویسد (به چنین کسی full-stack می‌گویند) و حتی کد مشترک (مثلاً تابع بررسی ایمیل) را در هر دو سمت استفاده کند.

۴. **Massive npm ecosystem for packages:** اکوسیستم عظیم **npm** (مدیر بسته‌ی Node) با میلیون‌ها پکیج آماده. در درس‌های بعدی مفصل می‌خوانیمش.

۵. **Turned JavaScript into a full platform:** JavaScript از «زبان اسکریپت‌نویسی مرورگر» به یک **پلتفرم کامل** تبدیل شد: سرور، برنامه‌ی کنسولی، ابزارهای ساخت پروژه و حتی برنامه‌ی دسکتاپ.

اولین برنامه‌ها با Node.js

یک فایل بساز و با دستور `node` اجرایش کن:

hello.js

```
console.log("Hello from Node.js!");
```

Terminal

```
node hello.js
# Hello from Node.js!
```

این یک **برنامه‌ی کنسولی** است؛ همان هدف a در اسلاید ۲. حالا یک وب‌سرور خیلی کوچک (هدف b):

server.js

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.end("Hello from a Node.js server!"); // the answer sent to the browser
});

server.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

بعد از `node server.js` برنامه تمام نمی‌شود؛ منتظر **رویداد** «رسیدن درخواست» می‌ماند. هر بار که آدرس `http://localhost:3000` را در مرورگر باز کنی، callback داده‌شده به `createServer` اجرا می‌شود و متن را برمی‌گرداند. این یعنی event-driven بودن.

در Node.js چه چیزهایی نیست و چه چیزهایی هست؟

Node.js صفحه‌ی وب ندارد، پس **DOM** هم ندارد: `window` ، `document` و `alert` تعریف نشده‌اند. به جایش چیزهای دیگری دارد:

- `globalThis` : شیء سراسری که در **هر دو** محیط هست (در Node.js همان `global` است و در مرورگر همان `window`).
- `process` : اطلاعات برنامه‌ی در حال اجرا؛ مثل `process.argv` (آرگومان‌های خط فرمان)، `process.env` (متغیرهای محیطی) و `process.platform` (نام سیستم‌عامل).
- ماژول‌های داخلی مثل `fs` (فایل)، `http` (سرور)، `path` (مسیر فایل‌ها) و `os` (اطلاعات سیستم‌عامل).

JavaScript

```
// Running in Node.js
console.log(typeof window);     // "undefined"
console.log(typeof globalThis); // "object"
console.log(typeof process);    // "object"
console.log(document.title);    // ReferenceError: document is not defined
```

نکته

Node.js **زبان** نیست؛ یک **runtime** است. زبانش همان JavaScript است و موتورش همان V8 مرورگر Chrome. فرق Node.js با مرورگر در **ابزارهای اطراف موتور** است: مرورگر DOM دارد و Node.js فایل و شبکه و `process` .

دام تستی

«Node.js تک‌نخی است، پس در هر لحظه فقط به یک کاربر جواب می‌دهد.» این جمله **غلط** است. چون I/O در Node.js غیرمسدودکننده است، همان یک thread هر درخواست را راه می‌اندازد و سراغ درخواست بعدی می‌رود؛ پس هزاران درخواست هم‌زمان را مدیریت می‌کند. ولی دقت کن: کارِ **محاسباتی سنگین** (مثل حلقه‌ی طولانی بالا) هنوز همان یک thread را **مسدود می‌کند** و همه منتظر می‌مانند.

⚠ دام تستی

`typeof window` در Node.js خطا **نمی‌دهد** و فقط رشته‌ی `"undefined"` برمی‌گرداند (`typeof` روی متغیر تعریف‌نشده خطا نمی‌دهد). ولی استفاده‌ی واقعی از آن، مثل `document.title` یا `window.innerWidth`، خطای `ReferenceError` می‌دهد.

جمع‌بندی

- اهداف جلسه: برنامه‌ی کنسولی با JS، اجرای JS روی سرور، حل مشکلات نوع‌دهی (با TypeScript).
- محدودیت‌های JS: فقط مرورگر، تک‌نخی و blocking، dynamic typing، نبودِ module system، ابزار و مقیاس‌پذیری ضعیف.
- جرقه‌ی اولیه: استفاده‌ی دوباره از مهارت JS در backend، تبادل داده‌ی یکپارچه، AJAX و JSON، موتور V8، JavaScript سمت سرور.
- Node.js: موتور I/O، V8، رویدادمحور و غیرمسدودکننده، یک زبان برای frontend و backend، اکوسیستم npm، پلتفرم کامل.

تست‌های تمرینی

۱۷ تست · شماره‌ی ۱۴۷ تا ۱۶۳

۱۴۷ آسان

کدام گزینه جزو اهداف جلسه‌ی «Beyond JavaScript» (اسلاید ۲) **نیست**؟

۱) ساختن برنامه‌های کنسولی (console application) با JavaScript

۲) یاد گرفتن اینکه JavaScript چطور روی سرور اجرا می‌شود

۳) طراحی ظاهر صفحه‌های وب با CSS

۴) حل مشکلات قدیمی نوع‌دهی (typing) در JavaScript

۱۴۸ آسان

طبق اسلاید ۳، کدام مورد از محدودیت‌های JavaScript در سال‌های اول **نبود**؟

۱) اجرای تک‌نخی (single-threaded) که در محاسبات سنگین یا عملیات I/O باعث مسدود شدن (blocking) می‌شد

۲) نوع‌دهی پویا (dynamic typing) که در پروژه‌های بزرگ خطاهای غیرمنتظره‌ی زمان اجرا می‌ساخت

۳) نبودِ یک سیستم ماژول (module system) درست برای سازمان‌دهی و استفاده‌ی دوباره از کد

۴) اینکه JavaScript در مرورگر اجرا نمی‌شد و فقط روی سرور کار می‌کرد

۱۴۹ آسان

طبق اسلایدهای ۴ و ۵، بهبود کدام موتور JavaScript آن را برای کارهای سمت سرور به‌اندازه‌ی کافی سریع کرد و Node.js هم بر پایه‌ی همان ساخته شد؟

۱) SpiderMonkey (موتور مرورگر Firefox)

۲) V8 ساخت Google

۳) JavaScriptCore (موتور مرورگر Safari)

۴) Chakra (موتور قدیمی مرورگر Edge)

۱۵۰ متوسط

یک رستوران فقط **یک گارسون** دارد. کدام رفتار گارسون، مدل **non-blocking I/O** در Node.js را درست نشان می‌دهد؟

۱) سفارش میز ۱ را به آشپزخانه می‌دهد، فوراً سراغ میزهای دیگر می‌رود و هر وقت آشپزخانه زنگ زد، غذای آماده را می‌برد.

۲) سفارش میز ۱ را به آشپزخانه می‌دهد و دم در آشپزخانه می‌ایستد تا غذا آماده شود؛ بعد سراغ میز ۲ می‌رود.

۳) برای هر مشتری تازه یک گارسون جدید استخدام می‌شود تا هیچ مشتری‌ای منتظر نماند.

۴) گارسون غذای همه‌ی میزها را خودش می‌پزد تا به آشپزخانه نیازی نباشد.

۱۵۱ متوسط

خروجی کد زیر در Node.js کدام است؟

```JavaScript
const fs = require("fs");

console.log("start");
fs.readFile(__filename, "utf8", () => {
  console.log("file read");
});
console.log("end");
```

۱) `start` `file read` `end`

۲) `start` `end` `file read`

۳) `file read` `start` `end`

۴) `start` `end`

۱۵۲ متوسط

در هر چهار گزینه، ماژول `fs` قبلاً با `const fs = require("fs");` بارگذاری شده است. کدام گزینه در Node.js **دقیقاً** خروجی زیر را چاپ می‌کند؟

```
start
end
done
```

۱
```JavaScript
console.log("start");
fs.readFile(__filename, () => console.log("done"));
console.log("end");
```

۲
```JavaScript
console.log("start");
fs.readFileSync(__filename);
console.log("done");
console.log("end");
```

۳
```JavaScript
fs.readFile(__filename, () => console.log("start"));
console.log("end");
console.log("done");
```

۴
```JavaScript
console.log("start");
fs.readFile(__filename, () => console.log("end"));
console.log("done");
```

۱۵۳ سخت

خروجی کد زیر در Node.js کدام است؟

```JavaScript
setTimeout(() => console.log("timer"), 0);
console.log("before loop");

const start = Date.now();
while (Date.now() - start < 100) {} // busy wait for about 100 ms
console.log("after loop");
```

۱ `before loop` `after loop` `timer`

۲ `timer` `before loop` `after loop`

۳ `before loop` `timer` `after loop`

۴ `before loop` `after loop`

۱۵۴ آسان

کدام تعریف درباره‌ی **Node.js** درست است؟

۱) یک زبان برنامه‌نویسی جدید که جایگزین JavaScript شده است

۲) یک فریم‌ورک (framework) برای ساختن رابط کاربری، مثل React

۳) یک محیط اجرا (runtime) برای JavaScript که روی موتور V8 ساخته شده و JS را بیرون از مرورگر اجرا می‌کند

۴) یک مرورگر وب سبک که Google برای سرورها ساخته است

۱۵۵ متوسط

خروجی کد زیر در **Node.js** کدام است؟

```JavaScript
console.log(typeof window);
console.log(typeof globalThis);
console.log(typeof process);
```

۱) `object` `object` `undefined`

۲) `Uncaught ReferenceError`

۳) `undefined` `undefined` `object`

۴) `undefined` `object` `object`

۱۵۶ سخت

اگر کد زیر را در **Node.js** اجرا کنیم، اولین خطا در کدام خط رخ می‌دهد؟

```JavaScript
1  console.log(typeof document);
2  const os = require("os");
3  console.log(typeof os.platform());
4  console.log(window.innerWidth);
5  console.log("finished");
```

۱) خط ۱ ۲) خط ۴

۳) خط ۵ ۴) هیچ خطایی رخ نمی‌دهد

۱۵۷ متوسط

طبق اسلاید ۴، ظهور **AJAX** و **JSON** چه چیزی را درباره‌ی JavaScript نشان داد؟

۱) اینکه JavaScript می‌تواند چند thread هم‌زمان داشته باشد.

۲) اینکه JavaScript بدون XML نمی‌تواند داده‌ای از سرور بگیرد.

۳) اینکه JavaScript می‌تواند عملیات ناهمگام (asynchronous) را به‌خوبی مدیریت کند.

۴) اینکه JavaScript فقط باید در مرورگر بماند و برای سرور مناسب نیست.

۱۵۸ متوسط

کدام گزینه منظور از **event-driven** (رویدادمحور) بودن Node.js را درست بیان می‌کند؟

۱) برنامه خط‌به‌خط اجرا می‌شود و برای هر عملیات I/O آن‌قدر صبر می‌کند تا تمام شود.

۲) برای هر درخواستی که به سرور می‌رسد، یک thread جدید ساخته می‌شود.

۳) برنامه فقط به کلیک‌ها و رویدادهای موس کاربر روی صفحه واکنش نشان می‌دهد.

۴) کد از قبل callbackهایی ثبت می‌کند و Node.js هر وقت رویدادی مثل «رسیدن درخواست» یا «تمام شدن خواندن فایل» رخ داد، آن‌ها را اجرا می‌کند.

۱۵۹ سخت

کدام عبارت درباره‌ی Node.js **نادرست** است؟

۱) چون Node.js روی سرور اجرا می‌شود، اشیای `document` و `window` در آن در دسترس‌اند تا صفحه را روی سرور بسازیم.

۲) Node.js از همان موتور V8 استفاده می‌کند که در مرورگر Chrome هست.

۳) با Node.js می‌توان برای frontend و backend از یک زبان استفاده کرد.

۴) وقتی یک حلقه‌ی محاسباتی سنگین در حال اجراست، callbackهای دیگر (مثل تایمرها) باید منتظر تمام شدن آن بمانند.

۱۶۰ متوسط

جای خالی را کامل کن:

«Node.js به لطف مدل I/O ..... و .....، فقط با یک thread به تعداد زیادی درخواست هم‌زمان پاسخ می‌دهد.»

۱) چندنخی (multi-threaded) و مسدودکننده (blocking)

۲) همگام (synchronous) و مسدودکننده (blocking)

۳) رویدادمحور (event-driven) و غیرمسدودکننده (non-blocking)

۴) همگام (synchronous) و بدون callback

۱۶۱ سخت

کدام جفتِ «محدودیت قدیمی JavaScript ← راه‌حلی که در این فصل می‌خوانیم» **نادرست** است؟

۱) نوع‌دهی پویا (dynamic typing) ← TypeScript

۲) اجرای تک‌نخی ← Node.js برای هر درخواست یک thread جدید می‌سازد تا هیچ کاری مسدود نشود

۳) نبودِ module system ← ماژول‌های CommonJS و ES Modules و پکیج‌های npm

۴) محدود بودن به مرورگر ← Node.js که JavaScript را روی سرور و در ترمینال اجرا می‌کند

۱۶۲ آسان

کدام مورد جزو ویژگی‌هایی که اسلاید ۵ («Then Came Node.js») برای Node.js می‌شمارد **نیست**؟

۱) اکوسیستم عظیم npm برای پکیج‌ها

۲) مدل I/O رویدادمحور و غیرمسدودکننده

۳) استفاده از یک زبان برای frontend و backend

۴) بررسی نوع‌ها (type checking) در زمان کامپایل

۱۶۳ سخت

خروجی کد زیر در Node.js کدام است؟ (در این برنامه هیچ درخواستی به سرور فرستاده نمی‌شود. پورت `0` یعنی «هر پورت آزادی»، و `server.close` سرور را می‌بندد تا برنامه تمام شود.)

```JavaScript
const http = require("http");

const server = http.createServer((req, res) => {
  console.log("request received");
  res.end("Hello");
});

server.listen(0, () => {
  console.log("listening");
  server.close(() => console.log("closed"));
});
console.log("setup done");
```

۱) `listening` `setup done` `closed`

۲) `setup done` `listening` `request received` `closed`

۳) `setup done` `listening` `closed`

۴) `request received` `listening` `setup done` `closed`

۳-۲ ویژگی‌های Node.js

منبع: NodeJS & TS، اسلایدهای ۶ و ۷

درسنامه

در درس قبل دیدیم Node.js چرا به وجود آمد. حالا وقت آن است که با آن کار کنیم و ویژگی‌هایش را یکی‌یکی بشناسیم. اسلاید ۶ فقط یک **تصویر** است (یک نمودار با هشت برچسب دور لوگوی Node.js) و اسلاید ۷ پنج ویژگی دیگر و یک میم دارد؛ هر دو را کامل اینجا آورده‌ایم.

اجرای Node.js در عمل

بعد از نصب Node.js، در ترمینال سه کار اصلی می‌توانی انجام دهی:

Terminal

```
node -v        # print the installed version, e.g. v24.22.2
node app.js    # run the file app.js
node           # open the REPL (interactive mode)
```

اجرای یک فایل

هر فایل `.js` را با `node <file>` اجرا می‌کنی؛ خروجی `console.log` در همان **ترمینال** چاپ می‌شود، نه در کنسول مرورگر. یک برنامه‌ی کنسولی کوچک که اسم کاربر را از **آرگومان‌های خط فرمان** (command-line arguments: کلمه‌هایی که بعد از نام فایل تایپ می‌کنی) می‌گیرد:

greet.js

```
const name = process.argv[2] || "stranger";
console.log("Hello, " + name + "!");
```

Terminal

```
node greet.js Ali   # Hello, Ali!
node greet.js       # Hello, stranger!
```

`process.argv` آرایه‌ای است که خانه‌ی `0` آن مسیر خودِ برنامه‌ی `node`، خانه‌ی `1` مسیر فایل و از خانه‌ی `2` به بعد آرگومان‌های ما هستند.

REPL

اگر فقط `node` را بزنی، وارد **REPL** (Read-Eval-Print Loop: حلقه‌ی «بخوان، ارزیابی کن، چاپ کن») می‌شوی؛ هر عبارتی تایپ کنی، فوراً اجرا و نتیجه‌اش نشان داده می‌شود. برای امتحان سریع یک تکه کد عالی است:

```
> 1 + 2
3
> "a" + 1
'a1'
> const x = 5
undefined
> x * 2
10
> .exit
```

REPL رشته‌ها را با کوتیشن نشان می‌دهد (`'a1'`) و برای دستوری که مقدار برنمی‌گرداند (مثل تعریف متغیر) `undefined` می‌نویسد. با `.exit` یا دو بار `Ctrl+C` بیرون می‌آیی.

هشت ویژگی Node.js (اسلاید ۶)

این اسلاید یک نمودار تصویری است: لوگوی Node.js در وسط و هشت برچسب رنگی دورش، چهار تا در هر طرف:

ستون چپ نمودار	ستون راست نمودار
Fast Data Streaming	Single-Threaded
Highly Scalable	Event-Driven
JavaScript	No Buffering
Cross-Platform Compatibility	Asynchronous

اسلاید هیچ توضیحی نمی‌دهد؛ معنی هر کدام را با هم ببینیم:

ویژگی	معنی ساده
Asynchronous (ناهمگام)	توابع Node.js منتظر تمام شدن کارهای کُند (فایل، شبکه) نمی‌مانند؛ کار را شروع می‌کنند و نتیجه بعداً با callback یا Promise می‌رسد.
Event-Driven (رویدادمحور)	برنامه حول رویدادها می‌چرخد: «درخواست رسید»، «داده آمد»، «کار تمام شد». برای هر رویداد یک تابع شنونده (listener) ثبت می‌کنیم.
Single-Threaded (تک‌نخی)	کد JavaScript ما روی **یک** thread اجرا می‌شود و event loop کارها را نوبت‌بندی می‌کند.
Highly Scalable (مقیاس‌پذیری بالا)	چون منتظر I/O نمی‌ماند، با منابع کم به تعداد خیلی زیادی اتصال هم‌زمان جواب می‌دهد.
No Buffering (بدون بافر کردن)	داده را **تکه‌تکه** (chunk) خروجی می‌دهد، نه اینکه اول همه‌اش را در حافظه جمع کند (**buffer**: حافظه‌ی موقتی که داده در آن جمع می‌شود).
Fast Data Streaming (جریان سریع داده)	موتور سریع V8 به‌اضافه‌ی پردازش داده به‌صورت **stream** (جریان: داده‌ای که ذره‌ذره می‌رسد و ذره‌ذره پردازش می‌شود).
JavaScript	همان زبان مرورگر؛ برنامه‌نویس frontend بدون یاد گرفتن زبان جدید، backend هم می‌نویسد.
Cross-Platform Compatibility (سازگاری چندسکویی)	همان کد روی Linux، Windows و macOS اجرا می‌شود. `process.platform` نام سیستم‌عامل را می‌دهد (`"win32"`، `"linux"`، `"darwin"`).

Asynchronous و Single-Threaded با هم چطور جور درمی‌آیند؟

شاید بپرسی «اگر فقط یک thread هست، چطور کارها هم‌زمان پیش می‌روند؟» جواب: کد **JavaScript** ما روی یک thread اجرا می‌شود، ولی کارهای I/O را سیستم‌عامل و کتابخانه‌ی داخلی Node.js (به نام libuv) **در پس‌زمینه** انجام می‌دهند. thread اصلی فقط کار را سفارش می‌دهد و وقتی نتیجه آماده شد، callback را اجرا می‌کند. برای همین Node.js با یک thread است؛ **Highly Scalable** ولی یادت باشد که یک محاسبه‌ی سنگین JavaScript همان thread را مسدود می‌کند.

Event-Driven در عمل: EventEmitter

قلب رویدادمحور بودن Node.js کلاسی به نام `EventEmitter` است. با `on` برای یک رویداد شنونده ثبت می‌کنی و با `emit` آن رویداد را «اعلام» می‌کنی:

JavaScript

```
const EventEmitter = require("events");
const shop = new EventEmitter();

shop.on("order", (item) => console.log(`Preparing ${item}`)); // register a listener
console.log("Waiting...");
shop.emit("order", "pizza"); // fire the event
console.log("Done");
// Output:
// Waiting...
// Preparing pizza
// Done
```

دقت کن `Preparing pizza` **قبل از** `Done` چاپ شد: `emit` شنونده‌ها را **همان لحظه و همگام** صدا می‌زند و تا کارشان تمام نشود، خط بعد اجرا نمی‌شود. اگر برای رویدادی هیچ شنونده‌ای ثبت نشده باشد، `emit` هیچ کاری نمی‌کند (فقط `false` برمی‌گرداند) و خطا هم نمی‌دهد.

No Buffering و Fast Data Streaming در عمل: stream

فرض کن می‌خواهی یک فیلم ۲ گیگابایتی را برای کاربر بفرستی. راه بد: کل فایل را در حافظه بخوانی و بعد بفرستی. راه Node.js: فایل را **تکه‌تکه** بخوانی و هر تکه را فوراً بفرستی؛ درست مثل تماشای آنلاین فیلم که لازم نیست صبر کنی کل فیلم دانلود شود. در مثال زیر، `highWaterMark: 4` اندازه‌ی هر تکه را ۴ بایت (اینجا ۴ حرف) گذاشته است:

JavaScript

```
const fs = require("fs");
fs.writeFileSync("data.txt", "abcdefghij");

const stream = fs.createReadStream("data.txt", { encoding: "utf8", highWaterMark: 4 });
stream.on("data", (chunk) => console.log("chunk:", chunk)); // fired for every chunk
stream.on("end", () => console.log("done"));                // fired once at the end
console.log("reading...");
// Output:
// reading...
// chunk: abcd
// chunk: efgh
// chunk: ij
// done
```

اول `reading...` آمد، چون خواندن stream ناهمگام است. بعد رویداد `data` سه بار (برای هر تکه) و در آخر رویداد `end` یک بار رخ داد. یعنی stream هم رویدادمحور است.

ادامه‌ی ویژگی‌ها (اسلاید ۷)

۱. Built on async/await for cleaner, modern concurrency. API های جدید Node.js به جای callback، **Promise** برمی‌گردانند، پس می‌شود با `async/await` کد ناهمگام را مثل کد معمولی و خطبه‌خط نوشت. (**concurrency** یعنی هم‌روندی: پیش بردن چند کار در یک بازه‌ی زمانی.)

```JavaScript
const fs = require("fs/promises"); // the Promise-based version of fs

async function main() {
  await fs.writeFile("note.txt", "Hello file");
  const text = await fs.readFile("note.txt", "utf8");
  console.log(text);
}

main();
console.log("main started");
// Output:
// main started
// Hello file
```

`main()` تا اولین `await` جلو می‌رود و بعد کنترل را پس می‌دهد؛ برای همین `main started` زودتر چاپ می‌شود.

۲. Node.js **Uses ES modules and import/export syntax natively.** سیستم ماژول استاندارد JavaScript یعنی **ES Modules** (ESM) را مستقیم و بدون ابزار اضافه پشتیبانی می‌کند (پایین‌تر مفصل می‌بینیم).

۳. Node.js **Embraces TypeScript, ESM, and modern tooling by default.** با TypeScript، ESM و ابزارهای مدرن به‌خوبی کنار می‌آید. مثلاً Node.js نسخه‌ی ۲۴ می‌تواند یک فایل `.ts` را مستقیم اجرا کند: فقط **نوع‌ها را پاک می‌کند** (type stripping) و اجرا می‌کند، ولی آن‌ها را **بررسی نمی‌کند**. پس برای پیدا کردن خطاهای نوع هنوز به کامپایلر TypeScript (`tsc`) نیاز داریم.

```Terminal
node app.ts   # Node.js 24 removes the types and runs the code (no type checking!)
```

۴. framework **Supports powerful frameworks like Express, NestJS, and Fastify.** (چارچوب: اسکلت آماده‌ای که ساختار برنامه را تعیین می‌کند و ما فقط بخش‌های خودمان را در آن می‌نویسیم) کار ساختن سرور را خیلی ساده می‌کند:

- **Express**: کمینه‌گرا (minimalist) و پرطرفدارترین فریم‌ورک وب Node.js (در درس «پکیج‌های محبوب» دوباره می‌بینیمش).
- **NestJS**: فریم‌ورکی ساختاریافته برای پروژه‌های بزرگ که با TypeScript نوشته می‌شود.
- **Fastify**: فریم‌ورکی با تمرکز بر سرعت بالا و سربار کم.

۵. Integrates seamlessly with containers, cloud, and edge runtimes. برنامه‌ی Node.js را راحت می‌شود داخل **container** (بسته‌ای که برنامه را با همه‌ی نیازهایش یکجا نگه می‌دارد تا همه‌جا یکسان اجرا شود؛ مثل Docker) گذاشت، روی سرویس‌های **cloud** (ابری) اجرا کرد یا در **edge runtime**ها (محیط‌هایی که کد را روی سرورهای نزدیک به محل کاربر اجرا می‌کنند) استفاده کرد.

میم اسلاید ۷

کنار این اسلاید یک میم با عنوان «**JavaScript programming**» هست: مردی که روی خودش نوشته «**app 300kb**» یک کوله‌پشتی نظامی غول‌پیکر به دوش دارد که رویش نوشته «**Node_modules, 12GB**». شوخی‌اش این است که کد خود برنامه کوچک است، ولی پوشه‌ی `node_modules` (جایی که npm همه‌ی پکیج‌های نصب‌شده را می‌گذارد) به‌خاطر **وابستگی‌ها** (dependency: پکیج‌هایی که برنامه به آن‌ها نیاز دارد) و وابستگی‌های آن وابستگی‌ها، **خیلی** بزرگ می‌شود. اسم این مشکل dependency bloat (پف کردن وابستگی‌ها) است؛ در درس بعد، «dependency overload» در اسلاید ۸ و «size» در اسلاید ۹ به همین اشاره دارند.

دو سیستم ماژول در Node.js: CommonJS و ES Modules

Node.js دو روش برای تقسیم کد به فایل‌های جدا (ماژول) دارد.

CommonJS (روش قدیمی و پیش‌فرض Node.js)

با `module.exports` چیزی را **صادر** (export) می‌کنی و با `require` **وارد** (import) می‌کنی:

math.js

```
console.log("math.js loaded");
function add(a, b) {
  return a + b;
}
module.exports = { add, PI: 3.14 };
```

app.js

```
const math = require("./math");       // the ".js" extension is optional here
const { add } = require("./math.js"); // destructuring works too
console.log(math.add(2, 3), add(1, 1), math.PI);
// Output:
// math.js loaded
// 5 2 3.14
```

`math.js loaded` فقط **یک بار** چاپ شد، با اینکه دو بار `require` کردیم: Node.js هر ماژول را فقط بار اول اجرا می‌کند و نتیجه را **cache** (ذخیره) می‌کند؛ `require` های بعدی همان شیء ذخیره‌شده را برمی‌گردانند.

ES Modules (روش استاندارد و جدید)

با `export` و `import` کار می‌کنی؛ همان سیستمی که مرورگرها و TypeScript هم استفاده می‌کنند:

math.mjs

```
export function add(a, b) {
  return a + b;
}
export const PI = 3.14;
export default function greet(name) {
  return "Hello " + name;
}
```

app.mjs

```
import greet, { add, PI } from "./math.mjs"; // the extension is required in ESM
console.log(add(2, 3), PI, greet("Ali"));    // 5 3.14 Hello Ali
```

Node.js از کجا می‌فهمد فایل CommonJS است یا ESM؟

فایل	Node.js آن را چطور اجرا می‌کند
`name.mjs`	همیشه ES Module
`name.cjs`	همیشه CommonJS
`name.js` و در `package.json` نوشته شده `"type": "module"`	ES Module
`name.js` و در `package.json` نوشته شده `"type": "commonjs"`	CommonJS (دستور `npm init` در نسخه‌های جدید همین را می‌نویسد)
`name.js` بدون فیلد `type`	CommonJS؛ ولی Node.js جدید اگر در فایل `import` / `export` ببیند، آن را ESM اجرا می‌کند (و اگر `package.json` وجود داشته باشد، یک هشدار هم می‌دهد)

	CommonJS	ES Modules
وارد کردن	`require("./math")`	`import { add } from "./math.mjs"`
صادر کردن	`module.exports = …` یا `exports.x = …`	`export` و `export default`
پسوند در مسیرهای نسبی	اختیاری	اجباری
`require` و `__dirname` و `__filename`	تعریف شده‌اند	تعریف **نشده‌اند**
`this` در بالاترین سطح فایل	`{}` (همان `module.exports`)	`undefined`
`await` در بالاترین سطح فایل (top-level await)	مجاز نیست	مجاز است
استاندارد بودن	مخصوص Node.js	استاندارد رسمی JavaScript (در مرورگر هم کار می‌کند)

⚠ دام تستی

- در CommonJS، `exports` فقط یک **میان‌بُر** به `module.exports` است. `exports.hello = "hi"` درست کار می‌کند، ولی `exports = { hello: "hi" }` فقط متغیر محلی `exports` را عوض می‌کند و چیزی صادر **نمی‌شود** (`require` همان شیء خالی `{}` را برمی‌گرداند). برای صادر کردن یک شیء کامل از `module.exports = {...}` استفاده کن.
- اگر در `package.json` نوشته باشد `"type": "module"`، استفاده از `require` در یک فایل `.js` خطای `ReferenceError: require is not defined in ES module scope` می‌دهد.
- برعکس، اگر `"type": "commonjs"` باشد (پیش‌فرض `npm init` جدید)، نوشتن `import` در فایل `.js` خطای `SyntaxError: Cannot use import statement outside a module` می‌دهد.

Node.js در برابر مرورگر

امکان	مرورگر	Node.js
`window`، `document`، `alert`، DOM	دارد	**ندارد**
`process`، `require`، `fs`، `http`	ندارد	دارد
`globalThis`	دارد (همان `window`)	دارد (همان `global`)
`console.log`	در کنسول DevTools	در ترمینال
`setTimeout`، `Promise`، `JSON`، `Math`، `fetch`	دارد	دارد
`import` / `export`	دارد (`<script type="module">`)	دارد (ESM)

JavaScript

```
// A CommonJS file running in Node.js
console.log(typeof window);         // "undefined"
console.log(globalThis === global); // true
console.log(typeof setTimeout);     // "function"
console.log(this);                  // {}
```

نکته

- **JavaScript** همان زبان است؛ چیزی که بین مرورگر و Node.js فرق می‌کند **ابزارهای محیط** است. قبل از حل هر تست، اول ببین کد در کدام محیط اجرا می‌شود.
- `typeof` روی چیزی که وجود ندارد خطا نمی‌دهد و `"undefined"` برمی‌گرداند؛ این ترفند معروف برای فهمیدن محیط است: `typeof window === "undefined"` یعنی احتمالاً داخل Node.js هستیم.

دام تستی

`emit` در `EventEmitter` **همگام** است: شنونده‌ها همان لحظه اجرا می‌شوند، نه «بعداً» مثل `setTimeout`. پس در تست‌های ترتیب خروجی، خروجی شنونده را درست در جای خط `emit` بنویس.

جمع‌بندی

- اجرای فایل با `node app.js`، REPL با `node` خالی، آرگومان‌ها در `process.argv`.
- هشت ویژگی اسلاید ۶: Asynchronous، Event-Driven، Single-Threaded، Highly Scalable، No Buffering، Fast، Data Streaming، JavaScript، Cross-Platform Compatibility.
- اسلاید ۷: TypeScript، ES Modules، async/await و ابزارهای مدرن، فریم‌ورک‌های Express و NestJS و Fastify، container و cloud و edge.
- میم اسلاید ۷: برنامه‌ی ۳۰۰ کیلوبایتی و `node_modules` دوازده‌گیگابایتی، یعنی پف کردن وابستگی‌ها.
- CommonJS (`require`، `module.exports`) در برابر ESM (`import`، `export`، `.mjs` یا `"type": "module"`).

تست‌های تمرینی

۲۰ تست · شماره‌ی ۱۶۴ تا ۱۸۳

۱۶۴ آسان

در نمودار تصویری اسلاید ۶ («Node.js Features») هشت ویژگی دور لوگوی Node.js نوشته شده است. کدام گزینه جزو این هشت ویژگی **نیست**؟

۱) No Buffering
۲) Multi-Threaded
۳) Cross-Platform Compatibility
۴) Highly Scalable

۱۶۵ آسان

منظور از ویژگی **No Buffering** در Node.js چیست؟

۱) Node.js اصلاً از حافظه‌ی RAM استفاده نمی‌کند.
۲) Node.js نمی‌تواند فایل‌های بزرگ را بخواند.
۳) داده‌ای که خوانده می‌شود هرگز در هیچ متغیری نگه داشته نمی‌شود.
۴) برنامه‌ها داده را تکه‌تکه (chunk) و به‌محض آماده شدن خروجی می‌دهند، نه اینکه اول همه‌ی داده را در حافظه جمع کنند.

۱۶۶ متوسط

در کد زیر `highWaterMark: 4` یعنی اندازه‌ی هر تکه (chunk) حداکثر ۴ بایت است (اینجا ۴ حرف). خروجی کد در Node.js کدام است؟

```JavaScript
const fs = require("fs");
fs.writeFileSync("data.txt", "abcdefghij");

const stream = fs.createReadStream("data.txt", { encoding: "utf8", highWaterMark: 4 });
stream.on("data", (chunk) => console.log("chunk:", chunk));
stream.on("end", () => console.log("done"));
console.log("reading...");
```

۱) `reading...` `chunk: abcd` `chunk: efgh` `chunk: ij` `done`
۲) `reading...` `chunk: abcdefghij` `done`
۳) `chunk: abcd` `chunk: efgh` `chunk: ij` `done` `reading...`
۴) `chunk: abcdefghij` `reading...` `done`

۱۶۷ سخت

خروجی کد زیر در Node.js کدام است؟

```JavaScript
const EventEmitter = require("events");
const shop = new EventEmitter();

shop.on("order", (item) => console.log(`Preparing ${item}`));
console.log("Waiting...");
shop.emit("order", "pizza");
shop.emit("cancel", "pizza");
console.log("Done");
```

۱) `Waiting...` `Done` `Preparing pizza`

۲) `Preparing pizza` `Waiting...` `Done`

۳) `Waiting...` `Preparing pizza` `Done`

۴) `Waiting...` `Preparing pizza` `Uncaught Error`

۱۶۸ آسان

ویژگی **Cross-Platform Compatibility** در نمودار اسلاید ۶ یعنی چه؟

۱) همان کد Node.js را می‌توان روی سیستم‌عامل‌های مختلف مثل Linux، Windows و macOS اجرا کرد.

۲) هر برنامه‌ی Node.js هم‌زمان در مرورگر کاربر و روی سرور اجرا می‌شود.

۳) برنامه‌ی Node.js را باید برای هر سیستم‌عامل از نو نوشت.

۴) Node.js کد زبان‌های دیگر مثل Python و Java را هم اجرا می‌کند.

۱۶۹ متوسط

خروجی کد زیر در Node.js کدام است؟

```JavaScript
const fs = require("fs/promises");

async function main() {
  await fs.writeFile("note.txt", "Hello file");
  const text = await fs.readFile("note.txt", "utf8");
  console.log(text);
}

main();
console.log("main started");
```

۱) `Hello file` `main started`

۲) `main started`

۳) `Hello file`

۴) `main started` `Hello file`

۱۷۰ آسان

فایلی به نام `app.js` داریم. با کدام دستور در ترمینال آن را با Node.js اجرا می‌کنیم؟

۱) `npm app.js`
۲) `run app.js`
۳) `node app.js`
۴) `js app.js`

۱۷۱ متوسط

اگر در ترمینال فقط دستور `node` را (بدون نام فایل) بزنیم چه اتفاقی می‌افتد؟

۱) نسخه‌ی نصب‌شده‌ی Node.js چاپ می‌شود و برنامه تمام می‌شود.
۲) محیط تعاملی REPL باز می‌شود که هر عبارت تایپ‌شده را می‌خواند، ارزیابی می‌کند و نتیجه را چاپ می‌کند.
۳) یک فایل `package.json` جدید در پوشه‌ی فعلی ساخته می‌شود.
۴) خطا می‌دهد، چون Node.js همیشه به نام یک فایل نیاز دارد.

۱۷۲ سخت

با اجرای `node app.js`، خروجی کدام است؟

math.js

```
console.log("math.js loaded");
function add(a, b) {
  return a + b;
}
module.exports = { add, PI: 3.14 };
```

app.js

```
const math = require("./math");
const { add } = require("./math.js");
console.log(math.add(2, 3), add(1, 1), math.PI);
```

۱) `math.js loaded` `math.js loaded` `5 2 3.14`
۲) `5 2 3.14`
۳) `math.js loaded` `5 2 3.14`
۴) `Uncaught Error`

۱۷۳ سخت

با اجرای `node app.js`، خروجی کدام است؟

bad.js

```
exports = { hello: "hi" };
```

good.js

```
exports.hello = "hi";
```

app.js

```
const bad = require("./bad");
const good = require("./good");
console.log(bad);
console.log(good);
```

۱) `{}` `{ hello: 'hi' }`

۲) `{ hello: 'hi' }` `{ hello: 'hi' }`

۳) `{ hello: 'hi' }` `{}`

۴) `undefined` `{ hello: 'hi' }`

۱۷۴ متوسط

با اجرای `node app.js` در پوشه‌ای که این دو فایل در آن هستند، چه اتفاقی می‌افتد؟

package.json

```
{
  "name": "demo",
  "type": "module"
}
```

app.js

```
const fs = require("fs");
console.log("file system ready");
```

۱) `file system ready`

۲) `Uncaught SyntaxError`

۳) `Uncaught TypeError`

۴) `Uncaught ReferenceError`

۱۷۵ سخت

پروژه را با `npm init -y` ساخته‌ایم (نسخه‌های جدید npm فیلد `"type": "commonjs"` را هم در `package.json` می‌نویسند). با اجرای `node index.js` چه اتفاقی می‌افتد؟

package.json

```
{
  "name": "demo",
  "version": "1.0.0",
  "type": "commonjs"
}
```

index.js

```
import fs from "fs";
console.log("file system ready");
```

۱) `file system ready`

۲) `Uncaught SyntaxError`

۳) `Uncaught ReferenceError`

۴) `Uncaught TypeError`

۱۷۶ متوسط

با کدام کار، Node.js یک فایل را به‌عنوان **ES Module** (با پشتیبانی از `import` و `export`) اجرا می‌کند؟

۱) گذاشتن پسوند `.cjs` برای فایل

۲) نوشتن `"type": "commonjs"` در `package.json`

۳) گذاشتن پسوند `.mjs` برای فایل، یا نوشتن `"type": "module"` در `package.json` برای فایل‌های `.js`

۴) نوشتن `"main": "index.js"` در `package.json`

۱۷۷ متوسط

فایل زیر با پسوند `.mjs` (یعنی به‌صورت ES Module) در Node.js اجرا می‌شود. خروجی کدام است؟

JavaScript

```
console.log(typeof require);
console.log(this);
console.log(typeof setTimeout);
```

۱) `undefined` `undefined` `function`

۲) `function` `{}` `function`

۳) `undefined` `{}` `undefined`

۴) `Uncaught ReferenceError`

۱۷۸ متوسط

در میم اسلاید ۷ با عنوان «JavaScript programming»، روی مرد نوشته شده «app 300kb» و روی کوله‌پشتی غول‌پیکرش «Node_modules, 12GB». این میم به کدام واقعیت اشاره دارد؟

۱) برنامه‌های Node.js همیشه دقیقاً ۱۲ گیگابایت حجم دارند.

۲) پوشه‌ی `node_modules` کد اصلی برنامه‌ی خود ما را نگه می‌دارد.

۳) Node.js اجازه نمی‌دهد حجم کد برنامه از ۳۰۰ کیلوبایت بیشتر شود.

۴) کد خود برنامه کوچک است، ولی وابستگی‌ها و وابستگی‌های آن‌ها در `node_modules` حجم بسیار بزرگی پیدا می‌کنند.

۱۷۹ متوسط

اسلاید ۷ می‌گوید Node.js از فریم‌ورک‌های قدرتمندی پشتیبانی می‌کند. کدام گزینه جزو فریم‌ورک‌های سمت سروری که این اسلاید نام می‌برد **نیست**؟

۱) Express ۲) React ۳) NestJS ۴) Fastify

۱۸۰ سخت

کدام عبارت درباره‌ی تفاوت Node.js و مرورگر **نادرست** است؟

۱) `globalThis` هم در مرورگر و هم در Node.js وجود دارد.

۲) در Node.js با ماژول `fs` می‌توان هر فایلی را با داشتن مسیرش از دیسک خواند؛ کاری که کد یک صفحه‌ی وب اجازه‌اش را ندارد.

۳) `setTimeout` و `Promise` مخصوص مرورگرند و در Node.js وجود ندارند.

۴) در Node.js عبارت `typeof window` برابر `"undefined"` است.

۱۸۱ سخت

اسلاید ۷ می‌گوید «Node.js Embraces TypeScript … by default». در **Node.js نسخه‌ی ۲۴** فایل `app.ts` زیر را مستقیم با دستور `node app.ts` اجرا می‌کنیم. چه اتفاقی می‌افتد؟

app.ts

```
let count: number = "ten";
console.log(count);
```

۱) برنامه اجرا نمی‌شود و خطای نوع (type error) می‌گیریم، چون `"ten"` از نوع `number` نیست.

۲) خطای `SyntaxError` می‌گیریم، چون Node.js نوشتن نوع (`: number`) را نمی‌شناسد.

۳) `NaN` چاپ می‌شود، چون `"ten"` به عدد تبدیل می‌شود.

۴) `ten` چاپ می‌شود؛ Node.js فقط نوع‌ها را پاک می‌کند و آن‌ها را بررسی نمی‌کند.

۱۸۲ آسان

در سیستم ماژول **CommonJS** (سیستم قدیمی و پیش‌فرض Node.js)، برای وارد کردن یک ماژول از چه چیزی و برای صادر کردن از چه چیزی استفاده می‌کنیم؟

۱) `require` برای وارد کردن و `module.exports` برای صادر کردن

۲) `import` برای وارد کردن و `export default` برای صادر کردن

۳) `include` برای وارد کردن و `module.import` برای صادر کردن

۴) `fetch` برای وارد کردن و `window.exports` برای صادر کردن

۱۸۳ متوسط

Node.js با اینکه **تک‌نخی** (single-threaded) است، چرا **Highly Scalable** شمرده می‌شود؟

۱) چون برای هر درخواست یک thread جدید می‌سازد و کارها را بین آن‌ها پخش می‌کند.

۲) چون I/O غیرمسدودکننده و رویدادمحور است و آن یک thread هیچ‌وقت بیکار منتظر فایل یا شبکه نمی‌ماند؛ پس به اتصال‌های هم‌زمان زیادی می‌رسد.

۳) چون کد سرور را در مرورگر هر کاربر اجرا می‌کند تا بار سرور کم شود.

۴) چون همه‌ی داده‌ها را اول کامل در حافظه buffer می‌کند و بعد یکجا می‌فرستد.

۳-۳ مشکلات تایپ در JavaScript و معرفی TypeScript

منبع: NodeJS & TS، اسلایدهای ۱۵ تا ۱۷

درسنامه

تا اینجای کتاب با خودِ JavaScript کار کردیم. در این درس می‌بینیم JavaScript در موضوع **type** (نوع داده؛ مثل عدد، رشته یا boolean) چه ضعف‌هایی دارد و چرا Microsoft زبانی به نام **TypeScript** ساخت. اسلاید ۱۵ فقط صفحه‌ی عنوانِ بخش TypeScript است و درس اصلی از اسلاید ۱۶ شروع می‌شود. این همان هدف سومی است که اسلاید ۲ برای این جلسه گفته بود: حل مشکلات قدیمی تایپ در JavaScript.

> **یک تشبیه ساده**
>
> فرض کن در یک انبار، روی هیچ جعبه‌ای برچسب نیست. هر چیزی را می‌شود در هر جعبه‌ای گذاشت و تا جعبه را باز نکنی (یعنی تا برنامه اجرا نشود) نمی‌فهمی داخلش چیست. JavaScript شبیه همین انبار است. TypeScript روی جعبه‌ها **برچسب** می‌زند و قبل از ارسال بار، یک بازرس همه‌ی برچسب‌ها را چک می‌کند.

۱. دو مفهوم پایه: زمان اجرا و زمان کامپایل

- **runtime** (زمان اجرا): لحظه‌ای که برنامه واقعاً اجرا می‌شود؛ مثلاً وقتی مرورگر یا Node.js کد را خط به خط اجرا می‌کند.
- **compile time** (زمان کامپایل): مرحله‌ای **قبل از اجرا** که یک ابزار به نام **compiler** (کامپایلر؛ برنامه‌ای که کد را می‌خواند، بررسی می‌کند و به شکل دیگری تبدیل می‌کند) روی کد کار می‌کند. خطایی که در این مرحله پیدا شود، هیچ‌وقت به دست کاربر نمی‌رسد.

بر همین اساس زبان‌ها از نظر نوع‌دهی دو دسته‌اند:

	dynamic typing (نوع‌دهی پویا)	static typing (نوع‌دهی ایستا)
نوع به چه چیزی تعلق دارد؟	به **مقدار**؛ یک متغیر هر بار می‌تواند مقداری با نوع دیگر بگیرد	به **متغیر**؛ نوع متغیر از قبل معلوم و ثابت است
خطای نوع کی پیدا می‌شود؟	در زمان اجرا (runtime)	در زمان کامپایل، قبل از اجرا
نمونه	JavaScript	C، Java، #TypeScript

۲. مشکلات تایپ در JavaScript (اسلاید ۱۶)

اسلاید ۱۶ پنج مشکل را نام می‌برد. بیا هر کدام را با مثال ببینیم.

مشکل ۱: نوع‌دهی پویا (dynamically typed)

در JavaScript نوعِ یک متغیر ثابت نیست و **در زمان اجرا می‌تواند عوض شود:**

```JavaScript
let x = 5;              // x holds a number
console.log(typeof x); // number
x = "hello";            // now a string: no complaint
x = true;               // now a boolean
console.log(typeof x); // boolean
```

این آزادی در برنامه‌ی کوچک راحت است، اما در برنامه‌ی بزرگ نمی‌دانی یک متغیر در لحظه‌ی استفاده چه نوعی دارد.

مشکل ۲: تبدیل نوع خودکار (type coercion)

وقتی دو مقدار با نوع متفاوت با هم ترکیب می‌شوند، JavaScript بی‌صدا یکی را به نوع دیگر تبدیل می‌کند (جزئیات کامل این قاعده‌ها را در فصل اول دیدیم). مثال خود اسلاید: `"5" + 2 → "52"`؛ علامت `→` یعنی «حاصلش می‌شود».

```JavaScript
console.log("5" + 2); // 52 (a string: "5" + "2")
console.log("5" - 2); // 3  (a number: "5" becomes 5)

function add(a, b) {
  return a + b;
}
console.log(add(2, 3));   // 5
console.log(add("2", 3)); // 23  <- a hidden bug, but no error!
```

مثلاً مقداری که از یک فیلد فرم HTML خوانده می‌شود همیشه رشته است؛ اگر آن را به این تابع بدهی، به‌جای جمع، دو مقدار به هم چسبانده می‌شوند و هیچ خطایی هم نمی‌بینی.

مشکل ۳: باگ‌های پنهان نوع در کدهای بزرگ

در پروژه‌ای با صدها فایل و چند برنامه‌نویس، هیچ‌چیز تضمین نمی‌کند که یک تابع با داده‌ی درست صدا زده شود. حتی یک غلط املایی ساده خطا نمی‌دهد؛ فقط `undefined` برمی‌گرداند و باگ جای دیگری از برنامه خودش را نشان می‌دهد. به همین دلیل نگه‌داری (maintain) کدهای بزرگ JavaScript سخت است:

```JavaScript
const user = { name: "Ali" };
console.log(user.nmae); // undefined (a typo, but no error)
```

مشکل ۴: نبودن بررسی‌های زمان کامپایل (compile-time checks)

JavaScript قبل از اجرا کد را از نظر نوع بررسی نمی‌کند؛ پس بسیاری از خطاها **فقط وقتی همان خط اجرا شود** پیدا می‌شوند:

```JavaScript
function shout(msg) {
  return msg.toUppercase(); // typo: the real method is toUpperCase
}
console.log("app started");
console.log(shout("hi"));   // TypeError: msg.toUppercase is not a function
```

خط `app started` بی‌مشکل چاپ می‌شود و برنامه تازه در خط آخر از کار می‌افتد. اگر `shout` فقط در یک حالت نادر صدا زده شود، این باگ ممکن است ماه‌ها پنهان بماند و یک روز برای کاربر واقعی رخ دهد.

مشکل ۵: رفتار ناسازگار در موتورها و نسخه‌های مختلف

کد JavaScript روی موتورهای مختلف (مثلاً V8 در Chrome و Node.js، SpiderMonkey در Firefox و JavaScriptCore در Safari) و نسخه‌های مختلفِ آن‌ها اجرا می‌شود. قابلیتی که در نسخه‌ی جدید هست ممکن است در مرورگر قدیمی نباشد و گاهی حتی رفتار یک تابع فرق کرده است؛ مثلاً در مرورگرهای خیلی قدیمی `parseInt("08")` به‌جای ۸ عدد ۰ را برمی‌گرداند. این تفاوت‌ها باگ‌های ظریفی می‌سازند که فقط روی بعضی سیستم‌ها دیده می‌شوند.

۳. معرفی TypeScript (اسلاید ۱۷)

- **سازنده و سال:** شرکت Microsoft در سال ۲۰۱۲ آن را معرفی کرد.
- **طراح:** طراحی آن را **Anders Hejlsberg** رهبری کرد؛ کسی که معمار اصلی (lead architect) زبان‌های **C#** و **Delphi** هم هست.
- **هدف:** اضافه کردن **static typing** (نوع‌دهی ایستا) و **tooling** (ابزارهای کمکی برنامه‌نویسی) به JavaScript برای پروژه‌های بزرگ.
- **superset تدریجی و اختیاری** (gradual, optional superset): TypeScript یک «ابرمجموعه»ی JavaScript است و می‌شود آن را کم‌کم وارد یک پروژه کرد.
- **محبوبیت:** خیلی زود در نرم‌افزارهای سازمانی (enterprise) و وب‌اپلیکیشن‌های پیچیده محبوب شد؛ برای نمونه ویرایشگر VS Code و فریم‌ورک Angular با TypeScript نوشته شده‌اند.
- **مسیر اجرا:** TypeScript می‌نویسی ← به JavaScript تبدیل می‌شود ← هر جایی که JavaScript اجرا شود (مرورگر، Node.js و ...) اجرا می‌شود.

superset یعنی چه؟

یعنی هر کد JavaScript از نظر **syntax** (نحو؛ قواعد نوشتن کد) یک کد TypeScript هم هست. TypeScript چیزی از JavaScript کم نمی‌کند، فقط **نوع‌ها** را به آن اضافه می‌کند: TypeScript = JavaScript + types. نوع با یک **type annotation** (برچسب نوع؛ دونقطه و نام نوع بعد از اسم متغیر) نوشته می‌شود:

```TypeScript
let age: number = 25; // ": number" is a type annotation
age = 26;             // OK
age = "twenty";       // Error TS2322: Type 'string' is not assignable to type 'number'.
```

اما «نحوِ معتبر داشتن» با «بی‌خطا بودن» فرق دارد. کد زیر هیچ نوعی ندارد و یک JavaScript کاملاً درست است که در Node.js بدون خطا اجرا می‌شود؛ با این حال tsc برایش خطا گزارش می‌کند، چون از روی مقدار اولیه فهمیده `count` عدد است:

```TypeScript
let count = 0;     // plain JavaScript: no types written
count = count + 1; // OK
count = "zero";    // Error TS2322: Type 'string' is not assignable to type 'number'.
```

پس **هر کد JavaScript نحوِ معتبر TypeScript را دارد، ولی ممکن است tsc در آن خطای نوع پیدا کند.**

تدریجی و اختیاری یعنی چه؟

- **اختیاری (optional):** نوشتن نوع اجباری نیست؛ هر جا ننویسی، TypeScript نوع را از روی مقدار تشخیص می‌دهد (به این کار type inference می‌گویند و در درس بعد کامل می‌آید).
- **تدریجی (gradual):** لازم نیست کل پروژه را یک‌جا بازنویسی کنی؛ می‌توانی فایل‌ها را یکی‌یکی از `.js` به `.ts` تغییر نام بدهی و کم‌کم نوع اضافه کنی.

tooling یعنی چه؟

چون ویرایشگر (مثلاً VS Code) نوع هر متغیر را می‌داند، می‌تواند:

- هنگام تایپ، متدها و ویژگی‌های درست را پیشنهاد بدهد (autocomplete)؛
- خطاها را همان لحظه زیر کد خط قرمز بکشد، قبل از اینکه برنامه را اجرا کنی؛
- نام یک متغیر یا تابع را در کل پروژه با خیال راحت عوض کند (rename / refactor) و با یک کلیک به محل تعریف هر چیز برود (go to definition).

۴. کامپایلر tsc: از TypeScript تا JavaScript

مرورگرها TypeScript را نمی‌فهمند. ابزاری به نام (TypeScript Compiler) **tsc** فایل `.ts` را بررسی می‌کند و به فایل `.js` تبدیل می‌کند:

app.ts

```ts
let age: number = 25;
function greet(name: string): string {
  return "Hello, " + name;
}
console.log(greet("Ali"), age);
```

Terminal

```
npm install -D typescript   # install the compiler in the project
npx tsc app.ts              # check app.ts and create app.js
node app.js                 # prints: Hello, Ali 25
```

app.js

```js
"use strict";
let age = 25;
function greet(name) {
    return "Hello, " + name;
}
console.log(greet("Ali"), age);
```

در خروجی **هیچ اثری از نوع‌ها نیست**؛ به این کار **type erasure** (پاک شدن نوع‌ها) می‌گویند. نوع‌ها فقط برای کامپایلر و ویرایشگرند: هیچ بررسی‌ای در زمان اجرا به کد اضافه نمی‌شود و برنامه نه سریع‌تر می‌شود و نه کندتر. (خط `"use strict";` را خود tsc در حالت strict اضافه می‌کند.)

خواندن پیام خطای tsc

اگر در فایل خطای نوع باشد، tsc چیزی شبیه این چاپ می‌کند:

```
app.ts(2,1): error TS2322: Type 'string' is not assignable to type 'number'.
```

یعنی در فایل `app.ts`، خط ۲، ستون ۱، خطایی با کد `TS2322` هست: «نوع string را نمی‌شود به نوع number نسبت داد». لازم نیست کد خطاها را حفظ کنی؛ در درس‌های بعد فقط برای آشنایی کنار مثال‌ها می‌آیند.

فایل tsconfig.json و حالت strict

در یک پروژه‌ی واقعی، تنظیمات کامپایلر در فایلی به نام `tsconfig.json` نوشته می‌شود. دستور `npx tsc --init` این فایل را با تنظیمات پیشنهادی می‌سازد که یکی از آن‌ها `"strict": true` است. بعد از آن کافی است فقط `npx tsc` (بدون نام فایل) را اجرا کنی تا همه‌ی فایل‌های پروژه طبق همین تنظیمات کامپایل شوند. بخشی از این فایل:

tsconfig.json

```ts
{
  "compilerOptions": {
    "target": "esnext",
    "strict": true
  }
}
```

حالت **strict** (سخت‌گیرانه) چند بررسی مهم را با هم روشن می‌کند؛ مثلاً اجازه نمی‌دهد پارامتر یک تابع بدون نوع بماند یا `null` در متغیری از نوع `number` قرار بگیرد. **در این کتاب همه‌جا فرض می‌کنیم strict روشن است** (همان پیش‌فرض `tsc --init`) و همه‌ی مثال‌ها و تست‌ها با همین فرض بررسی شده‌اند.

۵. TypeScript چطور مشکلات اسلاید ۱۶ را حل می‌کند؟

مشکل در JavaScript	پاسخ TypeScript
متغیر در زمان اجرا نوع عوض می‌کند	نوع متغیر ثابت می‌ماند؛ `count = "zero"` خطای کامپایل دارد
تبدیل نوع خودکار و نتایج عجیب	عملیات مشکوک مثل `"5" - 2`، `true + true` و `null + 1` گزارش می‌شوند
باگ‌های پنهان در کدهای بزرگ	غلط‌های املایی و فراخوانی‌های اشتباه قبل از اجرا پیدا می‌شوند؛ نوع‌ها مثل مستندات عمل می‌کنند
خطاها فقط در زمان اجرا دیده می‌شوند	tsc و ویرایشگر خطا را قبل از اجرا نشان می‌دهند
تفاوت موتورها و نسخه‌ها	با تنظیم `target` خروجی برای نسخه‌ی موردنظر JavaScript ساخته می‌شود

TypeScript

```
const a = "5" + 2;     // OK: type string (concatenation is allowed)
const b = "5" - 2;     // Error TS2362: The left-hand side of an arithmetic operation must be of type 'any', 'number', 'bigint' or an enum type.
const c = true + true; // Error TS2365: Operator '+' cannot be applied to types 'boolean' and 'boolean'.
const d = null + 1;    // Error TS18050: The value 'null' cannot be used here.
```

و همان باگ تابع `shout` این بار **قبل از اجرا** پیدا می‌شود، حتی اگر `shout` هیچ‌وقت صدا زده نشود:

TypeScript

```
function shout(msg: string): string {
  return msg.toUppercase(); // Error TS2551: Property 'toUppercase' does not exist on type 'string'. Did you mean 'toUpperCase'?
}
```

> **نکته**
>
> نوع‌ها فقط برای کامپایلر و ویرایشگرند. بعد از کامپایل هیچ اثری از آن‌ها در JavaScript نمی‌ماند؛ پس TypeScript نه برنامه را سریع‌تر می‌کند و نه در زمان اجرا نوع‌ها را چک می‌کند. تمام کمکش **قبل از اجرا** است.

> **دام تستی**
>
> - `"5" + 2` در TypeScript **خطا نیست**! چون چسباندن رشته کار مجازی است، tsc آن را قبول می‌کند و نوع حاصل را `string` می‌داند. اما `"5" - 2` خطای کامپایل دارد.
> - superset یعنی هر کد JavaScript نحوِ معتبر TypeScript را دارد؛ **نه** اینکه حتماً بدون خطای نوع کامپایل شود.
> - tsc به‌طور پیش‌فرض حتی وقتی خطای نوع پیدا کند، باز هم فایل `.js` را می‌سازد و فقط خطا را گزارش می‌کند.

بیشتر بدانیم

- اگر بخواهی در صورت وجود خطا هیچ فایل `.js` ساخته نشود، گزینه‌ی `noEmitOnError` را در `tsconfig.json` روشن کن.
- با گزینه‌ی `target` تعیین می‌کنی خروجی برای کدام نسخه‌ی JavaScript ساخته شود؛ مثلاً با `"target": "es2015"` عملگر جدید `??` به یک عبارت شرطی معمولی تبدیل می‌شود تا موتورهای قدیمی‌تر هم آن را بفهمند.
- نسخه‌های جدید Node.js (مثلاً نسخه‌ی ۲۴) فایل `.ts` را مستقیم هم اجرا می‌کنند (`node app.ts`)، اما فقط نوع‌ها را پاک می‌کنند و **هیچ بررسی نوعی انجام نمی‌دهند**؛ برای بررسی نوع همچنان tsc لازم است.

جمع‌بندی

JavaScript نوع‌دهی پویا دارد، تبدیل نوع خودکار انجام می‌دهد و خطاهای نوع را فقط در زمان اجرا نشان می‌دهد.
TypeScript (Microsoft، سال ۲۰۱۲، با طراحی Anders Hejlsberg) یک superset تدریجی و اختیاری از JavaScript است که static typing و tooling را اضافه می‌کند. کد TypeScript با tsc بررسی و به JavaScript ساده تبدیل می‌شود، نوع‌ها در خروجی پاک می‌شوند و نتیجه هر جا که JavaScript اجرا شود، اجرا می‌شود.

تست‌های تمرینی

۱۹ تست · شماره‌ی ۱۸۴ تا ۲۰۲

۱۸۴ آسان

TypeScript در چه سالی و توسط کدام شرکت معرفی شد؟

۱) Google، سال ۲۰۰۹
۲) Facebook، سال ۲۰۱۲
۳) Microsoft، سال ۲۰۱۲
۴) Mozilla، سال ۲۰۱۵

۱۸۵ آسان

در جای خالی کدام گزینه قرار می‌گیرد؟ «طراحی TypeScript را Anders Hejlsberg رهبری کرد؛ کسی که معمار اصلی زبان‌های ..... است.»

۱) Java و Kotlin
۲) C# و Delphi
۳) Python و Ruby
۴) C و C++

۱۸۶ آسان

جمله‌ی «JavaScript یک زبان **dynamically typed** است» یعنی چه؟

۱) نوع متغیرها قبل از اجرا توسط کامپایلر بررسی می‌شود.
۲) در JavaScript مقدارها اصلاً نوع ندارند.
۳) نوع یک متغیر فقط با توابع تبدیل مثل `Number()` قابل تغییر است.
۴) نوع به مقدار تعلق دارد و یک متغیر می‌تواند در زمان اجرا مقدارهایی با نوع‌های مختلف بگیرد.

۱۸۷ آسان

در JavaScript حاصل `"5" + 2` و `"5" - 2` به ترتیب کدام است؟ (دو حاصل در قالب یک آرایه نوشته شده‌اند.)

۱) `["52", 3]`
۲) `[7, 3]`
۳) `["52", "3"]`
۴) `["7", 3]`

۱۸۸ آسان

«TypeScript یک **superset** (ابرمجموعه) از JavaScript است.» منظور از این جمله کدام است؟

۱) هر کد JavaScript از نظر نحو (syntax) یک کد TypeScript هم هست و TypeScript نوع‌ها را به آن اضافه می‌کند.

۲) TypeScript زبانی کاملاً جدا با نحو متفاوت است که باید آن را از صفر یاد گرفت.

۳) هر کد TypeScript بدون هیچ تغییری مستقیماً در مرورگر اجرا می‌شود.

۴) TypeScript بخش کوچکی از JavaScript است که قسمت‌های خطرناک آن حذف شده است.

۱۸۹ آسان

دستور `npx tsc --init` چه کاری انجام می‌دهد؟

۱) TypeScript را روی سیستم نصب می‌کند.

۲) همه‌ی فایل‌های `.ts` پروژه را اجرا می‌کند.

۳) فایل `tsconfig.json` را با تنظیمات پیشنهادی (از جمله `"strict": true`) می‌سازد.

۴) فایل `package.json` پروژه را می‌سازد.

۱۹۰ متوسط

خروجی کد JavaScript زیر در Node.js کدام است؟

```JavaScript
let data = 10;
console.log(typeof data);
data = "ten";
console.log(typeof data);
data = data + 5;
console.log(data);
```

۱) `number` `number` `15`

۲) `number` `string` `ten5`

۳) `number` `Uncaught TypeError`

۴) `number` `string` `NaN`

۱۹۱ متوسط

کدام خط از کد JavaScript زیر باعث خطا می‌شود؟

```JavaScript
1  const user = { name: "Ali" };
2  console.log(user.nmae);
3  console.log(user.name.toUppercase());
4  console.log("done");
```

۱) خط ۲

۲) خط ۳

۳) خط ۴

۴) هیچ خطایی رخ نمی‌دهد

۱۹۲ سخت

خروجی کد JavaScript زیر در Node.js چیست؟

```javascript
function shout(msg) {
  return msg.toUppercase();
}
console.log("start");
if (Math.random() > 2) {
  console.log(shout("hi"));
}
console.log("end");
```

۱) `start` `Uncaught TypeError`
۲) `Uncaught SyntaxError`
۳) `start` `end`
۴) `start` `HI` `end`

۱۹۳ متوسط

کد TypeScript زیر را با tsc بررسی می‌کنیم. کدام خط خطای کامپایل دارد؟

```typescript
function shout(msg: string): string {
  return msg.toUppercase();
}
console.log("start");
if (Math.random() > 2) {
  console.log(shout("hi"));
}
console.log("end");
```

۱) خط ۲
۲) خط ۵
۳) خط ۶
۴) هیچ خطایی رخ نمی‌دهد

۱۹۴ سخت

در TypeScript (حالت strict) کدام دستور **بدون خطا** کامپایل می‌شود؟

۱) `const b = "5" - 2;`
۲) `const c = true + true;`
۳) `const d = null + 1;`
۴) `const a = "5" + 2;`

۱۹۵ متوسط

هر چهار کد زیر در JavaScript بدون خطا اجرا می‌شوند. tsc برای کدام‌یک خطای نوع گزارش می‌کند؟

۱) `let n = 5; n = n + 1;`
۲) `let s = "a"; s = s + 1;`
۳) `const list = [1, 2]; list.push(3);`
۴) `let flag = true; flag = "yes";`

۱۹۶ متوسط

کدام گزینه درباره‌ی «تدریجی و اختیاری» (gradual, optional) بودن TypeScript **نادرست** است؟

۱) برای استفاده از TypeScript باید کل پروژه یک‌جا بازنویسی شود و هیچ فایل JavaScript نباید باقی بماند.

۲) می‌توان یک پروژه‌ی JavaScript را فایل به فایل به TypeScript تبدیل کرد.

۳) نوشتن type annotation اجباری نیست و TypeScript نوع بسیاری از متغیرها را خودش تشخیص می‌دهد.

۴) کد TypeScript در نهایت به JavaScript تبدیل می‌شود و هر جا JavaScript اجرا شود، اجرا می‌شود.

۱۹۷ متوسط

TypeScript برای اضافه کردن static typing و **tooling** به JavaScript ساخته شد. کدام مورد از مزایای این tooling **نیست**؟

۱) پیشنهاد خودکار متدها و ویژگی‌ها هنگام تایپ (autocomplete)

۲) اجرای سریع‌تر برنامه در مرورگر به خاطر دانستن نوع‌ها

۳) نمایش خطا در ویرایشگر، پیش از اجرای برنامه

۴) تغییر نام امن یک تابع در همه‌ی فایل‌های پروژه

۱۹۸ متوسط

کدام گزینه جزو «مشکلات تایپ در JavaScript» که در اسلاید ۱۶ آمده **نیست**؟

۱) متغیرها می‌توانند در زمان اجرا نوع عوض کنند.

۲) تبدیل نوع خودکار نتایج غیرمنتظره می‌دهد؛ مثل `"5" + 2` .

۳) نبودن بررسی‌های زمان کامپایل باعث می‌شود بسیاری از خطاها فقط هنگام اجرا دیده شوند.

۴) اجرای تک‌نخی (single-threaded) باعث می‌شود محاسبات سنگین، اجرای برنامه را متوقف کنند.

۱۹۹ سخت

فایل `app.ts` یک خطای نوع دارد. در پوشه‌ی پروژه فایل `tsconfig.json` نیست و هیچ تنظیم خاصی (مثل `noEmitOnError`) هم فعال نشده است. با اجرای `npx tsc app.ts` چه اتفاقی می‌افتد؟

۱) خطا گزارش می‌شود، اما فایل `app.js` باز هم ساخته می‌شود.

۲) خطا گزارش می‌شود و فایل `app.js` ساخته نمی‌شود.

۳) هیچ خطایی گزارش نمی‌شود؛ خطا فقط هنگام اجرای `app.js` دیده می‌شود.

۴) tsc خطا را خودش اصلاح می‌کند و `app.js` درست را می‌سازد.

۲۰۰ متوسط

کدام گزینه درباره‌ی رابطه‌ی TypeScript و زمان اجرا **درست** است؟

۱) نوع‌هایی که در TypeScript می‌نویسیم در فایل JavaScript خروجی باقی می‌مانند و مرورگر آن‌ها را بررسی می‌کند.

۲) tsc نوع‌ها را پاک می‌کند و هیچ بررسی‌ای به کد خروجی اضافه نمی‌کند؛ پس در زمان اجرا چیزی جلوی مقدار نادرست را نمی‌گیرد.

۳) چون موتور JavaScript نوع‌ها را از قبل می‌داند، برنامه‌ی TypeScript همیشه سریع‌تر اجرا می‌شود.

۴) مرورگرها فایل `.ts` را مستقیماً و بدون هیچ تبدیلی اجرا می‌کنند.

۲۰۱ سخت

کد TypeScript زیر بدون خطا کامپایل می‌شود (نوع `any` یعنی «هر نوعی؛ بررسی نکن»). خروجی اجرای آن چیست؟

```TypeScript
function double(n: number): number {
  return n * 2;
}
const input: any = "21";
console.log(double(input));
```

۱) `2121`

۲) `Uncaught TypeError`

۳) `42`

۴) `NaN`

۲۰۲ سخت

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
1  const qty = 3;
2  const price = 2;
3  const msg = "Total: " + qty * price;
4  const half = msg / 2;
5  console.log(msg, half);
```

۱) خط ۳

۲) خط ۵

۳) خط ۴

۴) هیچ خطایی رخ نمی‌دهد

۳-۴ متغیرها و انواع داده در TypeScript

منبع: NodeJS & TS، اسلاید ۱۸

درسنامه

در درس قبل دیدیم TypeScript همان JavaScript است به‌علاوه‌ی نوع‌ها. حالا وقتش است ببینیم این نوع‌ها را دقیقاً کجا و چطور می‌نویسیم. موضوع اسلاید ۱۸ متغیرهاست. یادآوری: در این کتاب فرض می‌کنیم حالت **strict** (سخت‌گیرانه) روشن است؛ همان پیش‌فرض `tsc --init` .

یک تشبیه ساده

متغیر را مثل یک ظرف با برچسب تصور کن. در JavaScript ظرف‌ها برچسب ندارند و هر چیزی داخلشان می‌رود. در TypeScript روی ظرف برچسب «فقط عدد» یا «فقط متن» می‌خورد و کامپایلر اجازه نمی‌دهد چیز دیگری داخلش بریزی.

۱. let و const با type annotation

طبق اسلاید ۱۸، متغیرها را دقیقاً مثل JavaScript با `let` و `const` تعریف می‌کنیم؛ فقط می‌توانیم (به‌طور اختیاری) بعد از نام متغیر یک دونقطه و نام نوع بنویسیم. به این بخش **type annotation** (برچسب نوع) می‌گویند:

```
let variableName: type = value;
```

مثال خود اسلاید، `let age: number = 25;` ، تضمین می‌کند که `age` همیشه عدد بماند:

```TypeScript
let age: number = 25;
age = 26;   // OK: still a number
age = "26"; // Error TS2322: Type 'string' is not assignable to type 'number'.
```

پیام خطا را این‌طور بخوان: «نوع string را نمی‌شود به نوع number نسبت داد». این رایج‌ترین خطای TypeScript است و کدش `TS2322` است.

۲. نوع‌های پایه

نوع	مقدارهای مجاز	مثال
`number`	همه‌ی عددها: صحیح، اعشاری، منفی و حتی `NaN`	`let price: number = 9.5;`
`string`	متن، با `"…"` یا `'…'` یا `` `…` ``	`let city: string = "Tehran";`
`boolean`	فقط `true` یا `false`	`let isOpen: boolean = true;`
`null` و `undefined`	فقط خودِ همان مقدار	توضیح در بخش ۹

یادداشت

نام نوع‌ها را با حرف **کوچک** بنویس: `number` ، `string` و `boolean` . نوع‌های `Number` و `String` (با حرف بزرگ) مربوط به اشیای پوششی (wrapper) JavaScript‌اند و نباید به‌جای این‌ها به کار بروند.

۳. const: اتصال تغییرناپذیر (immutable binding)

اسلاید ۱۸ می‌گوید `const` یک **immutable binding** (اتصال تغییرناپذیر بین نام و مقدار) می‌سازد و جلوی **reassignment** (انتساب دوباره) را می‌گیرد. در JavaScript این اشتباه در زمان اجرا `TypeError` می‌داد؛ در TypeScript همان موقع کامپایل گزارش می‌شود:

```TypeScript
const PI = 3.14;
PI = 3; // Error TS2588: Cannot assign to 'PI' because it is a constant.
```

دقت کن: `const` فقط خودِ متغیر را قفل می‌کند، **نه محتوای** آرایه یا شیئی که متغیر به آن اشاره می‌کند:

```TypeScript
const list = [1, 2];
list.push(3);  // OK: changing the contents is allowed
list[0] = 10;  // OK
list = [4, 5]; // Error TS2588: Cannot assign to 'list' because it is a constant.
```

هر `const` هم باید هنگام تعریف مقدار بگیرد؛ نوشتن `const x: number;` این خطا را دارد: `'const' declarations must be initialized`.

۴. استنتاج نوع (type inference)

آخرین بولت اسلاید ۱۸: اگر نوع را ننویسی، TypeScript **خودش آن را از روی مقدار اولیه تشخیص می‌دهد**. به این کار **type inference** (استنتاج نوع) می‌گویند. پس نوشتن نوع اختیاری است، ولی بررسی نوع همچنان انجام می‌شود:

```TypeScript
let city = "Tehran"; // inferred type: string
city = "Shiraz";     // OK
city = 42;           // Error TS2322: Type 'number' is not assignable to type 'string'.
```

جدول زیر نوع استنتاج‌شده را در حالت‌های مهم نشان می‌دهد:

تعریف	نوع استنتاج‌شده	توضیح
`let x = 5;`	`number`	با `let` نوع «عمومی» انتخاب می‌شود
`const y = 5;`	`5`	با `const` نوع **literal** (خودِ همان مقدار)
`let s = "hi";`	`string`	
`const g = "hi";`	`"hi"`	literal type
`let flag = true;`	`boolean`	
`let nums = [1, 2, 3];`	`number[]`	آرایه‌ی عددی
`const arr = [1, 2, 3];`	`number[]`	`const` روی آرایه هم `number[]` می‌دهد؛ محتوا قابل تغییر است
`let mixed = [1, "a"];`	`(string | number)[]`	آرایه‌ای که هر خانه‌اش عدد یا رشته است
`const user = { name: "Ali" };`	`{ name: string }`	ویژگی‌های شیء هم نوع عمومی می‌گیرند

چرا `const y = 5` نوع `5` می‌گیرد؟ چون `y` هرگز عوض نمی‌شود، TypeScript دقیق‌ترین نوع ممکن یعنی خودِ مقدار `5` را برایش در نظر می‌گیرد. اما `let x = 5` بعداً می‌تواند عدد دیگری بگیرد، پس نوعش `number` است.

کِی نوع را خودمان بنویسیم؟ (۱) وقتی متغیر را **بدون مقدار اولیه** تعریف می‌کنیم؛ (۲) وقتی نوعی **بازتر** از مقدار اولیه می‌خواهیم (مثلاً union). اگر متغیری بدون مقدار تعریف شود و پیش از مقداردهی خوانده شود، خطا می‌گیریم:

```TypeScript
let later: number;  // declared without a value
console.log(later); // Error TS2454: Variable 'later' is used before being assigned.
later = 10;
```

۵. آرایه‌ها

نوع آرایه را به دو شکل می‌نویسیم که **کاملاً معادل‌اند**: `number[]` و `Array<number>`.

```TypeScript
let scores: number[] = [18, 20];
let tags: Array<string> = ["ts", "js"];
scores.push(19.5); // OK
tags.push("node"); // OK
scores.push("19"); // Error TS2345: Argument of type 'string' is not assignable to parameter of type 'number'.
```

۶. Union type (نوع اجتماع)

طبق اسلاید ۱۸، TypeScript از **union type** (نوع اجتماع؛ «این یا آن») پشتیبانی می‌کند: `let id: number | string;` یعنی `id` می‌تواند عدد **یا** رشته باشد. علامت `|` را «یا» بخوان.

```TypeScript
let id: number | string;
id = 101;     // OK
id = "A-101"; // OK
id = true;    // Error TS2322: Type 'boolean' is not assignable to type 'string | number'.
```

tsc در پیام خطا این union را `string | number` چاپ می‌کند؛ ترتیب اعضای union اهمیتی ندارد.

استفاده از مقدار union: روی یک مقدار union فقط کارهایی مجاز است که برای **همه‌ی** اعضای آن درست باشد. مثلاً `toString()` هم روی عدد هست هم روی رشته، اما `toUpperCase()` فقط مال رشته است:

```TypeScript
function show(id: number | string) {
  console.log(id.toString());    // OK: both types have toString
  console.log(id.toUpperCase()); // Error TS2339: Property 'toUpperCase' does not exist on type
'string | number'.
}
```

راه‌حل، **narrowing** (باریک کردن نوع) است: با یک شرط مثل `typeof` به TypeScript نشان می‌دهیم در هر شاخه کدام نوع را داریم:

```TypeScript
function format(id: number | string): string {
  if (typeof id === "string") {
    return id.toUpperCase(); // here id is a string
  }
  return id.toFixed(1);      // here id is a number
}
console.log(format("a-7")); // A-7
console.log(format(5));     // 5.0
```

union فقط در زمان کامپایل وجود دارد. در زمان اجرا، `typeof` نوع **واقعیِ** مقدار فعلی را برمی‌گرداند (`"number"` یا `"string"`)، نه چیزی مثل `"number | string"`.

> **بیشتر بدانیم**
>
> TypeScript آخرین مقداری را که به یک متغیر union داده‌ای هم دنبال می‌کند. مثلاً بعد از `let id: number | string = 5;` در خط‌های بعدی می‌داند `id` **فعلاً** عدد است و `id.toFixed(1)` را قبول می‌کند، ولی همچنان می‌توانی بعداً رشته در آن بگذاری. برای همین مثال‌های union را با پارامتر تابع می‌نویسیم که مقدارش از قبل معلوم نیست.

۷. Literal type (نوع لفظی)

نوع می‌تواند یک **مقدار مشخص** باشد؛ به آن **literal type** (نوع لفظی) می‌گویند. وقتی چند literal را با `|` ترکیب کنیم، فهرستی از مقدارهای مجاز ساخته‌ایم:

```TypeScript
let role: "admin" | "user" = "user";
role = "admin"; // OK
role = "guest"; // Error TS2322: Type '"guest"' is not assignable to type '"admin" | "user"'.
let dice: 1 | 2 | 3 | 4 | 5 | 6 = 3;
```

نتیجه‌ی جالب: چون `const status = "ok"` نوع `"ok"` دارد، می‌شود آن را در متغیری از نوع `"ok" | "error"` گذاشت؛ اما `let message = "ok"` نوع `string` دارد و نمی‌شود.

۸. any و unknown

- `any` (هر نوعی): بررسی نوع را برای آن مقدار **کاملاً خاموش** می‌کند. هر کاری با آن مجاز است و خطاها دوباره به زمان اجرا برمی‌گردند. فقط به‌عنوان راه فرار موقت (مثلاً هنگام تبدیل تدریجی یک پروژه) از آن استفاده کن.
- `unknown` (ناشناخته): نسخه‌ی امنِ `any` . هر مقداری را می‌پذیرد، اما **تا نوعش را با یک شرط مشخص نکنی** اجازه‌ی استفاده از آن را نمی‌دهد.

```TypeScript
let a: any = "hello";
a.toUpperCase(); // OK: no checking at all
a.foo.bar();     // also accepted by tsc, but crashes at runtime with a TypeError

let u: unknown = "hello";
u.toUpperCase(); // Error TS18046: 'u' is of type 'unknown'.
if (typeof u === "string") {
  console.log(u.toUpperCase()); // OK: narrowed to string
}
```

	`any`	`unknown`
چه مقداری می‌گیرد؟	هر مقداری	هر مقداری
استفاده‌ی مستقیم (صدا زدن متد و …)	مجاز، بدون هیچ بررسی	ممنوع تا وقتی narrowing نشود
نسبت دادن به متغیر نوع‌دار (`let n: number = …`)	مجاز	خطا (TS2322)
ایمنی	بررسی خاموش است	امن

۹. null و undefined در حالت strict

در حالت strict، `null` و `undefined` نوع‌های جداگانه‌اند و **خودبه‌خود** داخل `number` یا `string` جا نمی‌شوند. اگر متغیری ممکن است خالی باشد، باید صریحاً با union بگویی:

```TypeScript
let n1: number = null;         // Error TS2322: Type 'null' is not assignable to type 'number'.
let n2: number = undefined;    // Error TS2322: Type 'undefined' is not assignable to type
'number'.
let n3: number | null = null;  // OK
let nick: string | undefined;  // OK
```

۱۰. Tuple (چندتایی)

tuple (چندتایی) آرایه‌ای با **طول ثابت** است که نوع هر خانه‌اش جداگانه مشخص شده است:

```TypeScript
let pair: [string, number] = ["Ali", 20];
pair[0] = "Sara";     // OK: index 0 is a string
pair = [20, "Ali"];   // Error TS2322 (twice): the order of the types is wrong
console.log(pair[2]); // Error TS2493: Tuple type '[string, number]' of length '2' has no element at index '2'.
```

نکته

هر جا مقدار اولیه داری، بگذار TypeScript خودش نوع را تشخیص دهد (`let city = "Tehran";`). نوع را وقتی بنویس که مقدار اولیه نداری یا نوعی بازتر مثل `number | string` می‌خواهی.

دام تستی

- `const` جلوی **انتساب دوباره** را می‌گیرد، نه تغییر محتوا: `const list = [1]; list.push(2);` مجاز است.
- نوع `const y = 5` برابر `5` (literal) است، نه `number`؛ اما `const arr = [1, 2]` باز هم `number[]` است.
- `string | number[]` یعنی «یک رشته **یا** یک آرایه‌ی عددی»، اما `(string | number)[]` یعنی «آرایه‌ای که هر خانه‌اش عدد یا رشته است». پرانتز مهم است!
- `any` بررسی را خاموش می‌کند؛ `unknown` تا قبل از narrowing اجازه‌ی هیچ استفاده‌ای نمی‌دهد.
- در حالت strict، `let n: number = null;` خطاست؛ باید بنویسی `number | null`.

جمع‌بندی

در TypeScript متغیرها را مثل JavaScript با `let` و `const` تعریف می‌کنیم و می‌توانیم با type annotation نوعشان را بنویسیم (`let age: number = 25;`). اگر ننویسیم، TypeScript نوع را از مقدار اولیه استنتاج می‌کند (`let` نوع عمومی می‌گیرد و `const` نوع literal). `const` جلوی انتساب دوباره را می‌گیرد، نه تغییر محتوا. آرایه‌ها `number[]` یا `Array<number>` نوشته می‌شوند. union مثل `number | string` چند نوع را مجاز می‌کند و با `typeof` باریک می‌شود. `any` بررسی را خاموش می‌کند و `unknown` نسخه‌ی امن آن است. در حالت strict، `null` و `undefined` فقط با union مجازند و tuple آرایه‌ای با طول و نوعِ ثابت برای هر خانه است.

تست‌های تمرینی

۲۵ تست · شماره‌ی ۲۰۳ تا ۲۲۷

۲۰۳ آسان

در دستور `let age: number = 25;` بخش `: number` چه نام دارد؟

۱) type inference (استنتاج نوع)
۲) type annotation (برچسب نوع)
۳) union type (نوع اجتماع)
۴) type coercion (تبدیل نوع خودکار)

۲۰۴ آسان

با فرض تعریف زیر، کدام دستور خطای کامپایل دارد؟

```TypeScript
let age: number = 25;
```

۱) `age = "26";`
۲) `age = 26;`
۳) `age = age + 1;`
۴) `age = 25.5;`

۲۰۵ آسان

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
let count = 1;
count = 2;
const PI = 3.14;
PI = 3;
console.log(count, PI);
```

۱) خط ۲
۲) خط ۳
۳) خط ۴
۴) هیچ خطایی رخ نمی‌دهد

۲۰۶ متوسط

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
const list = [1, 2];
list.push(3);
list[0] = 10;
list = [4, 5];
```

۱) خط ۴
۲) خط ۲
۳) خط ۳
۴) هیچ خطایی رخ نمی‌دهد

۲۰۷ آسان

با فرض تعریف زیر (مثال اسلاید ۱۸)، کدام دستور خطای کامپایل دارد؟

```TypeScript
let id: number | string;
```

۱) `id = 101;`
۲) `id = "A-101";`
۳) `id = true;`
۴) `id = -5;`

۲۰۸ متوسط

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
let city = "Tehran";
city = "Shiraz";
let year = 1403;
city = year;
```

۱) خط ۲
۲) خط ۳
۳) هیچ خطایی رخ نمی‌دهد
۴) خط ۴

۲۰۹ سخت

کدام گزینه **بدون خطا** کامپایل می‌شود؟ (نوع `5` یعنی «فقط و فقط عدد ۵».)

۱) `let x = 5; const five: 5 = x;`
۲) `let s = "hi"; const h: "hi" = s;`
۳) `const y = 5; const five: 5 = y;`
۴) `const n = 5; n = 6;`

۲۱۰ متوسط

TypeScript نوع متغیر `x` را در کد زیر چه تشخیص می‌دهد؟

```TypeScript
let x = [1, 2, 3];
```

۱) `[1, 2, 3]`
۲) `any[]`
۳) `number`
۴) `number[]`

۲۱۱ آسان

کدام نوع **دقیقاً** با `number[]` معادل است؟

۱) `[number]`
۲) `Array<number>`
۳) `number | []`
۴) `Array[number]`

۲۱۲ متوسط

با فرض تعریف‌های زیر، کدام دستور خطای کامپایل دارد؟

```TypeScript
let scores: number[] = [18, 20];
let tags: Array<string> = ["ts"];
```

۱) `scores.push("19");`
۲) `scores.push(19.5);`
۳) `tags.push("node");`
۴) `tags = [];`

۲۱۳ سخت

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
function show(id: number | string) {
  console.log(id.toString());
  console.log(id.toUpperCase());
  if (typeof id === "string") {
    console.log(id.toUpperCase());
  }
}
```

۱) خط ۲
۲) خط ۳
۳) خط ۵
۴) هیچ خطایی رخ نمی‌دهد

۲۱۴ متوسط

خروجی کد TypeScript زیر پس از کامپایل و اجرا چیست؟

```TypeScript
function format(value: number | string): string {
  if (typeof value === "number") {
    return value.toFixed(1);
  }
  return value.toUpperCase();
}
console.log(format(5));
console.log(format("ok"));
```

۱) `5` `OK`
۲) `5.0` `ok`
۳) `5.0` `OK`
۴) `5` `ok`

۲۱۵ متوسط

با فرض تعریف زیر، کدام دستور خطای کامپایل دارد؟

```TypeScript
let role: "admin" | "user" = "user";
```

۱) `role = "admin";`
۲) `role = "guest";`
۳) `const r: string = role;`
۴) `role = "user";`

۲۱۶ متوسط

کدام گزینه درباره‌ی `any` و `unknown` **درست** است؟

۱) هر دو بررسی نوع را به‌طور کامل خاموش می‌کنند.

۲) `unknown` فقط می‌تواند مقدار `undefined` بگیرد.

۳) هر مقداری را می‌توان در هر دو گذاشت؛ اما از مقدار `unknown` فقط پس از مشخص کردن نوعش (مثلاً با `typeof`) می‌شود استفاده کرد، در حالی که `any` هر استفاده‌ای را بدون بررسی می‌پذیرد.

۴) `any` امن‌تر از `unknown` است، چون خطاهای بیشتری را پیدا می‌کند.

۲۱۷ متوسط

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
let a: any = "hello";
let u: unknown = "hello";
console.log(a.toUpperCase());
console.log(u.toUpperCase());
```

۱) خط ۲

۲) خط ۳

۳) خط ۴

۴) هیچ خطایی رخ نمی‌دهد

۲۱۸ سخت

کد TypeScript زیر بدون خطا کامپایل می‌شود. خروجی اجرای آن چیست؟

```TypeScript
let data: any = 42;
console.log("before");
data.toUpperCase();
console.log("after");
```

۱) `before` `Uncaught TypeError`

۲) `before` `after`

۳) `before` `42` `after`

۴) `Uncaught TypeError`

۲۱۹ متوسط

در حالت strict، کدام تعریف **بدون خطا** کامپایل می‌شود؟

۱) `let c: number | null = null;`

۲) `let a: number = null;`

۳) `let b: string = undefined;`

۴) `let d: boolean = null;`

۲۲۰ متوسط

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
let pair: [string, number] = ["Ali", 20];
pair[0] = "Sara";
pair[1] = 21;
console.log(pair[2]);
```

۱) خط ۲
۲) خط ۴
۳) خط ۳
۴) هیچ خطایی رخ نمی‌دهد

۲۲۱ آسان

با فرض تعریف زیر، کدام دستور خطای کامپایل دارد؟

```TypeScript
let user: [string, number];
```

۱) `user = ["Ali", 20];`
۲) `user = ["", 0];`
۳) `user = ["Sara", -1];`
۴) `user = [20, "Ali"];`

۲۲۲ سخت

کدام گزینه **بدون خطا** کامپایل می‌شود؟

۱) `let b: (string | number)[] = [1, "x"];`
۲) `let a: string | number[] = [1, "x"];`
۳) `let c: string[] | number[] = [1, "x"];`
۴) `let d: number[] = [1, "x"];`

۲۲۳ سخت

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
let total: number;
let count = 0;
count++;
console.log(total + count);
total = 100;
```

۱) خط ۱
۲) خط ۳
۳) هیچ خطایی رخ نمی‌دهد
۴) خط ۴

۲۲۴ سخت

کدام گزینه **نادرست** است؟

۱) در `let x = 5;` نوع استنتاج‌شده‌ی `x` برابر `number` است.

۲) در `const y = 5;` نوع استنتاج‌شده‌ی `y` برابر `5` است.

۳) در `let names = ["Ali", "Sara"];` نوع استنتاج‌شده‌ی `names` برابر `string[]` است.

۴) در `const arr = [1, 2];` چون `arr` با `const` تعریف شده، نمی‌توان با `push` عضوی به آن اضافه کرد.

۲۲۵ آسان

در جای خالی کدام نوع قرار گیرد تا کد زیر **بدون خطا** کامپایل شود؟

```TypeScript
let id: ______;
id = 101;
id = "A-101";
```

۱) `number` ۲) `string` ۳) `boolean` ۴) `number | string`

۲۲۶ سخت

tsc در کدام خط کد زیر خطا گزارش می‌کند؟

```TypeScript
1  const status = "ok";
2  let state: "ok" | "error" = status;
3  let message = "ok";
4  state = message;
```

۱) خط ۴ ۲) خط ۲

۳) هیچ خطایی رخ نمی‌دهد ۴) خط ۱

۲۲۷ متوسط

خروجی کد TypeScript زیر پس از کامپایل و اجرا چیست؟

```TypeScript
let id: number | string = 7;
console.log(typeof id);
id = "seven";
console.log(typeof id);
```

۱) `number | string` `number | string`

۲) `number` `string`

۳) `number` `number`

۴) `string` `string`

۳-۵ کلاس‌ها در TypeScript

منبع: NodeJS & TS، اسلاید ۲۱

درسنامه

کلاس یعنی چه؟ (یک یادآوری سریع)

فرض کن در یک کارخانه‌ی خودروسازی کار می‌کنی. قبل از ساختن خودروها یک **نقشه** می‌کشی: «هر خودرو رنگ دارد، مدل دارد و می‌تواند حرکت کند». بعد از روی همین یک نقشه، صدها خودروی واقعی می‌سازی. در برنامه‌نویسی به این نقشه **class** (کلاس) و به هر چیزی که از روی آن ساخته می‌شود **object** یا **instance** (شیء یا نمونه) می‌گویند.

> **یک تشبیه ساده**
>
> class مثل نقشه‌ی یک ساختمان است و هر object یک ساختمانِ واقعی که از روی آن نقشه ساخته شده. خودِ نقشه جای زندگی نیست؛ تا وقتی با `new` چیزی از روی آن نسازی، هیچ اتفاقی نمی‌افتد.

کلاس‌ها از نسخه‌ی ES6 بخشی از خودِ JavaScript هستند (در فصل اول با آن‌ها و با `this` آشنا شدی). TypeScript همان کلاس‌های JavaScript را برمی‌دارد و چند امکان مهم به آن‌ها اضافه می‌کند: نوع (type) برای propertyها و پارامترها، **access modifier**ها، `implements` ، `readonly` و چند امکان دیگر. برای همین زیر کد اسلاید ۲۱ نوشته شده: **«Just like JS, but better!»** (درست مثل JS، ولی بهتر!).

اسلاید ۲۱ چهار نکته‌ی اصلی دارد:

1. کلاس‌ها از **OOP** (Object-Oriented Programming، برنامه‌نویسی شیءگرا) با **constructor** (سازنده)، **property** (ویژگی) و **method** (متد) پشتیبانی می‌کنند.
2. کلاس می‌تواند یک **interface** را **implement کند** (پیاده‌سازی کند) تا ساختارش تضمین شود.
3. **inheritance** (وراثت) با `extends` و **method overriding** (بازنویسی متد) پشتیبانی می‌شود.
4. **access modifier**ها (تعیین‌کننده‌های سطح دسترسی: `public` ، `private` ، `protected`) مشخص می‌کنند هر property و method از کجا قابل دسترسی است.

کد اسلاید ۲۱: سه جزء اصلی یک کلاس

weee.ts

```ts
class Person {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
  greet() {
    console.log(`Hello, ${this.name}`);
  }
}
```

بیا خط به خط جلو برویم:

- `class Person { ... }` : کلاسی به اسم `Person` تعریف می‌کند. رسم است اسم کلاس با حرف بزرگ شروع شود.

- `name: string;` : **اعلان property**. در TypeScript هر property را باید (همراه با نوعش) در بدنه‌ی کلاس معرفی کنی. این خط یعنی «هر `Person` یک `name` از نوع `string` دارد».
- `constructor(name: string)` : **constructor** تابع ویژه‌ای است که **هر بار که با** `new` **یک شیء جدید ساخته می‌شود** اجرا می‌شود و شیء را مقداردهی اولیه می‌کند. پارامترش هم نوع دارد.
- `this.name = name;` : `this` یعنی «همین شیئی که الان در حال ساخته شدن است». مقدار پارامتر را در property آن شیء می‌گذاریم.
- `greet() { ... }` : یک **method** (تابعی که مال شیء است). داخلش با **template literal** (رشته‌ای بین دو بک‌تیک که `${...}` را با مقدار واقعی جایگزین می‌کند) سلام می‌کند.

> ⚠ **کد اسلاید به‌تنهایی هیچ خروجی‌ای ندارد**
>
> این کد فقط **نقشه** را تعریف می‌کند: نه شیئی ساخته شده و نه `greet` صدا زده شده است. پس اجرای آن هیچ چیزی چاپ نمی‌کند. برای دیدن خروجی باید شیء بسازیم و متد را صدا بزنیم.

TypeScript

```
class Person {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
  greet() {
    console.log(`Hello, ${this.name}`);
  }
}
const p = new Person("Alice");     // the constructor runs here
p.greet();                         // Hello, Alice
console.log(p.name);               // Alice
console.log(p instanceof Person); // true
```

اسم کلاس خودش یک **نوع** هم هست؛ یعنی می‌توانی بنویسی `const p: Person = new Person("Ali");` یا تابعی بنویسی که پارامترش از نوع `Person` باشد.

وقتی `tsc` (کامپایلر TypeScript) این کد را به JavaScript تبدیل می‌کند، نوع‌ها پاک می‌شوند و یک کلاس معمولی JavaScript باقی می‌ماند (خروجی با target ES2022):

JavaScript

```
class Person {
    name;
    constructor(name) {
        this.name = name;
    }
    greet() {
        console.log(`Hello, ${this.name}`);
    }
}
```

«Just like JS, but better!» یعنی چه؟

در JavaScript معمولی اشتباه‌های زیر تا لحظه‌ی اجرا دیده نمی‌شوند (یا اصلاً خطا نمی‌دهند و فقط نتیجه‌ی غلط می‌دهند). TypeScript همان موقعِ نوشتن، یعنی در **compile time** (زمان کامپایل)، جلویشان را می‌گیرد:

```TypeScript
class Person {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
  greet() {
    console.log(`Hello, ${this.name}`);
  }
}
const p = new Person("Ali");
p.gret();              // Error TS2551: Property 'gret' does not exist. Did you mean 'greet'?
const q = new Person(); // Error TS2554: Expected 1 arguments, but got 0.
p.name = 42;           // Error TS2322: Type 'number' is not assignable to type 'string'.
```

اعلان و مقداردهی propertyها (خطای TS2564)

در حالت **strict** (سخت‌گیرانه؛ این کتاب همیشه آن را فرض می‌کند) TypeScript مطمئن می‌شود هیچ propertyای بی‌مقدار نمی‌ماند: هر property اعلان‌شده باید یا **کنار اعلانش** مقدار اولیه بگیرد، یا **داخل constructor** مقداردهی شود. در غیر این صورت خطای **TS2564** می‌گیری.

```TypeScript
class Product {
  title: string;       // OK: assigned in the constructor
  price: number = 0;   // OK: has an initializer
  stock?: number;      // OK: optional (number | undefined)
  code: string;        // Error TS2564: no initializer and not assigned in the constructor
  constructor(title: string) {
    this.title = title;
  }
}
```

روش	مثال	توضیح
مقدار اولیه کنار اعلان	`price: number = 0;`	ساده‌ترین راه
مقداردهی در constructor	`this.title = title;`	باید در **همه‌ی** مسیرهای constructor انجام شود
property اختیاری	`stock?: number;`	نوعش `number | undefined` می‌شود
علامت `!`	`code!: string;`	به TypeScript قول می‌دهی بعداً مقدار می‌گیرد؛ فقط وقتی واقعاً مطمئنی

> ⚠ **دام تستی: مقداردهی «گاهی‌اوقات» کافی نیست**
>
> اگر property را فقط داخل یک `if` در constructor مقدار بدهی (`if (n) this.name = n;`) یا آن را در یک متد معمولی (مثلاً `init()`) مقدار بدهی، باز هم خطای TS2564 می‌گیری؛ چون ممکن است شیء ساخته شود و آن property بی‌مقدار بماند.

پیاده‌سازی interface با `implements`

یادت هست interface (اسلاید ۱۹) **شکل** یک شیء را تعریف می‌کرد. کلاس می‌تواند با کلیدواژه‌ی `implements` اعلام کند: «من این قرارداد را رعایت می‌کنم». آن‌وقت TypeScript بررسی می‌کند همه‌ی اعضای interface، با نوع درست، در کلاس وجود داشته باشند.

```TypeScript
interface Shape {
  name: string;
  area(): number;
}
class Square implements Shape {   // OK: has name and area()
  name = "square";
  side: number;
  constructor(side: number) {
    this.side = side;
  }
  area(): number {
    return this.side * this.side;
  }
}
class Circle implements Shape {   // Error TS2420: Property 'area' is missing in type 'Circle'
  name = "circle";
}
class Blob implements Shape {
  name = "blob";
  area(): string {                // Error TS2416: '() => string' is not assignable to '() =>
number'
    return "big";
  }
}
```

- یک کلاس می‌تواند چند interface را با هم پیاده کند: `class Duck implements Walker, Swimmer { ... }`.
- `implements` فقط یک **بررسی** است؛ هیچ کدی به کلاس اضافه نمی‌کند و متدها را باید خودت بنویسی. کلاس می‌تواند اعضای اضافه هم داشته باشد (مثل `side` در `Square`).

وراثت با `extends` و بازنویسی متد

با `class Student extends Person`، کلاس `Student` (به آن **subclass** یا کلاس فرزند می‌گویند) همه‌ی propertyها و متدهای `Person` (**superclass** یا کلاس والد) را **به ارث می‌برد** و می‌تواند چیزهای تازه هم اضافه کند. رابطه‌ی وراثت یعنی «**is-a**»: هر دانشجو **یک** انسان است.

TypeScript

```typescript
interface Greeter {
  greet(): void;
}
class Person implements Greeter {
  public name: string;
  private secret: string;
  protected age: number;
  constructor(name: string, secret: string, age: number) {
    this.name = name;
    this.secret = secret;
    this.age = age;
  }
  greet() {
    console.log(`Hello, ${this.name}`);
  }
}
class Student extends Person {
  school: string;
  constructor(name: string, age: number, school: string) {
    super(name, "hidden", age);   // call the parent constructor first
    this.school = school;
  }
  greet() {                       // overrides Person.greet
    console.log(`Hi, I am ${this.name} (${this.age}) from ${this.school}`);
  }
}
const people: Person[] = [new Person("Ali", "s3cr3t", 30), new Student("Sara", 20, "Sharif")];
for (const x of people) x.greet();
// Hello, Ali
// Hi, I am Sara (20) from Sharif
console.log(people[1] instanceof Person); // true
```

قانون‌های constructor در کلاس فرزند:

1. اگر کلاس فرزند constructor خودش را دارد، **باید** `super(...)` را صدا بزند (`super` یعنی constructor کلاس والد). در غیر این صورت: خطای **TS2377**.
2. `super(...)` باید **قبل از هر استفاده از** `this` صدا زده شود؛ وگرنه خطای **TS17009**. (در JavaScript خالی همین اشتباه در زمان اجرا `ReferenceError` می‌دهد.)
3. اگر کلاس فرزند اصلاً constructor ننویسد، constructor والد خودکار استفاده می‌شود: با `class Cat extends Animal {}` می‌توانی بنویسی `new Cat("Tom")`.

TypeScript

```
class Animal {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
}
class Dog extends Animal {
  breed: string;
  constructor(name: string, breed: string) {
    this.breed = breed;   // Error TS17009: 'super' must be called before accessing 'this'
    super(name);
  }
}
class Fish extends Animal {
  constructor(name: string) {   // Error TS2377: Constructors for derived classes must contain a
'super' call.
    console.log("creating a fish");
  }
}
class Cat extends Animal {}     // OK: uses Animal's constructor
const tom = new Cat("Tom");
```

Method overriding (بازنویسی متد) یعنی کلاس فرزند متدی با **همان اسم** متد والد تعریف کند؛ آن‌وقت برای اشیای فرزند، نسخه‌ی فرزند اجرا می‌شود. در مثال بالا با این‌که نوع آرایه `Person[]` است، برای `Sara` متد `greet` اجرا شد؛ چون مهم این است که **شیء واقعاً از چه کلاسی ساخته شده** (به این رفتار **polymorphism** یا چندریختی می‌گویند). اگر بخواهی داخل نسخه‌ی جدید، نسخه‌ی والد را هم اجرا کنی، از `super.method()` استفاده کن:

TypeScript

```
class Animal {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
  speak(): string {
    return `${this.name} makes a sound`;
  }
}
class Dog extends Animal {
  override speak(): string {
    return `${super.speak()} - Woof`;
  }
}
console.log(new Dog("Rex").speak()); // Rex makes a sound - Woof
```

کلیدواژه‌ی `override` اختیاری است و به TypeScript می‌گوید «عمداً دارم متد والد را بازنویسی می‌کنم». فایده‌اش این است که اگر والد چنین متدی نداشته باشد (مثلاً اسم را غلط نوشته باشی)، خطای **TS4113** می‌گیری. (گزینه‌ی `noImplicitOverride` نوشتن `override` را اجباری می‌کند؛ این گزینه جزو `strict` نیست.) همچنین متد بازنویسی‌شده باید با نسخه‌ی والد **سازگار** باشد؛ مثلاً اگر والد `string` برمی‌گرداند و فرزند `number`، خطای **TS2416** می‌گیری.

Access modifierها: `public` ، `private` ، `protected`

modifier	داخل خودِ کلاس	داخل کلاس فرزند	بیرون کلاس (روی شیء)
`public` (پیش‌فرض)	مجاز	مجاز	مجاز
`protected`	مجاز	مجاز	ممنوع (TS2445)
`private`	مجاز	ممنوع (TS2341)	ممنوع (TS2341)

- اگر هیچ modifierی ننویسی، عضو `public` است؛ پس `name: string` همان `public name: string` است.
- `private` یعنی «فقط خودِ همین کلاس». حتی کلاس فرزند هم نمی‌تواند به آن دست بزند.
- `protected` یعنی «خودِ کلاس و فرزندانش»، ولی نه بیرون.
- متدها هم می‌توانند `private` یا `protected` باشند (مثلاً یک متد کمکی که نباید از بیرون صدا زده شود).

با همان کلاس‌های `Person` و `Student` بالا:

TypeScript

```
class Person {
  public name: string;
  private secret: string;
  protected age: number;
  constructor(name: string, secret: string, age: number) {
    this.name = name;
    this.secret = secret;
    this.age = age;
  }
}
class Student extends Person {
  showAge() {
    return this.age;      // OK: protected is visible in subclasses
  }
  leak() {
    return this.secret;   // Error TS2341: 'secret' is private and only accessible within class
'Person'
  }
}
const p = new Person("Ali", "s3cr3t", 30);
console.log(p.name);      // OK: public
console.log(p.secret);    // Error TS2341: 'secret' is private
console.log(p.age);       // Error TS2445: 'age' is protected
```

`private` فقط در زمان کامپایل است!

TypeScript بعد از بررسی، همه‌ی modifierها را پاک می‌کند. یعنی در JavaScript خروجی، `secret` یک property کاملاً معمولی است و در **runtime** (زمان اجرا) هر کسی می‌تواند آن را ببیند:

TypeScript

```
class Account {
  public owner: string;
  private pin: string;
  #balance = 100;          // a real JavaScript private field
  constructor(owner: string, pin: string) {
    this.owner = owner;
    this.pin = pin;
  }
}
const acc = new Account("Sara", "1234");
console.log(Object.keys(acc));    // [ 'owner', 'pin' ]
console.log(JSON.stringify(acc)); // {"owner":"Sara","pin":"1234"}
console.log(acc["pin"]);          // 1234 (bracket access is allowed by TypeScript)
```

اگر **حریم خصوصی واقعی در زمان اجرا** بخواهی، از فیلدهای `#` خودِ JavaScript استفاده کن: `#balance` نه در `Object.keys` دیده می‌شود، نه در `JSON.stringify`، و نوشتن `acc.#balance` بیرون از کلاس هم در TypeScript خطای **TS18013** دارد و هم در JavaScript خالی `SyntaxError` است.

	`private` (TypeScript)	`#field` (JavaScript)
چه کسی بررسی می‌کند؟	فقط کامپایلر TypeScript	خودِ موتور JavaScript
در JS خروجی	property معمولی	فیلد خصوصی واقعی
`Object.keys` / `JSON.stringify`	دیده می‌شود	دیده نمی‌شود

`readonly`: فقط‌خواندنی

propertyای که `readonly` باشد **فقط دو جا** می‌تواند مقدار بگیرد: کنار اعلانش یا داخل constructor. هر جای دیگری (حتی داخل یک متد از همان کلاس) خطای **TS2540** می‌دهد.

TypeScript

```
class Config {
  readonly apiUrl: string;
  readonly retries = 3;
  constructor(url: string) {
    this.apiUrl = url;      // OK: inside the constructor
  }
  setUrl(url: string) {
    this.apiUrl = url;      // Error TS2540: Cannot assign to 'apiUrl' because it is a read-only
property.
  }
}
const cfg = new Config("https://api.example.com");
cfg.retries = 5;            // Error TS2540: Cannot assign to 'retries' because it is a read-only
property.
```

Parameter properties: خلاصه‌نویسی در constructor

الگوی «اعلان property + گرفتن پارامتر + `this.x = x`» آن‌قدر رایج است که TypeScript یک میان‌بر دارد: اگر **جلوی پارامتر constructor یک modifier** (`public`، `private`، `protected` یا `readonly`) بنویسی، آن پارامتر خودکار به property تبدیل و مقداردهی می‌شود. این دو کلاس دقیقاً یک کار می‌کنند:

```TypeScript
class UserLong {
  private name: string;
  public age: number;
  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }
}
class UserShort {
  constructor(private name: string, public age: number) {}
}
```

⚠ دام تستی: پارامتر بدون modifier، property نیست

در `constructor(public name: string, age: number) {}` فقط `name` property می‌شود. `age` یک پارامتر معمولی است که بعد از تمام شدن constructor از بین می‌رود؛ پس `this.age` در یک متد خطای **TS2339** (`Property 'age' does not exist`) می‌دهد.

`static` : عضوِ خودِ کلاس، نه شیء

عضو `static` به **خودِ کلاس** تعلق دارد، نه به تک‌تک اشیاء؛ همه‌ی اشیاء یک نسخه‌ی مشترک از آن دارند. آن را با **اسم کلاس** صدا می‌زنیم:

```TypeScript
class Counter {
  static count = 0;
  id: number;
  constructor() {
    Counter.count++;          // shared by all instances
    this.id = Counter.count;
  }
}
const a = new Counter();
const b = new Counter();
console.log(a.id, b.id, Counter.count); // 1 2 2
console.log(a.count);  // Error TS2576: Did you mean to access the static member 'Counter.count' instead?
```

نکته: کلاس TypeScript بعد از کامپایل همان کلاس JavaScript است

نوع‌ها، `implements` ، `public` / `private` / `protected` و `readonly` فقط برای بررسی در زمان کامپایل هستند و در JavaScript خروجی اثری از آن‌ها نمی‌ماند. ولی `extends` ، `super` ، `static` ، متدها و فیلدهای `#` امکانات خودِ JavaScript هستند و در زمان اجرا هم کار می‌کنند.

⚠ دام تستی: گم شدن `this` وقتی متد را جدا می‌کنی

اگر متد را از شیء جدا کنی و تنها صدا بزنی (`const f = p.greet; f();`)، دیگر شیئی قبل از نقطه نیست و `this` برابر `undefined` می‌شود (بدنه‌ی کلاس همیشه در strict mode اجرا می‌شود). نتیجه: `TypeError: Cannot read properties of undefined`. TypeScript این اشتباه را **نمی‌گیرد** و کد بدون خطا کامپایل می‌شود.

جمع‌بندی

- کلاس = نقشه؛ object = نمونه‌ای که با `new` ساخته می‌شود. constructor هنگام `new` اجرا می‌شود.
- در strict هر property باید کنار اعلان یا در constructor مقدار بگیرد (وگرنه TS2564).
- `implements` = تعهد به یک interface (TS2420 اگر عضوی کم باشد، TS2416 اگر نوعی ناسازگار باشد).
- `extends` = وراثت؛ constructor فرزند باید قبل از `this`، `super(...)` را صدا بزند (TS17009/TS2377). بازنویسی متد + `super.method()` + `override` (TS4113).
- `public` (پیش‌فرض) همه‌جا؛ `protected` خود و فرزندان (بیرون: TS2445)؛ `private` فقط خودِ کلاس (TS2341) و آن هم فقط در زمان کامپایل؛ حریم واقعی = `#`.
- `readonly` فقط کنار اعلان یا در parameter property = modifier constructor (TS2540). جلوی پارامتر constructor. `static` با اسم کلاس صدا زده می‌شود.

تست‌های تمرینی

۲۵ تست · شماره‌ی ۲۲۸ تا ۲۵۲

۲۲۸ آسان

کد زیر همان کد اسلاید ۲۱ است. اگر این فایل را کامپایل و اجرا کنیم، چه چیزی چاپ می‌شود؟

```TypeScript
class Person {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
  greet() {
    console.log(`Hello, ${this.name}`);
  }
}
```

۱) `Hello, undefined`
۲) `Hello, name`
۳) (بدون خروجی)
۴) `Uncaught ReferenceError`

۲۲۹ آسان

خروجی کد زیر چیست؟

```typescript
class Person {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
  greet() {
    console.log(`Hello, ${this.name}`);
  }
}
const p = new Person("Alice");
p.greet();
```

۱) `Hello, Alice`
۲) `Hello, ${this.name}`
۳) `Hello, undefined`
۴) `Hello, name`

۲۳۰ آسان

اگر برای یک property یا method در کلاس TypeScript هیچ access modifierی ننویسیم، آن عضو چه وضعیتی دارد؟

۱) `private` است و فقط داخل همان کلاس قابل استفاده است.
۲) `protected` است و فقط در همان کلاس و کلاس‌های فرزند قابل استفاده است.
۳) `readonly` است و بعد از ساخته شدن شیء قابل تغییر نیست.
۴) `public` است و از همه‌جا (حتی بیرون کلاس) قابل دسترسی است.

۲۳۱ آسان

کدام گزینه درباره‌ی access modifierهای TypeScript **درست** است؟

۱) عضو `private` در کلاس فرزند (کلاسی که با `extends` از آن ارث برده) قابل استفاده است.
۲) عضو `protected` داخل کلاس‌های فرزند قابل استفاده است، ولی از بیرون کلاس (روی شیء) نه.
۳) عضو `protected` را از بیرون کلاس می‌توان خواند، ولی نمی‌توان تغییر داد.
۴) نوشتن `public` اجباری است و اگر آن را ننویسیم خطای کامپایل می‌گیریم.

۲۳۲ آسان

زیر کد اسلاید ۲۱ نوشته شده «Just like JS, but better!». کدام قابلیت زیر در کلاس‌های JavaScript معمولی **وجود ندارد** و TypeScript آن را اضافه کرده است؟

۱) تابع ویژه‌ی `constructor` برای مقداردهی اولیه‌ی شیء
۲) وراثت با کلیدواژه‌ی `extends`
۳) کلیدواژه‌های `public`، `private` و `protected` برای تعیین سطح دسترسی
۴) تعریف متد (مثل `greet()`) داخل بدنه‌ی کلاس

۲۳۳ آسان

می‌خواهیم کلاس `Student` از `Person` **ارث ببرد** تا property `name` را از آن بگیرد و کد بدون خطا کامپایل شود. در جای خالی کدام کلیدواژه قرار می‌گیرد؟

```TypeScript
class Person {
  constructor(public name: string) {}
}
class Student ____ Person {}
const s = new Student("Sara");
console.log(s.name);
```

۱) `extends`
۲) `implements`
۳) `inherits`
۴) `super`

۲۳۴ آسان

نوشتن `class Person implements Greeter` (که `Greeter` یک interface است) چه کاری انجام می‌دهد؟

۱) TypeScript هنگام کامپایل بررسی می‌کند که کلاس همه‌ی اعضای `Greeter` را با نوع درست داشته باشد.

۲) کد متدهای `Greeter` به‌طور خودکار به کلاس `Person` اضافه می‌شود.

۳) در زمان اجرا (runtime) بررسی می‌شود که هر شیء ساخته‌شده شکل `Greeter` را داشته باشد.

۴) دقیقاً مثل `extends` است و `Person` از `Greeter` ارث می‌برد.

۲۳۵ متوسط

در حالت strict، کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
1  class Product {
2    title: string;
3    price: number = 0;
4    stock?: number;
5    constructor(title: string) {
6      this.title = title;
7    }
8  }
9  class Order {
10   id: number;
11   items: string[] = [];
12 }
```

۱) خط ۲
۲) خط ۴
۳) خط ۱۰
۴) خط ۱۱

۲۳۶ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
class Person {
  public name: string;
  private secret: string;
  protected age: number;
  constructor(name: string, secret: string, age: number) {
    this.name = name;
    this.secret = secret;
    this.age = age;
  }
}
class Student extends Person {
  info() {
    return `${this.name} (${this.age})`;
  }
}
const s = new Student("Sara", "x", 20);
console.log(s.name);
console.log(s.info());
console.log(s.age);
```

۱) خط ۱۳
۲) خط ۱۷
۳) خط ۱۸
۴) خط ۱۹

۲۳۷ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
class Account {
  private balance = 0;
  protected owner = "Ali";
  deposit(amount: number) {
    this.balance += amount;
  }
}
class Savings extends Account {
  addInterest() {
    this.balance *= 1.1;
  }
  whoOwns() {
    return this.owner;
  }
}
const acc = new Savings();
acc.deposit(100);
```

۱) خط ۱۰
۲) خط ۱۳
۳) خط ۱۷
۴) هیچ خطایی رخ نمی‌دهد

۲۳۸ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
class Config {
  readonly apiUrl: string;
  readonly retries = 3;
  constructor(url: string) {
    this.apiUrl = url;
  }
  setUrl(url: string) {
    this.apiUrl = url;
  }
}
const cfg = new Config("https://api.example.com");
console.log(cfg.apiUrl, cfg.retries);
```

۱) خط ۵
۲) خط ۸
۳) خط ۱۱
۴) هیچ خطایی رخ نمی‌دهد

۲۳۹ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
interface Shape {
  name: string;
  area(): number;
}
class Square implements Shape {
  name = "square";
  constructor(public side: number) {}
  area(): number {
    return this.side * this.side;
  }
}
class Circle implements Shape {
  name = "circle";
  constructor(public r: number) {}
}
```

۱) خط ۵
۲) خط ۷
۳) خط ۸
۴) خط ۱۲

۲۴۰ متوسط

خروجی کد زیر چیست؟

```TypeScript
class Counter {
  static count = 0;
  id: number;
  constructor() {
    Counter.count++;
    this.id = Counter.count;
  }
}
const a = new Counter();
const b = new Counter();
const c = new Counter();
console.log(a.id, c.id, Counter.count);
```

۱) `1 1 1`
۲) `1 3 3`
۳) `3 3 3`
۴) `0 2 3`

۲۴۱ متوسط

خروجی کد زیر چیست؟

```TypeScript
class Point {
  constructor(public x: number, private y: number, readonly z: number = 0) {}
  sum() {
    return this.x + this.y + this.z;
  }
}
const pt = new Point(1, 2);
console.log(pt.x, pt.z, pt.sum());
console.log(JSON.stringify(pt));
```

۱) `1 0 3` `{"x":1,"z":0}`
۲) `1 undefined NaN` `{}`
۳) `1 0 3` `{}`
۴) `1 0 3` `{"x":1,"y":2,"z":0}`

۲۴۲ متوسط

در حالت strict، کدام کلاس **بدون خطا** کامپایل می‌شود؟

۱
```TypeScript
class A {
  name: string;
}
```

۲
```TypeScript
class B {
  name: string;
  constructor(n?: string) {
    if (n) this.name = n;
  }
}
```

۳
```TypeScript
class C {
  name: string = "";
}
```

۴
```TypeScript
class D {
  name: string;
  init() {
    this.name = "x";
  }
}
```

۲۴۳ متوسط

کدام گزینه درباره‌ی `readonly` در کلاس‌های TypeScript **نادرست** است؟

۱ propertyای که `readonly` است را می‌توان همان‌جا که اعلان می‌شود مقدار داد.

۲ propertyای که `readonly` است را می‌توان داخل constructor مقدار داد.

۳ اگر یک property `readonly` آرایه باشد، باز هم می‌توان با `push` عضو جدیدی به همان آرایه اضافه کرد.

۴ propertyای که `readonly` است را می‌توان داخل یک متد معمولی از همان کلاس دوباره مقداردهی کرد.

۲۴۴ متوسط

کدام دستور را می‌توان در جای خالی نوشت تا کد **بدون خطا** کامپایل شود و شمارنده‌ی `static` یکی زیاد شود؟

```TypeScript
class Counter {
  static count = 0;
}
const c = new Counter();
// ____
```

۱) `Counter.count++;`
۲) `c.count++;`
۳) `this.count++;`
۴) `count++;`

۲۴۵ سخت

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
1  class Animal {
2    constructor(public name: string) {}
3  }
4  class Dog extends Animal {
5    breed: string;
6    constructor(name: string, breed: string) {
7      this.breed = breed;
8      super(name);
9    }
10 }
11 const d = new Dog("Rex", "Husky");
```

۱) خط ۵
۲) خط ۷
۳) خط ۸
۴) هیچ خطایی رخ نمی‌دهد

۲۴۶ سخت

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
1  class User {
2    constructor(public name: string, age: number) {}
3    describe() {
4      return `${this.name} is ${this.age} years old`;
5    }
6  }
7  const u = new User("Ali", 20);
8  console.log(u.name);
```

۱) خط ۲
۲) خط ۴
۳) خط ۷
۴) خط ۸

۲۴۷ سخت

با کلاس `Animal` زیر، کدام تعریف **خطای کامپایل** دارد؟

```TypeScript
class Animal {
  speak(): string {
    return "...";
  }
}
```

۱) `class Dog extends Animal { speak(): string { return "Woof"; } }`

۲) `class Cat extends Animal { override speak(): string { return "Meow"; } }`

۳) `class Bird extends Animal { fly(): string { return "flying"; } }`

۴) `class Fish extends Animal { override swim(): string { return "swimming"; } }`

۲۴۸ سخت

با کلاس `Animal` زیر، کدام تعریف **خطای کامپایل** دارد؟

```TypeScript
class Animal {
  speak(): string {
    return "...";
  }
}
```

۱) `class Fish extends Animal { speak(): number { return 0; } }`

۲) `class Dog extends Animal { speak(): string { return "Woof"; } }`

۳) `class Cow extends Animal { speak() { return "Moo"; } }`

۴) `class Cat extends Animal { override speak(): string { return super.speak() + "!"; } }`

۲۴۹ سخت

خروجی کد زیر چیست؟

```typescript
class Animal {
  constructor(public name: string) {}
  speak(): string {
    return `${this.name} makes a sound`;
  }
}
class Dog extends Animal {
  speak(): string {
    return `${super.speak()} - Woof`;
  }
}
const pets: Animal[] = [new Animal("Generic"), new Dog("Rex")];
for (const p of pets) console.log(p.speak());
```

۱) `Generic makes a sound` `Rex makes a sound`

۲) `Generic makes a sound` `Rex makes a sound - Woof`

۳) `Generic makes a sound` `Woof`

۴) `Generic makes a sound` `Uncaught TypeError`

۲۵۰ سخت

خروجی کد زیر چیست؟

```typescript
class Account {
  public owner: string;
  private pin: string;
  #balance = 100;
  constructor(owner: string, pin: string) {
    this.owner = owner;
    this.pin = pin;
  }
}
const acc = new Account("Sara", "1234");
console.log(Object.keys(acc));
console.log(JSON.stringify(acc));
```

۱) `['owner']` `{"owner":"Sara"}`

۲) `['owner', 'pin', '#balance']` `{"owner":"Sara","pin":"1234","#balance":100}`

۳) `['owner', 'pin']` `{"owner":"Sara","pin":"1234"}`

۴) `['owner', 'pin', 'balance']` `{"owner":"Sara","pin":"1234","balance":100}`

۲۵۱ سخت

کد زیر بدون خطا کامپایل می‌شود. خروجی آن در زمان اجرا چیست؟

```typescript
class Person {
  constructor(public name: string) {}
  greet() {
    console.log(`Hello, ${this.name}`);
  }
}
const p = new Person("Alice");
p.greet();
const f = p.greet;
f();
```

۱) `Hello, Alice` `Hello, Alice`

۲) `Hello, Alice` `Hello, undefined`

۳) `Uncaught TypeError`

۴) `Hello, Alice` `Uncaught TypeError`

۲۵۲ سخت

کدام گزینه درباره‌ی `private` در TypeScript و فیلدهای `#` در JavaScript **درست** است؟

۱) `private` باعث می‌شود آن property در کد JavaScript خروجی حذف شود.

۲) فیلد `#balance` در خروجی `Object.keys` دیده می‌شود، ولی property `private` دیده نمی‌شود.

۳) `private` فقط هنگام کامپایل بررسی می‌شود؛ برای پنهان ماندن واقعی در زمان اجرا باید از فیلدهای `#` خودِ JavaScript استفاده کرد.

۴) اگر در زمان اجرا از بیرون کلاس به یک property `private` دسترسی پیدا کنیم (مثلاً با `obj["pin"]`)، خطای `TypeError` رخ می‌دهد.

۳-۶ Generics در TypeScript

منبع: NodeJS & TS، اسلاید ۲۲

درسنامه

یک تشبیه: جعبه‌ی برچسب‌دار

یک تشبیه ساده

فرض کن یک کارخانه‌ی جعبه‌سازی داری. نمی‌خواهی برای کتاب، کفش و لیوان سه خط تولید جدا بسازی؛ یک **جعبه‌ی عمومی** می‌سازی و روی هر جعبه یک **برچسب** می‌زنی: «جعبه‌ی کتاب» یا «جعبه‌ی کفش». وقتی برچسب «کتاب» خورد، فقط کتاب داخلش می‌رود و وقتی بازش کنی مطمئنی کتاب بیرون می‌آید. **Generics** همین است: یک کد عمومی که با یک «برچسب نوع» برای هر نوع داده‌ای کار می‌کند، بدون این‌که TypeScript نوع را گم کند.

پارامتر معمولی تابع، جای خالی برای یک **مقدار** است. **generic** (عام) یعنی تابع، کلاس یا interface یک جای خالی هم برای **نوع** داشته باشد که هنگام استفاده پر می‌شود.

مشکل: یا تکرار کد، یا گم شدن نوع

فرض کن تابعی می‌خواهیم که هر چه بگیرد، همان را برگرداند. بدون generics دو راه بد داریم:

TypeScript

```
// Way 1: one copy per type (code duplication)
function identityNumber(value: number): number {
  return value;
}
function identityString(value: string): string {
  return value;
}

// Way 2: any (type information is lost)
function identityAny(value: any): any {
  return value;
}
const y = identityAny("hi");  // y: any
console.log(y.length);        // 2
console.log(y.toFixed(2));    // compiles, but crashes at runtime: TypeError: y.toFixed is not a function
```

راه اول برای هر نوع یک نسخه‌ی تکراری می‌خواهد. راه دوم از `any` (نوعی که یعنی «هر چیزی») و بررسی نوع را خاموش می‌کند) استفاده می‌کند: TypeScript دیگر نمی‌داند `y` رشته است، پس خطای `y.toFixed` را نمی‌گیرد و برنامه در زمان اجرا می‌ترکد.

راه‌حل: تابع generic (کد اسلاید ۲۲)

weee.ts

```ts
function identity<T>(value: T): T
{
  return value;
}

let num = identity<number>(42);
let str = identity<string>("Hello");
```

- `<T>` یک **type parameter** (پارامتر نوع) است: جای خالی برای یک نوع. اسمش دلخواه است؛ رسم است از `T` (اول کلمه‌ی Type) استفاده شود.
- `(value: T): T` یعنی «ورودی از نوع `T` است و خروجی **همان** نوع `T`». نوع خروجی به نوع ورودی **وصل** است.
- در `identity<number>(42)`، عبارت `<number>` یک **type argument** (آرگومان نوع) است: این‌جا `T` برابر `number` می‌شود، پس `num` از نوع `number` است. به همین ترتیب `str` از نوع `string` است.
- کد اسلاید هیچ `console.log` ی ندارد، پس **خروجی ندارد**. اگر `console.log(num, str, typeof num, typeof str);` را اضافه کنیم، `42 Hello number string` چاپ می‌شود.

فراخوانی	`T` چه می‌شود؟	نوع نتیجه
`identity<number>(42)`	`number`	`number`
`identity<string>("Hello")`	`string`	`string`
`identity<boolean>(true)`	`boolean`	`boolean`

generics فقط برای بررسی در زمان کامپایل هستند؛ در JavaScript خروجی اثری از آن‌ها نمی‌ماند:

JavaScript

```js
function identity(value) {
    return value;
}
let num = identity(42);
let str = identity("Hello");
```

چهار نکته‌ی اسلاید ۲۲ حالا معنا پیدا می‌کنند:

1. generics امکان ساختن کامپوننت‌ها و تابع‌های **قابل استفاده‌ی دوباره (reusable)** و **type-safe** (ایمن از نظر نوع) را می‌دهند: یک `identity` برای همه‌ی نوع‌ها.
2. روی **تابع‌ها، کلاس‌ها و interfaceها** به کار می‌روند (پایین‌تر هر سه را می‌بینیم).
3. **انعطاف** می‌دهند و در عین حال **ایمنی نوع** را حفظ می‌کنند.
4. با کار کردن روی هر نوع داده، **جلوی تکرار کد** را می‌گیرند **بدون این‌که بررسی نوع از دست برود** (برخلاف `any`).

بررسی نوع کاملاً برقرار است:

```TypeScript
function identity<T>(value: T): T {
  return value;
}
let num = identity<number>(42);
let str = identity<string>("Hello");
let wrong = identity<number>("Hello"); // Error TS2345: Argument of type 'string' is not assignable to parameter of type 'number'.
num = str;                             // Error TS2322: Type 'string' is not assignable to type 'number'.
```

استنتاج نوع (type inference): لازم نیست همیشه `<number>` بنویسی

اگر آرگومان نوع را ننویسی، TypeScript آن را از روی آرگومان‌های تابع **استنتاج** (حدس دقیق) می‌کند. یک ظرافت مهم: نتیجه به `let` و `const` بستگی دارد.

```TypeScript
function identity<T>(value: T): T {
  return value;
}
let a = identity(42);          // a: number
const b = identity(42);        // b: 42 (a literal type)
let c = identity("Hello");     // c: string
const d = identity("Hello");   // d: "Hello"
a = 100;                       // OK: a is a number
let e: 42 = b;                 // OK: b is exactly 42
let f: 42 = a;                 // Error TS2322: Type 'number' is not assignable to type '42'.
```

چرا؟ متغیر `let` بعداً می‌تواند عوض شود، پس TypeScript نوع را **گسترده** (`number`) در نظر می‌گیرد. اما `const` هرگز عوض نمی‌شود، پس TypeScript همان **نوع دقیق (literal type)** `42` را نگه می‌دارد (مثل `const PI = 3.14` که نوعش `3.14` است).

آرگومان نوع را وقتی صریح بنویس که نوعی **گسترده‌تر** از حدس TypeScript بخواهی:

```TypeScript
function identity<T>(value: T): T {
  return value;
}
let g = identity<number | string>(5); // g: string | number
g = "five";                           // OK
```

چرا generic بهتر از `any` است؟

	`any`	`<T>` **generic**
نوع خروجی	`any` (اطلاعات نوع گم می‌شود)	همان نوع ورودی
اشتباهی مثل `toFixed` روی رشته	در **زمان اجرا** (`TypeError`)	در **زمان کامپایل**
پیشنهاد خودکار (autocomplete) ویرایشگر	ندارد	دارد

```TypeScript
function identity<T>(value: T): T {
  return value;
}
const x = identity("hi");
x.toUpperCase();  // OK: x is a string
x.toFixed(2);     // Error TS2551: Property 'toFixed' does not exist on type '"hi"'.
```

آرایه‌ها: `Array<T>`

همین حالا هم بدون این‌که بدانی از generics استفاده می‌کردی: `number[]` همان `Array<number>` است.

```TypeScript
let nums: Array<number> = [1, 2, 3];   // same as number[]
nums.push(4);                          // OK
nums.push("5");                        // Error TS2345: Argument of type 'string' is not assignable
to parameter of type 'number'.

function firstItem<T>(items: T[]): T | undefined {
  return items[0];
}
const h = firstItem([10, 20]);    // h: number | undefined
const i = firstItem(["x", "y"]);  // i: string | undefined
```

(`Promise` فصل اول هم generic است: `Promise<string>` یعنی Promiseای که در پایان یک `string` می‌دهد.)

interface generic: `Box<T>`

```TypeScript
interface Box<T> {
  value: T;
}
const b1: Box<string> = { value: "hi" };
const b2: Box<number[]> = { value: [1, 2] };
const b3: Box<number> = { value: "hi" };  // Error TS2322: Type 'string' is not assignable to type
'number'.
const b4: Box = { value: 1 };             // Error TS2314: Generic type 'Box<T>' requires 1 type
argument(s).
```

چند پارامتر نوع: `<K, V>`

می‌توان بیش از یک پارامتر نوع داشت؛ رسم است برای کلید (Key) از `K` و برای مقدار (Value) از `V` استفاده شود:

TypeScript

```
interface Pair<K, V> {
  key: K;
  value: V;
}
const p1: Pair<string, number> = { key: "age", value: 30 };    // OK
const p2: Pair<string, number> = { key: "age", value: "30" };  // Error TS2322: Type 'string' is
not assignable to type 'number'.

function makePair<K, V>(key: K, value: V): Pair<K, V> {
  return { key, value };
}
const p3 = makePair("id", 7);                    // Pair<string, number>
const p4 = makePair<string, number>("id", "7");  // Error TS2345: Argument of type 'string' is not
assignable to parameter of type 'number'.
```

کلاس generic: `Stack<T>`

Stack (پشته) مثل یک دسته بشقاب است: آخرین بشقابی که می‌گذاری، اولین بشقابی است که برمی‌داری (LIFO: Last In, First Out). یک بار کلاس را می‌نویسیم و برای هر نوعی استفاده می‌کنیم:

TypeScript

```
class Stack<T> {
  private items: T[] = [];
  push(item: T): void {
    this.items.push(item);
  }
  pop(): T | undefined {
    return this.items.pop();
  }
  size(): number {
    return this.items.length;
  }
}
const s = new Stack<string>();
s.push("a");
s.push("b");
s.push("c");
console.log(s.pop(), s.size());  // c 2
const n = new Stack<number>();
console.log(n.pop(), n.size());  // undefined 0
n.push(1);                        // OK
n.push("2");                      // Error TS2345: Argument of type 'string' is not assignable to
parameter of type 'number'.
```

`pop` نوع `T | undefined` برمی‌گرداند، چون پشته ممکن است خالی باشد (`pop` آرایه‌ی خالی `undefined` می‌دهد).

قید (constraint) با `extends`

داخل یک تابع generic، TypeScript هیچ چیزی درباره‌ی `T` نمی‌داند (ممکن است عدد باشد، ممکن است رشته). پس اجازه نمی‌دهد به property خاصی دست بزنی:

```TypeScript
function size<T>(x: T): number {
  return x.length;  // Error TS2339: Property 'length' does not exist on type 'T'.
}
```

راه‌حل، **قید** گذاشتن روی `T` است: `T extends { length: number }` یعنی «`T` هر نوعی می‌تواند باشد، **به شرط این‌که** یک `length` عددی داشته باشد».

```TypeScript
function logLength<T extends { length: number }>(x: T): T {
  console.log(x.length);
  return x;
}
logLength("hello");        // 5
logLength([1, 2, 3]);      // 3
logLength({ length: 7 });  // 7
logLength(42);             // Error TS2345: Argument of type 'number' is not assignable to
parameter of type '{ length: number; }'.
```

چون خروجی `T` است، `logLength("hello")` همچنان یک `string` برمی‌گرداند و می‌توانی روی نتیجه `toUpperCase()` را صدا بزنی.

«You remember generic from Java, right?»

زیر کد اسلاید ۲۲ نوشته «generic را از Java یادت هست، نه؟». ایده دقیقاً همان است:

	Java	TypeScript
نوشتن	`List<Integer>` ، `class Box<T>`	`Array<number>` ، `class Box<T>`
استنتاج آرگومان نوع	محدودتر (مثلاً در `new ArrayList<>()`)	قوی؛ معمولاً لازم نیست بنویسی
نوع‌های پایه	`List<int>` مجاز نیست (باید `Integer` نوشت)	`Array<number>` مجاز است
در زمان اجرا	type erasure (اطلاعات generic پاک می‌شود)	کاملاً پاک می‌شود؛ در JS خروجی هیچ اثری نیست

نکته: اسم پارامتر نوع و شکل‌های دیگر

- اسم پارامتر نوع دلخواه است: `T` (Type)، `K` (Key)، `V` (Value)، `E` (Element) یا حتی اسم توصیفی مثل `TItem` .
- arrow function هم می‌تواند generic باشد: `const identity2 = <T>(value: T): T => value;` (در فایل `.ts` معمولی).
- generic روی تابع، کلاس و interface یک شکل دارد: `<...>` بعد از اسم.

دام تستی: generic مقدار را تبدیل نمی‌کند

`identity<number>("5")` رشته‌ی `"5"` را به عدد `5` تبدیل **نمی‌کند**؛ فقط یک خطای کامپایل (TS2345) است.

generics فقط بررسی نوع در زمان کامپایل‌اند و در زمان اجرا وجود ندارند؛ پس `typeof T` یا هر بررسی دیگری روی `T` در زمان اجرا ممکن نیست.

⚠ دام تستی: `let` یا `const`؟ و معنای `extends`

- `let a = identity(42)` نوع `number` می‌گیرد، ولی `const b = identity(42)` نوع دقیق `42` .
- `extends` در `<T extends { length: number }>` به معنی **قید** است («باید این شکل را داشته باشد»)، نه وراثت کلاس.

جمع‌بندی

- generic = جای خالی برای **نوع**: `function identity<T>(value: T): T` .
- `<number>` در `identity<number>(42)` آرگومان نوع است؛ اگر ننویسی، استنتاج می‌شود (`let` → `number` ، `const` → `42`).
- آرگومان ناسازگار با آرگومان نوع صریح → generic .TS2345 اطلاعات نوع را حفظ می‌کند، `any` نه.
- روی تابع (`identity<T>`)، interface (`Box<T>` ، `Pair<K, V>`) و کلاس (`Stack<T>`) به کار می‌رود؛ `Array<T>` همان `T[]` است.
- قید: `<T extends { length: number }>` ؛ بدون قید، `x.length` خطای TS2339 دارد.
- در JavaScript خروجی هیچ اثری از generics نمی‌ماند.

تست‌های تمرینی

۲۱ تست · شماره‌ی ۲۵۳ تا ۲۷۳

۲۵۳ آسان

طبق اسلاید ۲۲، مهم‌ترین فایده‌ی generics چیست؟

۱) نوشتن کدی که برای انواع مختلف داده دوباره قابل استفاده است، **بدون** از دست رفتن بررسی نوع

۲) سریع‌تر شدن اجرای برنامه در زمان اجرا

۳) خاموش کردن بررسی نوع برای انعطاف بیشتر، درست مثل `any`

۴) تبدیل خودکار مقدارها به نوع `T` در زمان اجرا

۲۵۴ آسان

در تعریف `function identity<T>(value: T): T` ، `T` چیست؟

۱) اسم کلاسی که باید از قبل جایی تعریف شده باشد.

۲) اسم دیگری برای همان پارامتر `value` .

۳) یک **پارامتر نوع** (type parameter)؛ جای خالی برای نوعی که هنگام فراخوانی مشخص می‌شود.

۴) کلیدواژه‌ای هم‌معنی `any` که بررسی نوع را خاموش می‌کند.

۲۵۵ آسان

خروجی کد زیر (کد اسلاید ۲۲ به همراه یک `console.log`) چیست؟

```typescript
function identity<T>(value: T): T
{
  return value;
}

let num = identity<number>(42);
let str = identity<string>("Hello");
console.log(num, str, typeof num, typeof str);
```

۱) `42 Hello T T`
۲) `42 Hello number string`
۳) `42 Hello object object`
۴) `42 Hello any any`

۲۵۶ آسان

طبق اسلاید ۲۲، generics را روی چه چیزهایی می‌توان به کار برد؟

۱) فقط روی تابع‌ها
۲) فقط روی کلاس‌ها (مثل Java)
۳) فقط روی interfaceها
۴) روی تابع‌ها، کلاس‌ها و interfaceها

۲۵۷ آسان

کدام خط کد زیر خطای کامپایل دارد؟

```typescript
function identity<T>(value: T): T {
  return value;
}
let num = identity<number>(42);
let str = identity<string>("Hello");
let wrong = identity<number>("Hello");
let auto = identity(true);
```

۱) خط ۴
۲) خط ۵
۳) خط ۶
۴) خط ۷

۲۵۸ آسان

می‌خواهیم `names` آرایه‌ای از رشته‌ها باشد (معادل `string[]`) و کد بدون خطا کامپایل شود. در جای خالی کدام گزینه قرار می‌گیرد؟

```typescript
let names: ____ = ["Ali", "Sara"];
```

۱) `Array[string]`
۲) `Array<string>`
۳) `Array(string)`
۴) `string<Array>`

۲۵۹ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
function identity<T>(value: T): T {
  return value;
}
let a = identity(42);
const b = identity(42);
a = 100;
let c: 42 = b;
let d: 42 = a;
```

۱) خط ۵
۲) خط ۶
۳) خط ۷
۴) خط ۸

۲۶۰ متوسط

TypeScript برای متغیر `s` در کد زیر چه نوعی استنتاج می‌کند؟

```TypeScript
function identity<T>(value: T): T {
  return value;
}
let s = identity("Hello");
```

۱) `string`
۲) `"Hello"`
۳) `any`
۴) `T`

۲۶۱ متوسط

خروجی کد زیر چیست؟

```typescript
class Stack<T> {
  private items: T[] = [];
  push(item: T): void {
    this.items.push(item);
  }
  pop(): T | undefined {
    return this.items.pop();
  }
  size(): number {
    return this.items.length;
  }
}
const s = new Stack<string>();
s.push("a");
s.push("b");
s.push("c");
console.log(s.pop(), s.size());
const e = new Stack<number>();
console.log(e.pop(), e.size());
```

۱) `a 2` `undefined 0`

۲) `c 3` `undefined 0`

۳) `c 2` `undefined 0`

۴) `c 2` `Uncaught TypeError`

۲۶۲ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

```typescript
class Stack<T> {
  private items: T[] = [];
  push(item: T): void {
    this.items.push(item);
  }
  pop(): T | undefined {
    return this.items.pop();
  }
}
const s = new Stack<number>();
s.push(1);
s.push(2 * 3);
s.push("4");
const top = s.pop();
```

۱) خط ۱۰

۲) خط ۱۲

۳) خط ۱۳

۴) خط ۱۴

۲۶۳ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

```typescript
function logLength<T extends { length: number }>(x: T): T {
  console.log(x.length);
  return x;
}
logLength("hello");
logLength([1, 2, 3]);
logLength({ length: 7 });
logLength(42);
```

۱) خط ۲ ۲) خط ۶ ۳) خط ۷ ۴) خط ۸

۲۶۴ متوسط

خروجی کد زیر چیست؟

```typescript
function logLength<T extends { length: number }>(x: T): T {
  console.log(x.length);
  return x;
}
const r = logLength("hello");
logLength([1, 2, 3]);
console.log(r.toUpperCase());
```

۱) `5` `3` `HELLO`
۲) `5` `3` `Uncaught TypeError`
۳) `hello` `1,2,3` `HELLO`
۴) `5` `undefined` `HELLO`

۲۶۵ متوسط

کدام گزینه درباره‌ی مقایسه‌ی `function f(x: any): any` با `function f<T>(x: T): T` **درست** است؟

۱) از نظر بررسی نوع هیچ فرقی با هم ندارند؛ هر دو هر ورودی‌ای را می‌پذیرند.

۲) نسخه‌ی generic فقط وقتی کار می‌کند که آرگومان نوع را صریحاً بنویسیم (مثل `f<number>(5)`).

۳) نسخه‌ی generic در زمان اجرا نوع مقدار را بررسی می‌کند و اگر نادرست باشد خطا می‌دهد.

۴) در نسخه‌ی generic نوع خروجی به نوع ورودی وصل می‌ماند؛ ولی با `any` اطلاعات نوع گم می‌شود و بعضی خطاها تا زمان اجرا دیده نمی‌شوند.

۲۶۶ متوسط

با interface زیر، کدام تعریف **خطای کامپایل** دارد؟

```TypeScript
interface Pair<K, V> {
  key: K;
  value: V;
}
```

۱) `const p: Pair<string, number> = { key: "age", value: 30 };`

۲) `const p: Pair<string, number> = { key: "age", value: "30" };`

۳) `const p: Pair<number, string> = { key: 1, value: "one" };`

۴) `const p: Pair<boolean, boolean> = { key: true, value: false };`

۲۶۷ متوسط

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
interface Box<T> {
  value: T;
}
const b1: Box<string> = { value: "hi" };
const b2: Box<number[]> = { value: [1, 2] };
const b3: Box<boolean> = { value: "true" };
const b4: Box<number> = { value: 0 };
```

۱) خط ۴ ۲) خط ۵ ۳) خط ۶ ۴) خط ۷

۲۶۸ سخت

کدام خط کد زیر خطای کامپایل دارد؟

```TypeScript
function first<T>(items: T[]): T | undefined {
  return items[0];
}
function size<T>(x: T): number {
  return x.length;
}
const n = first([10, 20]);
const s = first(["a", "b"]);
```

۱) خط ۲ ۲) خط ۵ ۳) خط ۷ ۴) خط ۸

۲۶۹ سخت

کد زیر **بدون خطا** کامپایل می‌شود. خروجی آن در زمان اجرا چیست؟

```typescript
function identityAny(value: any): any {
  return value;
}
const y = identityAny("hi");
console.log(y.length);
console.log(y.toFixed(2));
console.log("done");
```

۱) `2` `Uncaught TypeError`

۲) `2` `hi` `done`

۳) `2` `NaN` `done`

۴) `2` `undefined` `done`

۲۷۰ سخت

کدام خط کد زیر خطای کامپایل دارد؟

```typescript
function identity<T>(value: T): T {
  return value;
}
function identityAny(value: any): any {
  return value;
}
const x = identity("hi");
const y = identityAny("hi");
console.log(x.toUpperCase());
console.log(y.toFixed(2));
console.log(x.toFixed(2));
```

۱) خط ۹

۲) خط ۱۰

۳) خط ۱۱

۴) هیچ خطایی رخ نمی‌دهد

۲۷۱ سخت

با تابع زیر، کدام فراخوانی **خطای کامپایل** دارد؟

```typescript
function longest<T extends { length: number }>(a: T, b: T): T {
  return a.length >= b.length ? a : b;
}
```

۱) `const r = longest("abc", "de");`

۲) `const r = longest([1, 2], [3]);`

۳) `const r = longest<string | number[]>("abc", [1, 2]);`

۴) `const r = longest("abc", [1, 2]);`

۲۷۲ سخت

زیر کد اسلاید ۲۲ نوشته شده «You remember generic from Java, right?». کدام گزینه درباره‌ی generics در TypeScript **نادرست** است؟

۱) مثل Java، پارامتر یا آرگومان نوع داخل `< >` نوشته می‌شود؛ مثل `List<Integer>` در Java و `Array<number>` در TypeScript.

۲) می‌توان در زمان اجرا با `typeof T` فهمید `T` چه نوعی است، چون اطلاعات generic در JavaScript خروجی باقی می‌ماند.

۳) اگر آرگومان نوع را ننویسیم، کامپایلر معمولاً آن را از روی آرگومان‌های تابع استنتاج می‌کند.

۴) مثل Java می‌توان کلاس generic (مانند `class Box<T>`) تعریف کرد.

۲۷۳ سخت

TypeScript برای متغیر `g` در کد زیر چه نوعی در نظر می‌گیرد؟

```typescript
function identity<T>(value: T): T {
  return value;
}
let g = identity<number | string>(5);
```

۱) `string | number`　۲) `number`　۳) `5`　۴) `any`

بخش پایانی

پاسخنامه‌ی تشریحی

برای هر تست، گزینه‌ی درست همراه با دلیل درستی آن و دلیل نادرستی گزینه‌های وسوسه‌انگیز آمده است. شماره‌ها با شماره‌ی تست‌ها در متن کتاب یکی است؛ روی شماره‌ی هر پاسخ بزنید تا به خود تست برگردید.

پاسخ‌های فصل اول: مقدمه‌ای بر JavaScript

کلید سریع فصل اول

۱	۴	۲	۱	۳	۱	۴	۱	۵	۲
۶	۲	۷	۴	۸	۴	۹	۳	۱۰	۱
۱۱	۲	۱۲	۱	۱۳	۳	۱۴	۲	۱۵	۴
۱۶	۲	۱۷	۳	۱۸	۴	۱۹	۴	۲۰	۲
۲۱	۳	۲۲	۱	۲۳	۳				

۱-۱ متغیرها: let، const و var

۱ گزینه‌ی ۴ — آسان

`const` (مخفف constant) انتساب دوباره را ممنوع می‌کند و هر تلاشی برای آن خطای `TypeError` می‌دهد. `let` و `var` هر دو اجازه می‌دهند بعداً مقدار تازه‌ای در متغیر بریزیم. `final` کلمه‌ی کلیدی زبان Java است و در JavaScript وجود ندارد؛ `final x = 1` خطای `SyntaxError` می‌دهد.

۲ گزینه‌ی ۱ — آسان

اسلاید ۲۳ درباره‌ی `var` می‌گوید: outdated (قدیمی)، function-scoped و «در کد مدرن از آن پرهیز کن». گزینه‌ی ۲ برعکس است: `let` و `const` در ES6 آمدند و از اول زبان وجود داشت. گزینه‌ی ۳ غلط است چون `var` انتساب دوباره (و حتی تعریف دوباره) را اجازه می‌دهد. گزینه‌ی ۴ ویژگی `let` و `const` است، نه `var`.

۳ گزینه‌ی ۱ — آسان

متغیر `let` قابل انتساب دوباره است: اول `21`، بعد `22` و در خط سوم مقدار فعلی (`22`) با ۱ جمع می‌شود و حاصل یعنی `23` دوباره در `age` ریخته می‌شود. گزینه‌ی ۴ مخصوص `const` است و گزینه‌های ۲ و ۳ فرض کرده‌اند انتساب‌های بعدی اثری ندارند.

۴ گزینه‌ی ۱ — آسان

متغیری که تعریف شده ولی هنوز مقدار نگرفته، مقدارش `undefined` است؛ بعد از `score = 10` مقدارش `10` می‌شود. `null` را JavaScript خودش در متغیر نمی‌گذارد (آن را برنامه‌نویس عمداً می‌گذارد) و مقدار پیش‌فرض `0` هم مخصوص زبان‌هایی مثل Java است. `ReferenceError` فقط برای متغیری است که اصلاً تعریف نشده باشد.

۵ گزینه‌ی ۲ — آسان

خط ۴ تلاش می‌کند به `pi` که با `const` تعریف شده مقدار تازه بدهد، پس `TypeError: Assignment to constant variable.` رخ می‌دهد و برنامه متوقف می‌شود. خط ۲ مشکلی ندارد چون `age` با `let` تعریف شده است. خط ۵ هرگز اجرا نمی‌شود، چون برنامه قبلاً روی خط ۴ متوقف شده است.

۶ گزینه‌ی ۲ — آسان

متن بند اسلاید ۲۴: «Use `const` by default, switch to `let` when reassignment is needed». با این روش اگر جایی به‌اشتباه مقدار متغیری را عوض کنی، فوراً `TypeError` می‌گیری. گزینه‌ی ۱ این محافظت را از بین می‌برد، گزینه‌ی ۳ خلاف توصیه‌ی اسلاید ۲۳ (پرهیز از `var`) است و گزینه‌ی ۴ کاربرد `const` را بی‌دلیل محدود می‌کند؛ هر متغیری که قرار نیست دوباره مقدار بگیرد باید `const` باشد.

۷ **گزینه‌ی ۴** متوسط

در گزینه‌ی ۴ به متغیر `const` مقدار تازه داده شده (انتساب دوباره)، پس `TypeError` رخ می‌دهد. در گزینه‌های ۱ و ۲ خود متغیر عوض نشده، فقط **محتوای** شیء یا آرایه‌ای که به آن اشاره می‌کند تغییر کرده و این برای `const` مجاز است. گزینه‌ی ۳ هم یک `let` است که انتساب دوباره‌اش آزاد است.

۸ **گزینه‌ی ۴** متوسط

`const` فقط اجازه نمی‌دهد `nums` به آرایه‌ی **دیگری** وصل شود؛ تغییر محتوای همان آرایه آزاد است. `push(4)` عدد ۴ را به انتها اضافه می‌کند و `nums[0] = 10` خانه‌ی اول را عوض می‌کند، پس `[10, 2, 3, 4]` چاپ می‌شود. گزینه‌ی ۳ همان تصور غلطِ «const یعنی محتوای غیرقابل‌تغییر» است و گزینه‌های ۱ و ۲ فرض کرده‌اند بعضی تغییرها نادیده گرفته می‌شوند.

۹ **گزینه‌ی ۳** سخت

خط ۲ محتوای همان شیء را تغییر می‌دهد که برای `const` مجاز است؛ پس خط ۳ مقدار `light` را چاپ می‌کند. خط ۴ می‌خواهد `settings` را به یک شیء **جدید** وصل کند؛ این انتساب دوباره است و `TypeError` می‌دهد. این خطا در زمان اجرا رخ می‌دهد، پس خروجی خط ۳ قبل از آن چاپ شده است (رد گزینه‌ی ۴). گزینه‌ی ۲ تغییر ویژگی را هم ممنوع فرض کرده و گزینه‌ی ۱ اصلاً ممنوعیت انتساب دوباره را ندیده است.

۱۰ **گزینه‌ی ۱** سخت

`const` باید همان لحظه‌ی تعریف مقدار بگیرد؛ `const b;` در خط ۲ خطای `SyntaxError: Missing initializer in const declaration` می‌دهد. این خطا هنگام خواندن (parse) کل فایل و **قبل از اجرای هر خطی** کشف می‌شود. گزینه‌ی ۲ وسوسه‌انگیز است، چون خط ۳ هم انتساب دوباره به `const` به نظر می‌رسد، ولی برنامه اصلاً به مرحله‌ی اجرا نمی‌رسد.

۱۱ **گزینه‌ی ۲** متوسط

یک دستور تعریف (declaration) مقداری تولید نمی‌کند و کنسول برای چنین دستوری `undefined` نشان می‌دهد. این `undefined` مقدار `a` **نیست**؛ مقدار `a` همان `12` است و اگر `a` را تایپ کنی، `12` می‌بینی. گزینه‌ی ۱ همان تصور غلطِ «کنسول مقدار متغیر را نشان می‌دهد» است و `null` و نام متغیر هم هیچ‌وقت نتیجه‌ی یک تعریف نیستند.

۱۲ **گزینه‌ی ۱** متوسط

انتساب برخلاف تعریف، یک **عبارت** (expression) است و مقدارش همان مقداری است که منتسب شد؛ برای همین کنسول اسلاید ۲۳ جلوی `a = "Hello!"` مقدار `"Hello!"` را نشان داد. نتیجه‌ی دستورهای تعریف مثل `let a = 12` و `undefined` است، `12` مقدار قبلی `a` است و انتساب هیچ‌وقت `true` (به معنای «موفق بود») برنمی‌گرداند.

۱۳ **گزینه‌ی ۳** سخت

انتساب دوباره به `const` در همه‌ی موتورها `TypeError` است و فقط متن پیام فرق می‌کند: SpiderMonkey در Firefox می‌گوید `invalid assignment to const 'b'` و V8 در Chrome و Node.js می‌گوید `Assignment to constant variable.`. گزینه‌ی ۱ غلط است چون Node.js هم جلوی این کار را می‌گیرد. `SyntaxError` برای `const` بدون مقدار اولیه است، نه انتساب دوباره، و `ReferenceError` برای متغیر تعریف‌نشده است، در حالی که `b` تعریف شده است.

۱۴ **گزینه‌ی ۲** متوسط

اسلاید ۲۴: «Both are block-scoped (only exist inside `{ ... }`)». گزینه‌ی ۱ غلط است چون `const` فقط انتساب دوباره را منع می‌کند و `push` یا تغییر خانه‌های آرایه مجاز است. گزینه‌ی ۳ توصیف `const` است، نه `let`. گزینه‌ی ۴ از دو جهت غلط است: `var` قدیمی‌تر است و function-scoped است.

۱۵ **گزینه‌ی ۴** سخت

`var` بلوک `{ }` را نادیده می‌گیرد (function-scoped است)، پس `a` بیرون از بلوک هم وجود دارد و `1` چاپ می‌شود. اما `let` block-scoped است و `b` فقط داخل بلوک وجود دارد؛ خواندن آن بیرون از بلوک `ReferenceError` می‌دهد. چون خطا روی خط آخر رخ می‌دهد، خروجی خط قبل چاپ شده است (رد گزینه‌ی ۲). گزینه‌ی ۱ هر دو را مثل `var` فرض کرده است.

۱۶ **گزینه‌ی ۲** سخت

`let level` داخل بلوک، تعریف دوباره نیست؛ یک متغیر **جدید** است که فقط در همان بلوک زندگی می‌کند و در آن‌جا متغیر بیرونی را می‌پوشاند. داخل بلوک `inner` و بیرون از آن متغیر اصلی یعنی `outer` چاپ می‌شود. `SyntaxError` فقط وقتی رخ می‌داد که هر دو `let` در **یک** محدوده بودند. گزینه‌ی ۱ فرض کرده متغیر داخلی، بیرونی را عوض کرده است.

۱۷ **گزینه‌ی ۳** سخت

`var` تعریف دوباره را بی‌صدا اجازه می‌دهد، پس خط ۲ مشکلی ندارد. خط ۴ انتساب دوباره به `let` است که مجاز است. خط ۵ متغیر `y` را در همان محدوده **دوباره تعریف** می‌کند و `SyntaxError: Identifier 'y' has already been declared` می‌دهد. چون این یک `SyntaxError` است، در واقع هیچ خطی از فایل اجرا نمی‌شود.

۱۸ **گزینه‌ی ۴** سخت

تعریف دوباره‌ی `let total` در همان محدوده یک `SyntaxError` است. خطاهای نحوی هنگام خواندن (parse) فایل و **قبل از اجرای اولین خط** کشف می‌شوند، پس حتی `start` هم چاپ نمی‌شود. گزینه‌ی ۲ فرض کرده خطا در زمان اجرا رخ می‌دهد (مثل `TypeError` انتساب به `const`). گزینه‌های ۱ و ۳ رفتار `var` را در نظر گرفته‌اند که تعریف دوباره را اجازه می‌دهد.

۱۹ **گزینه‌ی ۴** متوسط

نام متغیر می‌تواند حرف، رقم، `_` و `$` داشته باشد، پس `_total$` کاملاً مجاز است. در گزینه‌ی ۱ خط تیره عملگر منها تفسیر می‌شود، در گزینه‌ی ۲ نام با رقم شروع شده و در گزینه‌ی ۳ `class` یک کلمه‌ی رزروشده است؛ هر سه `SyntaxError` می‌دهند.

۲۰ **گزینه‌ی ۲** متوسط

JavaScript به بزرگی و کوچکی حروف حساس است (case-sensitive)؛ پس `username` و `userName` دو متغیر جدا هستند و تعریف دوباره‌ای رخ نداده است (رد گزینه‌ی ۴). خط ۳ فقط `userName` را عوض می‌کند و `username` همان `Reza` می‌ماند. گزینه‌ی ۳ دو نام را یکی فرض کرده است.

۲۱ **گزینه‌ی ۳** آسان

در camelCase کلمه‌ی اول با حرف کوچک و هر کلمه‌ی بعدی با حرف بزرگ شروع می‌شود: `totalOrders` . هر چهار نام از نظر زبان مجازند و خطا نمی‌دهند، ولی `total_orders` سبک snake_case (رایج در Python) است، `TotalOrders` سبک PascalCase است که برای نام کلاس‌ها به کار می‌رود و `totalorders` مرز کلمه‌ها را نشان نمی‌دهد و خواندنش سخت است.

۲۲ **گزینه‌ی ۱** متوسط

در JavaScript نوع به **مقدار** تعلق دارد، نه به متغیر (dynamic typing). وقتی `data` عدد است `typeof` می‌گوید `number` و بعد از انتساب رشته می‌گوید `string` . گزینه‌ی ۲ فرض کرده نوع متغیر هنگام تعریف قفل می‌شود و گزینه‌ی ۳ رفتار زبان‌هایی مثل Java را در نظر گرفته که ریختن رشته در متغیر عددی را منع می‌کنند.

۲۳ **گزینه‌ی ۳** متوسط

خط دوم به `count` مقدار تازه می‌دهد، پس متغیر باید قابل انتساب دوباره باشد: `let` . با `const` همان خط دوم `TypeError` می‌دهد. `int` و `final` کلمه‌های Java هستند و در JavaScript برای تعریف متغیر وجود ندارند؛ هر دو `SyntaxError` می‌دهند.

پاسخ‌های فصل دوم: JavaScript در HTML (کار با DOM)

کلید سریع فصل دوم

۲۴	۳	۲۵	۴	۲۶	۱	۲۷	۲	۲۸	۳	۲۹	۱	۳۰	۲	۳۱	۳	۳۲	۴	۳۳	۱
۳۴	۲	۳۵	۴	۳۶	۳	۳۷	۱	۳۸	۲	۳۹	۳	۴۰	۴	۴۱	۴	۴۲	۲	۴۳	۳
۴۴	۴	۴۵	۲	۴۶	۱	۴۷	۴	۴۸	۲	۴۹	۳	۵۰	۱	۵۱	۴	۵۲	۳	۵۳	۲
۵۴	۱	۵۵	۲	۵۶	۳	۵۷	۴	۵۸	۱	۵۹	۱	۶۰	۳	۶۱	۱	۶۲	۲	۶۳	۳
۶۴	۴	۶۵	۱	۶۶	۳	۶۷	۲	۶۸	۱	۶۹	۳	۷۰	۲	۷۱	۳	۷۲	۳	۷۳	۴
۷۴	۱	۷۵	۴	۷۶	۱	۷۷	۲	۷۸	۴	۷۹	۴	۸۰	۳	۸۱	۲	۸۲	۲	۸۳	۴
۸۴	۱	۸۵	۱	۸۶	۲	۸۷	۳	۸۸	۴	۸۹	۴	۹۰	۲	۹۱	۴	۹۲	۲	۹۳	۱
۹۴	۳	۹۵	۱	۹۶	۱	۹۷	۴	۹۸	۳	۹۹	۳	۱۰۰	۲	۱۰۱	۳	۱۰۲	۴	۱۰۳	۲
۱۰۴	۱	۱۰۵	۲	۱۰۶	۱	۱۰۷	۳	۱۰۸	۴	۱۰۹	۲	۱۱۰	۳	۱۱۱	۱	۱۱۲	۴	۱۱۳	۲
۱۱۴	۳	۱۱۵	۱	۱۱۶	۳	۱۱۷	۴	۱۱۸	۲	۱۱۹	۴	۱۲۰	۱	۱۲۱	۴	۱۲۲	۲	۱۲۳	۱
۱۲۴	۳	۱۲۵	۴	۱۲۶	۲	۱۲۷	۳	۱۲۸	۴	۱۲۹	۲	۱۳۰	۱	۱۳۱	۳	۱۳۲	۲	۱۳۳	۱
۱۳۴	۳	۱۳۵	۴	۱۳۶	۴	۱۳۷	۴	۱۳۸	۱	۱۳۹	۴	۱۴۰	۲	۱۴۱	۳	۱۴۲	۱	۱۴۳	۲
۱۴۴	۳	۱۴۵	۱	۱۴۶	۴														

۲-۱ مرور پیش‌نیازها: سه لایه‌ی وب، شیء، this و async

۲۴ **گزینه‌ی ۳** آسان

طبق اسلاید ۳، لایه‌ی Behaviour (رفتار) به «How it reacts» پاسخ می‌دهد و فناوری آن JavaScript است. Structure به «محتوا چیست» (HTML) و Style به «چه شکلی است» (CSS) پاسخ می‌دهند. گزینه‌ی ۴ نام لایه را درست گفته ولی CSS زبان ظاهر است و منطق واکنش به کلیک و تایپ را نمی‌سازد.

۲۵ **گزینه‌ی ۴** آسان

گزینه‌ی ۴ نادرست است: JavaScript (لایه‌ی رفتار) می‌تواند ظاهر را هم تغییر دهد؛ مثلاً در اسلاید ۱۹ با `title.style.color = 'red'` رنگ عنوان پس از کلیک قرمز می‌شود. سه گزینه‌ی دیگر دقیقاً تعریف‌های اسلاید ۳ هستند: محتوا (HTML)، ظاهر (CSS) و واکنش (JavaScript).

۲۶ **گزینه‌ی ۱** آسان

`name` یک property با مقدار رشته‌ای است، پس `typeof` آن `"string"` است؛ `greet` یک method است و `typeof` هر تابعی `"function"` است (نه `"object"`؛ گزینه‌ی ۲). `typeof` هیچ‌وقت کلمه‌هایی مثل `"property"` یا `"method"` برنمی‌گرداند (گزینه‌ی ۳). چون `greet` به‌صورت `student.greet()` صدا زده شده، `this` همان `student` است و نتیجه `Hi, I am Ali` می‌شود، نه `undefined` (گزینه‌ی ۴).

۲۷ **گزینه‌ی ۲** متوسط

`JSON.stringify` فقط داده‌ها را به متن JSON تبدیل می‌کند و **متدها را حذف می‌کند**؛ پس `greet` در خروجی نیست (گزینه‌ی ۱ غلط است، چون تابع اجرا نمی‌شود و نتیجه‌اش هم ذخیره نمی‌شود). در JSON کلیدها حتماً داخل `"` هستند (رد گزینه‌ی ۳). خروجی `stringify` رشته است و `JSON.parse` آن را دوباره به شیء تبدیل می‌کند که `age` در آن **عدد** است؛ پس `20 + 1` برابر `21` است، نه `"201"` (گزینه‌ی ۴).

۲۸ **گزینه‌ی ۳** متوسط

آرایه‌ها شیء هستند و `typeof []` برابر `"object"` است. رشته‌ی `"abc"` یک primitive است و `typeof` آن `"string"` است (برای همین اسلاید می‌گوید «تقریباً» همه‌چیز). `typeof null` به‌خاطر یک باگ تاریخی `"object"` است، نه `"null"`. تابع‌ها هم با اینکه نوع خاصی از شیء هستند، `typeof` آن‌ها `"function"` است.

۲۹ **گزینه‌ی ۱** سخت

`s` یک primitive است، ولی وقتی روی آن نقطه می‌گذاریم JavaScript یک **wrapper object** موقت (`String`) می‌سازد؛ پس `length` و `toUpperCase()` کار می‌کنند و خطایی رخ نمی‌دهد (رد گزینه‌ی ۴). مقدار `42` روی همان شیء موقت نوشته می‌شود که بلافاصله دور انداخته می‌شود؛ بنابراین `s.extra` برابر `undefined` است (رد گزینه‌های ۲ و ۳). `s` همچنان primitive است و `typeof` آن `"string"` می‌ماند. (در حالت strict، خط `s.extra = 42` خطای `TypeError` می‌داد.)

۳۰ **گزینه‌ی ۲** متوسط

طبق اسلاید ۵، `this` به **نحوه‌ی فراخوانی** بستگی دارد، نه محل تعریف. `obj.regular()` با `obj` صدا زده شده و `other.regular()` با `other`؛ با اینکه هر دو یک تابع‌اند، نتیجه `obj` و `other` است (رد گزینه‌های ۱ و ۳ که فکر می‌کنند `this` به محل تعریف چسبیده است). `f()` بدون شیء صدا زده شده؛ در حالت sloppy در Node.js، `this` شیء سراسری است که `name` ندارد، پس `undefined` چاپ می‌شود (رد گزینه‌ی ۴).

۳۱ **گزینه‌ی ۳** سخت

`f()` یک فراخوانی ساده است؛ در اسکریپت معمولی مرورگر (sloppy) `this` برابر `window` می‌شود. `window` از قبل propertyای به نام `name` دارد که مقدار پیش‌فرضش رشته‌ی خالی است؛ پس `f()` مقدار `""` برمی‌گرداند و `JSON.stringify("")` متن `""` را چاپ می‌کند. گزینه‌ی ۲ جواب همین کد در **Node.js** است (آنجا `undefined` می‌شود). گزینه‌ی ۱ فکر کرده تابع شیء‌اش را به خاطر می‌سپارد، و خطایی هم رخ نمی‌دهد چون `this` در حالت sloppy `undefined` نیست (رد گزینه‌ی ۴).

۳۲ **گزینه‌ی ۴** متوسط

`regular` با `counter.regular()` صدا زده شده و `this` همان `counter` است، پس `10`. ولی arrow function خودش را ندارد و آن را از scope بیرونیِ object literal می‌گیرد؛ در یک ماژول CommonJS در Node.js، `this` بیرونی شیء خالی `module.exports` است که `count` ندارد، پس `undefined` (در مرورگر هم `this` بیرونی `window` است و باز `undefined` می‌شود). خطایی رخ نمی‌دهد، چون خواندن property ناموجود از یک شیء فقط `undefined` می‌دهد (رد گزینه‌ی ۳).

۳۳ **گزینه‌ی ۱** سخت

در `t.report()` مقدار `this` نمونه‌ی `t` است و arrow function `show` همان `this` را به ارث می‌برد، پس `T1` چاپ می‌شود (رد گزینه‌ی ۴). اما `r()` یک فراخوانی ساده است و کدِ داخل کلاس همیشه **strict** است؛ پس `this` برابر `undefined` می‌شود و خواندن `undefined.label` خطای `TypeError` می‌دهد. گزینه‌ی ۲ فکر کرده متد به نمونه چسبیده است و گزینه‌ی ۳ قانون حالت sloppy (شیء سراسری) را اشتباهی به کلاس تعمیم داده است.

۳۴ **گزینه‌ی ۲** متوسط

در listenerای که با `function` معمولی نوشته شده، مرورگر `this` را همان عنصری قرار می‌دهد که listener روی آن ثبت شده؛ پس `this.textContent` برابر `First` است (رد گزینه‌ی ۳). arrow function `this` خودش را ندارد و `this` بیرونی را برمی‌دارد که در سطح بالای اسکریپت مرورگر `window` است؛ پس `true` چاپ می‌شود (رد گزینه‌ی ۱). گزینه‌ی ۴ فکر کرده arrow function هم `this` را دکمه می‌گیرد، در حالی که اصلاً `textContent` چاپ نمی‌شود.

۳۵ **گزینه‌ی ۴** آسان

اسلاید ۶ صریحاً می‌گوید: «callbacks → manual, promises → .catch(), async → try...catch». در callbackها معمولاً اولین پارامتر خطاست و باید آن را دستی بررسی کرد؛ Promise خطا را به `.catch()` می‌فرستد؛ و در تابعِ async، Promise ردشده در خط `await` مثل یک خطای عادی پرتاب می‌شود و `try...catch` آن را می‌گیرد. گزینه‌های ۱ و ۲ جای روش‌ها را جابه‌جا کرده‌اند، و گزینه‌ی ۳ غلط است چون `try...catch` معمولی خطای callbackی را که بعداً اجرا می‌شود و خطای `.then` را نمی‌گیرد.

۳۶ **گزینه‌ی ۳** سخت

بدنه‌ی تابع async تا اولین `await` همگام اجرا می‌شود؛ پس `load start` بلافاصله بعد از `script start` چاپ می‌شود (رد گزینه‌ی ۲). `await` فقط همان تابع را معلق می‌کند، نه کل برنامه، پس `data ready` قبل از `script end` می‌آید (رد گزینه‌ی ۱). ادامه‌ی تابع بعد از `await` یک microtask است و قبل از (macrotask) `setTimeout` callback اجرا می‌شود، حتی با تأخیر `0` (رد گزینه‌ی ۴).

۳۷ **گزینه‌ی ۱** متوسط

اولین `await` مقدار `user loaded` را می‌گیرد و چاپ می‌کند (رد گزینه‌ی ۴). دومین Promise رد می‌شود و `await` آن را مثل یک خطا پرتاب می‌کند؛ اجرا مستقیم به `catch` می‌پرد و خط `after` هرگز اجرا نمی‌شود (رد گزینه‌ی ۲). چون خطا با `try...catch` گرفته شده، برنامه crash نمی‌کند (رد گزینه‌ی ۳) و در پایان بلوک `finally` هم `done` را چاپ می‌کند.

۳۸ **گزینه‌ی ۲** آسان

JavaScript در مرورگر روی یک thread اصلی اجرا می‌شود که پاسخ به کاربر و رسم صفحه هم کار همان thread است؛ اگر کدی همگام منتظر شبکه بماند، صفحه قفل می‌شود. پس کار را شروع می‌کنیم و callback یا Promise را برای «بعداً» ثبت می‌کنیم. گزینه‌ی ۱ برعکس واقعیت است (یک thread اصلی). async کار را سریع‌تر نمی‌کند، فقط منتظر ماندن را غیرمسدودکننده (non-blocking) می‌کند (رد گزینه‌ی ۳). گزینه‌ی ۴ هم ساختگی است؛ کد همگام همیشه مجاز است.

۳۹ **گزینه‌ی ۳** آسان

callbackهای تو در تو که کد را به شکل هرم درمی‌آورند همان «callback hell» اسلاید ۶ هستند؛ خواندن و مدیریت خطای آن‌ها (بررسی دستی `err` در هر سطح) سخت است. راه‌حل، Promiseهایی است که با `.then()` زنجیر می‌شوند، یا `async/await` که کد را شبیه کد همگام می‌کند. هم‌نام بودن `err` ها مشکل اصلی نیست، و این کد ربطی به TDZ یا DOM ندارد.

۴۰ **گزینه‌ی ۴** متوسط

callbackهای `.then` و `.catch` همیشه ناهمگام (microtask) هستند، پس اول `sync` چاپ می‌شود (رد گزینه‌های ۱ و ۳). در زنجیره، `5 * 2` برابر `10` است و چون `10 > 5` است خطا پرتاب می‌شود؛ خطا از `.then` بعدی رد می‌شود و مستقیم به `.catch` می‌رسد. پس `value: 10` هرگز چاپ نمی‌شود (رد گزینه‌ی ۲) و خروجی `error: too big` است.

۲-۲ زبان‌های نشانه‌گذاری و درخت DOM

۴۱ **گزینه‌ی ۴** آسان

اسلاید ۷ HTML، XML و Markdown را نمونه‌های زبان نشانه‌گذاری می‌آورد؛ این زبان‌ها فقط ساختار سند را با تگ یا نشانه توصیف می‌کنند. JavaScript یک زبان **برنامه‌نویسی** است که شرط، حلقه و منطق دارد و لایه‌ی رفتار صفحه را می‌سازد. Markdown با اینکه به‌جای `<...>` از نشانه‌هایی مثل `#` استفاده می‌کند، باز هم زبان نشانه‌گذاری است.

۴۲ **گزینه‌ی ۲** آسان

طبق اسلاید ۹، DOM مخفف **Document Object Model** است: «Document» یعنی صفحه‌ی وب، «Object» یعنی هر بخش صفحه به یک شیء تبدیل می‌شود و «Model» یعنی این اشیاء با هم یک مدل درختی از صفحه می‌سازند. بقیه‌ی گزینه‌ها ساختگی‌اند؛ DOM نه درباره‌ی «Data» است، نه یک زبان نشانه‌گذاری (Markup) و نه یک «Method».

۴۳ **گزینه‌ی ۳** آسان

برگ گره‌ای است که **هیچ فرزندی ندارد**. `C` و `D` با اینکه در ردیف دوم هستند، فرزندی ندارند، پس برگ‌اند؛ `G` و `H` هم پایین‌ترین گره‌ها و بدون فرزند هستند. گزینه‌ی ۱ فقط ردیف آخر را دیده و `C` و `D` را جا انداخته است. `E` و `F` هر کدام یک فرزند دارند (رد گزینه‌ی ۲)، و `A` ریشه است و `B` دو فرزند دارد (رد گزینه‌ی ۴).

۴۴ **گزینه‌ی ۴** متوسط

هم‌نیاها باید **والد مشترک** داشته باشند. والد `F` گره‌ی `B` است ولی والد `G` گره‌ی `E`؛ پس آن‌ها هم‌نیا نیستند (هم‌نیای `F` فقط `E` است). گزینه‌ی ۱ درست است چون هر سه فرزند `A` اند؛ گزینه‌ی ۲ درست است چون مسیر `A` به `B` به `F` به `H` نشان می‌دهد جدِ `H` است؛ و گزینه‌ی ۳ هم دقیقاً رابطه‌ی نمودار است.

۴۵ **گزینه‌ی ۲** آسان

اسلاید ۸ می‌گوید «Attributes become node properties»: attribute به خودِ عنصر چسبیده است و با `a.href` یا `a.getAttribute("href")` خوانده می‌شود. در نمودار W3Schools هم جعبه‌ی `href` کنار `<a>` آمده، نه زیر آن؛ یعنی فرزند نیست (رد گزینه‌ی ۱). attribute عنصر جداگانه هم نیست (رد گزینه‌ی ۳)، و حذف هم نمی‌شود؛ اگر حذف می‌شد لینک‌ها کار نمی‌کردند (رد گزینه‌ی ۴).

۴۶ **گزینه‌ی ۱** متوسط

تنها فرزند `<a>` گره‌ی متنی `"Home page"` است، پس `childNodes.length` برابر `1` و `nodeName` آن `#text` است. attributeها (`id` و `href`) فرزند نیستند و در مجموعه‌ی جداگانه‌ی `attributes` قرار دارند که طولش `2` است. گزینه‌ی ۲ و ۳ attributeها را فرزند حساب کرده‌اند، و گزینه‌ی ۴ هم attribute را اولین فرزند فرض کرده و `id` را جا انداخته است.

۴۷ **گزینه‌ی ۴** متوسط

`<p>` سه فرزند دارد: گره‌ی متنی `"Hello "`، عنصر `<b>` و گره‌ی متنی `" world"`؛ پس `childNodes.length` برابر `3` است. `children` فقط عنصرها را می‌شمارد که فقط `<b>` است، پس `1`. گزینه‌ی ۱ متن‌ها را گره حساب نکرده، گزینه‌ی ۲ متن قبل و بعد از `<b>` را یکی گرفته، و گزینه‌ی ۳ فکر کرده `children` هم گره‌های متنی را می‌شمارد.

۴۸ **گزینه‌ی ۲** متوسط

خط جدید و فاصله‌های بین تگ‌ها گره‌ی متنی می‌سازند؛ پس فرزندان `ul` این‌ها هستند: فاصله، `li`، فاصله، `li`، فاصله، یعنی `5` گره. `children` فقط عنصرها را می‌شمارد: `2`. گزینه‌ی ۱ فاصله‌ها را نادیده گرفته، گزینه‌ی ۳ فقط فاصله‌ی بین دو `li` را دیده، و گزینه‌ی ۴ فکر کرده `children` هم گره‌های متنی را شامل می‌شود.

۴۹ **گزینه‌ی ۳** سخت

اولین گره‌ی فرزند `ul` متنِ فاصله (خط جدید و دو فاصله) قبل از اولین `li` است؛ پس `firstChild.nodeName` برابر `#text` است (رد گزینه‌ی ۱). `firstElementChild` گره‌های متنی را رد می‌کند و به `LI` می‌رسد. آخرین `li` `lastElementChild` است که متنش `B` است (رد گزینه‌ی ۲). گزینه‌ی ۴ جای دو property را برعکس گرفته است.

۵۰ **گزینه‌ی ۱** متوسط

`nodeType` برای `document` برابر `9`، برای هر عنصر (مثل `body`) برابر `1` و برای گره‌ی متنی برابر `3` است. `p.firstChild` متن `"Hi"` است، نه یک عنصر (رد گزینه‌ی ۴). `document` عنصر نیست، پس `1` نیست (رد گزینه‌ی ۲)، و `2` کد attributeهاست، نه متن (رد گزینه‌ی ۳).

۵۱ **گزینه‌ی ۴** متوسط

`nodeName` سند `#document` و گره‌ی متنی `#text` است (با علامت `#`). برای عنصرها `nodeName` نام تگ با حروف بزرگ است و `document.documentElement` همان عنصر ریشه، یعنی `<html>` است؛ پس `HTML`. گزینه‌ی ۲ `documentElement` را با `body` اشتباه گرفته و گزینه‌های ۱ و ۳ قالب `nodeName` را درست نمی‌دانند.

۵۲ **گزینه‌ی ۳** آسان

اسلاید ۱۱ صریحاً می‌گوید «Root node is `document`». عنصر `<html>` **عنصر ریشه** (root element) است که فرزند `document` است، نه `window` (رد گزینه‌ی ۲). اصلاً جزو درخت DOM نیست؛ شیء پنجره است که `document` یکی از propertyهای آن است (رد گزینه‌ی ۱). `<body>` هم فرزند `<html>` است.

۵۳ **گزینه‌ی ۲** سخت

`document.documentElement` عنصر ریشه، یعنی `<html>` است و والدش خودِ `document` است؛ پس `HTML true` (رد گزینه‌های ۳ و ۴). `document` دو گره‌ی فرزند دارد: گره‌ی `<!DOCTYPE html>` (که خودش یک گره از نوع DocumentType است) و عنصر `<html>`؛ پس `2`. گزینه‌ی ۱ گره‌ی DOCTYPE را فراموش کرده است.

۵۴ **گزینه‌ی ۱** سخت

`div.sampleclass` دو عنصر فرزند دارد (`p` و `h3`) و والدش `body` است (رد گزینه‌ی ۲ که والد را `html` گرفته). عنصر هم‌نیای قبلی `h3` همان `p` است، نه `h1` که بیرون `div` قرار دارد (رد گزینه‌ی ۳). `head` دو عنصر فرزند دارد: `title` و `link` (رد گزینه‌ی ۴). attributeهایی مثل `class` و `id` هم در `children` شمرده نمی‌شوند.

۵۵ **گزینه‌ی ۲** سخت

محتوای `<title>` تا رسیدن به یک `</title>` واقعی متن خام حساب می‌شود و تگ‌های داخلش تشخیص داده نمی‌شوند. چون `</title>` ی وجود ندارد، کل باقی فایل (از جمله `<h1>` ، `<div>` و `<h3>`) متنِ عنوان می‌شود؛ پس `body` خالی است (`0`)، عنصری با id برابر `2223` وجود ندارد (`true`) و متن `SAMPLE PAGE!` داخل `document.title` است (`true`). parser برای HTML نادرست هیچ‌وقت خطا نمی‌دهد (رد گزینه‌ی ۳)، و گزینه‌های ۱ و ۴ فرض کرده‌اند مرورگر تگ اشتباه را نادیده می‌گیرد و بقیه‌ی صفحه را عادی می‌سازد.

۵۶ **گزینه‌ی ۳** متوسط

در نمودار اسلاید ۱۱، `Text` زیر `CharacterData` قرار دارد، نه زیر `Element` ؛ داخل تگ بودن متن یعنی «فرزند یک عنصر در درخت DOM بودن»، نه «زیرکلاس Element بودن». این دو را قاطی نکن: درخت DOM نشان می‌دهد کدام گره داخل کدام است، و نمودار کلاس‌ها نشان می‌دهد هر گره از چه نوعی است. سه گزینه‌ی دیگر دقیقاً روابط نمودار هستند: زنجیره‌ی `HTMLParagraphElement` به `Node` ، دو زیرکلاس `CharacterData` ، و `Attr` زیر `Node` .

۵۷ **گزینه‌ی ۴** سخت

`<p>` یک `HTMLParagraphElement` است که از `HTMLElement` ارث می‌برد، پس `true` (رد گزینه‌ی ۳). `p.firstChild` گره‌ی متنی `"Hi"` است: زیرکلاس `CharacterData` هست (`true`) ولی زیرکلاس `Element` نیست (`false` ؛ رد گزینه‌های ۱ و ۲). `document` گره است ولی عنصر نیست (`nodeType` آن `9` است)، پس `instanceof Element` برای آن `false` است.

۵۸ **گزینه‌ی ۱** متوسط

طبق اسلاید ۹، DOM یک نمایش درختی از صفحه است که HTML را به JavaScript وصل می‌کند و اجازه‌ی خواندن، تغییر، اضافه و حذف عنصرها را می‌دهد. DOM زبان نیست، بلکه مدلی از اشیاء است (رد گزینه‌ی ۲). مرورگر DOM را از روی فایل در حافظه می‌سازد و تغییر آن فایل را عوض نمی‌کند؛ با refresh همه‌چیز از نو ساخته می‌شود (رد گزینه‌ی ۳). گزینه‌ی ۴ هم با «read, modify, add, or remove» اسلاید ۹ تناقض دارد.

۵۹ **گزینه‌ی ۱** آسان

متن داخل هر تگ، گره‌ی متنیِ **فرزندِ همان تگ** می‌شود؛ پس والد `"My title"` عنصر `<title>` است. `<head>` والدِ `<title>` و در نتیجه **جدِ** این متن است، نه والدش (رد گزینه‌ی ۲). `<html>` و `Document` هم اجداد بالاتر هستند.

۶۰ **گزینه‌ی ۳** سخت

comment (`<!-- note -->`) هم یک گره در DOM است (از نوع `Comment` با `nodeType` برابر `8`). پس فرزندان `div` سه گره‌اند: comment، عنصر `span` و گره‌ی متنی `"text"` ؛ یعنی `childNodes.length` برابر `3` و `children.length` برابر `1` . اولین فرزند همان comment است با `nodeType` برابر `8` . گزینه‌ی ۱ comment را گره ندانسته ولی نوعش را درست گفته، گزینه‌ی ۲ comment را متن (`3`) فرض کرده، و گزینه‌ی ۴ فقط عنصرها را دیده است.

۶۱ **گزینه‌ی ۱** متوسط

این همان درخت نمودار Milne در اسلاید ۹ است: `<h1>` و `<a>` هم‌نیا هستند و والد هر دو `body` است (رد گزینه‌ی ۲ که فکر کرده `<a>` فرزند `<h1>` است). عنصر هم‌نیای قبلی `<a>` همان `h1` با id برابر `head` است (رد گزینه‌ی ۳)، و عنصر هم‌نیای بعدی `h1` همان `<a>` است که `href` آن `"#"` است، نه `null` (رد گزینه‌ی ۴).

۳-۲ سلسله‌مراتب DOM در JavaScript و شیء document

۶۲ گزینه‌ی ۲ — آسان

اسلاید ۱۲ می‌گوید `window` بالاترین شیء مرورگر و همان global context است و `document` (صفحه‌ی داخل پنجره) یکی از propertyهای آن است. `<html>` عنصر ریشه‌ی درخت DOM است که زیر `document` قرار دارد، و `<body>` هم فرزند `<html>` است؛ هیچ‌کدام بالاترین شیء نیستند.

۶۳ گزینه‌ی ۳ — آسان

اسلاید ۱۳: `document` «Represents the entire web page» و «Acts as the entry point to the DOM tree». پنجره، تاریخچه و آدرس مربوط به `window` و فرزندانش (`history` و `location`) هستند (رد گزینه‌ی ۱). `document` شامل `<head>` هم می‌شود، نه فقط `<body>` (رد گزینه‌ی ۲)، و DOM در حافظه است و تغییرش فایل روی دیسک را عوض نمی‌کند (رد گزینه‌ی ۴).

۶۴ گزینه‌ی ۴ — متوسط

در نمودار، `forms` (و زیرمجموعه‌اش `elements`) زیر `document` قرار دارد، چون فرم‌ها بخشی از خودِ صفحه هستند. `location` (آدرس صفحه)، `history` (تاریخچه‌ی زبانه) و `navigator` (اطلاعات مرورگر) مستقیماً زیر `window` کشیده شده‌اند؛ این‌ها به پنجره‌ی مرورگر مربوط‌اند، نه به محتوای صفحه.

۶۵ گزینه‌ی ۱ — متوسط

در نمودار، `navigator` (اطلاعات خودِ مرورگر) دو فرزند دارد: `plugins` و `mimeTypes` (نوع فایل‌هایی که مرورگر می‌شناسد). `document` هم یک `plugins` دارد ولی `mimeTypes` ندارد (رد گزینه‌ی ۲). `screen` (اطلاعات مانیتور) و `location` (آدرس صفحه) در نمودار فرزندی ندارند.

۶۶ گزینه‌ی ۳ — آسان

`document` یکی از propertyهای `window` است، پس `window.document === document` برابر `true` است (رد گزینه‌ی ۱). `document.documentElement` عنصر ریشه، یعنی `<html>` است، نه `<body>` (رد گزینه‌ی ۲) و نه خودِ `document` (رد گزینه‌ی ۴). `document.body` و `document.head` هم مستقیماً به `<body>` و `<head>` اشاره می‌کنند.

۶۷ گزینه‌ی ۲ — آسان

با هر کلیک، listener مقدار `textContent` عنوان را **جایگزین** می‌کند (`=`، نه `+=`)؛ پس کلیک اول متن را عوض می‌کند و کلیک دوم همان متن را دوباره می‌نویسد. نتیجه `Text Changed via document!` است، نه دو بار تکرار آن (رد گزینه‌ی ۳). اسکریپت بعد از `<h1>` آمده، پس `heading` برابر `null` نیست و خطایی رخ نمی‌دهد (رد گزینه‌ی ۴)، و متن هم تغییر کرده است (رد گزینه‌ی ۱).

۶۸ گزینه‌ی ۱ — متوسط

همان‌طور که اسلاید ۱۵ نشان می‌دهد، **خودِ عنصر** چاپ می‌شود (`▶ <h1 id="title">` همراه با لینک `doc.html:8:15`)، چون خودِ عنصر را به `console.log` داده‌ایم؛ برای دیدن متن باید `heading.textContent` را چاپ کرد (رد گزینه‌ی ۲). اسکریپت بعد از `<h1>` قرار دارد، پس عنصر از قبل در درخت هست و `null` نمی‌گیریم (رد گزینه‌ی ۳). نبودن `;` هم خطا نیست، چون JavaScript سمی‌کالن را خودکار درج می‌کند (رد گزینه‌ی ۴).

۶۹ گزینه‌ی ۳ — سخت

اسکریپت داخل `<head>` است و وقتی اجرا می‌شود، مرورگر هنوز به `<h1>` نرسیده؛ پس `getElementById` چیزی پیدا نمی‌کند و `null` برمی‌گرداند (نه `undefined`؛ رد گزینه‌ی ۲). خط بعد می‌خواهد روی `null` property بنویسد و `TypeError` می‌دهد، پس `done` هرگز چاپ نمی‌شود. گزینه‌ی ۱ فرض کرده عنصر پیدا می‌شود، و `ReferenceError` هم نیست چون متغیر `heading` تعریف شده است، فقط مقدارش `null` است (رد گزینه‌ی ۴).

۷۰ **گزینه‌ی ۲** سخت

اسکریپت `<head>` قبل از ساخته شدن `<h1>` اجرا می‌شود، پس `null` می‌بیند (رد گزینه‌ی ۳). اسکریپت انتهای `<body>` هنگام parse اجرا می‌شود، یعنی قبل از اینکه parse کل سند تمام شود؛ پس پیش از `DOMContentLoaded` می‌آید (رد گزینه‌ی ۱). `DOMContentLoaded` به محض آماده شدن درخت DOM رخ می‌دهد، ولی `load` منتظر همه‌ی منابع می‌ماند و همیشه بعد از آن است (رد گزینه‌ی ۴).

۷۱ **گزینه‌ی ۳** متوسط

مشکل، زمان اجراست: کد قبل از ساخته شدن `<body>` اجرا می‌شود. `DOMContentLoaded` اجرای کد را تا آماده شدن کامل درخت DOM عقب می‌اندازد، پس عنصر پیدا می‌شود. `defer` فقط روی اسکریپت‌های خارجی (با `src`) اثر دارد و روی اسکریپت درون‌خطی نادیده گرفته می‌شود (رد گزینه‌ی ۱). نوع تعریف متغیر ربطی به وجود عنصر ندارد (رد گزینه‌ی ۲)، و `getElementById` نام id را **بدون** `#` می‌گیرد؛ با `#` حتماً `null` برمی‌گرداند (رد گزینه‌ی ۴).

۷۲ **گزینه‌ی ۳** متوسط

`heading` یک شیء از کلاس `HTMLHeadingElement` است؛ پس `typeof` آن `"object"` است (`typeof` هیچ‌وقت `"element"` برنمی‌گرداند؛ رد گزینه‌ی ۴). تبدیل یک شیء DOM به رشته با `String(...)` فقط `[object HTMLHeadingElement]` می‌دهد، نه متن یا کد HTML آن (رد گزینه‌های ۱ و ۲). متن داخل عنصر را باید از `textContent` خواند که `Hello World` است.

۷۳ **گزینه‌ی ۴** آسان

طبق اسلاید ۱۲، attributeها و متن propertyهای عنصر هستند: `id` مقدار `title` را بدون `#` می‌دهد و `className` مقدار attribute `class` را بدون نقطه (رد گزینه‌ی ۱). property کلاس در DOM `className` نام دارد و وجود دارد، پس `undefined` نمی‌دهد (رد گزینه‌ی ۲). `textContent` فقط متن را می‌دهد، نه کد HTML عنصر (رد گزینه‌ی ۳)، و `tagName` نام تگ را با حروف بزرگ (`H1`) برمی‌گرداند.

۷۴ **گزینه‌ی ۱** متوسط

`document.title` متن عنصر `<title>` (عنوان زبانه‌ی مرورگر) را می‌دهد، نه متن `<h1>`؛ پس اول `Shop` چاپ می‌شود (رد گزینه‌های ۲ و ۴). نوشتن در `document.title` خودِ عنصر `<title>` را به‌روز می‌کند؛ بنابراین `textContent` آن هم `Cart (2)` می‌شود (رد گزینه‌ی ۳). سایت‌ها از همین ترفند برای نشان دادن تعداد پیام یا سبد خرید در عنوان زبانه استفاده می‌کنند.

۷۵ **گزینه‌ی ۴** آسان

رابطه برعکس است: طبق اسلاید ۱۲، `window` بالاترین شیء است و `document` یکی از فرزندان (propertyهای) آن؛ یعنی `window.document` درست است، نه `document.window`. سه گزینه‌ی دیگر عیناً بندهای اسلاید ۱۳ هستند: نقطه‌ی ورود به DOM، متدهای پیدا کردن عنصر، و امکان ساختن، حذف و به‌روزرسانی گره‌ها.

۷۶ **گزینه‌ی ۱** سخت

اسکریپت وسط صفحه وقتی اجرا می‌شود که `<h1>` ساخته شده ولی مرورگر هنوز به `<h2>` نرسیده؛ پس `a` پیدا می‌شود و `b` نه (`true false`؛ رد گزینه‌های ۲ و ۳). اسکریپت دوم در انتهای `<body>` است و آن موقع `<h2>` هم ساخته شده، پس متنش `B` چاپ می‌شود و خطایی رخ نمی‌دهد (رد گزینه‌ی ۴). هر اسکریپت فقط عنصرهای بالای خودش را می‌بیند.

۷۷ **گزینه‌ی ۲** متوسط

`DOMContentLoaded` روی `document` به محض کامل شدن درخت DOM رخ می‌دهد و منتظر تصاویر نمی‌ماند؛ `load` روی `window` تا بارگیری همه‌ی منابع صبر می‌کند، پس همیشه دیرتر است (رد گزینه‌های ۱ و ۳). اسکریپت‌های معمولی صفحه هنگام parse اجرا می‌شوند، یعنی **قبل** از `DOMContentLoaded`، نه بعد از آن (رد گزینه‌ی ۴).

۷۸ **گزینه‌ی ۴** سخت

دکمه داخل `DOMContentLoaded` جست‌وجو شده و پیدا می‌شود، پس listener ثبت می‌شود و کلیک بی‌اثر نیست (رد گزینه‌ی ۲). اما `heading` **بیرون** از `DOMContentLoaded` و زمانی جست‌وجو شده که `<h1>` هنوز ساخته نشده بود؛ پس مقدارش برای همیشه `null` مانده است. با کلیک، `heading.textContent = ...` روی `null` اجرا می‌شود و `TypeError` می‌دهد، پس `changed` چاپ نمی‌شود (رد گزینه‌های ۱ و ۳).

۷۹ **گزینه‌ی ۴** متوسط

ویژگی `defer` روی اسکریپت خارجی (دارای `src`) باعث می‌شود فایل هم‌زمان با parse دانلود شود ولی بعد از پایان parse (و قبل از `DOMContentLoaded`) اجرا شود. روی اسکریپت درون‌خطی، `defer` نادیده گرفته می‌شود (رد گزینه‌ی ۱). `<script src>` معمولی parse را متوقف می‌کند و همان لحظه اجرا می‌شود (رد گزینه‌ی ۲)، و ویژگی‌ای به نام `delay` در HTML وجود ندارد (رد گزینه‌ی ۳).

۲-۴ ساختن و افزودن عناصر جدید

۸۰ **گزینه‌ی ۳** آسان

`createElement` فقط یک عنصر **جدید** در حافظه می‌سازد؛ این عنصر پدر ندارد و تا وقتی با متدی مثل `appendChild` یا `append` به درخت اضافه نشود دیده نمی‌شود. گزینه‌های ۱ و ۴ فرض کرده‌اند افزودن خودکار انجام می‌شود که درست نیست، و گزینه‌ی ۲ کارِ `querySelector("li")` را توصیف می‌کند (پیدا کردن، نه ساختن).

۸۱ **گزینه‌ی ۲** آسان

پاراگراف جدید ساخته و متن‌دار شده ولی به صفحه اضافه نشده است؛ پس `isConnected` برابر `false` است و `querySelectorAll("p")` فقط همان یک `<p>` قدیمی صفحه را پیدا می‌کند. گزینه‌های ۱ و ۳ فرض کرده‌اند `createElement` یا مقداردهی `textContent` عنصر را خودکار وارد صفحه می‌کند؛ گزینه‌ی ۴ هم با شمارش درست، `isConnected` را اشتباه گفته است.

۸۲ **گزینه‌ی ۲** متوسط

ورودی `createElement` فقط نام تگ است؛ علامت‌های `<` و `>` در نام تگ مجاز نیستند و مرورگر خطای `InvalidCharacterError` می‌دهد. گزینه‌ی ۱ یک `<section>` می‌سازد، گزینه‌ی ۳ دو گره‌ی متنی به انتهای body اضافه می‌کند (`append` رشته قبول می‌کند) و در گزینه‌ی ۴ مرجعِ `null` یعنی افزودن به انتها، که کاملاً مجاز است.

۸۳ **گزینه‌ی ۴** آسان

`appendChild` گره را آخرین فرزندِ عنصری می‌کند که روی آن صدا زده شده (این‌جا `<body>`)؛ این همان خط آخرِ مثال اسلاید ۲۴ است. `createElement` روی `document` صدا زده می‌شود و عنصر **می‌سازد**، `querySelector` عنصر **پیدا** می‌کند و `setAttribute` دو آرگومان (نام و مقدار attribute) می‌گیرد و چیزی به صفحه اضافه نمی‌کند.

۸۴ **گزینه‌ی ۱** آسان

`append(...items)` هر تعداد گره یا رشته می‌گیرد و رشته‌ها را به text node تبدیل می‌کند؛ `appendChild(node)` فقط یک گره می‌گیرد. گزینه‌ی ۲ برعکس است: `appendChild` با رشته خطای `TypeError` می‌دهد. گزینه‌ی ۳ هم غلط است، چون مقدار `append` برمی‌گرداند (این `appendChild` است که خود گره را برمی‌گرداند). هر دو متد به **انتها** اضافه می‌کنند، نه ابتدا؛ برای ابتدا `prepend` داریم.

۸۵ **گزینه‌ی ۱** متوسط

`append` برای **هر** رشته یک text node جداگانه می‌سازد؛ پس `<ul>` یک فرزندِ عنصری (`<li>`) و دو گره‌ی متنی (`"note"` و `"!"`) دارد. `children` فقط عنصرها را می‌شمارد (۱) و `childNodes` همه‌ی گره‌ها را (۳). گزینه‌ی ۲ گره‌های متنی را عنصر حساب کرده، گزینه‌ی ۳ فکر کرده رشته‌ها اضافه نمی‌شوند و گزینه‌ی ۴ دو رشته را در یک گره ادغام کرده است.

۸۶ **گزینه‌ی ۲** سخت

`appendChild` خودِ گره را برمی‌گرداند، پس `first` همان `<li>` است و خط اول `One` چاپ می‌شود. اما `append` مقدار `undefined` برمی‌گرداند؛ پس `second` برابر `undefined` است و `undefined.textContent = "Two"` خطای `TypeError` می‌دهد و اجرا متوقف می‌شود. گزینه‌ی ۱ فکر کرده `append` هم گره را برمی‌گرداند، گزینه‌ی ۳ خطا را از همان ابتدا دانسته و گزینه‌ی ۴ فرض کرده مقداردهی به `undefined` بی‌صدا نادیده گرفته می‌شود.

۸۷ **گزینه‌ی ۳** متوسط

خط ۳ مجاز است، چون `append` هم رشته و هم گره می‌گیرد. در خط ۵ هم آرگومان اول یک گره است و مرجعِ `null` یعنی افزودن به انتها، پس مشکلی نیست. اما در خط ۷ به `appendChild` یک **رشته** داده شده و این متد فقط Node قبول می‌کند؛ پس `TypeError` رخ می‌دهد و `"C"` هرگز چاپ نمی‌شود. اگر به‌جای آن `box.append("Done")` یا `box.appendChild(document.createTextNode("Done"))` می‌نوشتیم، خطایی نبود.

---

۸۸ **گزینه‌ی ۴** آسان

متد درست `document.createTextNode(text)` است و خروجی‌اش یک گره است، پس `appendChild` آن را قبول می‌کند. گزینه‌ی ۱ یک **عنصر** با تگ عجیبِ `<text>` می‌سازد، نه گره‌ی متنی. متدهای گزینه‌های ۲ و ۳ اصلاً وجود ندارند و صدا زدنشان خطای `TypeError` (is not a function) می‌دهد.

---

۸۹ **گزینه‌ی ۴** متوسط

محتوای یک گره‌ی متنی هیچ‌وقت به‌عنوان HTML تفسیر نمی‌شود؛ پس عنصر `<b>` ساخته نمی‌شود (`children.length` برابر ۰) و خودِ نویسه‌های `<b>Hi</b>` متنِ box می‌شوند. گزینه‌های ۱ و ۳ فرض کرده‌اند تگ تفسیر شده است (این رفتار `innerHTML` است) و گزینه‌ی ۲ عنصر را ساخته‌شده فرض کرده ولی متن را خام گزارش کرده که با هم تناقض دارد.

---

۹۰ **گزینه‌ی ۲** آسان

`insertBefore` روی **پدر** صدا زده می‌شود و ترتیب آرگومان‌ها «اول گره‌ی جدید، بعد مرجع» است: `ul.insertBefore(newLi, oldLi)`. گزینه‌ی ۱ آرگومان‌ها را جابه‌جا کرده (و چون `newLi` فرزند `ul` نیست خطای `NotFoundError` می‌دهد). گزینه‌ی ۳ متد را روی خودِ مرجع صدا زده و گزینه‌ی ۴ روی گره‌ی جدید؛ در هر دو، پدرِ درست مشخص نشده است.

---

۹۱ **گزینه‌ی ۴** متوسط

`list.children[1]` همان `<li>C</li>` است و `insertBefore` گره‌ی جدید را درست **قبل** از آن می‌گذارد؛ پس ترتیب A، B، C می‌شود و `textContent` متن همه را پشت سر هم می‌دهد: `ABC`. گزینه‌ی ۱ فکر کرده درج در ابتدای لیست انجام می‌شود، گزینه‌ی ۲ آن را مثل `appendChild` در انتها گذاشته و گزینه‌ی ۳ فراموش کرده که `insertBefore` واقعاً گره را به صفحه اضافه می‌کند.

---

۹۲ **گزینه‌ی ۲** متوسط

`list.insertBefore(newItem, list.firstChild)` گره‌ی جدید قبل از **اولین** فرزند فعلی قرار می‌گیرد، پس خودش اولین فرزند می‌شود: (معادل `list.prepend(newItem)`). اگر لیست خالی باشد `firstChild` برابر `null` است و درج در انتها همان درج در ابتداست، پس باز هم درست کار می‌کند. گزینه‌ی ۱ همیشه به **انتها** اضافه می‌کند، گزینه‌ی ۳ گره را قبل از آخرین فرزند می‌گذارد و گزینه‌ی ۴ خطای `NotFoundError` می‌دهد، چون `list` فرزند خودش نیست.

---

۹۳ **گزینه‌ی ۱** سخت

در خط سوم مرجع `null` است، پس `<em>` به انتهای box اضافه می‌شود و تعداد فرزندان ۲ می‌شود. در خط پنجم مرجع (`s`) فرزند `box` **نیست** (هم‌سطح آن است)، پس مرورگر خطای `NotFoundError` می‌دهد. گزینه‌ی ۲ فرض کرده مرجع می‌تواند هر عنصری باشد، گزینه‌ی ۳ فکر کرده `null` مجاز نیست و گزینه‌ی ۴ اثرِ درجِ اول را نادیده گرفته است.

---

۹۴ **گزینه‌ی ۳** متوسط

`b.before(A)` عنصر A را درست قبل از B می‌گذارد (A، B)، `b.after(C)` عنصر C را بعد از B (A، B، C) و `list.prepend(0)` عنصر 0 را **اولین** فرزند لیست می‌کند؛ نتیجه `0ABC` است. گزینه‌ی ۲ فکر کرده `prepend` به انتها اضافه می‌کند، گزینه‌ی ۱ آن را قبل از B گذاشته و گزینه‌ی ۴ معنای `before` و `after` را جابه‌جا فهمیده است.

---

۹۵ **گزینه‌ی ۱** متوسط

`task` از قبل در صفحه (داخل `#todo`) است. append کردن یک گره‌ی موجود آن را از جای قبلی **برمی‌دارد** و به جای جدید **منتقل** می‌کند؛ پس `#todo` خالی می‌شود و `#done` یک فرزند پیدا می‌کند. گزینه‌ی ۲ تصور کرده کپی ساخته می‌شود، گزینه‌ی ۳ فکر کرده چون عنصر جای دیگری است اضافه نمی‌شود و گزینه‌ی ۴ فکر کرده عنصر گم می‌شود.

---

۹۶ **گزینه‌ی ۱** متوسط

هر گره فقط یک جا در درخت دارد؛ افزودن دوباره‌ی یک گره‌ی موجود آن را **منتقل** می‌کند و کپی نمی‌سازد (برای کپی باید `cloneNode` صدا زده شود). پس گزینه‌ی ۱ نادرست و پاسخ است. گزینه‌های ۲، ۳ و ۴ همگی رفتار واقعی این متدها هستند.

۹۷ **گزینه‌ی ۴** سخت

`cloneNode` کپی می‌سازد و عنصر اصلی سر جایش می‌ماند؛ پس `#a` هنوز ۱ فرزند دارد و دو کپی به `#b` اضافه می‌شوند (۲). کپی عمیق (`true`) متن داخلی `X` را هم دارد، ولی کپی سطحی فقط خود تگ `<li>` است و متنش خالی است؛ پس `true` چاپ می‌شود. گزینه‌های ۱ و ۳ کپی را با جابه‌جایی اشتباه گرفته‌اند و گزینه‌های ۲ و ۳ فکر کرده‌اند کپی سطحی متن را هم دارد.

۹۸ **گزینه‌ی ۳** سخت

ابتدا Z قبل از `items[1]` (یعنی B) درج می‌شود: A، Z، B، C. حالا `items[0]` هنوز A است و `insertBefore(A, null)` آن را به انتها **منتقل** می‌کند (کپی نمی‌سازد): Z، B، C، A. گزینه‌ی ۱ تصور کرده A کپی می‌شود، گزینه‌ی ۲ مرجعِ `null` را بی‌اثر دانسته و گزینه‌ی ۴ فکر کرده Z قبل از A درج شده است.

۹۹ **گزینه‌ی ۳** آسان

`document.body.appendChild(div)` یک div تازه را **آخرین فرزند** body می‌کند؛ عنوان دست نمی‌خورد و چون div یک عنصر بلوکی است، در خط جدیدی زیر دکمه نمایش داده می‌شود. گزینه‌ی ۱ رفتار تغییر `textContent` عنوان را توصیف می‌کند، گزینه‌ی ۲ رفتار `prepend` را، و گزینه‌ی ۴ اشتباه است چون نبودن CSS فقط ظاهر خاصی به div نمی‌دهد، ولی متنش طبق معمول دیده می‌شود.

۱۰۰ **گزینه‌ی ۲** متوسط

`createElement` داخل listener است، پس هر کلیک یک div **جدید** با کلاس `box` می‌سازد؛ بعد از سه کلیک سه `.box` داریم. `appendChild` هر div را آخرین فرزند body می‌کند، یعنی بعد از تگ `<script>` ؛ پس آخرین عنصرِ body یک `DIV` است. گزینه‌ی ۱ فکر کرده فقط یک div ساخته می‌شود، گزینه‌ی ۳ تصور کرده div قبل از script قرار می‌گیرد و گزینه‌ی ۴ اثر کلیک‌ها را نادیده گرفته است.

۱۰۱ **گزینه‌ی ۳** سخت

این‌بار `createElement` **بیرون** از listener و فقط یک بار اجرا شده است؛ پس در کل یک div وجود دارد. کلیک اول آن را به body اضافه می‌کند و کلیک‌های بعدی همان div را دوباره append می‌کنند که فقط آن را به انتهای body **جابه‌جا** می‌کند؛ تعداد divها همیشه ۱ می‌ماند. گزینه‌ی ۱ اشتباه رایج است (فکر کردن به کپی شدن)، گزینه‌ی ۲ ترتیب اجرا را غلط فهمیده و گزینه‌ی ۴ تصور کرده append دوباره‌ی یک گره خطاست.

۱۰۲ **گزینه‌ی ۴** سخت

بعد از اولین Add، `Read` اضافه و کادر خالی می‌شود. کلیک دوم به شرط `trim() === ""` می‌خورد و `return` می‌کند، پس آیتم خالی ساخته نمی‌شود. کلیک سوم `Write` را با `prepend` به **ابتدای** لیست می‌برد؛ پس ۲ آیتم داریم و اولی `Write` است. گزینه‌های ۱ و ۲ کلیک با کادر خالی را هم شمرده‌اند و گزینه‌های ۲ و ۳ `prepend` را با افزودن به انتها اشتباه گرفته‌اند.

۱۰۳ **گزینه‌ی ۲** متوسط

`forEach` برای هر عضو یک `<li>` تازه می‌سازد و به انتهای لیست اضافه می‌کند؛ پس ۳ فرزند داریم. در آخرین دور `i` برابر ۲ است و عبارت از چپ به راست حساب می‌شود: `2 + 1` برابر عدد `3` و بعد `"3. JS"` . گزینه‌ی ۱ فکر کرده `+` از همان ابتدا رشته می‌چسباند، گزینه‌ی ۳ تصور کرده هر `appendChild` قبلی‌ها را پاک می‌کند و گزینه‌ی ۴ اندیسِ صفرمبنا را بدون `+ 1` در نظر گرفته است.

۱۰۴ **گزینه‌ی ۱** سخت

`map` یک **آرایه** برمی‌گرداند و `innerHTML` آن را به رشته تبدیل می‌کند؛ تبدیل آرایه به رشته عضوها را با **کاما** به هم می‌چسباند: `"<li>Apple</li>,<li>Banana</li>"` . مرورگر این را تفسیر می‌کند: دو `<li>` واقعی و یک گره‌ی متنیِ `","` بینشان؛ پس ۲ فرزند عنصری داریم و متن کل `Apple,Banana` است. گزینه‌ی ۲ کاما را فراموش کرده (این نتیجه با `join("")` به دست می‌آید)، گزینه‌ی ۳ کل رشته را یک عنصر دانسته و گزینه‌ی ۴ فکر کرده `innerHTML` تگ‌ها را تفسیر نمی‌کند.

۲-۵ حذف و جایگزینی عناصر

۱۰۵ **گزینه‌ی ۲** آسان

`el.remove()` روی خودِ عنصر صدا زده می‌شود و آن را با همه‌ی فرزندانش از DOM برمی‌دارد. `removeChild` و `replaceChild` روی **پدر** صدا زده می‌شوند و آرگومان لازم دارند. `delete` فقط برای حذف property از یک شیء است و روی یک متغیر هیچ اثری روی DOM ندارد.

۱۰۶ **گزینه‌ی ۱** آسان

`removeChild` همان گره‌ای را که حذف کرده **برمی‌گرداند**؛ پس `removed === a` برقرار است. لیست یک فرزند کمتر دارد (۱) و گره‌ی حذف‌شده هنوز متنش (`A`) را دارد، چون حذف از DOM آن را نابود نمی‌کند. گزینه‌ی ۲ فرض کرده مقدار برگشتی `undefined` است، گزینه‌ی ۳ فکر کرده محتوای گره پاک می‌شود و گزینه‌ی ۴ تصور کرده پدر برگردانده می‌شود.

۱۰۷ **گزینه‌ی ۳** آسان

امضای درست `parent.replaceChild(newChild, oldChild)` است: **اول جدید، بعد قدیمی** (درست مثل `insertBefore(جدید, مرجع)`). گزینه‌ی ۱ ترتیب را برعکس کرده که معمولاً به `NotFoundError` می‌رسد، گزینه‌ی ۲ پدر را بی‌دلیل آرگومان کرده و گزینه‌ی ۴ مشخص نکرده کدام فرزند باید عوض شود (این شکلِ تک‌آرگومانی مال `oldChild.replaceWith(newChild)` است).

۱۰۸ **گزینه‌ی ۴** متوسط

حذف `#a` درست است، چون فرزند مستقیم لیست است؛ پس `0` چاپ می‌شود. اما `#note` فرزند `list` نیست (هم‌سطح آن است) و `removeChild` در این حالت خطای `NotFoundError` می‌دهد؛ پس `done` چاپ نمی‌شود. گزینه‌ی ۱ فرض کرده هر عنصری را حذف می‌کند، گزینه‌ی ۲ خطا را به خط اول نسبت داده و گزینه‌ی ۳ نوع خطا را اشتباه گفته است (`note` پیدا شده و `null` نیست).

۱۰۹ **گزینه‌ی ۲** متوسط

`replaceChild(fresh, old)` عنصر B را برمی‌دارد و X را **در همان جای** آن می‌گذارد؛ پس متن لیست `AXC` است. مقدار برگشتی **گره‌ی قدیمی** است، یعنی B. گزینه‌ی ۱ فکر کرده گره‌ی جدید برگردانده می‌شود، گزینه‌ی ۳ جای آرگومان‌ها را برعکس فهمیده و گزینه‌ی ۴ تصور کرده X فقط قبل از B درج می‌شود و B می‌ماند.

۱۱۰ **گزینه‌ی ۳** سخت

آرگومان‌ها جابه‌جا نوشته شده‌اند: `replaceChild(old, fresh)` یعنی «`fresh` را با `old` عوض کن». اما `fresh` اصلاً فرزند `list` نیست، پس مرورگر خطای `NotFoundError` می‌دهد و `console.log` اجرا نمی‌شود. گزینه‌ی ۱ نتیجه‌ی نسخه‌ی درست (`replaceChild(fresh, old)`) است، گزینه‌ی ۲ فرض کرده خطا بی‌صدا نادیده گرفته می‌شود و گزینه‌ی ۴ نوع خطا را اشتباه گفته است (هر دو آرگومان گره‌اند و هیچ‌کدام `null` نیست).

۱۱۱ **گزینه‌ی ۱** سخت

C با A عوض می‌شود. اما A از قبل در صفحه است، پس از جای قبلی‌اش **منتقل** می‌شود (کپی نمی‌شود): ابتدا از اول لیست برداشته می‌شود و بعد جای C می‌نشیند. نتیجه B، A و دو فرزند است. گزینه‌ی ۲ تصور کرده A کپی می‌شود، گزینه‌ی ۳ فکر کرده چیزی تغییر نمی‌کند و گزینه‌ی ۴ ترتیب را اشتباه گفته است.

۱۱۲ **گزینه‌ی ۴** آسان

`replaceWith` روی **عنصر قدیمی** صدا زده می‌شود و آن را با گره‌ی جدید عوض می‌کند. در نسخه‌ی قدیمی باید روی پدرِ `old` صدا بزنیم و اول جدید، بعد قدیمی را بدهیم: `old.parentNode.replaceChild(fresh, old)`. گزینه‌ی ۱ ترتیب آرگومان‌ها را برعکس کرده، گزینه‌ی ۲ هم پدر و هم ترتیب را اشتباه گرفته و گزینه‌ی ۳ اصلاً جایگزین نمی‌کند (و چون `fresh` فرزند `old` نیست، خطا هم می‌دهد).

۱۱۳ **گزینه‌ی ۲** آسان

listener عنصر `#title` (یعنی `<h1>`) را پیدا و حذف می‌کند؛ دکمه دست نمی‌خورد و چون جای عنوان خالی شده، بالای صفحه قرار می‌گیرد. گزینه‌ی ۱ اثر `remove()` را نادیده گرفته، گزینه‌ی ۳ عنصر حذف‌شده را اشتباه گرفته و گزینه‌ی ۴ تصور کرده کل body حذف می‌شود.

۱۱۴ **گزینه‌ی ۳** متوسط

کلیک اول عنوان را حذف می‌کند و خروجی ندارد. در کلیک دوم `getElementById("title")` مقدار `null` برمی‌گرداند و `null.remove()` خطای `TypeError: Cannot read properties of null (reading 'remove')` می‌دهد. این خطا فقط همان اجرای listener را متوقف می‌کند؛ دکمه سر جایش است، پس در پایان ۰ عنوان و ۱ دکمه داریم. گزینه‌های ۱ و ۴ خطای کلیک دوم را ندیده‌اند و گزینه‌های ۲ و ۴ فکر کرده‌اند دکمه هم حذف می‌شود.

---

۱۱۵ **گزینه‌ی ۱** متوسط

با optional chaining (`?.`)، وقتی `element` برابر `null` است متد صدا زده نمی‌شود و خطایی رخ نمی‌دهد؛ در کلیک اول هم عنوان مثل قبل حذف می‌شود. در گزینه‌ی ۲ آرگومان اضافه هیچ اثری ندارد و `null.remove` باز هم `TypeError` می‌دهد. در گزینه‌ی ۳ عنوان فرزند مستقیم `document` نیست (فرزند body است)، پس همان کلیک اول `NotFoundError` می‌دهد. گزینه‌ی ۴ عنوان را حذف نمی‌کند (فقط خالی‌اش می‌کند) و روی `null` هم خطا می‌دهد.

---

۱۱۶ **گزینه‌ی ۳** سخت

خط ۳ اولین `<li>` را حذف می‌کند. در خط ۵ عنصر دیگر در صفحه نیست، ولی `remove()` روی عنصرِ جداشده **بی‌صدا هیچ کاری نمی‌کند**. اما در خط ۷، `first` دیگر فرزند `list` نیست و `removeChild` سخت‌گیر است: خطای `NotFoundError` می‌دهد و `C` چاپ نمی‌شود. گزینه‌ی ۲ همین سخت‌گیری را به `remove()` نسبت داده و گزینه‌ی ۴ آن را از `removeChild` هم انتظار نداشته است.

---

۱۱۷ **گزینه‌ی ۴** متوسط

`remove()` عنصر را از صفحه برمی‌دارد ولی متغیر `msg` هنوز به همان شیء اشاره می‌کند و متنش سالم است؛ پس `0 Saved!` چاپ می‌شود. با `append` همان عنصر دوباره به صفحه برمی‌گردد و `getElementById` دقیقاً همان شیء را پیدا می‌کند (`true`). گزینه‌ی ۱ فکر کرده متغیر بعد از حذف `null` می‌شود، گزینه‌ی ۲ حذف را بی‌اثر دانسته و گزینه‌ی ۳ تصور کرده عنصرِ برگردانده‌شده شیء دیگری است.

---

۱۱۸ **گزینه‌ی ۲** سخت

حذف یک عنصر یعنی حذف **کل زیردرختش**؛ پس `<h2>` و `<p>` هم دیگر در صفحه نیستند (۰) و `title.isConnected` برابر `false` است. اما در حافظه زیردرخت سالم می‌ماند: `card` هنوز دو فرزندش را دارد. گزینه‌ی ۱ فکر کرده فرزندان در صفحه باقی می‌مانند، گزینه‌ی ۳ تصور کرده فرزندان نابود می‌شوند و گزینه‌ی ۴ `title` را هنوز متصل به صفحه دانسته است.

---

۱۱۹ **گزینه‌ی ۴** متوسط

`remove()` فقط عنصر را از درخت DOM جدا می‌کند و به متغیرهای ما دست نمی‌زند؛ `el` همچنان به همان شیء اشاره می‌کند و می‌شود آن را دوباره به صفحه اضافه کرد. پس گزینه‌ی ۴ نادرست است. سه جمله‌ی دیگر رفتار واقعی این متدها را توصیف می‌کنند.

---

۱۲۰ **گزینه‌ی ۱** آسان

فقط `remove()` عنصر را واقعاً از درخت DOM بیرون می‌برد، پس جست‌وجوها دیگر آن را پیدا نمی‌کنند. `display = "none"` و `hidden = true` فقط عنصر را **نامرئی** می‌کنند و `textContent = ""` فقط محتوایش را خالی می‌کند؛ در این سه حالت عنصر هنوز در DOM است و `querySelector` پیدایش می‌کند.

---

۱۲۱ **گزینه‌ی ۴** آسان

`list.remove()` خودِ لیست را (همراه فرزندانش) از صفحه برمی‌دارد، در حالی که خواسته‌ی ما ماندن `<ul>` بود. سه گزینه‌ی دیگر فقط محتوا را خالی می‌کنند: `innerHTML = ""` همه‌ی فرزندان را دور می‌ریزد، `replaceChildren()` بدون آرگومان یعنی «هیچ فرزندی نداشته باش» و حلقه‌ی `while` آن‌قدر اولین فرزند را حذف می‌کند تا چیزی نماند.

---

۱۲۲ **گزینه‌ی ۲** سخت

`firstChild` همه‌ی انواع گره را حساب می‌کند، از جمله گره‌های متنیِ فاصله و خط جدید بین تگ‌ها. این لیست ۵ گره‌ی فرزند دارد: متنِ فاصله، `<li>A`، متنِ فاصله، `<li>B` و متنِ فاصله‌ی آخر. حلقه همه را یکی‌یکی حذف می‌کند؛ پس `count` برابر ۵ و در پایان هیچ گره‌ای نمانده است. گزینه‌ی ۱ فقط عنصرها را شمرده (منطق `children`)، گزینه‌ی ۳ فکر کرده گره‌های متنی حذف نمی‌شوند و گزینه‌ی ۴ فاصله‌ی آخر را جا انداخته است.

---

۱۲۳ **گزینه‌ی ۱** سخت

`getElementsByClassName` یک مجموعه‌ی **زنده** است. وقتی `i = 0` است، آیتم ۱ حذف می‌شود و مجموعه کوچک می‌شود: حالا `items[0]` آیتم ۲ است، ولی `i` به ۱ می‌رود و آیتم ۳ حذف می‌شود. بعد `i = 2` است و طول مجموعه هم ۲؛ حلقه تمام می‌شود و آیتم‌های ۲ و ۴ می‌مانند. گزینه‌ی ۲ همان چیزی است که برنامه‌نویس انتظار داشت (با `querySelectorAll(...).forEach` به دست می‌آید)، گزینه‌ی ۳ آیتم‌های حذف‌شده را جابه‌جا گفته و گزینه‌ی ۴ فرض کرده فقط اولین آیتم حذف می‌شود.

---

۱۲۴ **گزینه‌ی ۳** متوسط

کلیک روی دکمه تا `<ul>` بالا می‌رود (bubbling) و listener پدر اجرا می‌شود. `e.target` خودِ دکمه است و `parentElement` آن، `<li>` «Code»؛ پس کل این آیتم همراه دکمه‌اش حذف می‌شود و ۲ آیتم و ۲ دکمه می‌ماند. گزینه‌ی ۱ فکر کرده فقط دکمه حذف می‌شود (این نتیجه‌ی `e.target.remove()` است)، گزینه‌ی ۲ دکمه‌ی آیتم حذف‌شده را هنوز شمرده و گزینه‌ی ۴ تصور کرده خودِ `<ul>` (صاحب listener) حذف می‌شود.

---

۱۲۵ **گزینه‌ی ۴** متوسط

callback تایمر حتی با تأخیر `0` بعد از تمام شدن کد همگامِ فعلی اجرا می‌شود (event loop در فصل اول). پس اولین `console.log` هنوز اعلان را در صفحه می‌بیند (`1`). بعد تایمر اول اعلان را حذف می‌کند و تایمر دوم (۱۰ میلی‌ثانیه بعد) `0` چاپ می‌کند. گزینه‌ی ۱ فکر کرده `setTimeout(..., 0)` فوراً اجرا می‌شود، گزینه‌ی ۲ حذف را بی‌اثر دانسته و گزینه‌ی ۳ ترتیب را برعکس کرده است.

۶-۲ ویژگی‌ها (Attributes) و کلاس‌ها: setAttribute و classList

---

۱۲۶ **گزینه‌ی ۲** آسان

`getAttribute` برای attributeی که روی عنصر وجود ندارد مقدار `null` برمی‌گرداند (و در غیر این صورت همیشه یک رشته). گزینه‌ی ۱ رفتار خواندن یک property ناموجود از شیء را با `getAttribute` اشتباه گرفته، گزینه‌ی ۳ خروجی `hasAttribute` را حدس زده و گزینه‌ی ۴ اشتباه است چون نبودن attribute خطا نیست.

---

۱۲۷ **گزینه‌ی ۳** آسان

فقط `removeAttribute` خودِ attribute را حذف می‌کند (بعد از آن `hasAttribute("title")` برابر `false` است). گزینه‌ی ۱ مقدار را رشته‌ی `"null"` می‌کند و گزینه‌ی ۲ آن را خالی می‌کند، ولی در هر دو attribute هنوز وجود دارد. گزینه‌ی ۴ با attributeها کاری ندارد و فقط کلاسی به نام `title` را (اگر باشد) از فهرست کلاس‌ها برمی‌دارد.

---

۱۲۸ **گزینه‌ی ۴** آسان

`setAttribute` اگر attribute وجود داشته باشد مقدارش را **عوض** می‌کند (`href` از `/home` به `/about`) و اگر نباشد آن را **اضافه** می‌کند (`target`). حالا لینک سه attribute دارد: `id`، `href` و `target`. گزینه‌ی ۱ فکر کرده مقدار قبلی عوض نمی‌شود، گزینه‌ی ۲ `id` را نشمرده و گزینه‌ی ۳ تصور کرده `setAttribute` فقط attributeهای موجود را تغییر می‌دهد.

---

۱۲۹ **گزینه‌ی ۲** متوسط

حذف attributeی که وجود ندارد **خطا نمی‌دهد** و فقط کاری انجام نمی‌شود. `hasAttribute` همیشه یک مقدار بولی برمی‌گرداند: `title` نیست (`false`) و `id` هست (`true`). گزینه‌ی ۱ رفتار سخت‌گیرانه‌ی `removeChild` را به `removeAttribute` نسبت داده، گزینه‌ی ۳ وجود `title` را فرض کرده و گزینه‌ی ۴ فکر کرده `hasAttribute` مقدار `undefined` برمی‌گرداند.

---

۱۳۰ **گزینه‌ی ۱** متوسط

`getAttribute` همیشه رشته (یا `null`) برمی‌گرداند، پس `"300"` از نوع `string` است. اما property متناظر، `img.width`، مقدار را با نوع واقعی‌اش یعنی عدد `300` می‌دهد. گزینه‌های ۲ و ۴ فکر کرده‌اند `getAttribute` عدد برمی‌گرداند و گزینه‌ی ۳ property را هم رشته دانسته است.

---

۱۳۱ **گزینه‌ی ۳** سخت

`disabled` یک attribute بولی است و فقط **حضورش** مهم است؛ مقدار `"false"` هم حضور حساب می‌شود، پس دکمه غیرفعال است و `true` چاپ می‌شود. با `removeAttribute` attribute حذف و دکمه فعال می‌شود (`false`). گزینه‌ی ۱ اشتباه رایج است (خیال کرده رشته‌ی `"false"` یعنی فعال)، گزینه‌ی ۲ کاملاً برعکس است و گزینه‌ی ۴ اثر `removeAttribute` را نادیده گرفته است.

۱۳۲ **گزینه‌ی ۲** آسان

`setAttribute("src", "dambooldor2.png")` فقط مقدار `src` همان `<img>` را عوض می‌کند؛ `width` و `height` دست نخورده‌اند، پس تصویر جدید هم ۳۰۰ در ۳۰۰ نمایش داده می‌شود و دکمه سر جایش است. هیچ عنصری حذف یا اضافه نمی‌شود؛ پس گزینه‌های ۱ و ۴ غلط‌اند، و گزینه‌ی ۳ اشتباه است چون attributeهای اندازه هنوز روی تگ هستند.

۱۳۳ **گزینه‌ی ۱** متوسط

listener مقدار attribute `src` را به `dambooldor2.png` تغییر داده و `getAttribute` همان متن نوشته‌شده را برمی‌گرداند؛ `width` هم دست نخورده و هنوز `"300"` است. گزینه‌ی ۲ تغییر را ندیده، گزینه‌ی ۳ فکر کرده تغییر `src` بقیه‌ی attributeها را پاک می‌کند و گزینه‌ی ۴ تصور کرده `setAttribute` یک attribute دوم کنار قبلی می‌سازد.

۱۳۴ **گزینه‌ی ۳** سخت

`getAttribute` همان متنی را می‌دهد که در attribute نوشته شده (`cat.png`)، ولی property `img.src` آدرس را **کامل و مطلق** می‌کند: آدرس نسبی `cat.png` نسبت به پوشه‌ی صفحه (`/gallery/`) حل می‌شود. گزینه‌ی ۱ فرقی بین این دو قائل نشده، گزینه‌ی ۲ جای آن‌ها را برعکس کرده و گزینه‌ی ۴ آدرس را نسبت به ریشه‌ی سایت حل کرده، در حالی که مبنا پوشه‌ی فعلی صفحه است.

۱۳۵ **گزینه‌ی ۴** متوسط

property `value` مقدار **فعلی** کادر (چیزی که کاربر تایپ کرده) را نگه می‌دارد، ولی attribute `value` همان مقدار **اولیه**‌ی نوشته‌شده در HTML است و با تایپ کاربر عوض نمی‌شود. گزینه‌ی ۱ این دو را همیشه یکی دانسته، گزینه‌ی ۲ تایپ کاربر را نادیده گرفته و گزینه‌ی ۳ نقش آن‌ها را برعکس کرده است.

۱۳۶ **گزینه‌ی ۴** سخت

`pic.src` آدرس کامل (`https://quiz.example/day.png`) را برمی‌گرداند، پس شرط `pic.src === "day.png"` **هیچ‌وقت** برقرار نیست و هر بار شاخه‌ی `else` اجرا می‌شود که `src` را دوباره `day.png` می‌کند. نتیجه این است که تصویر هرگز عوض نمی‌شود. گزینه‌ی ۱ همان رفتاری است که برنامه‌نویس می‌خواست (با `pic.getAttribute("src") === "day.png"` به دست می‌آید)؛ گزینه‌های ۲ و ۳ هم شرط را دست‌کم یک بار درست فرض کرده‌اند.

۱۳۷ **گزینه‌ی ۴** سخت

`data-product-id` در `dataset` با نام camelCase یعنی `productId` دیده می‌شود و مقدارش **رشته‌ی** `"42"` است؛ پس `"42" + 1` به‌صورت رشته‌ای `"421"` می‌شود. مقداردهی `dataset.inStock` هم یک attribute واقعی به نام `data-in-stock` روی تگ می‌سازد. گزینه‌ی ۱ مقدار را عدد فرض کرده، گزینه‌ی ۲ فکر کرده نام خط‌تیره‌دار در `dataset` پیدا نمی‌شود و گزینه‌های ۲ و ۳ تصور کرده‌اند `dataset` روی attributeها اثری ندارد.

۱۳۸ **گزینه‌ی ۱** آسان

`classList.add` کلاس‌های قبلی (`card`) را نگه می‌دارد، می‌تواند چند کلاس را یک‌جا بگیرد و کلاس **تکراری** اضافه نمی‌کند؛ پس فهرست نهایی `card active big` با ۳ کلاس است. گزینه‌های ۲ و ۴ تکرار را مجاز دانسته‌اند و گزینه‌های ۳ و ۴ فکر کرده‌اند `add` کلاس‌های قبلی را پاک می‌کند (کاری که `className =` انجام می‌دهد).

۱۳۹ **گزینه‌ی ۴** متوسط

حذف کلاسی که وجود ندارد (`ghost`) خطا نمی‌دهد و `remove` می‌تواند چند کلاس را یک‌جا حذف کند؛ پس فقط `b` می‌ماند و `contains("b")` برابر `true` است. گزینه‌ی ۱ فکر کرده حذف کلاس ناموجود خطاست، گزینه‌ی ۲ فرض کرده `remove` با چند آرگومان کار نمی‌کند و چیزی حذف نمی‌شود و گزینه‌ی ۳ نتیجه‌ی `contains` را اشتباه گفته است.

۱۴۰ **گزینه‌ی ۲** متوسط

`toggle` **وضعیت جدید** را برمی‌گرداند: بار اول کلاس اضافه می‌شود (`true`)، بار دوم حذف (`false`). با آرگومان دوم `true` ، `toggle` فقط **اضافه** می‌کند؛ پس کلاس اضافه و `true` چاپ می‌شود و در پایان `className` برابر `dark` است. گزینه‌ی ۱ فکر کرده `toggle` چیزی برنمی‌گرداند، گزینه‌ی ۳ تصور کرده خروجی، وضعیتِ **قبل** از تغییر است و گزینه‌ی ۴ `toggle` را مثل `add` فرض کرده که هیچ‌وقت حذف نمی‌کند.

۱۴۱ **گزینه‌ی ۳** متوسط

`setAttribute("class", "red")` **کل** کلاس‌ها را جایگزین می‌کند و `note` و `big` از بین می‌روند. بعد `classList.add` کلاس `bold` را اضافه می‌کند و `+= " small"` رشته‌ی کلاس را ادامه می‌دهد: `red bold small` . گزینه‌ی ۱ فکر کرده `setAttribute` کلاس اضافه می‌کند، گزینه‌ی ۲ تصور کرده `+=` کلاس‌های `classList` را پاک می‌کند و گزینه‌ی ۴ `red` را جا انداخته است.

۱۴۲ **گزینه‌ی ۱** سخت

نقطه فقط در **انتخابگرهای CSS** (مثل `querySelector(".active")`) معنای «کلاس» دارد. `classList.add` نام خام کلاس را می‌گیرد؛ پس کلاسی دقیقاً به نام `.active` (با نقطه) ساخته می‌شود و کلاس `active` وجود ندارد. گزینه‌ی ۲ فکر کرده نقطه خودکار حذف می‌شود، گزینه‌ی ۳ دو نام متفاوت را یکی دانسته و گزینه‌ی ۴ تصور کرده نقطه در نام کلاس خطاست، در حالی که فقط فاصله (whitespace) و رشته‌ی خالی ممنوع‌اند.

۱۴۳ **گزینه‌ی ۲** سخت

هر آرگومانِ `classList.add` باید **یک** نام کلاس بدون فاصله باشد. رشته‌ی `"btn primary"` فاصله دارد، پس مرورگر خطای `InvalidCharacterError` می‌دهد و `console.log` اجرا نمی‌شود. شکل درست `btn.classList.add("btn", "primary")` است. گزینه‌ی ۱ رفتار `className = "btn primary"` را توصیف می‌کند و گزینه‌های ۳ و ۴ حدس‌هایی هستند که هیچ‌کدام رخ نمی‌دهند.

۱۴۴ **گزینه‌ی ۳** متوسط

`toggle` دقیقاً همین کار را می‌کند: اگر کلاس نباشد اضافه و اگر باشد حذف می‌کند، و وضعیت جدید را هم برمی‌گرداند. `add` فقط اضافه می‌کند (کلیک دوم حالت تاریک را خاموش نمی‌کند)، `contains` فقط بررسی می‌کند و چیزی را تغییر نمی‌دهد و `replace` دو آرگومان (کلاس قدیم و جدید) می‌گیرد و یک کلاس را با دیگری عوض می‌کند.

۱۴۵ **گزینه‌ی ۱** سخت

`contains` فقط **بررسی** می‌کند و `true` یا `false` برمی‌گرداند؛ هیچ تغییری در کلاس‌ها نمی‌دهد. پس گزینه‌ی ۱ نادرست و پاسخ است. گزینه‌ی ۲ درست است، چون `className` کل رشته‌ی کلاس را جایگزین می‌کند؛ گزینه‌ی ۳ هم درست است و گزینه‌ی ۴ همان رفتار واقعی `toggle` است (وضعیت جدید، یعنی «نیست»، را برمی‌گرداند).

۱۴۶ **گزینه‌ی ۴** آسان

با کلاس‌ها، «ظاهر» (CSS) از «رفتار» (JavaScript) جدا می‌ماند؛ همان تفکیک سه‌لایه‌ی اسلاید ۳. برگرداندن وضعیت با یک `remove` یا `toggle` انجام می‌شود و تغییر ظاهر فقط در یک جای CSS لازم است. گزینه‌ی ۱ غلط است چون `el.style` کاملاً کار می‌کند، گزینه‌ی ۲ هم غلط است (با `style.color` هم می‌شود رنگ را عوض کرد) و گزینه‌ی ۳ برعکس است: کلاس بدون تعریف در CSS هیچ اثری ندارد.

پاسخ‌های فصل سوم: Node.js و TypeScript

کلید سریع فصل سوم

۱۴۷	۳	۱۴۸	۴	۱۴۹	۲	۱۵۰	۱	۱۵۱	۲
۱۵۲	۱	۱۵۳	۱	۱۵۴	۳	۱۵۵	۴	۱۵۶	۲
۱۵۷	۳	۱۵۸	۴	۱۵۹	۱	۱۶۰	۳	۱۶۱	۲
۱۶۲	۴	۱۶۳	۳	۱۶۴	۲	۱۶۵	۴	۱۶۶	۱
۱۶۷	۳	۱۶۸	۱	۱۶۹	۴	۱۷۰	۳	۱۷۱	۲
۱۷۲	۳	۱۷۳	۱	۱۷۴	۴	۱۷۵	۲	۱۷۶	۳
۱۷۷	۱	۱۷۸	۴	۱۷۹	۲	۱۸۰	۳	۱۸۱	۴
۱۸۲	۱	۱۸۳	۲	۱۸۴	۳	۱۸۵	۲	۱۸۶	۴
۱۸۷	۱	۱۸۸	۱	۱۸۹	۳	۱۹۰	۲	۱۹۱	۲
۱۹۲	۳	۱۹۳	۱	۱۹۴	۴	۱۹۵	۴	۱۹۶	۱
۱۹۷	۲	۱۹۸	۴	۱۹۹	۱	۲۰۰	۲	۲۰۱	۳
۲۰۲	۳	۲۰۳	۲	۲۰۴	۱	۲۰۵	۳	۲۰۶	۱
۲۰۷	۳	۲۰۸	۴	۲۰۹	۳	۲۱۰	۴	۲۱۱	۲
۲۱۲	۱	۲۱۳	۲	۲۱۴	۳	۲۱۵	۲	۲۱۶	۳
۲۱۷	۳	۲۱۸	۱	۲۱۹	۱	۲۲۰	۲	۲۲۱	۴
۲۲۲	۱	۲۲۳	۴	۲۲۴	۴	۲۲۵	۴	۲۲۶	۱
۲۲۷	۲	۲۲۸	۳	۲۲۹	۱	۲۳۰	۴	۲۳۱	۲
۲۳۲	۳	۲۳۳	۱	۲۳۴	۱	۲۳۵	۳	۲۳۶	۴
۲۳۷	۱	۲۳۸	۲	۲۳۹	۴	۲۴۰	۲	۲۴۱	۴
۲۴۲	۳	۲۴۳	۴	۲۴۴	۱	۲۴۵	۲	۲۴۶	۲
۲۴۷	۴	۲۴۸	۱	۲۴۹	۲	۲۵۰	۳	۲۵۱	۴
۲۵۲	۳	۲۵۳	۱	۲۵۴	۳	۲۵۵	۲	۲۵۶	۴
۲۵۷	۳	۲۵۸	۲	۲۵۹	۴	۲۶۰	۱	۲۶۱	۳
۲۶۲	۳	۲۶۳	۴	۲۶۴	۱	۲۶۵	۴	۲۶۶	۲
۲۶۷	۳	۲۶۸	۲	۲۶۹	۱	۲۷۰	۳	۲۷۱	۴
۲۷۲	۲	۲۷۳	۱						

۳-۱ محدودیت‌های JavaScript و تولد Node.js

۱۴۷ **گزینه‌ی ۳** آسان

سه کارت اسلاید ۲ این‌ها هستند: «Make console applications with JS» (گزینه‌ی ۱)، «Learn how JS runs on servers» (گزینه‌ی ۲) و «Solve the old JS typing issues» (گزینه‌ی ۴). طراحی ظاهر صفحه با CSS اصلاً موضوع این جلسه نیست؛ این جلسه درباره‌ی بردن JavaScript به بیرون از مرورگر (Node.js) و TypeScript است.

۱۴۸ **گزینه‌ی ۴** آسان

ماجرا برعکس است: JavaScript در اصل **برای مرورگر** و کارهای ساده‌ی آن طراحی شده بود و مشکلش این بود که برای برنامه‌های کامل و منطق backend ساخته نشده بود. گزینه‌های ۱، ۲ و ۳ دقیقاً سه مورد از پنج محدودیت اسلاید ۳ هستند (دو مورد دیگر: محدود بودن به کارهای ساده‌ی مرورگر، و ابزار محدود و مقیاس‌پذیری ضعیف).

۱۴۹ **گزینه‌ی ۲** آسان

اسلاید ۴ می‌گوید «Engine improvements like Google's V8 made JavaScript fast enough for server workloads» و اسلاید ۵ می‌گوید «Powered by Google's fast V8 engine» Node.js است. بقیه‌ی گزینه‌ها هم موتورهای واقعی JavaScript هستند (مال Firefox، Safari و Edge قدیمی)، ولی Node.js از هیچ‌کدام استفاده نمی‌کند.

۱۵۰ **گزینه‌ی ۱** متوسط

در non-blocking I/O، تنها thread برنامه (گارسون) کار کُند I/O را به سیستم (آشپزخانه) می‌سپارد، منتظر نمی‌ماند و وقتی نتیجه آماده شد، با یک رویداد (زنگ) callback اجرا می‌شود. گزینه‌ی ۲ همان مدل blocking است. گزینه‌ی ۳ مدل «یک thread برای هر درخواست» است که Node.js از آن استفاده نمی‌کند. در گزینه‌ی ۴ گارسون خودش همه‌ی کار را انجام می‌دهد، پس دوباره همه منتظر یک نفر می‌مانند.

۱۵۱ **گزینه‌ی ۲** متوسط

`fs.readFile` غیرمسدودکننده (non-blocking) است: خواندن فایل را شروع می‌کند و منتظر نمی‌ماند، پس اول `end` چاپ می‌شود. وقتی خواندن فایل تمام شد، callback اجرا می‌شود و `file read` می‌آید. گزینه‌ی ۱ رفتار `readFileSync` (مسدودکننده) است. گزینه‌ی ۴ غلط است چون callback بالاخره اجرا می‌شود و برنامه تا تمام شدن کارهای در جریان بسته نمی‌شود.

---

۱۵۲ **گزینه‌ی ۱** متوسط

در گزینه‌ی ۱، `start` و `end` همگام چاپ می‌شوند و callback غیرمسدودکننده‌ی `readFile` بعداً `done` را چاپ می‌کند. گزینه‌ی ۲ با `readFileSync` منتظر می‌ماند، پس اول `start`، بعد `done` و در آخر `end` چاپ می‌شود. در گزینه‌ی ۳، `start` داخل callback است و آخر از همه می‌آید (اول `end`، بعد `done`، بعد `start`). در گزینه‌ی ۴ هم `end` داخل callback است، پس `end` بعد از `done` چاپ می‌شود.

---

۱۵۳ **گزینه‌ی ۱** سخت

JavaScript تک‌نخی است. حتی با زمان `0`، callback تایمر فقط وقتی اجرا می‌شود که کد همگام فعلی تمام شده باشد. حلقه‌ی `while` حدود ۱۰۰ میلی‌ثانیه تنها thread را **مسدود** می‌کند، پس `timer` نمی‌تواند وسط حلقه (گزینه‌ی ۳) یا قبل از همه (گزینه‌ی ۲) چاپ شود. گزینه‌ی ۴ هم غلط است، چون تایمر بالاخره اجرا می‌شود. این دقیقاً همان مشکل blocking در اسلاید ۳ است.

---

۱۵۴ **گزینه‌ی ۳** آسان

Node.js یک **runtime** است: موتور V8 به‌اضافه‌ی ابزارهایی مثل `fs` و `http` که اجرای JavaScript را روی سرور و در ترمینال ممکن می‌کند. زبانش همان JavaScript است (رد گزینه‌ی ۱)، کتابخانه‌ی رابط کاربری نیست (React یک کتابخانه‌ی UI است؛ رد گزینه‌ی ۲) و مرورگر هم نیست، چون اصلاً صفحه و DOM ندارد (رد گزینه‌ی ۴).

---

۱۵۵ **گزینه‌ی ۴** متوسط

Node.js شیء `window` ندارد، ولی `typeof` روی متغیر تعریف‌نشده خطا نمی‌دهد و فقط `"undefined"` برمی‌گرداند (رد گزینه‌ی ۲). `globalThis` شیء سراسری استاندارد است که در هر دو محیط وجود دارد (رد گزینه‌ی ۳) و `process` هم مخصوص Node.js است و وجود دارد. گزینه‌ی ۱ جای مرورگر و Node.js را عوض کرده است.

---

۱۵۶ **گزینه‌ی ۲** سخت

خط ۱ فقط `undefined` چاپ می‌کند، چون `typeof` روی متغیر تعریف‌نشده خطا نمی‌دهد. خط‌های ۲ و ۳ هم سالم‌اند، چون `os` یک ماژول داخلی Node.js است. در خط ۴ واقعاً از `window` استفاده شده و Node.js چنین شیئی ندارد، پس `ReferenceError: window is not defined` رخ می‌دهد و برنامه همان‌جا متوقف می‌شود. برای همین خط ۵ هرگز اجرا نمی‌شود و گزینه‌ی «بدون خطا» هم غلط است.

---

۱۵۷ **گزینه‌ی ۳** متوسط

اسلاید ۴: «The rise of AJAX and JSON showed JS could efficiently handle asynchronous operations». با AJAX صفحه در پس‌زمینه و بدون بارگذاری دوباره داده می‌گیرد، یعنی کاری ناهمگام. این ربطی به چندنخی شدن ندارد (رد گزینه‌ی ۱)؛ امروزه بیشتر داده با JSON جابه‌جا می‌شود نه XML (رد گزینه‌ی ۲)؛ و همین موفقیت‌ها الهام‌بخش بردن JS به سرور شد، نه برعکس (رد گزینه‌ی ۴).

---

۱۵۸ **گزینه‌ی ۴** متوسط

در مدل رویدادمحور، برنامه برای رویدادها (درخواست، پایان I/O، تایمر و...) تابع callback ثبت می‌کند و Node.js وقتی رویداد رخ داد، آن تابع را صدا می‌زند. گزینه‌ی ۱ تعریف مدل blocking است و گزینه‌ی ۲ مدل «یک thread برای هر درخواست». گزینه‌ی ۳ رویداد را فقط به کلیک موس در صفحه محدود کرده، در حالی که Node.js اصلاً صفحه‌ای ندارد و رویدادهایش از جنس شبکه، فایل و تایمرند.

---

۱۵۹ **گزینه‌ی ۱** سخت

Node.js صفحه‌ی وب و DOM ندارد؛ `document` و `window` در آن تعریف نشده‌اند و استفاده از آن‌ها `ReferenceError` می‌دهد (شیء سراسری‌اش `globalThis` یا `global` است). گزینه‌های ۲ و ۳ مستقیماً در اسلاید ۵ آمده‌اند. گزینه‌ی ۴ هم درست است، چون JavaScript تک‌نخی است و کار محاسباتی سنگین همان یک thread را مسدود می‌کند.

---

۱۶۰ **گزینه‌ی ۳** متوسط

اسلاید ۵: «Event-driven, non-blocking I/O model». چون Node.js منتظر تمام شدن I/O نمی‌ماند و با رویداد و callback از نتیجه باخبر می‌شود، یک thread برای درخواست‌های زیاد کافی است. گزینه‌ی ۱ با «یک thread» تناقض دارد و گزینه‌های ۲ و ۴ (همگام و مسدودکننده) دقیقاً همان چیزی هستند که باعث معطل ماندن درخواست‌ها می‌شود.

۱۶۱ **گزینه‌ی ۲** سخت

Node.js همچنان **تک‌نخی** است و برای هر درخواست thread نمی‌سازد؛ راه‌حلش برای مشکل blocking، مدل **رویدادمحور و غیرمسدودکننده** است: I/O در پس‌زمینه انجام می‌شود و نتیجه با callback می‌رسد. سه جفت دیگر درست‌اند: TypeScript برای مشکلات نوع، ماژول‌ها و npm برای سازمان‌دهی و استفاده‌ی دوباره از کد، و خود Node.js برای بیرون آمدن از مرورگر.

۱۶۲ **گزینه‌ی ۴** آسان

پنج مورد اسلاید ۵ این‌هاست: موتور V8، I/O رویدادمحور و غیرمسدودکننده، یک زبان برای frontend و backend، اکوسیستم عظیم npm، و تبدیل JavaScript به یک پلتفرم کامل. بررسی نوع در زمان کامپایل کار **TypeScript** است، نه Node.js. خود Node.js کد را بدون بررسی نوع اجرا می‌کند.

۱۶۳ **گزینه‌ی ۳** سخت

`createServer` و `listen` فقط callbackها را **ثبت** می‌کنند و صبر نمی‌کنند، پس اول `setup done` چاپ می‌شود (رد گزینه‌های ۱ و ۴). وقتی سرور آماده‌ی گوش دادن شد، رویدادِ آماده شدن رخ می‌دهد و `listening` چاپ می‌شود؛ بعد سرور بسته می‌شود و `closed` می‌آید. callback `createServer` فقط با رسیدن یک **درخواست** اجرا می‌شود و چون درخواستی نیامده، هرگز چاپ نمی‌شود `request received` (رد گزینه‌ی ۲). این همان event-driven بودن است: کد فقط به رویدادهایی که واقعاً رخ می‌دهند واکنش نشان می‌دهد.

۳-۲ ویژگی‌های Node.js

۱۶۴ **گزینه‌ی ۲** آسان

هشت برچسب نمودار این‌ها هستند: Single-Threaded، Event-Driven، No Buffering، Asynchronous. Node.js **Single-Threaded** (تک‌نخی) است، نه Multi-Threaded؛ کد JavaScript روی یک thread اجرا می‌شود. گزینه‌های ۱، ۳ و ۴ عیناً روی نمودار هستند.

۱۶۵ **گزینه‌ی ۴** آسان

No Buffering یعنی Node.js به‌جای جمع کردن کل داده در حافظه (buffer کردن)، آن را به‌صورت stream و تکه‌تکه خروجی می‌دهد. این به معنای استفاده نکردن از حافظه نیست (رد گزینه‌ی ۱) و نگه داشتن هر تکه در یک متغیر هم کاملاً ممکن است (رد گزینه‌ی ۳). اتفاقاً همین روش باعث می‌شود Node.js فایل‌های خیلی بزرگ را راحت پردازش کند (رد گزینه‌ی ۲).

۱۶۶ **گزینه‌ی ۱** متوسط

خواندن stream ناهمگام است، پس اول کد همگام اجرا می‌شود و `reading...` می‌آید (رد گزینه‌های ۳ و ۴). بعد فایل ۱۰ حرفی در تکه‌های حداکثر ۴ حرفی خوانده می‌شود: `abcd`، `efgh` و `ij`، و برای هر تکه یک رویداد `data` رخ می‌دهد. در پایان رویداد `end` و `done` می‌آید. گزینه‌ی ۲ فرض کرده کل فایل یکجا (buffer شده) خوانده می‌شود، که برخلاف ایده‌ی stream است.

۱۶۷ **گزینه‌ی ۳** سخت

`on` فقط شنونده را ثبت می‌کند و چیزی چاپ نمی‌کند، پس اول `Waiting...` می‌آید (رد گزینه‌ی ۲). `emit` شنونده‌ها را **همگام** و همان لحظه اجرا می‌کند، پس `Preparing pizza` قبل از `Done` چاپ می‌شود (رد گزینه‌ی ۱ که `emit` را مثل `setTimeout` فرض کرده). برای رویداد `cancel` هیچ شنونده‌ای ثبت نشده؛ در این حالت `emit` فقط `false` برمی‌گرداند و خطایی رخ نمی‌دهد (رد گزینه‌ی ۴).

۱۶۸ **گزینه‌ی ۱** آسان

Cross-Platform یعنی «چندسکویی»: Node.js برای Windows، Linux و macOS وجود دارد و همان کد JavaScript روی همه‌شان اجرا می‌شود (`process.platform` نام سیستم‌عامل فعلی را می‌دهد). گزینه‌ی ۲ «سکو» را با مرورگر و سرور اشتباه گرفته؛ گزینه‌ی ۳ دقیقاً برعکس این ویژگی است؛ و Node.js فقط JavaScript اجرا می‌کند، نه زبان‌های دیگر (رد گزینه‌ی ۴).

۱۶۹ **گزینه‌ی ۴** متوسط

`fs/promises` نسخه‌ی Promiseمحور `fs` است. `main()` تا اولین `await` جلو می‌رود؛ چون نوشتن فایل کاری ناهمگام است، کنترل به بیرون برمی‌گردد و `main started` چاپ می‌شود. بعد از کامل شدن نوشتن و خواندن، `Hello file` چاپ می‌شود. گزینه‌ی ۱ فرض کرده `await` کل برنامه را متوقف می‌کند، در حالی که فقط ادامه‌ی همان تابع `async` منتظر می‌ماند. گزینه‌های ۲ و ۳ هم یکی از دو خروجی را جا انداخته‌اند؛ هر دو خط چاپ می‌شوند.

---

۱۷۰ **گزینه‌ی ۳** آسان

برنامه‌ی اجرایی Node.js در ترمینال `node` نام دارد و نام فایل را به آن می‌دهیم: `node app.js` . دستور `npm` مدیر بسته است و `npm app.js` خطای «Unknown command» می‌دهد. `run` و `js` هم دستورهای Node.js نیستند.

---

۱۷۱ **گزینه‌ی ۲** متوسط

`node` بدون آرگومان، (Read-Eval-Print Loop) **REPL** را باز می‌کند؛ مثلاً اگر `1 + 2` بنویسی، `3` نشان می‌دهد و با `.exit` بیرون می‌آیی. چاپ نسخه کار `node -v` است (گزینه‌ی ۱)، ساختن `package.json` کار `npm init` است (گزینه‌ی ۳) و نیاز به نام فایل فقط برای اجرای یک فایل است (گزینه‌ی ۴).

---

۱۷۲ **گزینه‌ی ۳** سخت

در CommonJS نوشتن پسوند `.js` در `require` اختیاری است، پس `./math` و `./math.js` هر دو به همان فایل می‌رسند و خطایی رخ نمی‌دهد (رد گزینه‌ی ۴). Node.js هر ماژول را فقط **بار اول** اجرا و نتیجه را cache می‌کند؛ پس `math.js loaded` فقط یک بار چاپ می‌شود (رد گزینه‌ی ۱). کد بالای فایل `math.js` هنگام اولین `require` واقعاً اجرا می‌شود، پس گزینه‌ی ۲ هم غلط است.

---

۱۷۳ **گزینه‌ی ۱** سخت

چیزی که `require` برمی‌گرداند همیشه `module.exports` است. `exports` در ابتدا فقط یک ارجاع (میان‌بُر) به همان شیء است. در `good.js` یک ویژگی به همان شیء اضافه شده، پس `{ hello: 'hi' }` صادر می‌شود. در `bad.js` متغیر محلی `exports` به یک شیء **جدید** اشاره داده شده و `module.exports` همان شیء خالی اولیه مانده است، پس `{}` برمی‌گردد. خروجی `undefined` (گزینه‌ی ۴) هم غلط است، چون `module.exports` از اول یک شیء خالی است.

---

۱۷۴ **گزینه‌ی ۴** متوسط

`"type": "module"` می‌گوید همه‌ی فایل‌های `.js` این پوشه ES Module هستند. در ESM تابع `require` تعریف نشده است، پس خط اول خطای `ReferenceError: require is not defined in ES module scope` می‌دهد و چیزی چاپ نمی‌شود. خطا از نوع SyntaxError نیست، چون صدا زدن یک تابع از نظر نحوی کاملاً درست است و فقط آن نام وجود ندارد. راه درست: `import fs from "fs";` یا تغییر پسوند فایل به `.cjs` .

---

۱۷۵ **گزینه‌ی ۲** سخت

با `"type": "commonjs"` ، فایل `index.js` به‌صورت CommonJS اجرا می‌شود و در CommonJS دستور `import` از نظر **نحوی** مجاز نیست؛ پس قبل از اجرای هر خطی، `SyntaxError: Cannot use import statement outside a module` رخ می‌دهد و هیچ چیزی چاپ نمی‌شود. `ReferenceError` (گزینه‌ی ۳) خطای حالت برعکس است، یعنی `require` داخل ESM. برای درست کردنش یا `"type": "module"` بنویس، یا پسوند را `.mjs` کن، یا به جای `import` از `require` استفاده کن.

---

۱۷۶ **گزینه‌ی ۳** متوسط

پسوند `.mjs` همیشه یعنی ES Module و `"type": "module"` هم فایل‌های `.js` آن پروژه را ESM می‌کند. پسوند `.cjs` و `"type": "commonjs"` دقیقاً برعکس، یعنی CommonJS را تعیین می‌کنند. فیلد `main` فقط فایل ورودی پکیج را مشخص می‌کند و ربطی به نوع ماژول ندارد.

---

۱۷۷ **گزینه‌ی ۱** متوسط

در ES Module تابع `require` وجود ندارد، ولی `typeof` روی نام تعریف‌نشده خطا نمی‌دهد و `"undefined"` برمی‌گرداند (رد گزینه‌ی ۴). مقدار `this` در بالاترین سطح یک ES Module برابر `undefined` است. `setTimeout` هم در Node.js یک تابع سراسری است (رد گزینه‌ی ۳). گزینه‌ی ۲ خروجی همین کد در یک فایل **CommonJS** است، که در آن `require` تابع و `this` همان `{}` (یعنی `module.exports`) است.

۱۷۸ **گزینه‌ی ۴** متوسط

این میم به dependency bloat اشاره دارد: npm همه‌ی پکیج‌هایی که برنامه لازم دارد، **به‌اضافه‌ی** پکیج‌هایی که خود آن‌ها لازم دارند، را در `node_modules` می‌ریزد و این پوشه از کد خود برنامه خیلی بزرگ‌تر می‌شود. اعداد میم اغراق طنز است، نه قانون (رد گزینه‌های ۱ و ۳). کد خود ما بیرون از `node_modules` است؛ این پوشه فقط پکیج‌های نصب‌شده را نگه می‌دارد (رد گزینه‌ی ۲).

۱۷۹ **گزینه‌ی ۲** متوسط

متن اسلاید ۷: «Supports powerful frameworks like Express, NestJS, and Fastify». هر سه برای ساختن سرور و API با Node.js هستند. React کتابخانه‌ای برای ساختن **رابط کاربری** (UI) در سمت frontend است (اسلاید ۱۳: «library for building interactive UIs with components») و فریم‌ورک سمت سرور نیست.

۱۸۰ **گزینه‌ی ۳** سخت

`setTimeout`، `Promise`، `console`، `JSON` و `Math` در **هر دو** محیط وجود دارند؛ اصلاً event loop و Promiseها ستون اصلی ناهمگام بودن Node.js هستند. گزینه‌ی ۱ درست است (`globalThis` در مرورگر همان `window` و در Node.js همان `global` است). گزینه‌ی ۲ درست است، چون صفحه‌ی وب برای امنیت کاربر به فایل‌های دیسک دسترسی آزاد ندارد. گزینه‌ی ۴ هم درست است، چون Node.js شیء `window` ندارد و `typeof` روی نام تعریف‌نشده `"undefined"` می‌دهد.

۱۸۱ **گزینه‌ی ۴** سخت

Node.js نسخه‌ی ۲۴ فایل `.ts` را با «پاک کردن نوع‌ها» (type stripping) اجرا می‌کند: `: number` حذف می‌شود و کد به `let count = "ten";` تبدیل می‌شود. پس `ten` چاپ می‌شود. Node.js نوع‌ها را **بررسی نمی‌کند**؛ خطای نوع را فقط کامپایلر TypeScript (`tsc`) گزارش می‌دهد (رد گزینه‌ی ۱). Node.js این نحو را می‌شناسد و پاکش می‌کند، پس SyntaxError هم رخ نمی‌دهد (رد گزینه‌ی ۲). هیچ تبدیل نوعی هم انجام نمی‌شود (رد گزینه‌ی ۳).

۱۸۲ **گزینه‌ی ۱** آسان

در CommonJS با `require("./file")` وارد می‌کنیم و با `module.exports` (یا میان‌بُر `exports.name`) صادر می‌کنیم. گزینه‌ی ۲ نحو **ES Modules** است، نه CommonJS. `include` و `module.import` در JavaScript وجود ندارند، `fetch` برای درخواست HTTP است و `window` هم اصلاً در Node.js نیست.

۱۸۳ **گزینه‌ی ۲** متوسط

راز مقیاس‌پذیری Node.js همان مدل event-driven و non-blocking است: thread اصلی کارهای I/O را به پس‌زمینه می‌سپارد و فوراً سراغ درخواست بعدی می‌رود. گزینه‌ی ۱ با تک‌نخی بودن تناقض دارد. کد سرور در مرورگر کاربر اجرا نمی‌شود (رد گزینه‌ی ۳). گزینه‌ی ۴ هم برعکس ویژگی No Buffering است.

۳-۳ مشکلات تایپ در JavaScript و معرفی TypeScript

۱۸۴ **گزینه‌ی ۳** آسان

طبق اسلاید ۱۷، TypeScript را Microsoft در سال ۲۰۱۲ معرفی کرد. گزینه‌ی ۲ سال درست را با شرکت نادرست ترکیب کرده است. Google سازنده‌ی موتور V8 است و Mozilla سازنده‌ی Firefox؛ هیچ‌کدام TypeScript را نساخته‌اند.

۱۸۵ **گزینه‌ی ۲** آسان

اسلاید ۱۷ می‌گوید: Anders Hejlsberg, lead architect of C# and Delphi. زبان‌های گزینه‌های دیگر را افراد دیگری طراحی کرده‌اند و در اسلاید هم نیامده‌اند.

۱۸۶ **گزینه‌ی ۴** آسان

در نوع‌دهی پویا نوع مال **مقدار** است، نه متغیر؛ پس `let x = 5; x = "hi";` در JavaScript کاملاً مجاز است و نوع `x` در زمان اجرا عوض می‌شود. گزینه‌ی ۱ تعریف static typing است. گزینه‌ی ۲ غلط است چون مقدارها نوع دارند (`typeof` همین را نشان می‌دهد). گزینه‌ی ۳ هم غلط است چون با یک انتساب ساده نوع متغیر عوض می‌شود و به تابع تبدیل نیازی نیست.

۱۸۷ **گزینه‌ی ۱** آسان

اگر یک طرف `+` رشته باشد، طرف دیگر هم رشته می‌شود و دو رشته به هم می‌چسبند: `"5" + 2` یعنی `"52"` (مثال اسلاید ۱۶). عملگر `-` فقط معنای ریاضی دارد، پس `"5"` به عدد ۵ تبدیل می‌شود و حاصل **عدد** `3` است، نه رشته‌ی `"3"` (گزینه‌ی ۳). گزینه‌های ۲ و ۴ فرض کرده‌اند `+` جمع ریاضی انجام می‌دهد.

۱۸۸ **گزینه‌ی ۱** آسان

superset یعنی TypeScript همه‌ی JavaScript را در بر دارد و فقط نوع‌ها را اضافه می‌کند (TypeScript = JavaScript + types). گزینه‌ی ۲ برعکس این است. گزینه‌ی ۳ غلط است چون مرورگر TypeScript را نمی‌فهمد و کد باید اول با tsc به JavaScript تبدیل شود. گزینه‌ی ۴ تعریف subset (زیرمجموعه) است، نه superset.

۱۸۹ **گزینه‌ی ۳** آسان

`tsc --init` فایل تنظیمات کامپایلر یعنی `tsconfig.json` را می‌سازد و در آن `"strict": true` را هم قرار می‌دهد. نصب TypeScript با `npm install -D typescript` انجام می‌شود (گزینه‌ی ۱). tsc اصلاً برنامه را اجرا نمی‌کند، فقط بررسی و ترجمه می‌کند (گزینه‌ی ۲). `package.json` را `npm init` می‌سازد (گزینه‌ی ۴).

۱۹۰ **گزینه‌ی ۲** متوسط

JavaScript نوع‌دهی پویا دارد: `data` اول عدد است و بعد از انتساب `"ten"` رشته می‌شود، بدون هیچ خطایی. سپس `"ten" + 5` چون یک طرف رشته است، به رشته‌ی `"ten5"` تبدیل می‌شود. گزینه‌ی ۳ رفتار TypeScript را با JavaScript قاطی کرده است (تازه TypeScript هم خطا را در زمان کامپایل می‌دهد، نه با TypeError). گزینه‌ی ۴ فکر کرده `+` جمع عددی انجام می‌دهد.

۱۹۱ **گزینه‌ی ۲** متوسط

خط ۲ یک غلط املایی دارد (`nmae`)، اما JavaScript فقط `undefined` چاپ می‌کند و خطایی نمی‌دهد؛ این همان «باگ پنهان» اسلاید ۱۶ است. در خط ۳، `toUppercase` وجود ندارد (درستش `toUpperCase` است)، پس `undefined` صدا زده می‌شود و در زمان اجرا `TypeError` رخ می‌دهد. خط ۴ دیگر اجرا نمی‌شود. در TypeScript هر دو غلط (خطوط ۲ و ۳) قبل از اجرا گزارش می‌شدند.

۱۹۲ **گزینه‌ی ۳** سخت

`Math.random()` همیشه عددی بین ۰ و ۱ است، پس شرط `> 2` هیچ‌وقت درست نمی‌شود و `shout` اصلاً اجرا نمی‌شود. چون JavaScript قبل از اجرا کد را بررسی نمی‌کند، غلط املایی `toUppercase` هیچ اثری نمی‌گذارد و فقط `start` و `end` چاپ می‌شوند. این یعنی باگ پنهان می‌ماند تا روزی که آن شاخه اجرا شود (گزینه‌ی ۱ فقط در آن حالت درست بود). اشتباه در نام متد خطای نحوی (SyntaxError) نیست و گزینه‌ی ۴ هم هم غلط املایی و هم شرط را نادیده گرفته است.

۱۹۳ **گزینه‌ی ۱** متوسط

tsc کل کد را **بدون اجرا** بررسی می‌کند؛ پس مهم نیست `shout` هیچ‌وقت صدا زده نشود. چون نوع `msg` رشته است، tsc می‌داند رشته متدی به نام `toUppercase` ندارد و در خط ۲ خطا می‌دهد (پیامش حتی پیشنهاد می‌کند: `Did you mean 'toUpperCase'?`). خط ۵ یک مقایسه‌ی عددی معمولی است و خط ۶ هم یک رشته را به تابعی می‌دهد که رشته می‌خواهد. گزینه‌ی ۴ طرز فکر JavaScript است که خطا را فقط در زمان اجرا می‌بیند.

۱۹۴ **گزینه‌ی ۴** سخت

چسباندن رشته با `+` کار مجاز و رایجی است، پس tsc عبارت `"5" + 2` را قبول می‌کند و نوع `a` را `string` می‌داند؛ یعنی TypeScript جلوی همین مثال اسلاید ۱۶ را **نمی‌گیرد**. اما تفریق رشته (`"5" - 2`) خطای TS2362 دارد، جمع دو boolean خطای TS2365 و استفاده از `null` در محاسبه خطای TS18050؛ هر سه در JavaScript بی‌صدا یک عدد می‌ساختند.

۱۹۵ **گزینه‌ی ۴** متوسط

tsc از مقدار اولیه نوع `flag` را `boolean` تشخیص می‌دهد و نسبت دادن رشته‌ی `"yes"` به آن خطای TS2322 دارد. این نمونه‌ی همان نکته است که هر کد JavaScript نحوِ معتبر TypeScript دارد ولی ممکن است خطای نوع داشته باشد. در گزینه‌ی ۱ حاصل `n + 1` عدد است. در گزینه‌ی ۲ حاصل `s + 1` رشته است و در متغیر رشته‌ای قرار می‌گیرد. در گزینه‌ی ۳ هم `const` فقط انتساب دوباره را منع می‌کند و `push` کردن عدد به آرایه‌ی عددی مجاز است.

۱۹۶ **گزینه‌ی ۱** متوسط

«تدریجی» دقیقاً یعنی لازم نیست همه‌چیز یک‌جا عوض شود؛ می‌شود فایل‌ها را کم‌کم از `.js` به `.ts` برد و فایل‌های JavaScript و TypeScript مدتی کنار هم باشند. گزینه‌ی ۲ معنای «تدریجی» است، گزینه‌ی ۳ معنای «اختیاری» (به کمک type inference) و گزینه‌ی ۴ همان بولت آخر اسلاید ۱۷.

۱۹۷ **گزینه‌ی ۲** متوسط

نوع‌ها در کامپایل پاک می‌شوند (type erasure) و مرورگر همان JavaScript معمولی را اجرا می‌کند؛ پس TypeScript سرعت اجرا را بیشتر نمی‌کند. سه گزینه‌ی دیگر دقیقاً کارهایی‌اند که ویرایشگر به لطف شناختن نوع‌ها انجام می‌دهد: autocomplete، نشان دادن خطا قبل از اجرا و rename امن.

۱۹۸ **گزینه‌ی ۴** متوسط

تک‌نخی بودن یک محدودیت اجرایی JavaScript است که در اسلاید ۳ (Limitations of JavaScript) آمده و ربطی به نوع داده ندارد. سه گزینه‌ی دیگر از بولت‌های اسلاید ۱۶اند: نوع‌دهی پویا، type coercion و نبودن compile-time checks. دو مشکل دیگر آن اسلاید هم باگ‌های پنهان در کدهای بزرگ و رفتار ناسازگار در موتورها و نسخه‌های مختلف است.

۱۹۹ **گزینه‌ی ۱** سخت

tsc به‌طور پیش‌فرض خطا را گزارش می‌کند ولی خروجی JavaScript را **باز هم** می‌سازد؛ چون نوع‌ها فقط پاک می‌شوند و کد باقی‌مانده قابل اجراست. گزینه‌ی ۲ فقط وقتی درست است که `noEmitOnError` روشن باشد. گزینه‌ی ۳ غلط است چون کار اصلی tsc گزارش خطا قبل از اجراست. tsc هم هیچ‌وقت کد را خودش اصلاح نمی‌کند (گزینه‌ی ۴).

۲۰۰ **گزینه‌ی ۲** متوسط

در کامپایل، همه‌ی type annotationها پاک می‌شوند (type erasure) و خروجی JavaScript ساده است؛ اگر داده‌ی نادرست از جایی که tsc بررسی نکرده (مثلاً یک مقدار `any` یا داده‌ی یک سرور) برسد، در زمان اجرا چیزی جلویش را نمی‌گیرد. گزینه‌ی ۱ غلط است چون نوع‌ها در خروجی نمی‌مانند. گزینه‌ی ۳ غلط است چون موتور همان JavaScript معمولی را اجرا می‌کند. گزینه‌ی ۴ غلط است چون مرورگر TypeScript را نمی‌فهمد و کد باید اول به JavaScript تبدیل شود.

۲۰۱ **گزینه‌ی ۳** سخت

چون `input` از نوع `any` است، tsc اجازه می‌دهد آن را به پارامتر `number` بدهیم. در خروجی JavaScript هم نوع‌ها پاک شده‌اند و هیچ بررسی‌ای در زمان اجرا نیست؛ پس رشته‌ی `"21"` واقعاً وارد تابع می‌شود و `"21" * 2` با تبدیل نوع خودکار `42` می‌شود. TypeScript کد بررسی نوع به خروجی اضافه نمی‌کند، پس TypeError رخ نمی‌دهد (گزینه‌ی ۲). عملگر `*` رشته نمی‌چسباند (گزینه‌ی ۱) و `"21"` یک عدد معتبر است، پس حاصل NaN نیست (گزینه‌ی ۴).

۲۰۲ **گزینه‌ی ۳** سخت

در خط ۳ اولویت `*` بیشتر است: اول `qty * price` برابر ۶ می‌شود و بعد `"Total: " + 6` یک رشته می‌سازد. چسباندن رشته در TypeScript مجاز است، پس خط ۳ خطا ندارد و نوع `msg` برابر `string` است. در خط ۴ تقسیم یک رشته بر عدد خطای TS2362 دارد (در JavaScript همین خط بی‌صدا `NaN` می‌ساخت). خط ۵ فقط دو مقدار را چاپ می‌کند. گزینه‌ی «بدون خطا» طرز فکر JavaScript است.

۳-۴ متغیرها و انواع داده در TypeScript

۲۰۳ گزینه‌ی ۲ — آسان

دونقطه و نام نوع بعد از نام متغیر، **type annotation** است؛ یعنی خودمان نوع را صریح نوشته‌ایم. type inference برعکس آن است: وقتی نوع را نمی‌نویسیم و TypeScript خودش تشخیص می‌دهد. union type با علامت `|` ساخته می‌شود و type coercion تبدیل نوع خودکار JavaScript در زمان اجراست.

۲۰۴ گزینه‌ی ۱ — آسان

`age` با annotation از نوع `number` است، پس رشته‌ی `"26"` (حتی با اینکه شبیه عدد است) قابل نسبت دادن نیست و خطای TS2322 می‌گیرد؛ TypeScript برخلاف JavaScript رشته را بی‌صدا به عدد تبدیل نمی‌کند. گزینه‌ی ۳ مجاز است چون `age + 1` عدد است. نوع `number` عدد اعشاری را هم شامل می‌شود، پس گزینه‌ی ۴ هم بی‌خطاست.

۲۰۵ گزینه‌ی ۳ — آسان

`const` یک immutable binding می‌سازد و انتساب دوباره به آن ممنوع است؛ پس خط ۴ خطای TS2588 دارد: `Cannot assign to 'PI' because it is a constant`. خط ۲ مجاز است چون `count` با `let` تعریف شده و مقدار جدید هم عدد است. خط ۳ یک تعریف معمولی است.

۲۰۶ گزینه‌ی ۱ — متوسط

`const` فقط جلوی **انتساب دوباره** به خودِ متغیر را می‌گیرد؛ پس تغییر محتوای آرایه با `push` (خط ۲) یا تغییر یک خانه (خط ۳) کاملاً مجاز است. فقط خط ۴ که می‌خواهد یک آرایه‌ی جدید به `list` نسبت دهد خطای TS2588 دارد. کسی که فکر کند `const` آرایه را «یخ‌زده» می‌کند، اشتباهاً خط ۲ را انتخاب می‌کند.

۲۰۷ گزینه‌ی ۳ — آسان

union type `number | string` یعنی `id` می‌تواند عدد **یا** رشته باشد؛ پس گزینه‌های ۱، ۲ و ۴ مجازند (عدد منفی هم `number` است). اما `true` از نوع `boolean` است که جزو این union نیست و خطای TS2322 می‌گیرد: `Type 'boolean' is not assignable to type 'string | number'`.

۲۰۸ گزینه‌ی ۴ — متوسط

هیچ annotationی نوشته نشده، ولی TypeScript از مقدار اولیه نوع `city` را `string` و نوع `year` را `number` استنتاج می‌کند (type inference). خط ۲ رشته را در رشته می‌گذارد و مجاز است. خط ۴ می‌خواهد یک عدد را در متغیر رشته‌ای بگذارد و خطای TS2322 می‌گیرد. گزینه‌ی «بدون خطا» طرز فکر JavaScript است که نوع متغیر را آزاد می‌داند.

۲۰۹ گزینه‌ی ۳ — سخت

متغیری که با `const` و یک مقدار ساده تعریف شود، نوع **literal** می‌گیرد؛ پس نوع `y` دقیقاً `5` است و در متغیری از نوع `5` جا می‌شود. اما `let x = 5` نوع عمومی `number` می‌گیرد و هر `number`ی لزوماً ۵ نیست، پس گزینه‌ی ۱ خطا دارد. گزینه‌ی ۲ به همین دلیل خطا دارد (`string` به `"hi"` نسبت داده نمی‌شود). گزینه‌ی ۴ هم انتساب دوباره به `const` است.

۲۱۰ گزینه‌ی ۴ — متوسط

از روی مقدار اولیه، TypeScript نوع `x` را «آرایه‌ای از عددها» یعنی `number[]` تشخیص می‌دهد؛ پس بعداً می‌شود هر عددی را `push` کرد ولی رشته را نه. نوع `[1, 2, 3]` یک tuple با مقدارهای ثابت است و TypeScript برای آرایه‌ی معمولی چنین نوعی نمی‌سازد. `any[]` یعنی بدون بررسی، که اینجا لازم نیست چون همه‌ی خانه‌ها عددند. `number` هم نوع یک عدد تنهاست، نه آرایه.

۲۱۱ گزینه‌ی ۲ — آسان

`number[]` و `Array<number>` دو شیوه‌ی نوشتن یک نوع‌اند: آرایه‌ای با هر تعداد عدد. `[number]` یک tuple با دقیقاً **یک** عدد است. `number | []` یعنی «یک عدد یا یک آرایه‌ی خالی». `Array[number]` اصلاً نوشتن درستی نیست و tsc برایش خطا می‌دهد (برای Array باید نوع را داخل `<>` نوشت).

۲۱۲ **گزینه‌ی ۱** متوسط

`scores` آرایه‌ای از عددهاست و `push` فقط عدد می‌پذیرد؛ رشته‌ی `"19"` خطای TS2345 می‌گیرد: `Argument of type 'string' is not assignable to parameter of type 'number'`. `19.5` یک `number` است (گزینه‌ی ۲). `Array<string>` همان `string[]` است و `"node"` را می‌پذیرد (گزینه‌ی ۳). نسبت دادن آرایه‌ی خالی به `tags` هم مجاز است، چون با `let` تعریف شده و آرایه‌ی خالی با `string[]` سازگار است (گزینه‌ی ۴).

۲۱۳ **گزینه‌ی ۲** سخت

روی مقداری از نوع `number | string` فقط کارهایی مجاز است که برای **هر دو** نوع درست باشد. `toString()` هم روی عدد هست هم روی رشته، پس خط ۲ مجاز است. `toUpperCase()` روی عدد وجود ندارد، پس خط ۳ خطای TS2339 دارد. در خط ۵، شرط `typeof id === "string"` نوع را به `string` باریک کرده (narrowing) و همان متد بدون خطاست.

۲۱۴ **گزینه‌ی ۳** متوسط

برای `5` شرط `typeof value === "number"` درست است و `toFixed(1)` عدد را با یک رقم اعشار به رشته‌ی `"5.0"` تبدیل می‌کند. برای `"ok"` شرط نادرست است و `toUpperCase()` حاصل `"OK"` را می‌دهد. گزینه‌ی ۱ اثر `toFixed(1)` را نادیده گرفته و گزینه‌های ۲ و ۴ اثر `toUpperCase()` را.

۲۱۵ **گزینه‌ی ۲** متوسط

نوع `role` یک union از دو **literal type** است، یعنی فقط دو مقدار `"admin"` و `"user"` مجازند؛ `"guest"` با اینکه رشته است، جزو این فهرست نیست و خطای TS2322 می‌گیرد. گزینه‌های ۱ و ۴ مقدارهای مجازند. گزینه‌ی ۳ هم مجاز است، چون هر کدام از این دو literal یک `string` هم هست و می‌شود آن را در متغیر `string` گذاشت (برعکسش ممکن نیست).

۲۱۶ **گزینه‌ی ۳** متوسط

هر دو هر مقداری را قبول می‌کنند، اما رفتارشان هنگام **استفاده** فرق دارد: `any` بررسی را خاموش می‌کند و `unknown` تا وقتی با شرطی مثل `typeof` نوع را باریک نکنی، اجازه‌ی صدا زدن متد یا نسبت دادن به متغیر نوع‌دار را نمی‌دهد. پس گزینه‌ی ۱ فقط درباره‌ی `any` درست است. `unknown` هر مقداری می‌گیرد، نه فقط `undefined` (گزینه‌ی ۲). گزینه‌ی ۴ برعکس است: `unknown` امن‌تر است.

۲۱۷ **گزینه‌ی ۳** متوسط

نسبت دادن هر مقداری به `unknown` مجاز است (خط ۲)، اما استفاده از آن پیش از narrowing ممنوع است؛ پس خط ۴ خطای TS18046 دارد: `'u' is of type 'unknown'`. خط ۳ بدون خطاست چون `any` بررسی را کاملاً خاموش می‌کند. اگر خط ۴ داخل `if (typeof u === "string") { … }` بود، خطا برطرف می‌شد.

۲۱۸ **گزینه‌ی ۱** سخت

چون `data` از نوع `any` است، tsc هیچ بررسی‌ای انجام نمی‌دهد و کد کامپایل می‌شود. اما در زمان اجرا مقدار `data` عدد ۴۲ است که متد `toUpperCase` ندارد؛ پس پس از چاپ `before`، خط ۳ با `TypeError` متوقف می‌شود و `after` هرگز چاپ نمی‌شود. گزینه‌ی ۲ فکر کرده چون کامپایل موفق بوده، اجرا هم بی‌خطاست؛ این دقیقاً خطر `any` است. خط ۳ چیزی چاپ نمی‌کند (گزینه‌ی ۳) و خطا هم بعد از اجرای خط ۲ رخ می‌دهد، نه قبل از آن (گزینه‌ی ۴).

۲۱۹ **گزینه‌ی ۱** متوسط

در حالت strict، `null` و `undefined` جزو نوع‌های `number`، `string` و `boolean` نیستند؛ پس گزینه‌های ۲ تا ۴ خطای TS2322 دارند. اگر متغیری ممکن است خالی بماند، باید آن را صریحاً در union بیاوری: `number | null` (گزینه‌ی ۱).

۲۲۰ **گزینه‌ی ۲** متوسط

`[string, number]` یک tuple با **دقیقاً دو خانه** است: خانه‌ی ۰ رشته و خانه‌ی ۱ عدد. خط‌های ۲ و ۳ در هر خانه مقداری با نوع درست می‌گذارند. اما tuple خانه‌ی شماره‌ی ۲ ندارد، پس خط ۴ خطای TS2493 می‌گیرد: `Tuple type '[string, number]' of length '2' has no element at index '2'`. در JavaScript همین کار فقط `undefined` چاپ می‌کرد.

۲۲۱ گزینه‌ی ۴ آسان

در tuple **ترتیب** نوع‌ها مهم است: خانه‌ی اول باید رشته و خانه‌ی دوم عدد باشد. گزینه‌ی ۴ ترتیب را برعکس کرده و خطای TS2322 می‌گیرد. رشته‌ی خالی و عدد صفر یا منفی هم رشته و عدد معتبرند، پس سه گزینه‌ی دیگر مجازند.

---

۲۲۲ گزینه‌ی ۱ سخت

`(string | number)[]` یعنی «آرایه‌ای که **هر خانه‌اش** می‌تواند عدد یا رشته باشد»، پس `[1, "x"]` در آن جا می‌شود. `string | number[]` (بدون پرانتز) یعنی «یک رشته **یا** یک آرایه‌ی تمام‌عددی». `string[] | number[]` یعنی «آرایه‌ای که یا همه‌اش رشته است یا همه‌اش عدد»؛ پس آرایه‌ی مخلوط در هیچ‌کدام جا نمی‌شود. `number[]` هم رشته‌ی `"x"` را نمی‌پذیرد.

---

۲۲۳ گزینه‌ی ۴ سخت

تعریف یک متغیر `let` با نوع ولی بدون مقدار اولیه مجاز است (خط ۱)؛ این کار فقط برای `const` ممنوع است. اما در خط ۴ از `total` پیش از مقداردهی استفاده شده و tsc خطای TS2454 می‌دهد: `Variable 'total' is used before being assigned`. مقداردهی در خط ۵ دیر است. خط ۳ هم یک عملیات عددی عادی روی `count` است.

---

۲۲۴ گزینه‌ی ۴ سخت

`const` فقط انتساب دوباره به `arr` را ممنوع می‌کند؛ نوع `arr` همان `number[]` است و `arr.push(3)` بدون خطا کامپایل و اجرا می‌شود؛ پس گزینه‌ی ۴ نادرست است. سه گزینه‌ی دیگر قاعده‌های درست type inference‌اند: `let` نوع عمومی (`number`) می‌گیرد، `const` با مقدار ساده نوع literal (`5`) و آرایه‌ی رشته‌ها نوع `string[]`.

---

۲۲۵ گزینه‌ی ۴ آسان

`id` یک بار عدد و یک بار رشته می‌گیرد، پس به union type `number | string` نیاز دارد (همان مثال اسلاید ۱۸). با `number` خط سوم و با `string` خط دوم خطا می‌دهد و با `boolean` هر دو خط.

---

۲۲۶ گزینه‌ی ۱ سخت

`status` با `const` تعریف شده، پس نوعش literal یعنی `"ok"` است و در متغیری از نوع `"ok" | "error"` جا می‌شود (خط ۲ مجاز است). اما `message` با `let` تعریف شده و نوعش `string` است؛ هر رشته‌ای لزوماً `"ok"` یا `"error"` نیست، پس خط ۴ خطای TS2322 دارد. مقدار هر دو متغیر یکی است، ولی **نوعشان** فرق دارد.

---

۲۲۷ گزینه‌ی ۲ متوسط

union type فقط در زمان کامپایل وجود دارد و در JavaScript خروجی پاک می‌شود. در زمان اجرا `typeof` نوع **مقدار فعلی** را برمی‌گرداند: اول `7` که `number` است و بعد `"seven"` که `string` است. گزینه‌ی ۱ فکر کرده union در زمان اجرا هم هست. گزینه‌ی ۳ فکر کرده نوع با اولین مقدار ثابت می‌ماند و گزینه‌ی ۴ آخرین نوع را برای هر دو خط در نظر گرفته است.

۳-۵ کلاس‌ها در TypeScript

---

۲۲۸ گزینه‌ی ۳ آسان

این کد فقط کلاس (نقشه) را تعریف می‌کند؛ هیچ شیئی با `new` ساخته نشده و `greet()` هم صدا زده نشده است، پس **هیچ خروجی‌ای** ندارد. constructor فقط هنگام `new` اجرا می‌شود و `console.log` فقط وقتی `greet` صدا زده شود. گزینه‌های ۱ و ۲ فرض کرده‌اند تعریف کلاس خودبه‌خود متد را اجرا می‌کند و گزینه‌ی ۴ هم دلیلی ندارد، چون هیچ متغیر تعریف‌نشده‌ای استفاده نشده است.

---

۲۲۹ گزینه‌ی ۱ آسان

`new Person("Alice")` constructor را اجرا می‌کند و `this.name` برابر `"Alice"` می‌شود. بعد `p.greet()` با template literal مقدار `${this.name}` را جایگزین می‌کند و `Hello, Alice` چاپ می‌شود. گزینه‌ی ۲ فراموش کرده که داخل بک‌تیک `${...}` با مقدار جایگزین می‌شود، گزینه‌ی ۳ فکر کرده constructor مقدار را ذخیره نکرده و گزینه‌ی ۴ اسم property را با مقدارش اشتباه گرفته است.

---

۲۳۰ **گزینه‌ی ۴** آسان

در TypeScript سطح دسترسی پیش‌فرض `public` است؛ یعنی `name: string` دقیقاً همان `public name: string` است. `private` و `protected` را باید صریحاً بنویسی (گزینه‌های ۱ و ۲). `readonly` هم ربطی به سطح دسترسی ندارد و فقط وقتی اعمال می‌شود که خودت آن را بنویسی (گزینه‌ی ۳).

---

۲۳۱ **گزینه‌ی ۲** آسان

`protected` یعنی «خودِ کلاس و فرزندانش»؛ استفاده از بیرون (مثل `s.age`) خطای TS2445 می‌دهد. گزینه‌ی ۱ غلط است چون `private` حتی در کلاس فرزند هم ممنوع است (TS2341). گزینه‌ی ۳ رفتار `protected` را با `readonly` قاطی کرده؛ از `protected` از بیرون نه خواندنی است نه نوشتنی. گزینه‌ی ۴ هم غلط است چون `public` پیش‌فرض است و نوشتنش اختیاری است.

---

۲۳۲ **گزینه‌ی ۳** آسان

access modifierها (`public`، `private`، `protected`) کلیدواژه‌های TypeScript هستند و در JavaScript وجود ندارند (JavaScript فقط فیلدهای `#` را دارد که سازوکار دیگری است). در مقابل، `constructor`، `extends` و متدها از ES6 بخشی از خودِ JavaScript هستند، پس گزینه‌های ۱، ۲ و ۴ امکان تازه‌ای نیستند.

---

۲۳۳ **گزینه‌ی ۱** آسان

وراثت در TypeScript (و JavaScript) با `extends` است؛ `Student` بدون constructor خودش، constructor `Person` را به کار می‌برد و `name` را هم به ارث می‌برد. `implements` فقط «قول رعایت شکل» است و چیزی به ارث نمی‌برد؛ این‌جا property `name` را `Student` ندارد، پس خطای TS2720 می‌دهد. `inherits` اصلاً کلیدواژه‌ی زبان نیست (خطای نحوی) و `super` برای صدا زدن constructor یا متد والد است، نه تعریف وراثت.

---

۲۳۴ **گزینه‌ی ۱** آسان

`implements` یک **قرارداد زمان کامپایل** است: اگر عضوی کم باشد خطای TS2420 و اگر نوعش ناسازگار باشد TS2416 می‌گیری. interface هیچ کدی ندارد که به کلاس اضافه شود (گزینه‌ی ۲)، و چون interface و `implements` در JavaScript خروجی پاک می‌شوند، هیچ بررسی‌ای در زمان اجرا انجام نمی‌شود (گزینه‌ی ۳). `extends` ارث‌بری واقعی از یک کلاس است، نه بررسی شکل (گزینه‌ی ۴).

---

۲۳۵ **گزینه‌ی ۳** متوسط

`id` در کلاس `Order` نه مقدار اولیه دارد و نه constructorی که آن را مقدار بدهد، پس خطای **TS2564** (`Property 'id' has no initializer and is not definitely assigned in the constructor`) می‌گیرد. خط ۲ مشکلی ندارد چون `title` در constructor مقدار می‌گیرد؛ خط ۴ اختیاری (`?`) است و می‌تواند `undefined` بماند؛ خط ۱۱ هم مقدار اولیه‌ی `[]` دارد.

---

۲۳۶ **گزینه‌ی ۴** متوسط

`age` `protected` فقط داخل `Person` و فرزندانش قابل استفاده است؛ پس `this.age` در خط ۱۳ (داخل متد `Student`) مجاز است، ولی `s.age` در خط ۱۹ از بیرون کلاس است و خطای **TS2445** می‌دهد. خط ۱۷ مشکلی ندارد چون `name` `public` است و خط ۱۸ هم یک متد `public` را صدا می‌زند.

---

۲۳۷ **گزینه‌ی ۱** متوسط

`balance` `private` فقط داخل خودِ `Account` قابل استفاده است؛ حتی کلاس فرزند (`Savings`) هم به آن دسترسی ندارد، پس خط ۱۰ خطای **TS2341** می‌دهد. خط ۱۳ مجاز است چون `owner` `protected` در فرزند قابل استفاده است، و خط ۱۷ متد `deposit` `public` را صدا می‌زند که خودش داخل `Account` به `balance` دست می‌زند. کسی که «بدون خطا» را انتخاب کند، فکر کرده `private` هم مثل `protected` به فرزند می‌رسد.

---

۲۳۸ **گزینه‌ی ۲** متوسط

property `readonly` فقط کنار اعلانش یا **داخل constructor** مقدار می‌گیرد؛ پس خط ۵ مجاز است ولی خط ۸ (داخل متد معمولی `setUrl`) خطای **TS2540** (`Cannot assign to 'apiUrl' because it is a read-only property`) می‌دهد. خط ۱۱ فقط شیء می‌سازد و خواندن propertyهای `readonly` در خط ۱۲ هم آزاد است. «متد همان کلاس است پس مجاز است» دام اصلی این تست است.

---

۲۳۹ **گزینه‌ی ۴** متوسط

`Circle` قول داده `Shape` را پیاده کند ولی متد `area()` را ندارد؛ پس روی خط ۱۲ خطای **TS2420** (`Class 'Circle' incorrectly implements interface 'Shape'`) گزارش می‌شود. `Square` هم `name` دارد و هم `area()` با خروجی `number`، پس خطوط ۵ تا ۸ درست‌اند. داشتن property اضافه (`side` یا `r`) هم برای `implements` مشکلی ندارد.

۲۴۰ **گزینه‌ی ۲** متوسط

`count` چون `static` است فقط **یک نسخه‌ی مشترک** دارد که مال خودِ کلاس است. با هر `new` یکی زیاد می‌شود و همان لحظه در `id` آن شیء کپی می‌شود: `a.id = 1`، `b.id = 2`، `c.id = 3` و در پایان `Counter.count = 3`. گزینه‌ی ۱ فکر کرده هر شیء شمارنده‌ی جدای خودش را دارد، گزینه‌ی ۳ فکر کرده `id` به شمارنده وصل می‌ماند و بعداً عوض می‌شود، و گزینه‌ی ۴ ترتیب افزایش و کپی را برعکس فرض کرده است.

۲۴۱ **گزینه‌ی ۴** متوسط

هر سه پارامتر modifier دارند، پس هر سه **parameter property** هستند و خودکار به property تبدیل می‌شوند؛ `z` هم مقدار پیش‌فرض `0` را می‌گیرد. پس `pt.sum()` برابر `1 + 2 + 0 = 3` است. در JavaScript خروجی `y` یک property معمولی است (`private` فقط در زمان کامپایل معنا دارد)، پس `JSON.stringify` هر سه را نشان می‌دهد. گزینه‌ی ۱ فکر کرده `private` در زمان اجرا پنهان می‌ماند و گزینه‌های ۲ و ۳ فکر کرده‌اند پارامترهای constructor به property تبدیل نمی‌شوند.

۲۴۲ **گزینه‌ی ۳** متوسط

فقط در `C` مطمئنیم `name` همیشه مقدار دارد (مقدار اولیه‌ی `""`). `A` اصلاً مقداردهی نمی‌کند، `B` فقط گاهی (داخل `if`) مقدار می‌دهد و `D` مقداردهی را به متدی سپرده که شاید هرگز صدا زده نشود؛ هر سه خطای **TS2564** می‌گیرند. strict mode می‌خواهد property کنار اعلان یا در **همه‌ی** مسیرهای constructor مقدار بگیرد.

۲۴۳ **گزینه‌ی ۴** متوسط

مقداردهی دوباره‌ی property `readonly` فقط کنار اعلان (گزینه‌ی ۱) یا در constructor (گزینه‌ی ۲) مجاز است؛ در یک متد معمولی، حتی از همان کلاس، خطای **TS2540** می‌دهد، پس گزینه‌ی ۴ نادرست است. گزینه‌ی ۳ درست است چون `readonly` «سطحی» است: فقط جلوی **عوض کردن خودِ property** (مثلاً `this.items = []`) را می‌گیرد، نه تغییر محتوای آرایه‌ای که به آن اشاره می‌کند.

۲۴۴ **گزینه‌ی ۱** متوسط

عضو `static` مال **خودِ کلاس** است و با اسم کلاس صدا زده می‌شود: `Counter.count++`. دسترسی از طریق شیء (`c.count`) خطای **TS2576** می‌دهد و TypeScript خودش پیشنهاد می‌کند `Counter.count` بنویسی. در سطح بالای یک ماژول `this` برابر `undefined` است (خطای TS2532)، و `count` به‌تنهایی هم متغیری تعریف‌نشده است (TS2304).

۲۴۵ **گزینه‌ی ۲** سخت

در constructor کلاس فرزند، **قبل از** صدا زدن `super(...)` شیء هنوز ساخته نشده و نمی‌توان از `this` استفاده کرد؛ پس خط ۷ خطای **TS17009** (`'super' must be called before accessing 'this'`) می‌دهد. خودِ `super(name)` در خط ۸ مشکلی ندارد، فقط باید جابه‌جا شود. خط ۵ هم خطا ندارد چون `breed` در constructor مقدار گرفته است. در JavaScript خالی همین ترتیب در زمان اجرا `ReferenceError` می‌دهد.

۲۴۶ **گزینه‌ی ۲** سخت

فقط پارامتری که **modifier** دارد (`public name`) به property تبدیل می‌شود. `age` یک پارامتر معمولی است که فقط داخل constructor زنده است، پس `this.age` در خط ۴ خطای **TS2339** (`Property 'age' does not exist on type 'User'`) می‌دهد. خط ۲ خودش درست است (نوشتن پارامتر بدون modifier مجاز است)، و خطوط ۷ و ۸ هم با دو آرگومان و property `public` `name` کاملاً درست‌اند.

۲۴۷ **گزینه‌ی ۴** سخت

`override` یعنی «این متد، متدی از والد را بازنویسی می‌کند»؛ چون `Animal` متدی به اسم `swim` ندارد، گزینه‌ی ۴ خطای **TS4113** (`This member cannot have an 'override' modifier because it is not declared in the base class 'Animal'`) می‌دهد. گزینه‌ی ۱ بدون `override` هم درست است (در strict نوشتن `override` اجباری نیست)، گزینه‌ی ۲ با `override` درست بازنویسی کرده و گزینه‌ی ۳ هم یک متد تازه اضافه کرده که اصلاً بازنویسی نیست.

---

۲۴۸ **گزینه‌ی ۱** سخت

متد بازنویسی‌شده باید با نسخه‌ی والد **سازگار** باشد. `Fish` نوع خروجی `speak` را از `string` به `number` عوض کرده و خطای **TS2416** (`Property 'speak' in type 'Fish' is not assignable to the same property in base type 'Animal'`) می‌گیرد. در گزینه‌ی ۳ نوع خروجی نوشته نشده ولی از `return "Moo"` نوع `string` استنتاج می‌شود و سازگار است؛ گزینه‌ی ۴ هم با `super.speak()` نسخه‌ی والد را صدا زده که کاملاً مجاز است.

---

۲۴۹ **گزینه‌ی ۲** سخت

این‌که نوع آرایه `Animal[]` است فقط برای بررسی در زمان کامپایل است؛ در زمان اجرا برای هر شیء، متدِ **کلاس واقعی‌اش** اجرا می‌شود (polymorphism). برای `Rex` که یک `Dog` است، `speak` `Dog` اجرا می‌شود که اول با `super.speak()` متن والد (`Rex makes a sound`) را می‌گیرد و `- Woof` را به آن اضافه می‌کند. گزینه‌ی ۱ فکر کرده نوع آرایه تعیین می‌کند کدام متد اجرا شود، گزینه‌ی ۳ نقش `super.speak()` را نادیده گرفته و `Dog` بدون constructor هم از constructor `Animal` استفاده می‌کند، پس خطایی (گزینه‌ی ۴) رخ نمی‌دهد.

---

۲۵۰ **گزینه‌ی ۳** سخت

`private` فقط در زمان کامپایل بررسی می‌شود و در JavaScript خروجی `pin` یک property معمولی است؛ پس هم در `Object.keys` و هم در `JSON.stringify` دیده می‌شود (گزینه‌ی ۱ غلط است). در مقابل، `#balance` یک فیلد خصوصی **واقعی** JavaScript است و نه جزو کلیدهای شیء حساب می‌شود و نه در JSON می‌آید (گزینه‌های ۲ و ۴ غلط‌اند).

---

۲۵۱ **گزینه‌ی ۴** سخت

`p.greet()` چون شیء قبل از نقطه است، `this` را `p` می‌گیرد و `Hello, Alice` چاپ می‌شود. اما `f()` بدون شیء صدا زده شده و چون بدنه‌ی کلاس همیشه **strict** است، `this` برابر `undefined` می‌شود؛ خواندن `this.name` خطای `TypeError` می‌دهد. گزینه‌ی ۱ فکر کرده متد همیشه به شیء اولش چسبیده است، گزینه‌ی ۲ رفتار حالت غیر strict را فرض کرده و گزینه‌ی ۳ فراموش کرده خط اولِ اجرا (`p.greet()`) قبل از خطا خروجی داده است. TypeScript این اشتباه را نمی‌گیرد.

---

۲۵۲ **گزینه‌ی ۳** سخت

access modifierهای TypeScript بعد از کامپایل پاک می‌شوند؛ property `private` در JavaScript خروجی **باقی می‌ماند** (گزینه‌ی ۱ غلط) و یک property معمولی است، پس `obj["pin"]` در زمان اجرا مقدارش را برمی‌گرداند و هیچ خطایی نمی‌دهد (گزینه‌ی ۴ غلط؛ TypeScript هم دسترسی با براکت را مجاز می‌داند). گزینه‌ی ۲ برعکس است: property `private` در `Object.keys` دیده می‌شود ولی فیلد `#` نه. پس فقط گزینه‌ی ۳ درست است.

۳-۶ Generics در TypeScript

۲۵۳ **گزینه‌ی ۱** آسان

اسلاید ۲۲ می‌گوید generics کامپوننت‌ها و تابع‌های **reusable** و **type-safe** می‌سازند و جلوی تکرار کد را می‌گیرند «بدون از دست رفتن بررسی نوع». generics فقط در زمان کامپایل وجود دارند و هیچ اثری روی سرعت اجرا یا تبدیل مقدارها ندارند (گزینه‌های ۲ و ۴). گزینه‌ی ۳ دقیقاً برعکس است: `any` بررسی نوع را خاموش می‌کند ولی generic آن را حفظ می‌کند.

---

۲۵۴ **گزینه‌ی ۳** آسان

`T` یک type parameter است: همان‌طور که `value` جای خالی برای مقدار است، `T` جای خالی برای **نوع** است و با `identity<number>(42)` (یا با استنتاج) پر می‌شود. `T` لازم نیست جایی تعریف شده باشد (گزینه‌ی ۱)، با `value` فرق دارد چون یکی نوع است و دیگری مقدار (گزینه‌ی ۲)، و برخلاف `any` نوع را حفظ می‌کند (گزینه‌ی ۴).

---

۲۵۵ **گزینه‌ی ۲** آسان

`identity` همان مقدار را برمی‌گرداند، پس `num` عدد `42` و `str` رشته‌ی `"Hello"` است و `typeof` نوع واقعی زمان اجرا را می‌دهد: `number` و `string`. `T` و `any` فقط در زمان کامپایل وجود دارند و در JavaScript خروجی پاک شده‌اند، پس `typeof` هرگز `T` یا `any` برنمی‌گرداند (گزینه‌های ۱ و ۴). `object` هم فقط برای اشیاء (و `null`) است، نه عدد و رشته (گزینه‌ی ۳).

---

۲۵۶ **گزینه‌ی ۴** آسان

اسلاید ۲۲ صریحاً می‌گوید «generics can be applied to functions, classes, and interfaces»: مثل `function identity<T>`، `class Stack<T>` و `interface Box<T>`. هر سه گزینه‌ی دیگر با کلمه‌ی «فقط» یکی از این سه را کنار گذاشته‌اند؛ حتی در Java هم generic مخصوص کلاس‌ها نیست.

---

۲۵۷ **گزینه‌ی ۳** آسان

در خط ۶ آرگومان نوع صریحاً `number` است، پس پارامتر `value` باید `number` باشد؛ دادن `"Hello"` خطای **TS2345** (`Argument of type 'string' is not assignable to parameter of type 'number'`) می‌دهد. خطوط ۴ و ۵ مقدار درست را با آرگومان نوع درست داده‌اند و در خط ۷ آرگومان نوع نوشته نشده و از روی `true` استنتاج می‌شود، پس هیچ‌کدام خطا ندارند.

---

۲۵۸ **گزینه‌ی ۲** آسان

`Array` یک نوع generic آماده است و آرگومان نوعش داخل `< >` می‌آید: `Array<string>` دقیقاً همان `string[]` است.
`Array[string]` شبیه دسترسی به عضو آرایه است و خطای TS2314 (`Generic type 'Array<T>' requires 1 type argument(s)`) می‌دهد، `Array(string)` شبیه فراخوانی تابع است و خطای نحوی دارد، و در `string<Array>` جای نوع و آرگومان عوض شده است (`string` اصلاً generic نیست).

---

۲۵۹ **گزینه‌ی ۴** متوسط

با `let`، نوع `a` گسترده‌ی `number` استنتاج می‌شود (پس خط ۶ مجاز است)، ولی با `const`، نوع `b` دقیقاً `42` است (پس خط ۷ هم مجاز است). در خط ۸ می‌خواهیم یک `number` کلی را در متغیری از نوع `42` بگذاریم و خطای **TS2322** (`Type 'number' is not assignable to type '42'`) می‌گیریم؛ TypeScript نمی‌تواند مطمئن باشد `a` (که قبلاً `100` شده) همیشه `42` است.

---

۲۶۰ **گزینه‌ی ۱** متوسط

آرگومان نوع نوشته نشده، پس TypeScript آن را از روی `"Hello"` استنتاج می‌کند؛ چون `s` با `let` تعریف شده و بعداً می‌تواند عوض شود، نوع **گسترده‌ی** `string` را می‌گیرد. اگر `const s` بود، نوع دقیق `"Hello"` می‌شد (گزینه‌ی ۲). generic اطلاعات نوع را حفظ می‌کند، پس نتیجه `any` نیست (گزینه‌ی ۳)، و `T` فقط داخل تعریف تابع معنا دارد و بعد از فراخوانی با یک نوع واقعی جایگزین می‌شود (گزینه‌ی ۴).

---

۲۶۱ **گزینه‌ی ۳** متوسط

پشته LIFO است: آخرین عضوی که وارد شده (`"c"`) اول بیرون می‌آید و بعد از آن دو عضو باقی می‌ماند، پس `c 2`. `pop` روی آرایه‌ی خالی خطا نمی‌دهد و فقط `undefined` برمی‌گرداند؛ برای همین نوع خروجی `T | undefined` است و خط دوم `undefined 0` چاپ می‌کند.
گزینه‌ی ۱ رفتار صف (FIFO) را فرض کرده، گزینه‌ی ۲ فراموش کرده `pop` عضو را حذف می‌کند و گزینه‌ی ۴ فکر کرده پشته‌ی خالی خطا می‌دهد.

---

۲۶۲ **گزینه‌ی ۳** متوسط

با `new Stack<number>()`، `T` برابر `number` است و `push` فقط `number` می‌پذیرد؛ پس `s.push("4")` در خط ۱۳ خطای **TS2345** می‌دهد. `2 * 3` یک عدد است (خط ۱۲ درست)، و خط ۱۴ فقط نتیجه را در `top` (با نوع `number | undefined`) می‌گذارد که مشکلی ندارد. خودِ کلاس هم با یک بار نوشتن برای هر نوعی کار می‌کند.

---

۲۶۳ **گزینه‌ی ۴** متوسط

قید `T extends { length: number }` یعنی هر آرگومانی باید یک `length` عددی داشته باشد. رشته، آرایه و حتی شیء `{ length: 7 }` این شرط را دارند، ولی عدد `42` ندارد؛ پس خط ۸ خطای **TS2345** (`Argument of type 'number' is not assignable to parameter of type '{ length: number; }'`) می‌دهد. خط ۲ هم دقیقاً به لطف همین قید مجاز است.

---

۲۶۴ **گزینه‌ی ۱** متوسط

`length` رشته‌ی `"hello"` برابر ۵ و `length` آرایه‌ی سه‌عضوی ۳ است. چون خروجی تابع از نوع `T` است، `r` همان رشته‌ی `"hello"` است (و TypeScript هم می‌داند `string` است)، پس `toUpperCase()` بدون مشکل `HELLO` می‌دهد. گزینه‌ی ۲ فکر کرده نتیجه رشته نیست، گزینه‌ی ۳ خودِ مقدار را به‌جای `length` چاپ کرده و گزینه‌ی ۴ فکر کرده آرایه `length` ندارد.

---

۲۶۵ **گزینه‌ی ۴** متوسط

هر دو هر ورودی‌ای می‌پذیرند، ولی فرق اصلی در **خروجی** است: `f<T>` می‌گوید خروجی همان نوع ورودی است (مثلاً `string`)، ولی `any` هر اطلاعاتی را دور می‌ریزد؛ پس گزینه‌ی ۱ غلط است. آرگومان نوع معمولاً خودکار استنتاج می‌شود (گزینه‌ی ۲ غلط) و generics در زمان اجرا وجود ندارند که بررسی‌ای انجام دهند (گزینه‌ی ۳ غلط).

---

۲۶۶ **گزینه‌ی ۲** متوسط

در `Pair<string, number>`، `K = string` و `V = number` است؛ پس `value` باید عدد باشد و رشته‌ی `"30"` خطای **TS2322** (`Type 'string' is not assignable to type 'number'`) می‌دهد. در بقیه‌ی گزینه‌ها هر مقدار با پارامتر نوع متناظرش جور است؛ ترتیب آرگومان‌های نوع مهم است (`<number, string>` در گزینه‌ی ۳ یعنی کلید عدد و مقدار رشته) و دو پارامتر نوع می‌توانند یکسان هم باشند (گزینه‌ی ۴).

---

۲۶۷ **گزینه‌ی ۳** متوسط

در `Box<boolean>`، نوع `value` برابر `boolean` است؛ `"true"` (بین کوتیشن) یک **رشته** است نه مقدار بولی، پس خط ۶ خطای **TS2322** می‌دهد. آرگومان نوع می‌تواند هر نوعی باشد، حتی آرایه (`number[]` در خط ۵)، و `0` هم یک `number` معتبر است (خط ۷).

---

۲۶۸ **گزینه‌ی ۲** سخت

در تابع `size` هیچ قیدی روی `T` نیست، پس TypeScript نمی‌داند `x` چیست (ممکن است عدد باشد) و `x.length` خطای **TS2339** (`Property 'length' does not exist on type 'T'`) می‌دهد؛ راه‌حلش `<T extends { length: number }>` است. در `first` پارامتر از نوع `T[]` (آرایه) است و دسترسی `items[0]` همیشه مجاز است؛ خطوط ۷ و ۸ هم به‌ترتیب `number | undefined` و `string | undefined` می‌گیرند.

---

۲۶۹ **گزینه‌ی ۱** سخت

چون خروجی `any` است، TypeScript هیچ بررسی‌ای روی `y` نمی‌کند و `y.toFixed(2)` را قبول می‌کند. ولی `y` در زمان اجرا یک رشته است و رشته‌ها متد `toFixed` ندارند: `y.toFixed` برابر `undefined` است و صدا زدن آن `TypeError: y.toFixed is not a function` می‌دهد و برنامه همان‌جا متوقف می‌شود (پس `done` چاپ نمی‌شود). این دقیقاً همان خطری است که generics جلویش را در زمان کامپایل می‌گیرند.

---

۲۷۰ **گزینه‌ی ۳** سخت

نوع `x` (خروجی generic) رشته است (دقیقاً `"hi"`)، پس `toUpperCase` مجاز است ولی `x.toFixed(2)` در خط ۱۱ خطای **TS2551** (`Property 'toFixed' does not exist on type '"hi"'`) می‌دهد. خط ۱۰ با این‌که در زمان اجرا می‌ترکد، **در کامپایل خطایی ندارد**، چون `y` از نوع `any` است و بررسی نوع برایش خاموش است. این تفاوت generic و `any` است.

---

۲۷۱ **گزینه‌ی ۴** سخت

هر دو پارامتر از **یک** نوع `T` هستند. در گزینه‌ی ۴، TypeScript نوع `T` را از آرگومان اول حدس می‌زند و بعد آرایه‌ی `[1, 2]` با آن جور نیست؛ خطای **TS2345** (`Argument of type 'number[]' is not assignable to parameter of type '"abc"'`) می‌دهد. در گزینه‌های ۱ و ۲ هر دو آرگومان هم‌نوع‌اند، و در گزینه‌ی ۳ آرگومان نوع صریحاً `string | number[]` است که هر دو آرگومان را می‌پذیرد و `length` هم دارد، پس خطایی نیست.

---

۲۷۲ **گزینه‌ی ۲** سخت

generics در TypeScript فقط در زمان کامپایل وجود دارند و در JavaScript خروجی **کاملاً پاک** می‌شوند (کد `identity` بعد از کامپایل هیچ `<T>`ی ندارد)؛ پس در زمان اجرا هیچ راهی برای فهمیدن `T` نیست و گزینه‌ی ۲ نادرست است. گزینه‌ی ۱ شباهت نحوی دو زبان، گزینه‌ی ۳ استنتاج آرگومان نوع (`identity(42)`) و گزینه‌ی ۴ کلاس‌های generic مثل `Stack<T>` را درست بیان می‌کنند.

---

۲۷۳ **گزینه‌ی ۱** سخت

وقتی آرگومان نوع **صریحاً** نوشته شود، استنتاج انجام نمی‌شود و `T` همان چیزی است که نوشته‌ایم: `number | string` (ترتیب اعضای union مهم نیست و TypeScript آن را `string | number` نمایش می‌دهد). پس `g` بعداً می‌تواند رشته هم بگیرد (`g = "five"`). اگر `<...>` را نمی‌نوشتیم، با `let` نوع `number` (گزینه‌ی ۲) و با `const` نوع `5` (گزینه‌ی ۳) استنتاج می‌شد؛ `any` هم هیچ‌وقت نتیجه‌ی generic نیست.
